Explicación del diagrama reducido – Defensa 5

Proyecto: The Legend of Zelda: Link's Awakening (versión simplificada) · Tecnología de Programación 2026 · UNS

Este archivo acompaña a `Diagrama-reducido-defensa-5.drawio` (y a su PNG y su PDF), el diagrama reducido para la defensa 5. Parte del explicativo del reducido puntuable de la defensa 4 y le suma el **modelo de hilos**: los hilos del juego, la transición como animación de la capa gráfica, las acciones programadas sin hilos extra y cómo se resuelven las colisiones cuando hay más de un hilo.

Explica el **diagrama reducido**: todas las clases, interfaces y relaciones (roles y multiplicidades), la interconexión lógica ↔ gráfica con el mapeo uno a uno entre entidad lógica y celda gráfica (Observer), el formato y parseo de niveles, los patrones de diseño, el funcionamiento de la partida con diagramas de secuencia, la posición en píxeles y en la matriz, los hilos y una guía para la defensa.

Junta en un solo documento toda la información disponible:

- el **enunciado** del proyecto, resumido en la parte A;
- los **requerimientos de la defensa 5** y las condiciones de entrega, en la parte B;
- lo que **cambió respecto de la defensa 4**, en C.1;
- la **explicación del diagrama** (secciones 1 a 14), con ejemplos de código, comunicación entre clases y diagramas de secuencia;
- la **posición en píxeles y en la matriz**, en la sección 15;
- los **hilos, las animaciones y las colisiones**, en la sección 16.

El diagrama extendido **no cambió** para esta entrega: su explicación sigue siendo la del extendido puntuable. Lo que habría que cambiarle para que coincida con este reducido está en 16.11. Las imágenes de las cajas de cada clase están en `Defensa 4/imagenes/clases/`.

> Los fragmentos de código son **esquemas** para explicar el diseño: usan los nombres del diagrama extendido de la defensa 4, salvo lo que cambia con los hilos (sección 16), que se marca en cada lugar. No son la implementación final.

Índice

- A. El enunciado en resumen
  - A.1 Reglas duras (su incumplimiento desaprueba)
  - A.2 Requisitos funcionales
  - A.3 Entidades del juego (valores del enunciado)
  - A.4 Ambigüedades del enunciado
- B. Qué pide la defensa 5
- C. El diagrama
  - C.1 Qué cambia respecto del reducido de la defensa 4
  - C.2 Historia: qué cambió de la defensa 3 a la defensa 4
- 1. Cómo leer el diagrama
- 2. Vista general
- 3. Vista y controladores

    - `GUI`
    - `Ventana`
    - `JPanel` / `JLabel`
    - `PanelVista*`
    - `PanelPantallaPrincipal`
    - `PanelPantallaRanking`
    - `PanelPantallaNivel`
    - `AnimadorTransicion`
    - `<<Interface>> ControladorJuego` (vista → lógica)
    - `<<Interface>> ControladorGrafico` (lógica → vista)
    - `<<Interface>> ControladorDeVistas`
- 4. Observer (lógica → gráfica)
    - `<<Interface>> Observer`
    - `<<Interface>> EntidadLogica`
    - `<<Interface>> EntidadJugador`
    - `CeldaGrafica`
    - `CeldaJugador`
- 5. Núcleo del juego
    - `Juego`
    - `Reloj`
    - Hilos: `HiloJuego*`, `HiloJugador`, `HiloMundo`
    - `ModoJuego*`
    - `Ranking`
    - `EntradaRanking`
    - `PersistenciaRanking`
- 6. Mapa y niveles
    - `GeneradorNivel`
    - `Nivel`
    - `Escena`
    - `Sector`
- 7. Formato y parseo de niveles
    - 7.1 Quién lee el archivo y con qué relaciones
    - 7.2 El formato en una pantalla
    - 7.3 Qué objeto y qué relación crea cada línea
    - 7.4 Qué crea cada símbolo
    - 7.5 Ejemplo: el sector (0,0) del nivel 1, línea por línea
    - 7.6 Ejemplos del nivel 2
    - 7.7 Lo que no está en el archivo y quién lo crea
    - 7.8 Código del parseo (esquema)
    - 7.9 Diagrama de secuencia de la carga
    - 7.10 El objeto que queda armado
    - 7.11 Errores del archivo y cómo se detectan
- 8. Fábricas y sprites (patrón Abstract Factory)
    - `FabricaEntidades`
    - `FabricaSprites*`
    - `Sprite`
- 9. Entidades
    - `Entidad*`
    - `<<Interface>> Colisionable`
    - `<<Interface>> Colisionador`
    - Obstáculos

    - Power-ups
    - Objetos
    - `EntidadAnimada*`
    - `AccionProgramada`
    - `Link`
    - `Proyectil`
    - `Enemigo*`
    - `EnemigoConDrop*`
    - `<<Interface>> GeneradorDrop`
    - `<<Interface>> GeneradorProyectil`
    - Factory Method: las relaciones «crea» y por qué son interfaces
    - Enemigos concretos
    - `Jefe*`
    - Posición y dirección (`Posicion` y `Direccion`)
- 10. Comportamientos (patrón Strategy)
    - `<<Interface>> ComportamientoMovimiento`
    - `<<Interface>> ComportamientoAtaque`
    - `<<Interface>> ComportamientoDefensa`
    - Estrategias una por una (con diagramas de secuencia)
    - Disparo de proyectiles: `AtaquePorProyectil` + `GeneradorProyectil`
    - Avalancha de StoneHinox: `AtaqueAvalancha`
    - Teletransporte del Wizzrobe: `MovimientoTeletransporte`
    - Efecto de la bellota: `DefensaReducida`
- 11. Funcionamiento de la partida
    - Puntaje: quién lo lleva y quién lo modifica
    - Tiempo y reglas del modo
    - Comunicación entre `Juego`, `Sector` y las entidades
    - Creación y asignación de entidades
- 12. Observaciones y puntos a revisar
    - 12.1 Observaciones de la defensa 3: estado en la defensa 4
    - 12.2 Observaciones nuevas de la defensa 4
    - 12.3 Resumen
    - 12.4 Observaciones de la defensa 5
- 13. Sugerencias y alternativas (no aplicadas)
    - 13.1 Mover `#miJuego` de `Entidad*` a `EntidadAnimada*`
    - 13.2 Alternativa de modelado E2: `Juego` con tres rankings con nombre
- 14. Guía para la defensa
    - 14.1 Qué pide la cátedra y dónde se cumple
    - 14.2 Patrones de diseño
    - 14.3 Escenarios (secuencias de mensajes)
    - 14.4 Preguntas probables y respuestas cortas
- 15. Posición en píxeles y posición en la matriz
    - 15.1 La idea en una frase
    - 15.2 Clases y métodos que participan
    - 15.3 Las medidas del sector: píxeles y celdas
    - 15.4 Entidades fijas: la posición en la matriz
    - 15.5 Entidades animadas: la posición por movimiento
    - 15.6 Dónde se cruzan: moverse sobre la matriz
    - 15.7 Casos especiales
    - 15.8 Pseudocódigo Java
    - 15.9 Observaciones: lo que faltaba en el diagrama

    - 15.10 Preguntas probables en la defensa
- 16. Hilos, animaciones y colisiones (defensa 5)
    - 16.1 Resumen
    - 16.2 Los cuatro hilos
    - 16.3 Clases nuevas, una por una
    - 16.4 Relaciones nuevas
    - 16.5 Animaciones: qué se toma del ejemplo de Candy Crush
    - 16.6 Acciones programadas (sin hilos extra)
    - 16.7 Colisiones con hilos
    - 16.8 Concurrencia: qué se sincroniza y por qué
    - 16.9 Transición entre sectores
    - 16.10 Cómo quedan los diagramas de secuencia 2 y 3 de la defensa 5
    - 16.11 Impacto en el extendido
    - 16.12 Preguntas probables en la defensa
- Glosario
- Registro de cambios

A. El enunciado en resumen

Resumen del enunciado oficial (`docs/Enunciado-proyecto-Zelda2026.pdf`, UNS, Tecnología de Programación, 2do cuatrimestre 2026) tomado de `CONTEXTO-proyecto-zelda.md`. Es lo que el diseño tiene que poder resolver.

A.1 Reglas duras (su incumplimiento desaprueba)

- **Prohibido `instanceof`** en cualquier parte del código. Las colisiones y los casos por tipo se resuelven con **doble despacho** (`Colisionable` / `Colisionador`) y polimorfismo.
- **Solo Java + Swing.** Sin librerías externas.
- **Movimientos y efectos visuales con `Thread`.** No usar `javax.swing.Timer` ni otras alternativas. Debe haber al menos 2 hilos simultáneos.
- Al menos **un patrón creacional y uno de comportamiento**, usados de forma clara y no forzada.
- **Los nombres de las entidades deben ser los del enunciado**, sin importar el dominio gráfico.
- Cada integrante debe tener commits reales desde la primera defensa (mínimo estimado 20% del desarrollo). Entrega siempre por el repo de GitHub de la comisión.
- Transiciones entre escenas al estilo del juego original (con desplazamiento).

A.2 Requisitos funcionales

1. **≥ 2 dominios de aplicación** (skins gráficas), seleccionables en el menú principal. El comportamiento es idéntico en todos.
2. **≥ 3 modos de juego**, elegibles en el menú: Clásico (llegar al final y vencer al jefe), Contrarreloj (tiempo límite por nivel), No hit (completar sin recibir daño).
3. **≥ 2 niveles**, con dificultad creciente, **cargados desde archivo de texto plano**. El formato es libre pero hay que documentarlo en `Formateo-de-niveles.pdf`.
4. **Enemigos**, **power-ups**, **obstáculos**, **objetos** y **jefes** (ver sección 3).
5. **Vidas:** el jugador empieza con 3. Pierde una al recibir daño o al caer al vacío. Al caer al vacío reinicia el nivel. Al perder todas, aparece en el inicio del escenario y **pierde los puntos acumulados**.

6. **Controles:** flechas = mover a Link en 4 direcciones; **A** = atacar con espada; **S** = usar escudo. Solo Link lo controla el jugador; el resto se mueve solo.
7. **Información visible siempre:** nivel actual, puntaje, tiempo transcurrido, vidas y rupias.
8. **Ranking persistente:** Top 5 por cada uno de los 3 modos (nombre + puntaje). Accesible desde el menú principal y al terminar una partida si el jugador entra al Top 5.
9. **Sonidos** variados (ataque, rupia, impacto, etc.).
10. Link empieza solo con el escudo y consigue la espada durante el primer nivel. Se alterna entre objetos con un botón, y debe verse cuál está equipado (o ninguno).

A.3 Entidades del juego (valores del enunciado)

Enemigos

Enemigo	Comportamiento	Golpes
Moblin	Camina libremente. Daño por contacto: ½ corazón.	2
MoblinConEspadaYEscudo	Más agresivo: se abalanza si Link está de frente. Contacto ½ corazón. Bloquea la espada si lo atacan de frente.	2
ErizoDeMar	Inmóvil. Daño por contacto.	1
Octorok	Dispara un proyectil cuando está frente a Link.	1
TrampaDeCuchillas	Inmóvil, se abalanza si Link pasa frente a ella. Contacto. **Indestructible.**	–
Wizzrobe	Se teletransporta cada pocos segundos a un lugar aleatorio y lanza proyectil a Link. Tras recibir un golpe es invulnerable 1 s hasta su próxima teletransportación.	2

Si no se indica que arroja proyectiles, el enemigo daña por contacto.

Jefes

- **MasterStalfos** (jefe del nivel 1): se mueve en horizontal y vertical. Contacto ½ corazón. Ataca con espada (1 corazón). Necesita ≥ 5 golpes; tras cada golpe es invulnerable un corto período.
- **StoneHinox** (jefe del nivel 2): el que más golpes necesita. Todos sus ataques hacen 1 corazón:
  - *Embestida:* embiste por contacto y queda vulnerable un breve lapso (único momento en que recibe daño).
  - *Lanza rocas:* tres rocas seguidas apuntando a Link.
  - *Avalancha:* va al centro, golpea el piso y caen entre 5 y 10 rocas en lugares aleatorios.
- Los jefes siempre aparecen al final de cada nivel, en todos los modos.

Power-ups

Power-up	Efecto
Rupia	Suma entre 1 y 5 rupias (aleatorio). Las sueltan los enemigos con cierta probabilidad.
Bellota	Reduce a ½ corazón el daño de los próximos 3 golpes. Aparece tras matar 4 enemigos consecutivos sin ser dañado.

Power-up	Efecto
Corazón	+1 corazón (hit point) a Link. Puede estar en lugares fijos del mapa o soltarlo un enemigo.
FragmentoDeCorazon	Cada 4 recogidos suben en 1 la vida máxima. **Exactamente 4 por nivel**, en ubicaciones aleatorias, **máx. 1 por escena**.

Obstáculos

Obstáculo	Efecto
Roca y Agua	Bloquean el paso. Funcionalmente son lo mismo.
Pasto	Reduce la velocidad y cambia la animación de Link. Se destruye con un golpe de espada.
CaidasAlVacio	Ver la ambigüedad de la caída al vacío en A.4.
SuelosQuebradizo	Se destruyen al pisarlos y quedan como caídas al vacío.

Objetos

- **Espada:** ataca al presionar el botón. *Ataque recargado*: mantener 3 s y soltar.
- **Escudo:** al usarlo, Link no recibe daño por proyectiles ni por colisión con enemigos.

Puntajes y drops

Elemento	Puntos		Enemigo	Prob. de soltar rupia o corazón
Moblin	+100		Moblin	20%
MoblinConEspadaYEscudo	+300		MoblinConEspadaYEscudo	50%
ErizoDeMar	+50		ErizoDeMar	20%
Octorok	+150		Octorok	40%
Wizzrobe	+500		Wizzrobe	100%
Jefe	+5000			
Rupia	+10 c/u			
Otros power-ups	+300			

Los jefes y la TrampaDeCuchillas no sueltan nada.

A.4 Ambigüedades del enunciado

- **Caída al vacío:** la Tabla 3 dice que pierde **media vida** y reaparece donde entró a la escena. El punto 9 dice que pierde **una vida** y **reinicia el nivel**.
- **Teclas vs. botón de intercambio:** el punto 7 habla de un botón para alternar objetos, pero el punto 13 asigna A a la espada y S al escudo. Además dice que A sirve para "acelerar", probablemente un error de redacción.
- **"Casos no contemplados":** el enunciado indica seguir la política del juego original y consultar antes de decidir.
- **Perder todas las vidas — RESUELTO (respuesta del ayudante):** la partida **no termina**. Link se reinicia con **puntaje 0** en el **nivel actual** y vuelve a su **posición inicial**.

Por eso no se registra en el ranking en ese momento: el ranking se registra solo cuando la partida termina (lo decide el `ModoJuego`).
    - Dudas que quedan para el ayudante: (1) ¿el nivel se reconstruye (enemigos y power-ups como al principio) o solo se reubica a Link? (2) ¿Link vuelve a tener 3 vidas? (3) En Contrarreloj (tiempo agotado) y NoHit (recibir un golpe), ¿termina la partida y se ofrece el ranking, o se reinicia como en Clásico?

B. Qué pide la defensa 5

Los requerimientos están en `Requerimientos-defensa-5.pdf` (el PDF dice "martes 20/05"; por el calendario es el martes 20/10). Para los dos diagramas de clases pide **lo mismo que la defensa 4**, con un agregado, y suma tres diagramas de secuencia:

- **Reducido:** solo nombres de clases y relaciones. Foco en los aspectos lógicos, el tratamiento de datos para el parseo de niveles y las cuestiones gráficas; interconexión lógica ↔ gráfica con mapeo uno a uno y cómo un cambio lógico se refleja en lo gráfico; todos los patrones requeridos definidos.
- **Extendido:** nombre, atributos y operaciones; comunicación entre el motor, el personaje, los power-ups y los enemigos; detección y resolución de colisiones. **Nuevo:** "que se comience a analizar el uso de hilos en el proyecto (con sus respectivas operaciones)".
- **Tres diagramas de secuencia, por separado:**
    1. Link colisiona con un power-up y se aplica su efecto (la cadena puede empezar en la operación que indica la colisión, en `Link` o en el power-up).
    2. Link equipa la espada y hace un ataque recargado, desde que la gráfica captura el evento (sin chequeo de colisiones).
    3. Un enemigo se desplaza solo y su gráfica se actualiza, desde que **el hilo que actualiza las entidades** empieza una vuelta hasta que el cambio se ve; **en simultáneo, el hilo que lleva el tiempo descuenta un segundo** al modo de juego vigente (sin chequeo de colisiones).

Los *tips* son los de la defensa 4 (clases vs. interfaces, factorización, clases parcial o totalmente implementadas, entidades abstractas como Juego, Mapa, Nivel y Ranking, patrones, modelo de 3 capas).

Dónde se cumple en este reducido:

Pedido	Dónde
Reducido con lo de la defensa 4	Se mantiene todo: ver 14.1.
Hilos con sus operaciones	El reducido ahora **muestra los hilos**: `Runnable`, `HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj`, `AnimadorTransicion` y `AccionProgramada` (C.1 y sección 16). Las operaciones, que van en el extendido, están en 16.3.
Secuencia 1 (Link y un power-up)	Escenarios de 14.3; con hilos, la colisión la resuelve `HiloJugador` (16.7.4).
Secuencia 2 (ataque recargado)	16.10: el EDT deja la intención y `HiloJugador` da el golpe.
Secuencia 3 (enemigo y reloj en simultáneo)	16.10: `HiloMundo` y `Reloj` en un fragmento `par`.

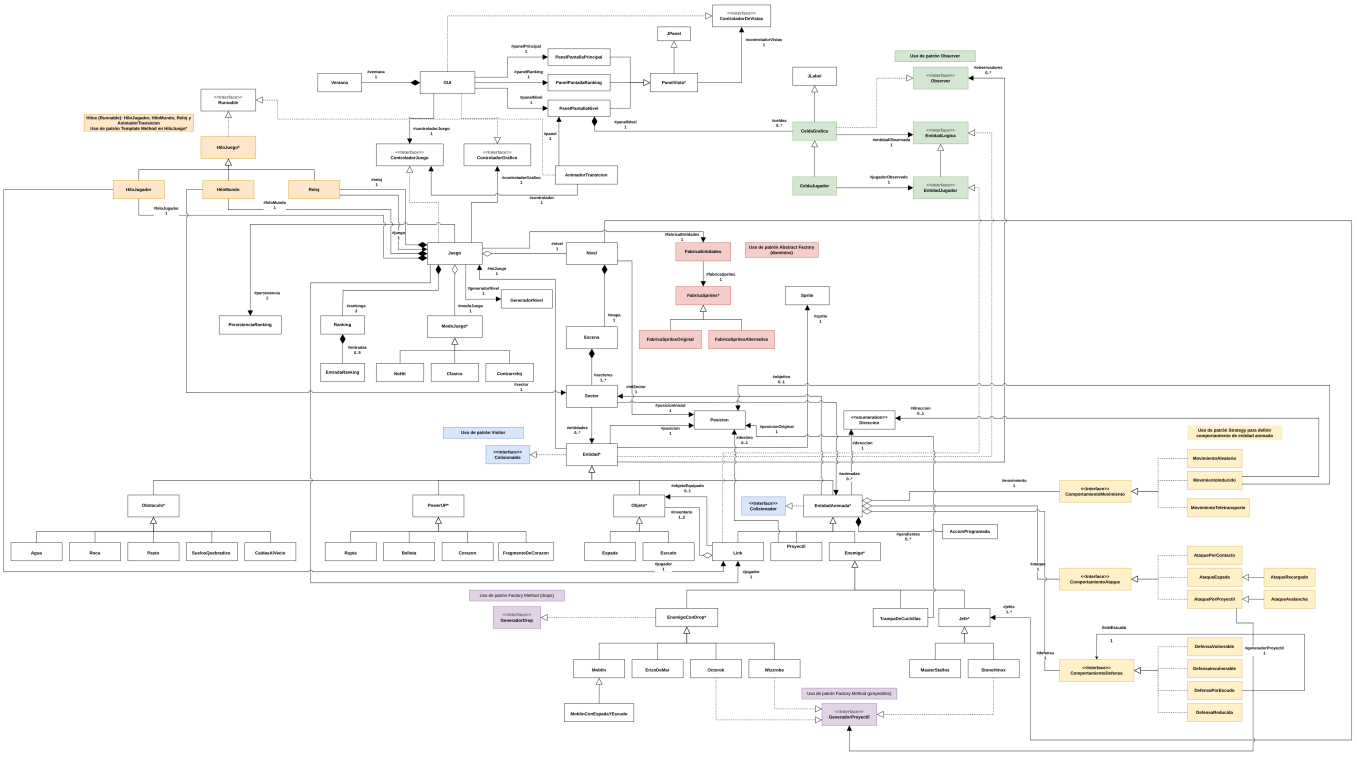
Condiciones de entrega:

Condición	Estado
Carpeta `Diagramas/Diagramas de secuencia` en la **rama principal** del repositorio de la comisión (respetar el nombre sin excepción)	Pendiente: los tres diagramas de secuencia tienen que ir ahí, por separado.
Diagramas de secuencia en PDF, en lo posible	Pendiente.

Veredicto: el reducido cubre lo pedido y ahora muestra los hilos. Lo que falta para la entrega es armar los tres diagramas de secuencia en PDF y actualizar el extendido con los hilos (16.11).

C. El diagrama

El diagrama es `Diagrama-reducido-defensa-5.drawio`, exportado a PNG y PDF:



C.1 Qué cambia respecto del reducido de la defensa 4

Se agregan **seis clases** y **doce relaciones**, y `Timer` pasa a llamarse `Reloj`. Las 85 clases y las 115 relaciones de la defensa 4 siguen iguales, en el mismo lugar y con los mismos roles (salvo `#timer 1`, que pasa a `#reloj 1`). Lo nuevo está arriba a la izquierda, junto a `Juego`, salvo `AnimadorTransicion`, que va en la zona de la vista, y `AccionProgramada`, que va junto a `EntidadAnimada*`.

Cajas:

Acción	Caja	Por qué
Agregar	`<<Interface>> Runnable`	Interfaz de Java (como `JPanel` y `JLabel` son de Swing). La realizan los hilos.

Acción	Caja	Por qué
Agregar	`HiloJuego*` (abstracta, naranja)	Base de los tres hilos de la lógica: `run()` fijo y `ciclo()` redefinido (Template Method).
Agregar	`HiloJugador` (naranja)	El hilo de Link.
Agregar	`HiloMundo` (naranja)	El hilo de los enemigos, jefes y proyectiles del sector visible.
Renombrar y mover	`Timer` → `Reloj` (naranja)	Ahora hereda de `HiloJuego*`. El enunciado prohíbe usar "Timers"; el nombre nuevo evita la pregunta.
Agregar	`AnimadorTransicion`	Capa gráfica: el desplazamiento entre sectores, en su propio hilo.
Agregar	`AccionProgramada`	Reemplaza los hilos que creaba `programar(ms, accion)`.
Agregar nota (naranja)	"Hilos (Runnable): HiloJugador, HiloMundo, Reloj y AnimadorTransicion / Uso de patrón Template Method en HiloJuego*"	Nombra los hilos y el patrón.

Relaciones (todas negras, como el resto del diagrama):

Relación	Tipo	Rol / multiplicidad
`HiloJuego*` ⇢▷ `Runnable`	Realización	—
`HiloJugador`, `HiloMundo`, `Reloj` ──▷ `HiloJuego*`	Herencia (3)	—
`Juego` ◆ `HiloJugador`	Composición	`#hiloJugador 1`
`Juego` ◆ `HiloMundo`	Composición	`#hiloMundo 1`
`HiloJugador` → `Link`	Asociación	`#jugador 1`
`HiloMundo` → `Sector`	Asociación	`#sector 1`
`EntidadAnimada*` ◆ `AccionProgramada`	Composición	`#pendientes 0..*`
`AnimadorTransicion` ⇢▷ `Runnable`	Realización	—
`AnimadorTransicion` → `PanelPantallaNivel`	Asociación	`#panel 1`
`AnimadorTransicion` → `ControladorJuego`	Asociación	`#controlador 1`
`Juego` ◆ `Reloj` (ya estaba)	Composición	`#reloj 1` (antes `#timer 1`)
`Reloj` → `Juego` (ya estaba)	Asociación	`#juego 1`

Color: las cuatro clases del Template Method de los hilos (`HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj`) y su nota van en **naranja**, siguiendo la regla del diagrama de que cada patrón tiene su color. `Runnable`, `AnimadorTransicion` y `AccionProgramada` van en blanco porque no forman parte de un patrón.

Orden: las flechas de los tres hilos a `Juego` entran por el costado izquierdo sin cruzarse entre sí, y `AccionProgramada` quedó a la derecha de `EntidadAnimada*`, fuera de las líneas que bajan a `Link` y a `Enemigo*`.

El diagrama tiene ahora **91 clases e interfaces** y **127 relaciones**: 120 dibujadas y las 7 dependencias que el equipo decidió no dibujar. La explicación completa está en la sección 16.

C.2 Historia: qué cambió de la defensa 3 a la defensa 4

> Esta parte describe el paso de la defensa 3 a la 4 y se deja como estaba. Los nombres `Timer` y `#timer` son los de ese momento (hoy `Reloj` y `#reloj`).

En el diagrama reducido hay un solo cambio. Las 85 clases e interfaces son las mismas, están en el mismo lugar, y las 115 relaciones son las mismas, con los mismos roles y multiplicidades, salvo una:

Relación	En el reducido de la defensa 3	En el reducido de la defensa 4
`Posicion → Direccion` «usa»	**Dependencia**: línea punteada con flecha abierta y estereotipo «usa».	Línea **continua** con flecha abierta y estereotipo «usa», sin rol ni multiplicidad. Se lee como una **asociación**.

¿Es correcto el cambio? No: conviene volver a la línea punteada. Explicación completa en Posición y dirección y en Observaciones. En resumen:

1. Una línea continua con flecha es una **asociación navegable**: dice que `Posicion` tiene un **atributo** de tipo `Direccion`. En el extendido de la defensa 4, `Posicion` tiene solo `- x : int` y `- y : int`: la dirección aparece únicamente como **parámetro** de `desplazada(d : Direccion, pasos : int)`. Usar un tipo como parámetro es una **dependencia**, que se dibuja punteada.
2. En este diagrama **todas** las asociaciones llevan rol y multiplicidad (`#nombre 1`) y punta negra pintada. La flecha nueva no tiene ni lo uno ni lo otro, y conserva la punta abierta de las dependencias: queda a mitad de camino.
3. En el propio archivo la relación se llama `rel-posicion-direccion-dep`: fue pensada como dependencia.
4. Si se quisiera que fuera asociación, habría que agregarle a `Posicion` un atributo `# direccion : Direccion` (con rol `#direccion 1` en la flecha), y eso no tiene sentido: un punto no tiene dirección; la dirección es de la entidad que se mueve (`EntidadAnimada* → Direccion`, `#direccion 1`).

Después de ese cambio, el equipo decidió no dibujar las dependencias en el reducido. Se sacaron del dibujo las siete («crea», «usa», «carga/guarda»), incluida `Posicion → Direccion`, así que el punto anterior ya no aplica: la relación no está dibujada ni como asociación ni como dependencia. El texto las sigue explicando como **dependencias no dibujadas** (ver 1. Cómo leer el diagrama). Además se agregaron notas de patrones: a las de Observer (verde), Strategy (amarillo), Abstract Factory (rojo) y Visitor (azul) se suman **Factory Method** (violeta, dos notas: drops junto a `GeneradorDrop` y proyectiles junto a `GeneradorProyectil`, que ahora también van en violeta). El Template Method (`AtaquePorProyectil` → `AtaqueAvalancha`) se explica en el texto, sin nota en el diagrama (decisión del equipo).

Más tarde se agregó una asociación: `Timer → Juego` (`#juego 1`, flecha negra pintada), además de la composición `Juego ◆ Timer` (`#timer 1`) que ya estaba. Es la forma que eligió el equipo para que el reloj le avise a `Juego` cada segundo (ver `Timer` y Tiempo y reglas del modo).

En el extendido sí hubo cambios que afectan a esta explicación (nombres de métodos que se usan en los ejemplos y las secuencias). Los principales:

Tema	Extendido de la defensa 3	Extendido de la defensa 4
Clase de imágenes	`Sprites` en constructores y retornos	`Sprite` en todos: el atributo de `Entidad*` es `# sprite : Sprite` y los constructores reciben `sprite : Sprite` (observación 5 resuelta)
`Posicion`	Sin `getX()`	`getX()` y `getY()`
`Direccion`	Sin sus valores	`ARRIBA`, `ABAJO`, `IZQUIERDA`, `DERECHA`
Ranking	`Juego.guardarRanking(r)`; `ModoJuego.getRanking(j)`	`ModoJuego.getRanking(j)` **y** `registrarPuntaje(j, e)`; `Juego.agregarAlRankingClasico/Contrarreloj/NoHit(e)` y `-agregar(i, e)`
Muerte	`alQuedarSinVida()`, `soltarDrop()`	Un solo nombre: `# morir()`, declarado en `EntidadAnimada*` y redefinido en `Enemigo*` (avisa `enemigoDerrotado(this)`, sin drop), `EnemigoConDrop*` (`super.morir()` y después `soltarDrop()`), `Jefe*` (avisa `jefeDerrotado(this)`), `Link` y `Proyectil`
Ganchos de `EntidadAnimada*`	`vigilar()`, `alTeletransportarse()`, `ataqueTerminado()`, `ataqueRealizado()`, `alSerGolpeado()`, `quiereAtacar()`, `estaEnElCentro()`, `run()`	`EntidadAnimada*` declara los ganchos que llaman las estrategias: `# morir()`, `# alSerGolpeado()`, `+ ataqueRealizado()`, `+ restaurarDefensa()`, `+ teletransportarse()` y la consulta `+ quiereAtacar() : boolean`; las subclases los redefinen (`Wizzrobe.teletransportarse()`, `Link.ataqueRealizado()`, `MasterStalfos.alSerGolpeado()` y `quiereAtacar()`…). Quedan en las subclases `Enemigo.vigilarJugador()` y `StoneHinox.ataqueTerminado()`. `run()` volvió a `EntidadAnimada*` con las correcciones del explicativo de posición. Además `EntidadAnimada*` tiene `# programar(ms : long, accion : Runnable)` (un `Thread` que duerme `ms` y ejecuta la acción) y `AtaquePorProyectil` volvió a tener el gancho `# alTerminar(atacante : EntidadAnimada)`, que redefine `AtaqueAvalancha`.
`Link`	`activarEscudo`, `caerAlVacio`, `romperSuelo`, `entrarPorBordeOpuesto`	Volvieron todos: `activarEscudo()`, `desactivarEscudo()`, `caerAlVacio()`, `romperSuelo(s : SuelosQuebradizo)` y `entrarPorBordeOpuesto(d)`. Link sigue delegando en el objeto equipado (`atacarCon`, `protegerCon`), redefine `recibirGolpeDeEspada(atacante, danio)` y su muerte es `# morir()`. `Juego` resuelve caída y cambio de sector (`jugadorCayoAlVacio()`, `cambiarDeSector(d)`).
`Timer`	`Timer(alTic : Runnable)`	`Timer(j : Juego)` con `# juego : Juego`; `Juego` lo crea con `new Timer(this)` y `+ tic(segundos : int)` es público
`Juego`	`getRankings() : Ranking[]`	`+ getRankings() : Ranking[]` (faltaba en la primera versión de la defensa 4 y volvió)
`Enemigo*`, `EnemigoConDrop*`	Sin constructor	`# Enemigo(sprite : Sprite, x : int, y : int, vida : int, danioContacto : int)` y `# EnemigoConDrop(` *(mismos parámetros)* `)`
`PanelPantallaNivel`	`desplazar(d, alTerminar)`	`desplazar(d)`

El diagrama de la defensa 4 tenía **85 clases e interfaces** y **115 relaciones**: 108 dibujadas y las 7 dependencias que el equipo decidió no dibujar (ver la tabla de 2. Vista general).

---

1. Cómo leer el diagrama

Notación	Significado
`Nombre*` (en cursiva)	Clase abstracta. No se instancia; define lo común a sus subclases.
`<<Interface>>`	Interfaz: contrato de métodos sin implementación.
`<<enumeration>>`	Enumeración: tipo con un conjunto fijo de valores (`Direccion`).
Línea continua con triángulo vacío	**Herencia** (`extends`).
Línea punteada con triángulo vacío	**Realización** (`implements`).
Línea continua con rombo lleno ◆ (del lado del todo)	**Composición**: el todo es dueño de la parte; la parte no vive sin el todo.
Línea continua con rombo vacío ◇ (del lado del todo)	**Agregación**: el todo usa la parte, pero la parte puede existir por separado.
Línea continua con **flecha negra pintada** ▶	**Asociación navegable**: la clase de origen tiene un atributo que referencia a la de destino.
Línea punteada con **flecha abierta** `⇢` y estereotipo (`«crea»`, `«carga/guarda»`, `«usa»`)	**Dependencia**: la clase de origen usa a la de destino (la crea, la devuelve o la recibe por parámetro), pero no guarda un atributo de ese tipo. **En este reducido no se dibujan**, por decisión del equipo; el texto las explica igual (ver el recuadro de abajo).
`#nombre multiplicidad`	Rol de la relación: el **atributo** que la implementa y su multiplicidad. Todas las asociaciones, agregaciones y composiciones lo tienen.
`1`, `0..*`, `1..2`, etc.	Multiplicidad: cuántas instancias participan.
Recuadro de color con el nombre de un patrón	Nota que indica qué patrón se usa en esa zona. Cada patrón tiene su color, y las clases que lo forman llevan ese mismo color: **verde** Observer, **amarillo** Strategy, **rojo** Abstract Factory, **azul** Visitor, **violeta** Factory Method (dos notas: drops y proyectiles) y **naranja** Template Method de los hilos.

> **Cómo están dibujadas hoy las dependencias. No están dibujadas.** El equipo decidió no reflejar las dependencias en el reducido: UML no obliga a dibujarlas todas, y así el diagrama queda con las relaciones estructurales (herencias, realizaciones, asociaciones, agregaciones y composiciones). Las siete dependencias que tenía el diagrama (las seis con estereotipo «crea» o «carga/guarda»: `FabricaSprites* → Sprite`, `GeneradorNivel → Nivel`, `GeneradorDrop → Rupia`, `GeneradorDrop → Corazon`, `GeneradorProyectil → Proyectil`, `PersistenciaRanking → Ranking`, y `Posicion → Direccion` «usa») se sacaron del dibujo, pero **este documento las sigue explicando** como **dependencias no dibujadas**: qué clase usa a cuál, por qué no es una asociación y en qué método aparece. Si se dibujaran, irían punteadas con flecha abierta y estereotipo.

Dónde va el rol. En las asociaciones el atributo es de la clase de origen y la etiqueta va junto a la flecha, del lado del destino (`Juego ▶ Link` con `#jugador 1` cerca de `Link`). En las agregaciones y composiciones el atributo es del todo (donde está el rombo) y la etiqueta va del lado de la parte (`Juego ◆ Reloj` con `#reloj 1` cerca de `Reloj`).

Convención de nombres de roles.

- Todos son **protegidos** (`#`), en **español** y en **camelCase**, con el nombre de lo que guardan (`#reloj`, `#juego`, `#sprite`, `#posicion`, `#modoJuego`, `#generadorNivel`).
- En **plural** cuando la multiplicidad es mayor que 1 (`#rankings 3`, `#sectores 1..*`, `#celdas 0..*`, `#observadores 0..*`, `#entradas 0..5`, `#jefes 1..*`).
- Con prefijo **`mi`** solo las referencias "hacia arriba", al contenedor o al contexto donde vive el objeto (`#miJuego`, `#miSector`).
- Los roles que indican una función se nombran por la función (`#jugador`, `#objetoEquipado`, `#inventario`, `#entidadObservada`, `#jugadorObservado`, `#persistencia`, `#generadorProyectil`, `#mapa`, `#sinEscudo`).

Estilo uniforme. Todas las relaciones son negras. Las clases que no forman parte de un patrón tienen fondo blanco. Las que sí forman parte de uno llevan el color de su patrón, igual que la nota que lo nombra:

Color	Patrón	Clases
Verde	Observer	`Observer`, `EntidadLogica`, `EntidadJugador`, `CeldaGrafica`, `CeldaJugador`
Amarillo	Strategy	`ComportamientoMovimiento`, `ComportamientoAtaque`, `ComportamientoDefensa` y sus implementaciones
Rojo	Abstract Factory	`FabricaEntidades`, `FabricaSprites*`, `FabricaSpritesOriginal`, `FabricaSpritesAlternativa`
Azul	Visitor	`Colisionable`, `Colisionador`
Violeta	Factory Method	`GeneradorDrop`, `GeneradorProyectil` (dos notas: "Uso de patrón Factory Method (drops)" y "Uso de patrón Factory Method (proyectiles)")
Naranja	Template Method (hilos)	`HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj` (nota "Hilos (Runnable)… Uso de patrón Template Method en HiloJuego*")

El Template Method de los ataques (`AtaqueAvalancha` redefine pasos del esquema de ataque de `AtaquePorProyectil`) **no tiene nota** en el diagrama, por decisión del equipo; se explica en 14.2. Sus clases siguen en **amarillo** porque forman parte del Strategy. El Template Method de los **hilos** (`HiloJuego*`) sí tiene nota, en naranja, junto con la lista de hilos. En total el diagrama tiene **seis patrones con nota**: Observer, Strategy, Abstract Factory, Visitor, Factory Method (dos notas) y Template Method (hilos).

Las relaciones de un mismo tipo tienen la misma punta y el mismo tamaño, y las etiquetas de roles el mismo estilo (negrita, centradas), con una excepción que viene de la defensa 3: las etiquetas de `#sinEscudo` (ver Observaciones). Las dependencias no se dibujan (ver el recuadro de arriba).

---

2. Vista general

El diagrama se organiza en zonas (de arriba hacia abajo):

1. **Vista y controladores** (arriba a la izquierda): `Ventana`, `GUI`, los paneles con `PanelVista*` y `JPanel`, y `ControladorDeVistas`. Debajo de `GUI` están `ControladorJuego` y `ControladorGrafico`, justo entre `GUI` y `Juego`, porque son las dos interfaces que los comunican.
2. **Observer** (arriba a la derecha): a la izquierda la parte gráfica (`JLabel`, `CeldaGrafica`, `CeldaJugador`) y a la derecha las interfaces (`Observer`, `EntidadLogica`, `EntidadJugador`).
3. **Núcleo del juego** (centro a la izquierda): `Juego`, `Ranking`/`EntradaRanking`, `PersistenciaRanking`, `ModoJuego*` y sus modos, `GeneradorNivel`, y la columna del mapa `Nivel`

→ `Escena` → `Sector`.

4. **Fábricas** (centro): Abstract Factory de sprites (`FabricaEntidades`, `FabricaSprites*` y sus dos dominios, `Sprite`).
5. **Entidades** (abajo): `Entidad*` debajo de `Sector`, y su jerarquía en árbol: obstáculos, power-ups, objetos y `EntidadAnimada*` (`Link`, `Proyectil`, `Enemigo*` y todos los enemigos y jefes).
6. **Comportamientos** (abajo a la derecha, junto a `EntidadAnimada*`): las tres interfaces Strategy con sus implementaciones.
7. **Posición y dirección** (entre `Sector` y `Entidad*`): `Posicion` y la enumeración `Direccion`, con flechas desde `Entidad*`, `Nivel`, `EntidadAnimada*`, `Proyectil`, `TrampaDeCuchillas` y `MovimientoInducido`.
8. **Hilos** (arriba a la izquierda, junto a `Juego`; defensa 5): `Runnable`, `HiloJuego*`, `HiloJugador`, `HiloMundo` y `Reloj`, con su nota. `AnimadorTransicion` está en la zona de la vista, debajo de `PanelPantallaNivel`, y `AccionProgramada` junto a `EntidadAnimada*`.

Las flechas son ortogonales (salvo tres, ver Observaciones); las herencias de una misma clase se agrupan en un árbol. Las líneas largas que se cruzan tienen un "salto" (arco) para distinguir el cruce.

Cuántas relaciones hay de cada tipo:

Tipo	Cantidad	Ejemplos
Herencia (incluye `EntidadJugador` ⟶▷ `EntidadLogica`, herencia entre interfaces)	45	`Link` ⟶▷ `EntidadAnimada*`, `AtaqueAvalancha` ⟶▷ `AtaquePorProyectil`
Realización	24	`Juego` ⇢▷ `ControladorJuego`, `Octorok` ⇢▷ `GeneradorProyectil`, `HiloJuego*` ⇢▷ `Runnable`
Asociación con rol	34	`Juego → Link (#jugador 1)`, `Sector → Entidad* (#entidades 0..*)`, `Reloj → Juego (#juego 1)`, `HiloMundo → Sector (#sector 1)`
Asociación con etiquetas sueltas	1	`DefensaPorEscudo → ComportamientoDefensa (#sinEscudo 1)`
Agregación	6	`Juego ◇ Nivel (#nivel 1)`, `EntidadAnimada* ◇ ComportamientoAtaque (#ataque 1)`
Composición	10	`Nivel ◆ Escena (#mapa 1)`, `Juego ◆ Ranking (#rankings 3)`, `Juego ◆ HiloMundo (#hiloMundo 1)`
Dependencias con estereotipo (**no dibujadas**, por decisión del equipo; se explican en el texto)	7	`GeneradorNivel «crea» Nivel`, `Posicion «usa» Direccion`
Total	**127**	120 dibujadas y 7 dependencias no dibujadas (en la defensa 4 eran 115)

> Los números por tipo salen de contar las flechas del `.drawio`; si el equipo agrega o corrige relaciones, hay que actualizarlos junto con el registro de cambios.

Principio rector: **la lógica nunca importa Swing**. Se comunica con la vista solo a través de interfaces.

---

3. Vista y controladores

`GUI`

- **Utilidad:** punto central de la capa gráfica. Arma la ventana y administra qué pantalla se muestra.
- **Comportamiento:** recibe las teclas del jugador en `procesarTecla(codigo, presionada)` y las traduce a *intenciones* que le envía a la lógica (`mover(d)`, `detenerMovimiento()`, `atacar()`, `soltarAtaque()`, `usarEscudo()`, `soltarEscudo()`, `cambiarObjeto()`) a través de `ControladorJuego`. Recibe de la lógica las órdenes de dibujo y de cambio de pantalla a través de `ControladorGrafico` (`mostrarEntidad`, `transicion`, `pedirNombre`, `mostrarRanking`…), y de los paneles las de navegación a través de `ControladorDeVistas`.
- **Relaciones:**
  - Compone `Ventana` (◆, `#ventana 1`).
  - Conoce los tres paneles (→): `PanelPantallaPrincipal` (`#panelPrincipal 1`), `PanelPantallaRanking` (`#panelRanking 1`) y `PanelPantallaNivel` (`#panelNivel 1`).
  - Referencia a `ControladorJuego` (→, `#controladorJuego 1`): es por donde manda las intenciones a `Juego`. Se la asigna `setControladorJuego(c)` al arrancar la aplicación.
  - Realiza `ControladorGrafico` y `ControladorDeVistas`.
  - En cada `transicion(d)` crea un `AnimadorTransicion` y lo arranca en un `Thread` (defensa 5). Es una dependencia «crea», no una relación dibujada: `GUI` no guarda el animador.

```
        // Arranque de la aplicación (main): se conectan las dos capas solo por interfaces
        GUI gui = new GUI();
        Juego juego = new Juego(gui);           // Juego recibe la vista como ControladorGrafico
        gui.setControladorJuego(juego);         // GUI recibe la lógica como ControladorJuego
        gui.mostrarPantallaPrincipal();
```

`Ventana`

- **Utilidad:** el `JFrame` de la aplicación (`Ventana(titulo)`).
- **Comportamiento:** muestra el panel que esté activo (`mostrarPanel(p)`). No tiene lógica de juego.
- **Relaciones:** es parte de `GUI` (◆, `#ventana 1`): se crea y se destruye con ella.

`JPanel` / `JLabel`

- Clases de Swing. Aparecen solo para mostrar de quién heredan `PanelVista*` y `CeldaGrafica`.

`PanelVista*`

- **Utilidad:** base común de todas las pantallas.
- **Comportamiento:** cuando el usuario quiere ir a otra pantalla (por ejemplo, del menú al ranking), lo pide a través de `ControladorDeVistas` en lugar de conocer a `GUI` directamente. Cada subclase arma sus componentes en `armarComponentes()`.
- **Relaciones:** hereda de `JPanel`; referencia a `ControladorDeVistas` (→, `#controladorVistas 1`).

`PanelPantallaPrincipal`

- **Utilidad:** menú principal.

- **Comportamiento:** permite elegir dominio gráfico (`elegirDominio(d)`), modo de juego (`elegirModo(m)`), empezar la partida (`jugar()`) y ver el ranking (`verRanking()`). Guarda lo elegido en `dominio : FabricaSprites` y `modo : ModoJuego` (atributos privados de la vista, no dibujados como relación).
- **Relaciones:** hereda de `PanelVista*`.

`PanelPantallaRanking`

- **Utilidad:** muestra el Top 5 de cada uno de los 3 modos (`mostrarRankings(r : Ranking[])`, tres tablas).
- **Comportamiento:** se muestra desde el menú y al terminar una partida. `volver()` regresa al menú.
- **Relaciones:** hereda de `PanelVista*`.

`PanelPantallaNivel`

- **Utilidad:** pantalla donde se juega.
- **Comportamiento:** dibuja las celdas de las entidades (`agregarCelda`, `quitarCelda`, `limpiar`) y el HUD (`actualizarHUD(j)`, `actualizarTiempo(s)`, `actualizarNivel(n)`: nivel, puntaje, tiempo, vidas, rupias y objeto equipado). Hace la transición con desplazamiento entre sectores (`desplazar(d)`).
- **Relaciones:** hereda de `PanelVista*`; compone las `CeldaGrafica` (◆, `#celdas 0..*`). Cada `CeldaGrafica` guarda el panel en `# panelNivel` (atributo del extendido); `CeldaJugador` lo hereda y lo usa para actualizar el HUD (ver `CeldaJugador`). Lo referencia `AnimadorTransicion` (`#panel 1`), que lo desplaza durante la transición.

`AnimadorTransicion`

- **Utilidad (defensa 5):** el hilo de la transición entre sectores. Es una animación de la **capa gráfica**, tomada de la idea de "animador" del ejemplo de Candy Crush.
- **Comportamiento:** realiza `Runnable`. `GUI.transicion(d)` lo crea con el panel, la dirección y el controlador, y lo arranca en un `Thread`. Su `run()` hace `panel.desplazar(d)` paso a paso y, al terminar, avisa `controlador.transicionTerminada()`.
- **Relaciones:** realiza `Runnable`; referencia a `PanelPantallaNivel` (→, `#panel 1`) y a `ControladorJuego` (→, `#controlador 1`). No conoce ninguna entidad lógica: solo la interfaz por la que avisa. Detalle en 16.9.

`<<Interface>> ControladorJuego` (vista → lógica)

- **Utilidad:** entrada a la lógica. Es lo único que la vista sabe de `Juego`.
- **Comportamiento (según el extendido):** expone intenciones, no eventos de Swing: `iniciar(modo, dominio)`, `mover(d)`, `detenerMovimiento()`, `atacar()`, `soltarAtaque()`, `usarEscudo()`, `soltarEscudo()`, `cambiarObjeto()`, `registrarNombre(nombre)`, `transicionTerminada()` y la consulta `getRankings() : Ranking[]` (para el menú). `iniciar` recibe la `FabricaSprites` del dominio que eligió el usuario (ver Elección del dominio gráfico).
- **Relaciones:** la realiza `Juego`; la referencian `GUI` (`#controladorJuego 1`) y `AnimadorTransicion` (`#controlador 1`), que avisa por acá `transicionTerminada()`.

`<<Interface>> ControladorGrafico` (lógica → vista)

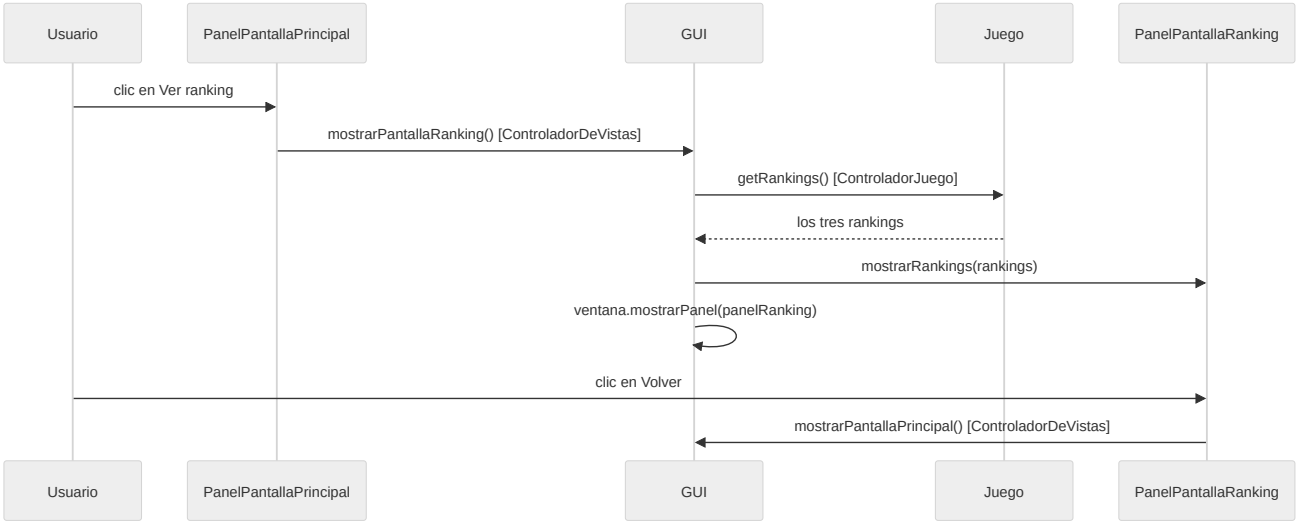
- **Utilidad:** salida de la lógica hacia la vista.

- **Comportamiento (según el extendido):** `mostrarEntidad(e : EntidadLogica) : Observer` y `mostrarJugador(j : EntidadJugador) : Observer` (crean la celda y la devuelven como `Observer`), `limpiarEscena()`, `transicion(d)`, `actualizarTiempo(segundos)`, `actualizarNivel(numero)`, `pedirNombre(puntaje)` y `mostrarRanking(r)`. Ver Creación y asignación de entidades.
- **Relaciones:** la realiza `GUI`; la referencia `Juego` (`#controladorGrafico 1`).

`<<Interface>> ControladorDeVistas`

- **Utilidad:** navegación entre pantallas.
- **Comportamiento:** `mostrarPantallaPrincipal()`, `mostrarPantallaRanking()` e `iniciarPartida(modo, dominio)`: los paneles piden "mostrame tal pantalla" o "empezá la partida" y `GUI` resuelve.
- **Relaciones:** la realiza `GUI`; la referencia `PanelVista*` (`#controladorVistas 1`).

Por qué hay tres interfaces y no una: cada una tiene un solo sentido y un solo cliente. `ControladorJuego` la usa la vista para hablarle a la lógica; `ControladorGrafico` la usa la lógica para hablarle a la vista; `ControladorDeVistas` la usan los paneles para hablarle a `GUI`. Si fueran una sola, `Juego` vería métodos de navegación de pantallas y los paneles verían métodos de dibujo de entidades.



Explicación: el panel no conoce a `GUI` ni a `Juego`: pide la navegación por `ControladorDeVistas`. `GUI` le pide los rankings a la lógica por `ControladorJuego` y se los pasa al panel.

---

4. Observer (lógica → gráfica)

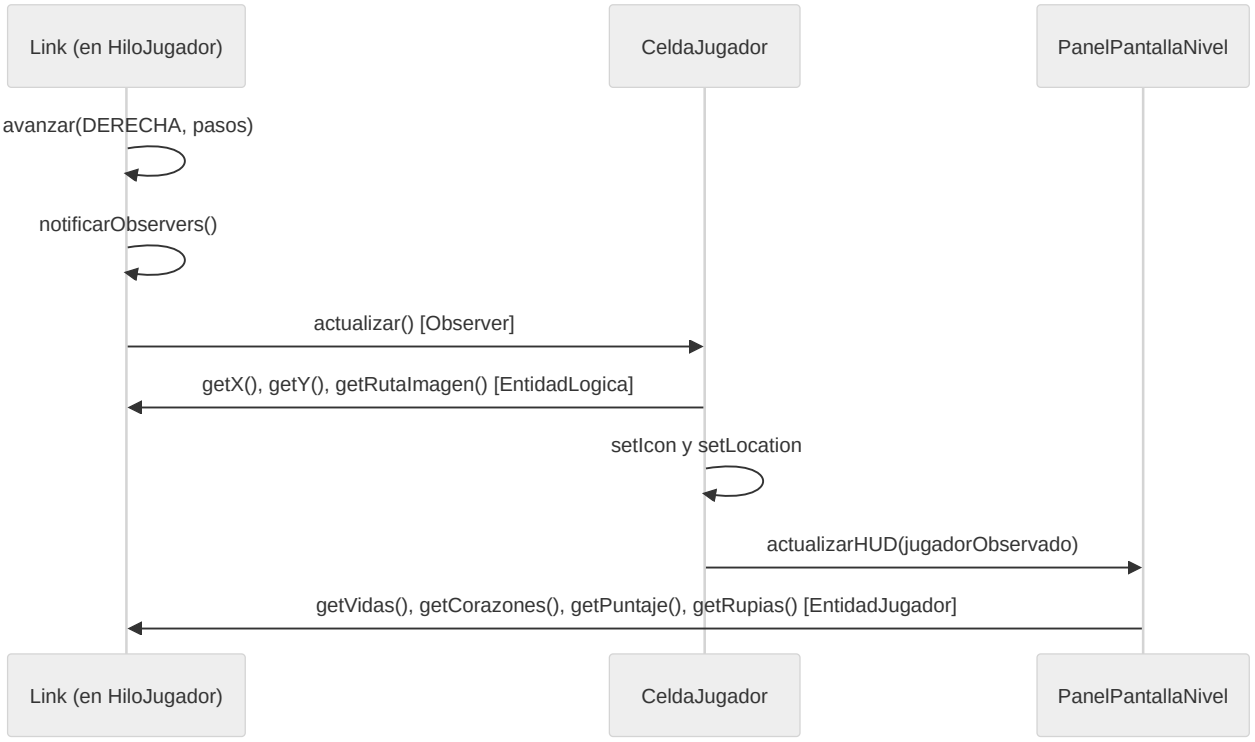
Hay **un solo** `Observer`. Los dos roles del patrón quedan separados:

- **Sujeto (observado):** `Entidad*`. Cuando cambia (se mueve, recibe daño, cambia de estado) recorre su lista `#observadores` y los notifica (`notificarObservers()`).
- **Observador:** `CeldaGrafica` (y por herencia `CeldaJugador`). Recibe el aviso (`actualizar()`) y se redibuja.

La gráfica lee los datos de la entidad a través de `EntidadLogica` / `EntidadJugador`, que son **vistas de solo lectura**: no son observers, solo exponen consultas. Así la lógica solo conoce `Observer` y la gráfica solo conoce esas dos interfaces; ninguna conoce las clases concretas de la otra.

Recorrido de una notificación:

1. Una entidad cambia (por ejemplo, el hilo que la actualiza, `HiloJugador` o `HiloMundo`, le cambia la posición).
2. La entidad llama a `actualizar()` de cada `Observer` de su lista.
3. La `CeldaGrafica` consulta a su `EntidadLogica` la posición (`getX()`, `getY()`), la imagen (`getRutaImagen()`) y si fue eliminada (`estaEliminada()`), y se actualiza.
4. Si es la `CeldaJugador`, además le pide a `PanelPantallaNivel` que actualice el HUD con los datos de `EntidadJugador` (`actualizarHUD(j)`).



Link se mueve distinto que el resto (lo dirige el teclado), pero esa diferencia se resuelve en la entrada (`ControladorJuego`) y en el Strategy (`MovimientoInducido`). Para el Observer, Link es una entidad más: por eso no hace falta un observer propio del jugador.

`<<Interface>> Observer`

- **Utilidad:** contrato del patrón Observer: "avisame cuando cambies".
- **Comportamiento:** define un único método, `actualizar()`. La entidad lo llama; el observer decide qué hacer (redibujarse, actualizar el HUD, sacarse del panel).
- **Relaciones:** la realiza `CeldaGrafica` (y `CeldaJugador` por herencia); la referencia `Entidad*` (→, `#observadores 0..*`).

`<<Interface>> EntidadLogica`

- **Utilidad:** vista de solo lectura de una entidad para la capa gráfica.
- **Comportamiento (según el extendido):** `getX()`, `getY()`, `getRutaImagen()` (tomada de su `Sprite`) y `estaEliminada()`. No tiene métodos que modifiquen la entidad.
- **Relaciones:** la realiza `Entidad*`; la referencia `CeldaGrafica` (→, `#entidadObservada 1`); la extiende `EntidadJugador`. **No** extiende `Observer`.

`<<Interface>> EntidadJugador`

- **Utilidad:** vista de solo lectura del jugador, con los datos que necesita el HUD.
- **Comportamiento (según el extendido):** además de lo de `EntidadLogica`, `getVidas()`, `getCorazones()`, `getCorazonesMaximos()`, `getPuntaje()`, `getRupias()` y `getNombreObjetoEquipado()`.
- **Relaciones:** extiende `EntidadLogica`; la realiza `Link`; la referencia `CeldaJugador` (→, `#jugadorObservado 1`).

`CeldaGrafica`

- **Utilidad:** representación en pantalla de una entidad (un `JLabel` con su imagen).
- **Comportamiento:** cuando su entidad la notifica (`actualizar()`), consulta su `EntidadLogica` y actualiza imagen y ubicación; si la entidad está eliminada, se saca del panel. La crea la vista (`GUI`, dentro de `mostrarEntidad`) y la lógica la registra como `Observer` en la entidad.
- **Relaciones:** hereda de `JLabel`; realiza `Observer`; referencia a `EntidadLogica` (`#entidadObservada 1`); es parte de `PanelPantallaNivel` (◆, `#celdas 0..*`). En el extendido guarda ese panel en `# panelNivel`, que hereda `CeldaJugador`.

```
        public class CeldaGrafica extends JLabel implements Observer {
            protected EntidadLogica entidadObservada;                // #entidadObservada 1
            protected PanelPantallaNivel panelNivel;                 // el panel que la
contiene (#celdas)
            public CeldaGrafica(EntidadLogica e) { entidadObservada = e; actualizar(); }
            public void actualizar() {
                if (entidadObservada.estaEliminada()) { getParent().remove(this); return; }
                setIcon(new
ImageIcon(getClass().getResource(entidadObservada.getRutaImagen())));
                setLocation(entidadObservada.getX(), entidadObservada.getY());
            }
        }
```

`CeldaJugador`

- **Utilidad:** celda especial para Link.
- **Comportamiento:** además de redibujar a Link, al ser notificada pide a `PanelPantallaNivel` que actualice el HUD con los datos de `EntidadJugador`.
- **Relaciones:** hereda de `CeldaGrafica` (por eso también es `Observer`); referencia a `EntidadJugador` (`#jugadorObservado 1`). El panel no lo guarda ella: usa `panelNivel`, heredado de `CeldaGrafica`, para pedir `actualizarHUD`.

```
        public class CeldaJugador extends CeldaGrafica {
            protected EntidadJugador jugadorObservado;               // #jugadorObservado 1
            // panelNivel lo hereda de CeldaGrafica
            public CeldaJugador(EntidadJugador j, PanelPantallaNivel p) { super(j);
jugadorObservado = j; panelNivel = p; }
            public void actualizar() { super.actualizar();
panelNivel.actualizarHUD(jugadorObservado); }
        }
```

Por qué `CeldaJugador` guarda dos referencias al mismo Link: `#entidadObservada` (heredada, tipo `EntidadLogica`) y `#jugadorObservado` (tipo `EntidadJugador`) apuntan al mismo objeto, pero con tipos distintos. Guardar el tipo más específico evita un *cast* cada vez que se actualiza el HUD.

---

5. Núcleo del juego

`Juego`

Juego

```
# controladorGrafico : ControladorGrafico
# jugador : Link
# nivel : Nivel
# numeroNivel : int
# modoJuego : ModoJuego
# rankings : Ranking[3]
# timer : Timer
# generadorNivel : GeneradorNivel
# fabricaEntidades : FabricaEntidades
# persistencia : PersistenciaRanking
```

«constructores»

```
+ Juego(vista : ControladorGrafico)
```

«comandos»

```
+ iniciar(modo : ModoJuego, dominio : FabricaSprites)
+ mover(d : Direccion)
+ detenerMovimiento()
+ atacar()
+ soltarAtaque()
+ usarEscudo()
+ soltarEscudo()
+ cambiarObjeto()
+ registrarNombre(nombre : String)
+ transicionTerminada()
+ enemigoDerrotado(e : Enemigo)
+ jefeDerrotado(j : Jefe)
+ powerUpRecogido(p : PowerUP)
+ jugadorDaniado()
+ jugadorSinVidas()
+ jugadorCayoAlVacio()
+ cambiarDeSector(d : Direccion)
+ mostrarEntidad(e : Entidad)
+ terminarPartida()
+ tic(segundos : int)
+ agregarAlRankingClasico(e : EntradaRanking)
+ agregarAlRankingContrarreloj(e : EntradaRanking)
+ agregarAlRankingNoHit(e : EntradaRanking)
- cargarNivel()
- activarSector(s : Sector)
```

```
- activarSector(s : Sector)
- aparecerBellota(x : int, y : int)
- agregar(i : int, e : EntradaRanking)
«consultas»
+ getRankingNoHit() : Ranking
+ getRankingContrarreloj() : Ranking
+ getRankingClasico() : Ranking
+ getRanking(m : ModoJuego) : Ranking
+ getJugador() : Link
+ getFabricaEntidades() : FabricaEntidades
+ getNumeroNivel() : int
+ getTiempo() : int
```

- **Utilidad:** centro de la lógica. Coordina la partida.
- **Comportamiento:** recibe las intenciones del jugador (como `ControladorJuego`), inicia la partida en el modo elegido (`iniciar(modo, dominio)`), carga el nivel que corresponde (`cargarNivel()`, con `GeneradorNivel`), lleva el tiempo (`tic(segundos)`, público: lo llama el `Reloj` cada segundo), le da los tres rankings al menú (`getRankings()`), recibe los avisos de las entidades (`enemigoDerrotado`, `jefeDerrotado`, `powerUpRecogido`, `jugadorDaniado`, `jugadorSinVidas`, `jugadorCayoAlVacio`, `cambiarDeSector`), decide cuándo se termina la partida (`terminarPartida()`), actualiza el ranking y avisa a la vista (por `ControladorGrafico`).
- **Relaciones:**
  - Realiza `ControladorJuego`.
  - Referencia a `ControladorGrafico` (→, `#controladorGrafico 1`).
  - Compone sus tres hilos (defensa 5): `Reloj` (◆, `#reloj 1`), `HiloJugador` (◆, `#hiloJugador 1`) y `HiloMundo` (◆, `#hiloMundo 1`). Los arranca en `iniciar(...)`, los pausa durante la transición y los detiene en `terminarPartida()`. Además el `Reloj` la referencia (`Reloj → Juego`, `#juego 1`) para avisarle `tic(segundos)` cada segundo: `Juego` lo crea con `new Reloj(this)`.
  - Agrega el modo de juego activo (◇ `ModoJuego*`, `#modoJuego 1`). El modo se crea en cada partida (lo crea el menú).
  - Compone los tres rankings (◆ `Ranking`, `#rankings 3`), uno por modo. Existen desde que arranca la aplicación, así el menú puede mostrarlos antes de jugar.
  - Agrega el nivel que se está jugando (◇ `Nivel`, `#nivel 1`). Los demás niveles no están en memoria: viven en sus archivos de texto y `GeneradorNivel` los construye cuando hacen falta. `Juego` lleva además el **número** del nivel actual (`numeroNivel : int`) para saber qué archivo cargar y para mostrarlo en el HUD.
  - Referencia a `GeneradorNivel` (→, `#generadorNivel 1`).
  - Referencia a `FabricaEntidades` (→, `#fabricaEntidades 1`), que arma en `iniciar(...)` con el dominio elegido. La fábrica, a su vez, guarda a `Juego` (atributo `# juego : Juego` del extendido) para asignar `#miJuego`. En el reducido de la defensa 4 esa flecha **sigue sin dibujarse** (ver Observaciones).
  - Referencia a `PersistenciaRanking` (→, `#persistencia 1`): carga los rankings al arrancar y los guarda al insertar (ver Persistencia del ranking).
  - Es el **único** que modifica el puntaje de Link (ver Puntaje).
  - Referencia a `Link` (→, `#jugador 1`): para pasarle las intenciones del jugador (ver Comunicación entre `Juego`, `Sector` y las entidades).

- La referencian todas las entidades (`Entidad* → Juego`, `#miJuego 1`): por ahí recibe los avisos de sucesos.

Qué guarda y qué no: `Juego` no guarda el sector activo como atributo (en el extendido de la defensa 4 no hay `sectorActivo`). Cuando lo necesita, lo obtiene de Link: `jugador.getSector()`, porque Link siempre está en el sector activo. Así no hay dos datos que mantener sincronizados.

`Reloj`

- **Utilidad:** el hilo que lleva el tiempo. En la defensa 4 se llamaba `Timer`; se renombró en la defensa 5 porque el enunciado prohíbe usar "Timers" (nunca fue `javax.swing.Timer`, pero el nombre invitaba a la pregunta).
- **Comportamiento:** hereda de `HiloJuego*` el arranque, la pausa y el `run()` fijo; en su `ciclo()`, una vez por segundo, suma `segundos` y llama `juego.tic(segundos)`, que es público. Además `reiniciar()` y `getSegundos()`. El tiempo se muestra en el HUD y lo usa el modo Contrarreloj.
- **Relaciones:** hereda de `HiloJuego*`; es parte de `Juego` (◆, `#reloj 1`) y referencia a `Juego` (→, `#juego 1`).
- **Por qué hay dos flechas entre `Juego` y `Reloj`:** son dos atributos distintos. `Juego ◆ Reloj` (`#reloj 1`) es el atributo de `Juego`: el reloj se crea con el juego y no vive sin él. `Reloj → Juego` (`#juego 1`) es el atributo de `Reloj`: a quién le avisa. `Juego` lo crea con `new Reloj(this)`. Ver Tiempo y reglas del modo.

```
        public class Reloj extends HiloJuego {                          // run() fijo en HiloJuego (Template Method)
            protected int segundos;
            protected Juego juego;                                      // #juego 1

            public Reloj(Juego j) { super(1000); juego = j; }           // Juego hace new Reloj(this)
            public void reiniciar() { segundos = 0; }
            protected void ciclo() {                                    // una vez por segundo
                segundos++;
                juego.tic(segundos);                                    // HUD y modo (Contrarreloj)
            }
        }
```

Hilos: `HiloJuego*`, `HiloJugador`, `HiloMundo`

- **`HiloJuego*`** (abstracta, naranja): realiza `Runnable`. Tiene el `run()` fijo (`while (activo) { if (!pausado) ciclo(); sleep(periodo); }`) y las operaciones `iniciar()`, `detener()`, `pausar()` y `reanudar()`. Cada subclase redefine solo `ciclo()`: es un **Template Method**.
- **`HiloJugador`:** cada ~30 ms hace `jugador.actualizarEstado()`. Referencia a `Link` (→, `#jugador 1`) y es parte de `Juego` (◆, `#hiloJugador 1`).
- **`HiloMundo`:** cada ~30 ms hace `sector.actualizarEntidades()` sobre el sector visible. Referencia al `Sector` (→, `#sector 1`), que cambia con `setSector(s)` en cada transición, y es parte de `Juego` (◆, `#hiloMundo 1`).
- Atributos, operaciones y código en 16.3.

`ModoJuego*`

- **Utilidad:** define las reglas que cambian entre modos.
- **Comportamiento (según el extendido):** `ModoJuego(cantidadNiveles)`; `esUltimoNivel(numero)`, `getNombre()`; ganchos `alRecibirDanio(j)` y `alPasarTiempo(j, segundos)` (por defecto no hacen nada; los redefinen `NoHit` y `Contrarreloj`); y los dos doble despachos del ranking: `getRanking(j)` (consulta) y `registrarPuntaje(j, e)` (comando). Ver Cómo llega el puntaje al ranking del modo.
- **Relaciones:** agregado en `Juego` (◇, `#modoJuego 1`). Usa a `Juego` solo como parámetro (dependencia, no dibujada en el reducido). No tiene relación con `Ranking`: no lo guarda, solo lo devuelve.
- **Subclases:**
  - `Clasico`: se gana llegando al final y venciendo al jefe del último nivel.
  - `Contrarreloj(tiempoLimite)`: cada nivel tiene un tiempo límite (`getTiempoLimite()`); redefine `alPasarTiempo`.
  - `NoHit`: hay que completar sin recibir daño; redefine `alRecibirDanio`.

`Ranking`

- **Utilidad:** Top 5 persistente de **un** modo de juego.
- **Comportamiento (según el extendido):** `MAXIMO = 5`; `insertar(e) : boolean`, `entraEnTop5(puntaje) : boolean`, `getEntradas()` y `cantidad()`. Guarda hasta 5 entradas ordenadas por puntaje. No sabe nada de archivos: la persistencia la hace `PersistenciaRanking`.
- **Relaciones:** es parte de `Juego` (◆, `#rankings 3`); compone `EntradaRanking` (◆, `#entradas 0..5`); lo carga y guarda `PersistenciaRanking` (dependencia «carga/guarda»).
- **Por qué cuelga directamente de `Juego`:** el menú principal tiene que mostrar el Top 5 de los tres modos **antes de jugar**, cuando todavía no existe ningún modo (el modo se crea al empezar la partida). Si el ranking estuviera dentro del modo (como en el diagrama válido), en el menú no habría de dónde sacarlo. Con los tres rankings en `Juego`, existen siempre. Además, las reglas del modo y la persistencia del ranking quedan separadas.
- **Por qué no hay relación `Ranking → ModoJuego`:** no hace falta. El ranking no necesita saber de qué modo es; el que sabe cuál es su ranking es el propio modo, por doble despacho.

Cómo llega el puntaje al ranking del modo

En el extendido de la defensa 4 los dos doble despachos trabajan **juntos**, cada uno con su función:

Método del modo	Tipo	Para qué	Qué llama en `Juego`
`getRanking(j) : Ranking`	Consulta	Saber **si el puntaje entra** al Top 5 antes de pedir el nombre, y qué ranking mostrar	`getRankingClasico()`, `getRankingContrarreloj()`, `getRankingNoHit()`
`registrarPuntaje(j, e)`	Comando	**Insertar** la entrada en el ranking del modo y guardarlo	`agregarAlRankingClasico(e)`, `agregarAlRankingContrarreloj(e)`, `agregarAlRankingNoHit(e)`

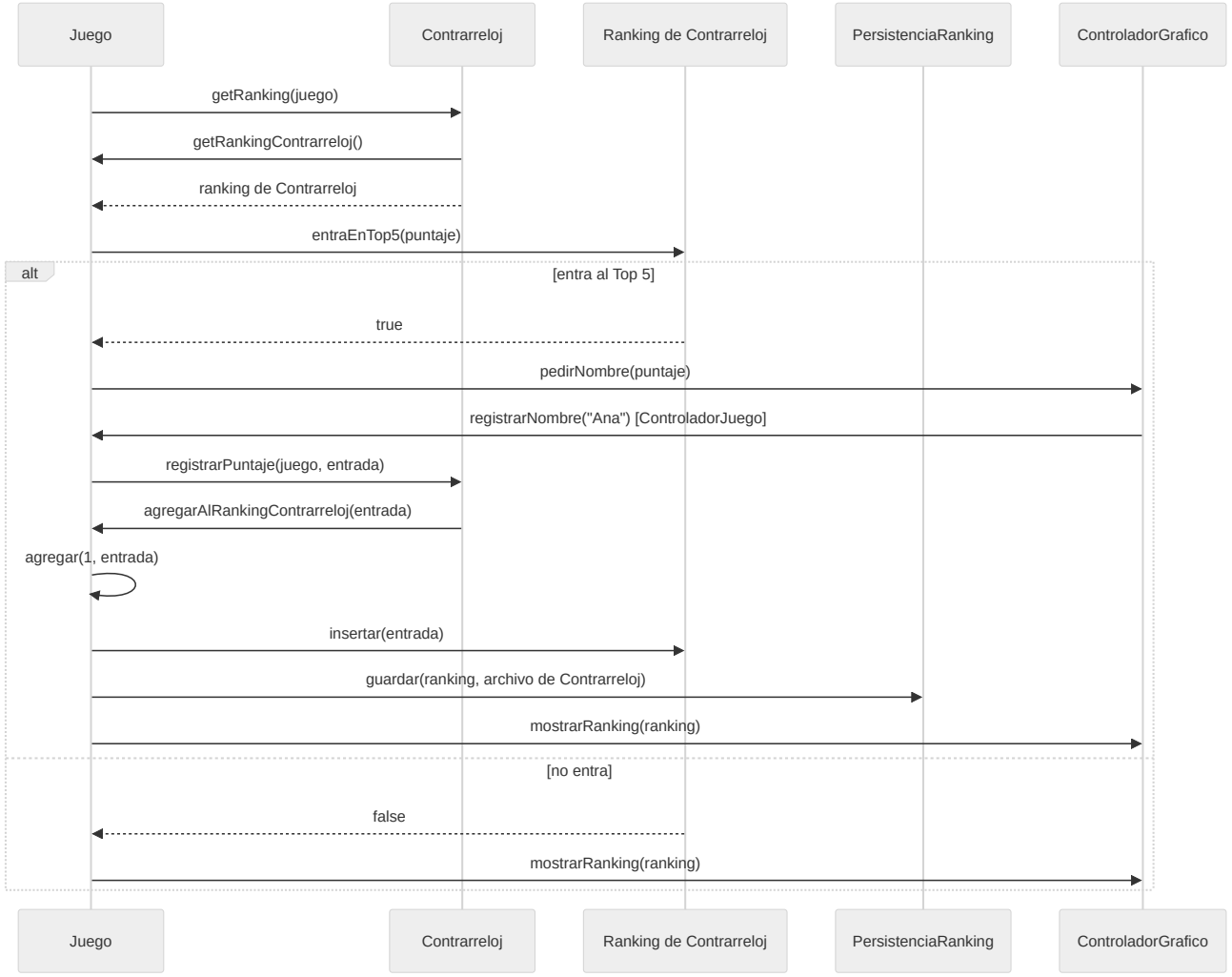
Los tres `agregarAlRanking…(e)` de `Juego` terminan en el método privado `agregar(i, e)`, que inserta en `rankings[i]` y le pide a `PersistenciaRanking` que guarde ese ranking. `Juego` tiene los tres rankings en `#rankings` (posiciones 0 = Clásico, 1 = Contrarreloj, 2 = NoHit). Nunca pregunta `if (modoJuego instanceof Clasico)`: le pregunta al modo.

```
        public class Juego implements ControladorJuego {
            protected Ranking[] rankings = new Ranking[3];         // #rankings 3 (se cargan al
arrancar)
            protected ModoJuego modoJuego;                         // #modoJuego 1

            public void terminarPartida() {
                reloj.detener(); hiloJugador.detener(); hiloMundo.detener();   // defensa 5:
los tres hilos
                Ranking r = modoJuego.getRanking(this);            // consulta: el ranking del
modo
                if (r.entraEnTop5(jugador.getPuntaje()))
                    controladorGrafico.pedirNombre(jugador.getPuntaje());   // solo si entra al
Top 5
                else
                    controladorGrafico.mostrarRanking(r);                   // si no, solo se
muestra
            }
            public void registrarNombre(String nombre) {           // lo llama la vista con el
nombre ingresado
                modoJuego.registrarPuntaje(this, new EntradaRanking(nombre,
jugador.getPuntaje()));   // comando
                controladorGrafico.mostrarRanking(modoJuego.getRanking(this));
            }

            // Segundo despacho: los llama cada modo
            public Ranking getRankingClasico()      { return rankings[0]; }
            public Ranking getRankingContrarreloj() { return rankings[1]; }
            public Ranking getRankingNoHit()        { return rankings[2]; }
            public void agregarAlRankingClasico(EntradaRanking e)      { agregar(0, e); }
            public void agregarAlRankingContrarreloj(EntradaRanking e) { agregar(1, e); }
            public void agregarAlRankingNoHit(EntradaRanking e)        { agregar(2, e); }
            private void agregar(int i, EntradaRanking e) {
                if (rankings[i].insertar(e)) persistencia.guardar(rankings[i], ARCHIVOS[i]);
            }
        }

        public class Contrarreloj extends ModoJuego {
            public Ranking getRanking(Juego j)                      { return
j.getRankingContrarreloj(); }
            public void registrarPuntaje(Juego j, EntradaRanking e) {
j.agregarAlRankingContrarreloj(e); }
        }
        // Clasico y NoHit son iguales, cada uno con su par de métodos de Juego.
```



Ejemplos:

1. **Ver los rankings desde el menú:** `GUI` llama a `getRankings()` por `ControladorJuego` y `Juego` devuelve los tres (`+ getRankings() : Ranking[]` devuelve `rankings`). No hace falta que exista ningún modo.
2. **Terminar una partida de Contrarreloj con 7300 puntos (entra al Top 5):** `modoJuego.getRanking(this)` ejecuta la versión de `Contrarreloj`, que devuelve `j.getRankingContrarreloj()`. `entraEnTop5(7300)` da `true`: se pide el nombre; cuando la vista llama a `registrarNombre("Ana")`, el modo inserta por `agregarAlRankingContrarreloj(e)`, se guarda el archivo y se muestra el ranking.
3. **Terminar con 900 puntos y el Top 5 lleno de puntajes mayores:** `entraEnTop5(900)` da `false`. **No se pide el nombre**: solo se muestra el ranking.
4. **Empezar otra partida en NoHit:** `iniciar(new NoHit(), dominio)`. El modo anterior se descarta y no hay que reiniciar ningún estado. Los rankings no se tocan: siguen en `Juego`.

Qué cambió respecto de la defensa 3: en la defensa 3 el extendido tenía `getRanking(j)` y un `guardarRanking(r)` que buscaba el índice comparando referencias. En la defensa 4 vuelve `registrarPuntaje(j, e)` y el índice lo fija cada `agregarAlRanking…`, así que el método privado `agregar(i, e)` ya sabe qué archivo escribir.

`EntradaRanking`

- **Utilidad:** una fila del ranking.

- **Comportamiento:** guarda nombre del jugador y puntaje (`EntradaRanking(nombre, puntaje)`, `getNombre()`, `getPuntaje()`).
- **Relaciones:** es parte de `Ranking` (◆, `#entradas 0..5`).

`PersistenciaRanking`

- **Utilidad:** guardar y leer los rankings en archivos, para que el Top 5 sobreviva al cerrar el juego.
- **Comportamiento:** `cargar(archivo) : Ranking` devuelve un `Ranking` con las entradas del archivo (vacío si no existe o está dañado); `guardar(r, archivo)` escribe una línea por entrada.
- **Relaciones:** la referencia `Juego` (`#persistencia 1`); carga y guarda `Ranking` (dependencia «carga/guarda»: lo recibe o lo devuelve, no lo guarda como atributo).

Persistencia del ranking

Decisión: un **archivo de texto por modo** (`ranking_clasico.txt`, `ranking_contrarreloj.txt`, `ranking_nohit.txt`), con una línea `nombre;puntaje` por entrada. Se puede leer a mano y es fácil de documentar. `Ranking` queda con su única responsabilidad (el Top 5) y la lectura y escritura de archivos queda en `PersistenciaRanking`.

```
Ana;7300
Juan;5400
Sofi;5100
```

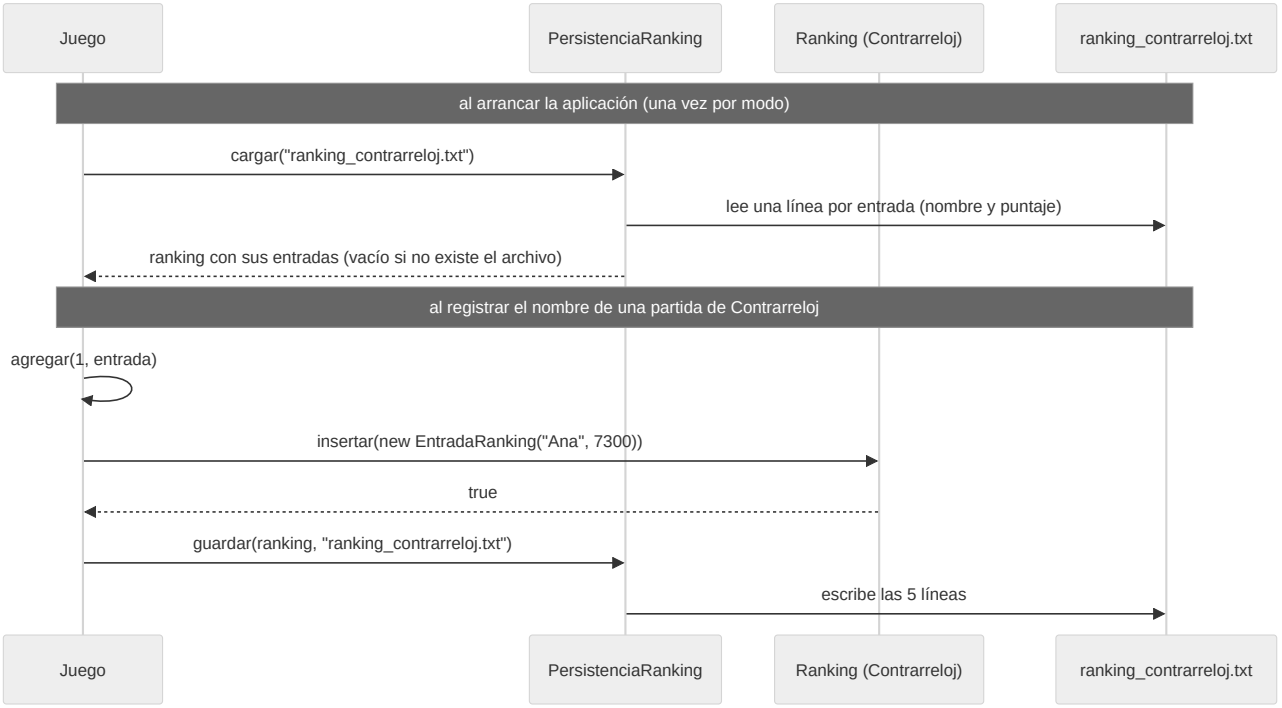
```
        public class Ranking {
            private static final int MAXIMO = 5;
            protected final List<EntradaRanking> entradas = new ArrayList<>();    // #entradas
0..5
            public boolean entraEnTop5(int puntaje) {
                return entradas.size() < MAXIMO || puntaje > entradas.get(entradas.size() -
1).getPuntaje();
            }
            public boolean insertar(EntradaRanking e) {
                if (!entraEnTop5(e.getPuntaje())) return false;
                entradas.add(e);
                entradas.sort((a, b) -> b.getPuntaje() - a.getPuntaje());          // de mayor
a menor
                if (entradas.size() > MAXIMO) entradas.remove(MAXIMO);
                return true;
            }
            public List<EntradaRanking> getEntradas() { return
Collections.unmodifiableList(entradas); }
            public int cantidad() { return entradas.size(); }
        }

        public class PersistenciaRanking {
            public Ranking cargar(String archivo) {
                Ranking r = new Ranking();
                File f = new File(archivo);
                if (!f.exists()) return r;                                          // primera
vez: vacío
```

```
                try (BufferedReader in = new BufferedReader(new FileReader(f))) {
                    String linea;
                    while ((linea = in.readLine()) != null) {
                        String[] partes = linea.split(";");                        //
"Ana;7300"
                        r.insertar(new EntradaRanking(partes[0], Integer.parseInt(partes[1])));
                    }
                } catch (IOException | RuntimeException ex) {
                    return new Ranking();                                          // archivo
dañado: vacío
                }
                return r;
            }
            public void guardar(Ranking r, String archivo) {
                try (PrintWriter out = new PrintWriter(new FileWriter(archivo))) {
                    for (EntradaRanking e : r.getEntradas()) out.println(e.getNombre() + ";" +
e.getPuntaje());
                } catch (IOException ex) { /* no se pudo guardar: el juego sigue */ }
            }
        }

        // En Juego: al arrancar se cargan los tres
        public Juego(ControladorGrafico vista) {
            controladorGrafico = vista;
            persistencia = new PersistenciaRanking();
            for (int i = 0; i < 3; i++) rankings[i] = persistencia.cargar(ARCHIVOS[i]);
        }
```

> En el extendido de la defensa 4 ya no aparecen la constante `ARCHIVOS` de `Juego` ni el `SEPARADOR` de `PersistenciaRanking`. El esquema los usa porque `agregar(i, e)` necesita saber qué archivo escribir; conviene volver a declararlos donde el equipo prefiera (ver Observaciones).



Explicación: al arrancar, `Juego` carga los tres rankings (así el menú puede mostrarlos antes de jugar). Al terminar una partida, el modo indica su ranking (doble despacho) y `Juego` consulta si el puntaje entra al Top 5: solo en ese caso pide el nombre; con el nombre, el modo inserta (otro doble despacho) y se reescribe el archivo.

Alternativa descartada: serialización de Java (`Ranking implements Serializable`). Menos código, pero el archivo no se puede leer a mano y se rompe si cambia la clase.

---

6. Mapa y niveles

Cómo se arma y se recorre el mapa: `GeneradorNivel` construye el `Nivel` desde un archivo; el nivel compone su `Escena`, la escena compone sus `Sector`, y cada sector referencia las entidades que contiene. El detalle de **qué línea del archivo crea qué objeto** está en la sección siguiente, 7. Formato y parseo de niveles.

`GeneradorNivel`

GeneradorNivel
`# carpetaNiveles : String`

«constructores»
`+ GeneradorNivel(carpetaNiveles : String)`
«consultas»
`+ generar(numero : int, fabrica : FabricaEntidades) : Nivel`
`- leerArchivo(numero : int) : List<String>`
`- crearEntidad(simbolo : char, x : int, y : int, fabrica : FabricaEntidades) : Entidad`
`- colocarFragmentosDeCorazon(e : Escena, fabrica : FabricaEntidades)`

- **Utilidad:** construye un `Nivel` a partir de un archivo de texto plano.
- **Comportamiento (según el extendido):** `GeneradorNivel(carpetaNiveles)`; `generar(numero, fabrica) : Nivel` es lo único público. Por dentro usa `leerArchivo(numero)`

(las líneas del archivo), `crearEntidad(simbolo, x, y, fabrica)` (traduce un carácter a una entidad pidiéndosela a la fábrica) y `colocarFragmentosDeCorazon(escena, fabrica)` (ubica los 4 `FragmentoDeCorazon`, como máximo 1 por sector). También sirve para reiniciar un nivel: se vuelve a llamar a `generar` con el mismo número.
- **Relaciones:** lo referencia `Juego` (→, `#generadorNivel 1`); crea `Nivel` (dependencia «crea»: lo arma y lo devuelve, pero no guarda un atributo de tipo `Nivel`; el que lo guarda es `Juego`, en `#nivel`). Usa `FabricaEntidades` solo como **parámetro** de `generar(...)`: no guarda una referencia a la fábrica y esa dependencia no se dibuja. Ver Cómo obtiene `GeneradorNivel` la fábrica.

Cómo obtiene `GeneradorNivel` la fábrica

`GeneradorNivel` necesita la `FabricaEntidades` para crear las entidades que lee del archivo, pero **no tiene una relación propia con ella**. Se la pasa `Juego`, que ya conoce a los dos (`Juego → GeneradorNivel` y `Juego → FabricaEntidades`), cada vez que le pide un nivel:

```
        // En Juego (cargarNivel)
        nivel = generadorNivel.generar(numeroNivel, fabricaEntidades);
```

Tipo de relación: como la fábrica llega por parámetro y no se guarda como atributo, entre `GeneradorNivel` y `FabricaEntidades` hay una **dependencia** (uso temporal), no una asociación. Se decidió **no dibujarla** para no recargar el diagrama: la conexión se entiende porque `Juego` conoce a los dos.

Por qué así:

- **Un solo dueño de la fábrica:** solo `Juego` guarda la `FabricaEntidades`. No hay dos referencias que mantener.
- **Respeta el dominio elegido:** el dominio gráfico se elige en el menú y `Juego` arma la fábrica con la `FabricaSprites` correspondiente (`FabricaSpritesOriginal` o `FabricaSpritesAlternativa`). Como se pasa en cada llamada, el generador siempre usa la del dominio actual, aunque se cambie entre partidas.
- **`GeneradorNivel` queda simple:** solo sabe leer el archivo y armar el mapa; no le importa de dónde sale la fábrica.

Referencias de contexto de las entidades creadas:

- `#miSector`: la asigna el sector al agregar la entidad (`sector.agregar(e)` llama a `e.setSector(this)`).
- `#miJuego`: la asigna la **fábrica**, que conoce a `Juego` desde que se crea (`new FabricaEntidades(this, dominio)` dentro de `Juego`). Así toda entidad que crea, tanto al cargar el nivel como en plena partida (proyectiles, drops, bellota), ya sale con su `#miJuego` puesto.

Ejemplos:

1. **Empezar la partida en el dominio alternativo:** en el menú se elige el dominio; `Juego` crea `new FabricaEntidades(this, new FabricaSpritesAlternativa())` y llama a `generar(1, fabricaEntidades)`. Todas las entidades del nivel 1 salen con los sprites alternativos y con `#miJuego` asignado.
2. **Pasar al nivel 2:** `Juego` llama a `generar(2, fabricaEntidades)` con la misma fábrica; el generador no guardó nada del nivel anterior.
3. **Reiniciar el nivel actual:** `Juego` llama otra vez a `generar(numeroNivel, fabricaEntidades)`; se reconstruye desde el archivo con el mismo dominio.

`Nivel`

Nivel
mapa : Escena # jefes : List<Jefe> # filaInicial : int # columnaInicial : int # posicionInicial : Posicion
«constructores» + Nivel(mapa : Escena, jefes : List<Jefe>, fila : int, columna : int, inicio : Posicion) *«consultas»* + getMapa() : Escena + getSectorInicial() : Sector + getPosicionInicial() : Posicion + getJefes() : List<Jefe> + jefesDerrotados() : boolean

- **Utilidad:** un nivel del juego (hay al menos 2, con dificultad creciente).
- **Comportamiento (según el extendido):** `Nivel(mapa, jefes, fila, columna, inicio)`; `getMapa()`, `getSectorInicial()`, `getPosicionInicial()`, `getJefes()` y `jefesDerrotados()`. Termina cuando sus jefes son derrotados. **No guarda la lista de entidades**: esas viven en cada `Sector`.
- **Relaciones:** agregado en `Juego` (◇, `#nivel 1`); compone su `Escena` (◆, `#mapa 1`); referencia a `Jefe*` (→, `#jefes 1..*`); referencia a `Posicion` (→, `#posicionInicial 1`): dónde aparece Link dentro del sector inicial. Cada jefe referenciado es **el mismo objeto** que está en algún `Sector` (es un atajo para saber cuándo termina el nivel sin recorrer todos los sectores).
- **Por qué `#nivel 1`:** `Juego` solo necesita el nivel que se está jugando. El diagrama válido tenía el rol `#nivelActual` con multiplicidad `2..*`, que se contradice; se dejó el rol `#nivel` con multiplicidad `1`.
  - El "≥ 2 niveles" del enunciado se cumple con **los archivos** de niveles, no con objetos en memoria.
  - Al pasar de nivel o al reiniciarlo, `GeneradorNivel` construye un `Nivel` nuevo desde el archivo y `Juego` reemplaza su referencia. Así siempre se arranca de un nivel limpio (enemigos y power-ups en su lugar).
  - Link no se recrea: se lo reubica en el nivel nuevo, así conserva vidas, puntaje y rupias.
  - Es **agregación** (◇) y no composición porque el `Nivel` lo construye `GeneradorNivel`, no `Juego`, y se reemplaza durante la partida.

```
        // Empezar la partida
        numeroNivel = 1;
        nivel = generadorNivel.generar(numeroNivel, fabricaEntidades);   // #nivel apunta al nivel 1

        // Vencer al jefe del nivel 1
        numeroNivel++;
        nivel = generadorNivel.generar(numeroNivel, fabricaEntidades);   // el nivel 1 se descarta
```

```
        // Reiniciar el nivel actual (por ejemplo, al caer al vacío)
        nivel = generadorNivel.generar(numeroNivel, fabricaEntidades);   // se vuelve a construir desde el archivo
```

Diferencia entre `posicionInicial` y `filaInicial` / `columnaInicial`

Los tres atributos dicen **dónde empieza Link**, pero a dos escalas distintas: **en qué pantalla** y **en qué lugar de esa pantalla**.

Atributo	Qué indica	Unidad	Ejemplo con `INICIO 0 0 2 5`
`filaInicial`, `columnaInicial`	**En qué sector** de la grilla del mapa empieza Link	Índices de la grilla de sectores	`0`, `0`: el sector de arriba a la izquierda
`posicionInicial`	**En qué lugar dentro de ese sector** aparece	Píxeles, relativos al sector	`(96, 240)`: la celda (2, 5) × 48 píxeles

```
Escena (MAPA 2 3)                  Sector (0,0): 768 x 528 px
+-------+-------+-------+         +----------------------+
| (0,0) | (0,1) | (0,2) |  --->   |                      |
+-------+-------+-------+         |   L <- (96, 240)     |
| (1,0) | (1,1) | (1,2) |         |                      |
+-------+-------+-------+         +----------------------+
 filaInicial, columnaInicial       posicionInicial
```

Por qué hacen falta los dos datos: cada `Sector` tiene su propio sistema de coordenadas, que empieza en (0,0) en su esquina. Una `Posicion` sola no alcanza, porque (96, 240) existe en **todos** los sectores. Por eso `Juego` usa los dos datos juntos:

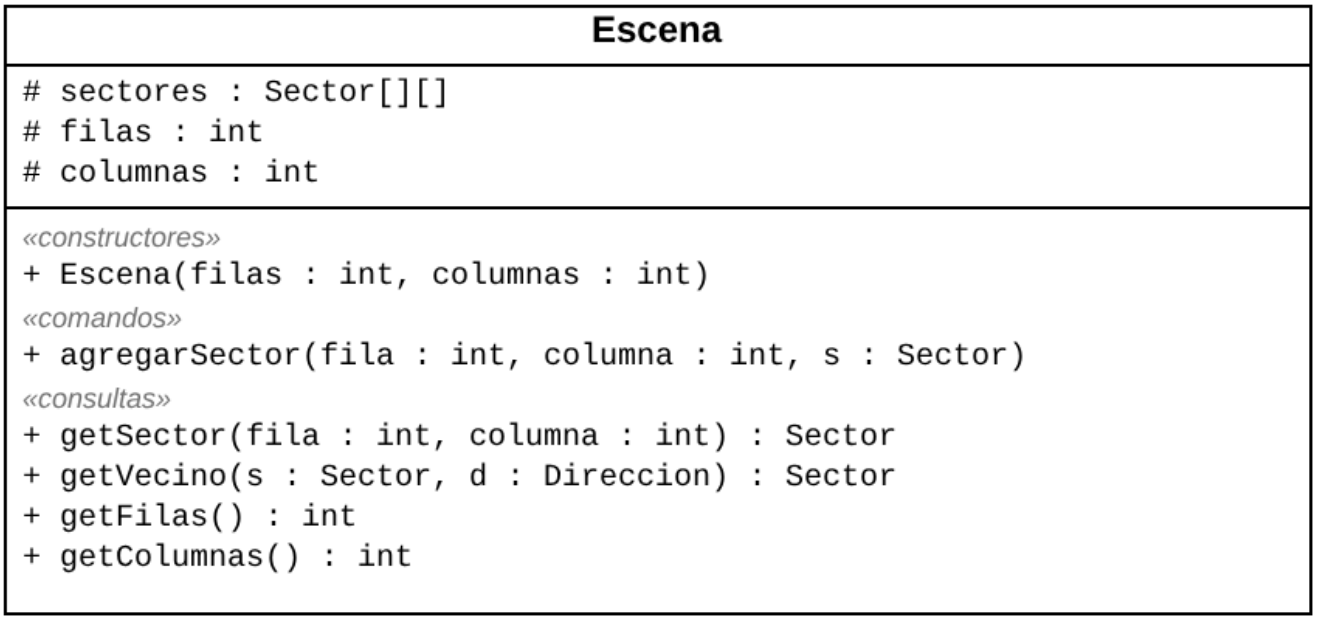
```
        Sector inicio = nivel.getSectorInicial();         // mapa.getSector(filaInicial, columnaInicial): en qué pantalla
        inicio.agregar(jugador);
        jugador.setPosicion(nivel.getPosicionInicial());  // en qué lugar de esa pantalla
```

Se usan al **empezar el nivel**, al **caer al vacío** (se reinicia el nivel) y al **perder todas las vidas** (Link vuelve al inicio del nivel actual, según el ayudante).

Por qué no se guarda directamente el `Sector` inicial: en el diagrama reducido `Nivel` no tiene una relación con `Sector`: solo conoce su `Escena` (`#mapa`), y la escena es la que conoce los sectores. Guardando fila y columna, `Nivel` obtiene el sector a través de `mapa.getSector(...)` sin agregar una relación nueva.

De dónde salen: de la línea `INICIO <fila> <columna> <x> <y>` del archivo del nivel (ver 7.3).

`Escena`



- **Utilidad:** el mapa completo de un nivel.
- **Comportamiento (según el extendido):** `Escena(filas, columnas)` con `sectores : Sector[][]`; `agregarSector(fila, columna, s)`, `getSector(fila, columna)`, `getVecino(s, d)`, `getFilas()` y `getColumnas()`. Contiene los sectores organizados en una **grilla** y responde "¿cuál es el sector vecino de este, hacia tal dirección?" (`getVecino(sector, direccion)`), que es lo que necesita `Juego` cuando Link cruza un borde.
- **Relaciones:** es parte de su `Nivel` (◆, `#mapa 1`); compone `Sector` (◆, `#sectores 1..*`). La composición es **unidireccional**: la escena conoce a sus sectores, pero los sectores no conocen a su escena (ver Por qué `Sector` no conoce a `Escena`).

Por qué `Nivel ◆ Escena` es composición

Una relación es composición cuando se cumplen tres condiciones, y las tres se cumplen para la escena:

Condición	¿Se cumple?	Por qué
La parte pertenece a **un solo** todo	Sí	Cada nivel tiene su propio mapa; las escenas no se comparten
La parte **no tiene sentido** sin el todo	Sí	El mapa del nivel 1 no sirve fuera del nivel 1
La parte **no pasa** de un todo a otro	Sí	Al pasar de nivel o reiniciarlo, `GeneradorNivel` construye un nivel nuevo con su propia escena; la anterior se descarta junto con su nivel

Que la escena la arme `GeneradorNivel` y no `Nivel` no cambia nada: la composición habla de **pertenencia y tiempo de vida**, no de quién hace el `new`. El generador arma la escena y se la entrega al nivel; desde ahí es del nivel y muere con él.

Coherencia con el resto del mapa:

Relación	Tipo	Motivo
`Juego ◇ Nivel` (`#nivel 1`)	Agregación	El nivel lo construye `GeneradorNivel` y se reemplaza durante la partida
`Nivel ◆ Escena` (`#mapa 1`)	Composición	La escena es parte exclusiva del nivel y muere con él
`Escena ◆ Sector` (`#sectores 1..*`)	Composición	El sector es parte exclusiva de la escena
`Sector → Entidad*` (`#entidades 0..*`)	Asociación	Las entidades no son exclusivas: Link pasa de un sector a otro

El mapa queda como una estructura de partes exclusivas (`Nivel ◆ Escena ◆ Sector`); la relación con las entidades es más "suelta" porque las entidades sí se mueven.

Ejemplo de grilla

El nivel 1 (`MAPA 2 3`) tiene 6 sectores en 2 filas y 3 columnas:

```
            columna 0          columna 1        columna 2
fila 0   [ (0,0) inicio ]   [ (0,1) espada ]  [ (0,2) ]
fila 1   [ (1,0) jefe   ]   [ (1,1)        ]  [ (1,2) ]
```

- `escena.getVecino((1,1), ARRIBA)` → `(0,1)`
- `escena.getVecino((0,1), DERECHA)` → `(0,2)`
- `escena.getVecino((0,0), IZQUIERDA)` → `null` (borde del mapa: Link no puede salir por ahí; además el archivo no tiene pasos hacia afuera)

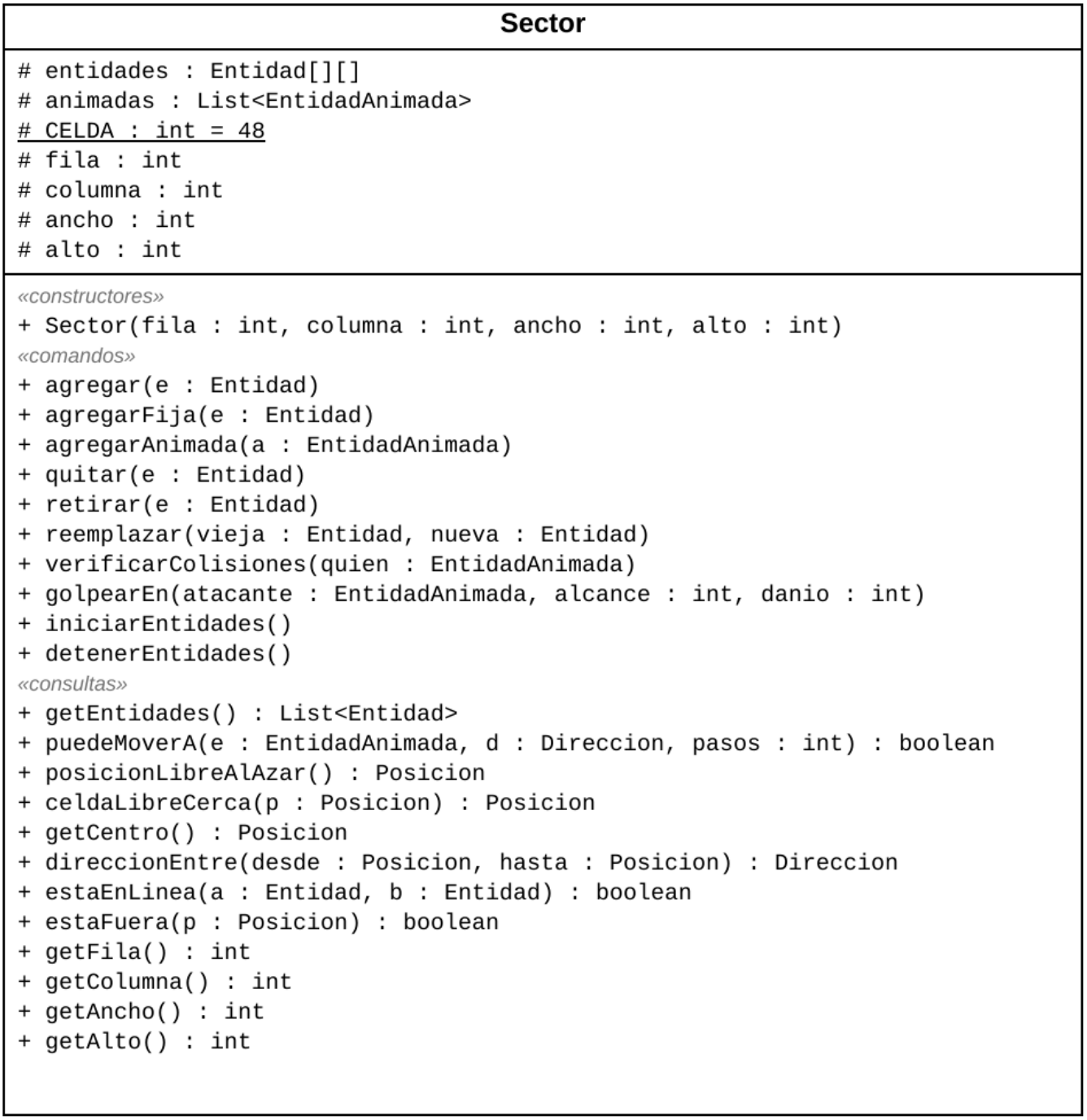
Por qué `Sector` no conoce a `Escena`

En el diagrama válido la composición `Sector ◆ Escena` tenía, además de `#sectores 1..*`, la etiqueta `#miMapa 1`, que sugería que cada sector conoce a su escena. Se sacó porque ningún caso lo necesita:

Caso	¿Necesita `Sector → Escena`?	Quién lo resuelve
Buscar el sector vecino al cruzar un borde	No	`Juego` le pregunta a la `Escena` (llega por `Nivel → Escena`)
Ubicar los 4 `FragmentoDeCorazon` (máx. 1 por sector)	No	`GeneradorNivel`, al construir el nivel
Colisiones, agregar, sacar o reemplazar entidades	No	El propio `Sector`
Fin de nivel, transición entre sectores	No	`Juego`

La coordinación del cambio de sector la hace `Juego`, que es quien conoce el nivel actual, a Link (`#jugador`) y la vista (`ControladorGrafico`). Si el sector conociera a su escena, la lógica del cambio quedaría repartida entre `Sector` y `Juego` sin ninguna ventaja.

`Sector`



- **Utilidad:** cada pantalla del mapa.
- **Comportamiento (según el extendido):** `Sector(fila, columna, ancho, alto)`. Es el responsable de todo lo **espacial** de su pantalla:
  - agregar, sacar, retirar y reemplazar entidades: `agregar(e)`, `quitar(e)` (la elimina), `retirar(e)` (la saca de `animadas` sin eliminarla, para Link al cambiar de sector), `reemplazar(vieja, nueva)` (`SuelosQuebradizo` → `CaidasAlVacio`);
  - colisiones y golpes: `verificarColisiones(quien)` (doble despacho contra cada entidad que se toca con `quien`) y `golpearEn(atacante, alcance, danio)` (golpe de espada a lo que esté delante);
  - consultas de espacio: `puedeMoverA(e, d, pasos)`, `posicionLibreAlAzar()`, `getCentro()`, `direccionEntre(desde, hasta)`, `estaEnLinea(a, b)`, `estaFuera(p)`;
  - actualización (defensa 5): `actualizarEntidades()`, que llama `HiloMundo` en cada vuelta y hace una vuelta de cada animada del sector (salvo Link, que lo actualiza

`HiloJugador`). Reemplaza a `iniciarEntidades()` y `detenerEntidades()` de la defensa 4: solo se mueven las entidades del sector visible.

- **Dónde guarda cada entidad (extendido):** lo **fijo** (obstáculos, power-ups, objetos) va en la matriz `entidades : Entidad[][]`, en la celda de su posición (`entidades[fila][columna]`, una por celda). Lo que **se mueve** (Link, enemigos, jefes, proyectiles) va en `animadas : List<EntidadAnimada>`, porque cambia de celda y puede quedar encima de algo fijo (Link sobre el pasto). `getEntidades()` arma una lista con las dos cosas.
- **Píxeles y casilleros (`CELDA = 48`, en el extendido):** las posiciones están en píxeles y la matriz en casilleros de 48 × 48. La constante estática `CELDA` traduce de uno a otro (`columna = x / 48`, `fila = y / 48`): con ella `Sector` arma la matriz (11 × 16), guarda cada entidad fija en su casillero y, cuando algo se mueve, mira solo los casilleros cercanos en vez de recorrer todas las entidades. Ejemplo: Link en (100, 240) está en `entidades[5][2]`. Más detalle en el explicativo de posición.
- **Coordenadas locales:** todos los sectores miden lo mismo (una pantalla) y las posiciones de sus entidades están en coordenadas **del sector**: el (0,0) es la esquina superior izquierda de esa pantalla. Por eso una posición sola no dice dónde está una entidad en el mapa: hace falta saber también en qué sector está (eso lo dan `#entidades` y `#miSector`).
- **Relaciones:** es parte de `Escena` (◆, `#sectores 1..*`); referencia a sus entidades (→ `Entidad*`, `#entidades 0..*`, asociación unidireccional) y a las que se mueven (→ `EntidadAnimada*`, `#animadas 0..*`); lo referencia cada `EntidadAnimada*` que está en él (`#miSector 1`). Para saber dónde guardar una entidad sin `instanceof`, `agregar(e)` usa doble despacho: `e.ubicarseEn(this)`, que `Entidad*` responde con `agregarFija(this)` y `EntidadAnimada*` con `agregarAnimada(this)` (ver la observación 26). Lo referencia `HiloMundo` (`#sector 1`) mientras es el sector visible. Ver Comunicación entre `Juego`, `Sector` y las entidades.

---

7. Formato y parseo de niveles

El enunciado pide que cada nivel se cargue desde un **archivo de texto plano** con un formato propio y documentado. La especificación completa del formato (reglas de un archivo válido, los dos niveles con su imagen y la comparación de dificultad) está en `Defensa 3/Formateo-de-niveles.md`, y la explicación con ejemplos en `Defensa 3/Explicacion-formateo-de-niveles.md`. Esta sección lo mira **desde el diagrama reducido**: qué clase lee el archivo, qué objeto se crea con cada línea y qué relación del diagrama queda armada.

7.1 Quién lee el archivo y con qué relaciones

Paso	Clase que actúa	Relación del diagrama que usa o arma
Pide el nivel	`Juego` (`cargarNivel()`)	`Juego → GeneradorNivel` (`#generadorNivel 1`), `Juego → FabricaEntidades` (`#fabricaEntidades 1`)
Lee el archivo y lo recorre	`GeneradorNivel` (`generar`, `leerArchivo`)	— (el archivo no es una clase)
Arma el mapa	`GeneradorNivel`	crea `Escena`, `Sector`; arma `Escena ◆ Sector` (`#sectores`)
Traduce cada símbolo	`GeneradorNivel` (`crearEntidad`) → `FabricaEntidades`	dependencia no dibujada `GeneradorNivel ⇢ FabricaEntidades` (parámetro)
Pide las imágenes	`FabricaEntidades` → `FabricaSprites*`	`FabricaEntidades → FabricaSprites*` (`#fabricaSprites 1`), `FabricaSprites* «crea» Sprite`

Paso	Clase que actúa	Relación del diagrama que usa o arma
Crea la entidad	`FabricaEntidades`	arma `Entidad* → Sprite` (`#sprite`), `Entidad* → Posicion` (`#posicion`), `Entidad* → Juego` (`#miJuego`)
Ubica la entidad	`Sector.agregar(e)` → `e.ubicarseEn(sector)`	si es fija arma `Sector → Entidad*` (`#entidades`); si es animada, `Sector → EntidadAnimada*` (`#animadas`) y `EntidadAnimada* → Sector` (`#miSector`)
Registra los jefes	`GeneradorNivel`	arma `Nivel → Jefe*` (`#jefes 1..*`)
Pone los fragmentos de corazón	`GeneradorNivel` (`colocarFragmentosDeCorazon`)	`Sector → Entidad*` en 4 sectores distintos
Devuelve el nivel	`GeneradorNivel`	`GeneradorNivel «crea» Nivel`; arma `Nivel ◆ Escena` (`#mapa`) y `Nivel → Posicion` (`#posicionInicial`)
Guarda el nivel y ubica a Link	`Juego`	arma `Juego ◇ Nivel` (`#nivel`); Link entra al sector inicial (`#entidades`, `#miSector`)

Ninguna clase de la vista participa en el parseo: el archivo se convierte en objetos de la lógica, y la vista recién se entera cuando `Juego` activa el sector inicial (`mostrarEntidad` por cada entidad, ver Activar un sector).

7.2 El formato en una pantalla

```
MAPA <filas> <columnas>          <- tamaño de la grilla de sectores (la Escena)
CELDAS <ancho> <alto> <tamaño>   <- celdas por sector y píxeles por celda
INICIO <fila> <columna> <x> <y>  <- sector donde empieza Link y celda dentro de ese sector
SECTOR <fila> <columna>          <- un bloque por sector, en cualquier orden
<alto líneas de exactamente ancho caracteres: un carácter por celda>
SECTOR <fila> <columna>
...
```

Las primeras líneas reales del nivel 1 (`Defensa 3/niveles/nivel1.txt`):

```
MAPA 2 3
CELDAS 16 11 48
INICIO 0 0 2 5
SECTOR 0 0
RRRRRRRRRRRRRRRR
R..............R
R..PPP......$..R
R..PPP.........R
R..............R
R.......M......R
R..............R
R...C.....PPP..R
R.........PPP..R
R..............R
RRRRRRRRRRRRRRRR
SECTOR 0 1
...
```

Conversión de celda a píxeles: la celda de la columna `k` y la fila `r` de un sector está en `x = k × tamaño`, `y = r × tamaño`, relativa al sector. Con `tamaño = 48`, la celda (2, 5) está en (96, 240) y la

(15, 10) en (720, 480). Cada sector mide `ancho × tamaño` por `alto × tamaño` = 768 × 528 píxeles.

7.3 Qué objeto y qué relación crea cada línea

Línea del archivo	Ejemplo (nivel 1)	Qué hace `GeneradorNivel`	Objeto que se crea	Relación que queda armada
`MAPA f c`	`MAPA 2 3`	`new Escena(2, 3)`	Una `Escena` con una grilla vacía de 2 × 3	Todavía ninguna; al final, `Nivel ◆ Escena` (`#mapa 1`)
`CELDAS a h t`	`CELDAS 16 11 48`	Guarda `ancho = 16`, `alto = 11`, `tamaño = 48` en variables locales	Ninguno	Ninguna: solo sirve para el tamaño de los `Sector` (768 × 528) y para convertir celdas a píxeles
`INICIO f c x y`	`INICIO 0 0 2 5`	Guarda `fila = 0`, `columna = 0` y crea `new Posicion(2 × 48, 5 × 48)`	Una `Posicion` (96, 240)	Al final, `Nivel → Posicion` (`#posicionInicial 1`); `filaInicial` y `columnaInicial` son atributos `int` de `Nivel`
`SECTOR f c`	`SECTOR 0 0`	`new Sector(0, 0, 768, 528)` y `escena.agregarSector(0, 0, sector)`	Un `Sector`	`Escena ◆ Sector` (`#sectores 1..*`)
Cada carácter distinto de `.`	`M` en la columna 8 de la fila 5	`crearEntidad('M', 384, 240, fabrica)` → `fabrica.crearMoblin(384, 240)` y `sector.agregar(moblin)`	Un `Moblin` con su `Sprite` y su `Posicion`	`Entidad* → Sprite` (`#sprite`), `Entidad* → Posicion` (`#posicion`), `Entidad* → Juego` (`#miJuego`), `Sector → Entidad*` (`#entidades`), `EntidadAnimada* → Sector` (`#miSector`)
Carácter `.`	`.`	No crea nada	Ninguno	Ninguna. En el borde de un sector, el `.` es un **paso** al vecino
Carácter de jefe (`S`, `H`)	`S` en el sector (1,0)	Igual que cualquier entidad y además lo agrega a la lista de jefes	Un `MasterStalfos`	Lo anterior y, al final, `Nivel → Jefe*` (`#jefes 1..*`)
Fin del archivo	—	`colocarFragmentosDeCorazon(escena, fabrica)` y `new Nivel(escena, jefes, 0, 0, inicio)`	4 `FragmentoDeCorazon` y el `Nivel`	`Sector → Entidad*` para cada fragmento; `GeneradorNivel «crea» Nivel`; `Nivel ◆ Escena`; `Nivel → Posicion`; `Nivel → Jefe*`

7.4 Qué crea cada símbolo

Símbolo	Clase	Método de `FabricaEntidades`	Método de `FabricaSprites*`	Tipo de entidad	¿Arma `#miSector`?
`R`	`Roca`	`crearRoca(x, y)`	`crearSpritesRoca()`	`Obstaculo*`	No
`A`	`Agua`	`crearAgua(x, y)`	`crearSpritesAgua()`	`Obstaculo*`	No
`P`	`Pasto`	`crearPasto(x, y)`	`crearSpritesPasto()`	`Obstaculo*`	No
`V`	`CaidasAlVacio`	`crearCaidasAlVacio(x, y)`	`crearSpritesCaidasAlVacio()`	`Obstaculo*`	No

Símbolo	Clase	Método de `FabricaEntidades`	Método de `FabricaSprites*`	Tipo de entidad	¿Arma `#miSector`?
`Q`	`SuelosQuebradizo`	`crearSuelosQuebradizo(x, y)`	`crearSpritesSuelosQuebradizo()`	`Obstaculo*`	No
`M`	`Moblin`	`crearMoblin(x, y)`	`crearSpritesMoblin()`	`EnemigoConDrop*`	Sí
`N`	`MoblinConEspadaYEscudo`	`crearMoblinConEspadaYEscudo(x, y)`	`crearSpritesMoblinConEspadaYEscudo()`	`EnemigoConDrop*`	Sí
`E`	`ErizoDeMar`	`crearErizoDeMar(x, y)`	`crearSpritesErizoDeMar()`	`EnemigoConDrop*`	Sí
`O`	`Octorok`	`crearOctorok(x, y)`	`crearSpritesOctorok()`	`EnemigoConDrop*`	Sí
`T`	`TrampaDeCuchillas`	`crearTrampaDeCuchillas(x, y)`	`crearSpritesTrampaDeCuchillas()`	`Enemigo*`	Sí
`W`	`Wizzrobe`	`crearWizzrobe(x, y)`	`crearSpritesWizzrobe()`	`EnemigoConDrop*`	Sí
`S`	`MasterStalfos`	`crearMasterStalfos(x, y)`	`crearSpritesMasterStalfos()`	`Jefe*` (va también a `#jefes`)	Sí
`H`	`StoneHinox`	`crearStoneHinox(x, y)`	`crearSpritesStoneHinox()`	`Jefe*` (va también a `#jefes`)	Sí
`$`	`Rupia`	`crearRupia(x, y)`	`crearSpritesRupia()`	`PowerUP*`	No
`C`	`Corazon`	`crearCorazon(x, y)`	`crearSpritesCorazon()`	`PowerUP*`	No
`X`	`Espada`	`crearEspada(x, y)`	`crearSpritesEspada()`	`Objeto*`	No
`.`	—	—	—	Vacío o paso	—

Por qué "¿Arma `#miSector`?" depende del tipo y no de un `if`: `Sector.agregar(e)` siempre llama a `e.setSector(this)`. En `Entidad*` ese método no hace nada; `EntidadAnimada*` lo redefine para guardar `#miSector`. El sector no pregunta qué tipo de entidad está agregando.

Por qué el `switch` sobre el carácter no viola la regla de no usar `instanceof`: traduce **texto a objetos**; no pregunta por el tipo de un objeto ya creado. Es el único lugar del juego donde aparece la lista de símbolos.

7.5 Ejemplo: el sector (0,0) del nivel 1, línea por línea

```
SECTOR 0 0
RRRRRRRRRRRRRRR      fila 0
R.............R      fila 1
R..PPP......$..R     fila 2
R..PPP.........R     fila 3
R..............      fila 4  (paso a la derecha)
R.......M.......     fila 5  (paso a la derecha)
R..............      fila 6  (paso a la derecha)
R...C.....PPP..R     fila 7
```

```
R.........PPP..R      fila 8
R.............R      fila 9
RRRRRRRRRRRRRRR      fila 10
```

Línea	Llamadas a la fábrica (x, y en píxeles)	Qué queda en `#entidades` del sector (0,0)
`SECTOR 0 0`	`new Sector(0, 0, 768, 528)`; `escena.agregarSector(0, 0, s)`	Nada todavía
Fila 0 `RRRRRRRRRRRRRRR`	`crearRoca(0, 0)`, `crearRoca(48, 0)`, …, `crearRoca(720, 0)`	16 rocas (borde superior)
Fila 1 `R.............R`	`crearRoca(0, 48)`, `crearRoca(720, 48)`	2 rocas (bordes laterales)
Fila 2 `R..PPP.....$..R`	`crearRoca(0, 96)`, `crearPasto(144, 96)`, `crearPasto(192, 96)`, `crearPasto(240, 96)`, `crearRupia(576, 96)`, `crearRoca(720, 96)`	2 rocas, 3 pastos y 1 rupia
Fila 3 `R..PPP.....R`	`crearRoca(0, 144)`, tres `crearPasto` en x = 144, 192, 240, `crearRoca(720, 144)`	2 rocas y 3 pastos
Fila 4 `R.............`	`crearRoca(0, 192)`	1 roca: la columna 15 es `.`, un **paso** al sector (0,1)
Fila 5 `R.......M.....`	`crearRoca(0, 240)`, `crearMoblin(384, 240)`	1 roca y el **Moblin**, que además queda con `#miSector` = (0,0)
Fila 6 `R.............`	`crearRoca(0, 288)`	1 roca (paso)
Fila 7 `R...C.....PPP..R`	`crearRoca(0, 336)`, `crearCorazon(192, 336)`, tres `crearPasto` en x = 480, 528, 576, `crearRoca(720, 336)`	2 rocas, 1 corazón y 3 pastos
Fila 8 `R.........PPP..R`	`crearRoca(0, 384)`, tres `crearPasto` en x = 480, 528, 576, `crearRoca(720, 384)`	2 rocas y 3 pastos
Fila 9 `R.............R`	`crearRoca(0, 432)`, `crearRoca(720, 432)`	2 rocas
Fila 10 `RRRRRRRRRRRRRRR`	`crearRoca(0, 480)`, …, `crearRoca(720, 480)`	16 rocas (borde inferior)

Total del sector (0,0): **62 entidades** (47 rocas, 12 pastos, 1 rupia, 1 corazón y 1 Moblin), más el fragmento de corazón si el azar elige este sector. Link no está en el archivo: con `INICIO 0 0 2 5`, `Juego` lo agrega a este sector en (96, 240), en la fila 5, a la izquierda del Moblin.

Cómo queda el sector como objetos (solo algunas entidades):

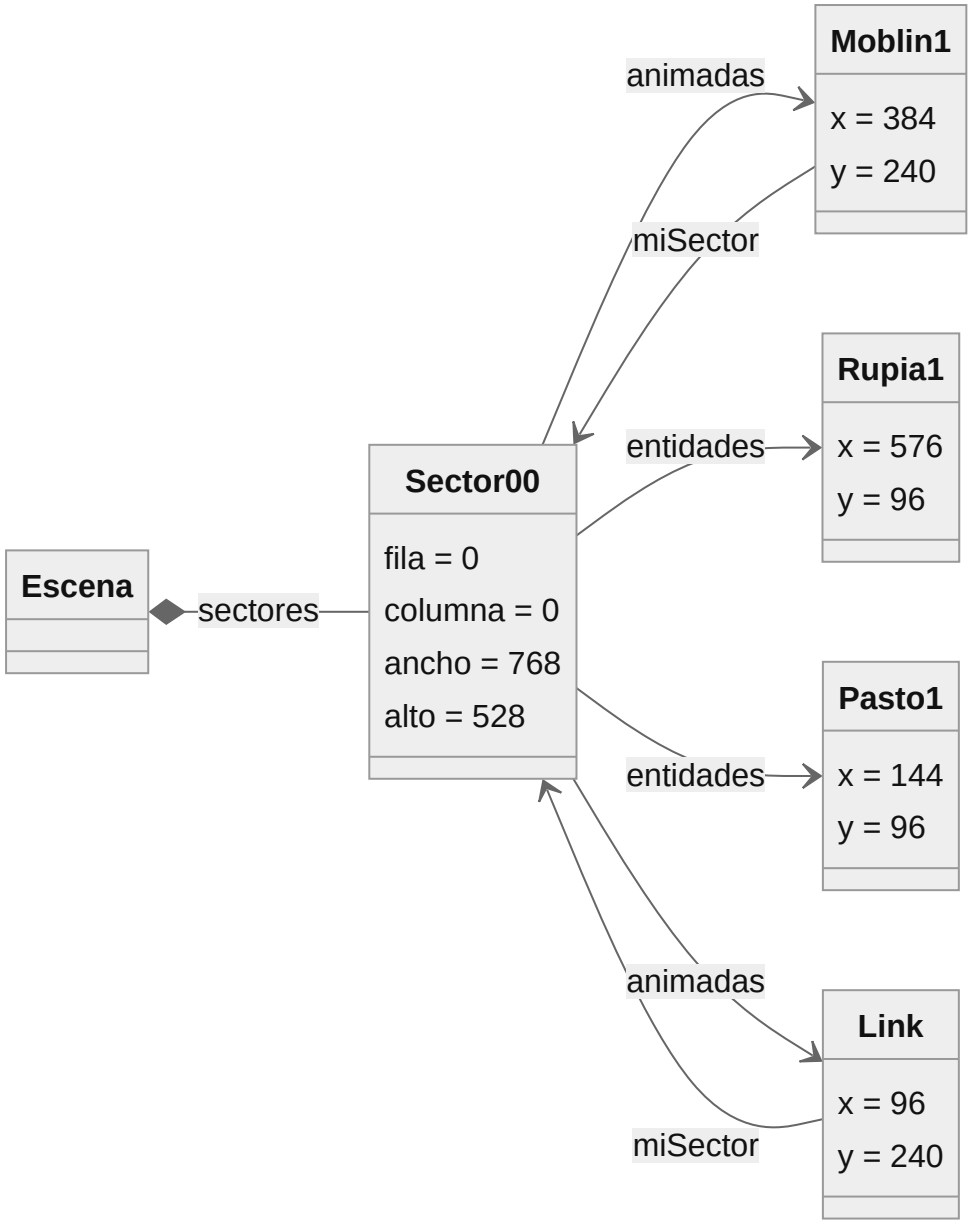


Diagrama de objetos simplificado: cada caja es un objeto (no una clase). La `Rupia` y el `Pasto` no tienen `#miSector` porque no son animados.

Más ejemplos del nivel 1:

Dónde	Línea	Qué crea
Sector (0,1), fila 2	`R...AAAA....E..R`	4 `Agua` en x = 192 a 336 (el lago) y un `ErizoDeMar` en (576, 96)
Sector (0,1), fila 8	`R......X...PP..R`	La **`Espada`** en (336, 384): el enunciado pide que Link la consiga en el nivel 1
Sector (0,2), fila 5	`.....QQQQ.....R`	4 `SuelosQuebradizo` en x = 240 a 384: un puente; la `R` final es el borde derecho del mapa (sin paso)
Sector (0,2), fila 10	`RRRRRRR..RRRRRRR`	Rocas con un **paso hacia abajo** en las columnas 7 y 8, que coincide con la fila 0 del sector (1,2)
Sector (1,2), fila 2	`R.T.........T.R`	Dos `TrampaDeCuchillas` en (96, 48) y (624, 48); cada una guarda su `#posicionOriginal`

Dónde	Línea	Qué crea
Sector (1,0), fila 5	`R.......S.....`	**`MasterStalfos`** en (384, 240): además de agregarse al sector, entra a la lista `jefes` del `Nivel`

7.6 Ejemplos del nivel 2

El nivel 2 (`Defensa 3/niveles/nivel2.txt`) empieza con:

```
MAPA 2 4
CELDAS 16 11 48
INICIO 0 0 2 5
```

Línea	Qué cambia respecto del nivel 1
`MAPA 2 4`	`new Escena(2, 4)`: 8 sectores. `Escena ◆ Sector` termina con 8 partes.
`CELDAS 16 11 48`	Igual: los sectores siguen midiendo 768 × 528.
`INICIO 0 0 2 5`	Igual: Link empieza en el sector (0,0), en (96, 240).

Fragmentos con algo nuevo:

```
SECTOR 0 1
RRRRRRRRRRRRRRR
R.T............R
R.....AA.......R
R.....AA...W...R
...............
.......N........
```

- `W` en la fila 3, columna 11: `crearWizzrobe(528, 144)`. El constructor del `Wizzrobe` le pone `MovimientoTeletransporte` y `AtaquePorProyectil(this, ...)` (las estrategias se asignan al crear, no al parsear).
- `N` en la fila 5, columna 7: `crearMoblinConEspadaYEscudo(336, 240)`, con `DefensaPorEscudo(true)`.
- Las filas 4 y 5 empiezan con `.`: hay pasos a la izquierda (al sector (0,0)) y a la derecha (al (0,2)).

```
SECTOR 0 2
...
......VVVV......
......QQQQ......
......VVVV......
```

- Un abismo de `CaidasAlVacio` con un único puente de `SuelosQuebradizo` (`crearSuelosQuebradizo(288, 240)` a `(432, 240)`). Cada `Q` es un objeto propio: cuando Link pisa uno, ese objeto se reemplaza en el sector por una `CaidasAlVacio` (`Sector.reemplazar(vieja, nueva)`); los demás siguen intactos.

```
SECTOR 1 0
...
R.......H.......
```

- `H` en (384, 240): `crearStoneHinox(384, 240)`; va al sector (1,0) y a la lista `jefes`. Cuando StoneHinox muere, `Juego.jefeDerrotado(j)` pregunta `nivel.jefesDerrotados()`; como es el

último nivel, termina la partida.

Cuántos objetos se crean: el nivel 1 crea 364 entidades desde el archivo más 4 fragmentos de corazón (368); el nivel 2, 507 más 4 (511). En los dos, 1 jefe. De todas esas entidades, solo las del **sector activo** tienen celda en pantalla e hilo corriendo.

7.7 Lo que no está en el archivo y quién lo crea

Elemento	Quién lo crea	Cuándo	Relación que usa
`Link`	`Juego`, con `fabricaEntidades.crearLink(x, y)`	Una sola vez, en `iniciar(...)`	`Juego → Link (#jugador 1)`
`Escudo`	`Juego` (o la fábrica al crear a Link)	Al crear a Link: empieza con él	`Link ◇ Objeto* (#inventario 1..2)`
`FragmentoDeCorazon` (4 por nivel)	`GeneradorNivel.colocarFragmentosDeCorazon(escena, fabrica)`	Al terminar de leer el archivo	`Sector → Entidad*` en 4 sectores distintos
`Bellota`	`Juego.aparecerBellota(x, y)`	Racha de 4 enemigos sin recibir daño	`Juego → FabricaEntidades`
Rupias y corazones que sueltan los enemigos	`EnemigoConDrop*.generarDrop()`	Al morir el enemigo	`GeneradorDrop «crea» Rupia / Corazon`
Proyectiles y rocas	`Octorok`, `Wizzrobe`, `StoneHinox` (`generarProyectil()`)	Durante la partida	`GeneradorProyectil «crea» Proyectil`
`CaidasAlVacio` que deja un suelo roto	`SuelosQuebradizo.afectar(l)`	Al pisarlo	`Sector.reemplazar(vieja, nueva)`

Fragmentos de corazón, en detalle: el enunciado pide exactamente 4 por nivel, en lugares aleatorios y como máximo 1 por escena (cada sector es una pantalla). `colocarFragmentosDeCorazon` elige 4 sectores distintos al azar con `escena.getSector(f, c)`, le pide a cada uno `posicionLibreAlAzar()` y agrega `fabrica.crearFragmentoDeCorazon(x, y)`. Los niveles tienen 6 y 8 sectores, así que siempre hay lugar.

7.8 Código del parseo (esquema)

```
        public class GeneradorNivel {
            protected String carpetaNiveles;                          // por ejemplo "/niveles/"
            public GeneradorNivel(String carpetaNiveles) { this.carpetaNiveles = carpetaNiveles; }

            public Nivel generar(int numero, FabricaEntidades fabrica) {
                List<String> lineas = leerArchivo(numero);            // líneas no vacías de nivel<numero>.txt
                String[] mapa = lineas.get(0).split(" ");             // MAPA 2 3
                String[] celdas = lineas.get(1).split(" ");           // CELDAS 16 11 48
                String[] inicio = lineas.get(2).split(" ");           // INICIO 0 0 2 5
                int ancho = Integer.parseInt(celdas[1]), alto = Integer.parseInt(celdas[2]), tam = Integer.parseInt(celdas[3]);
```

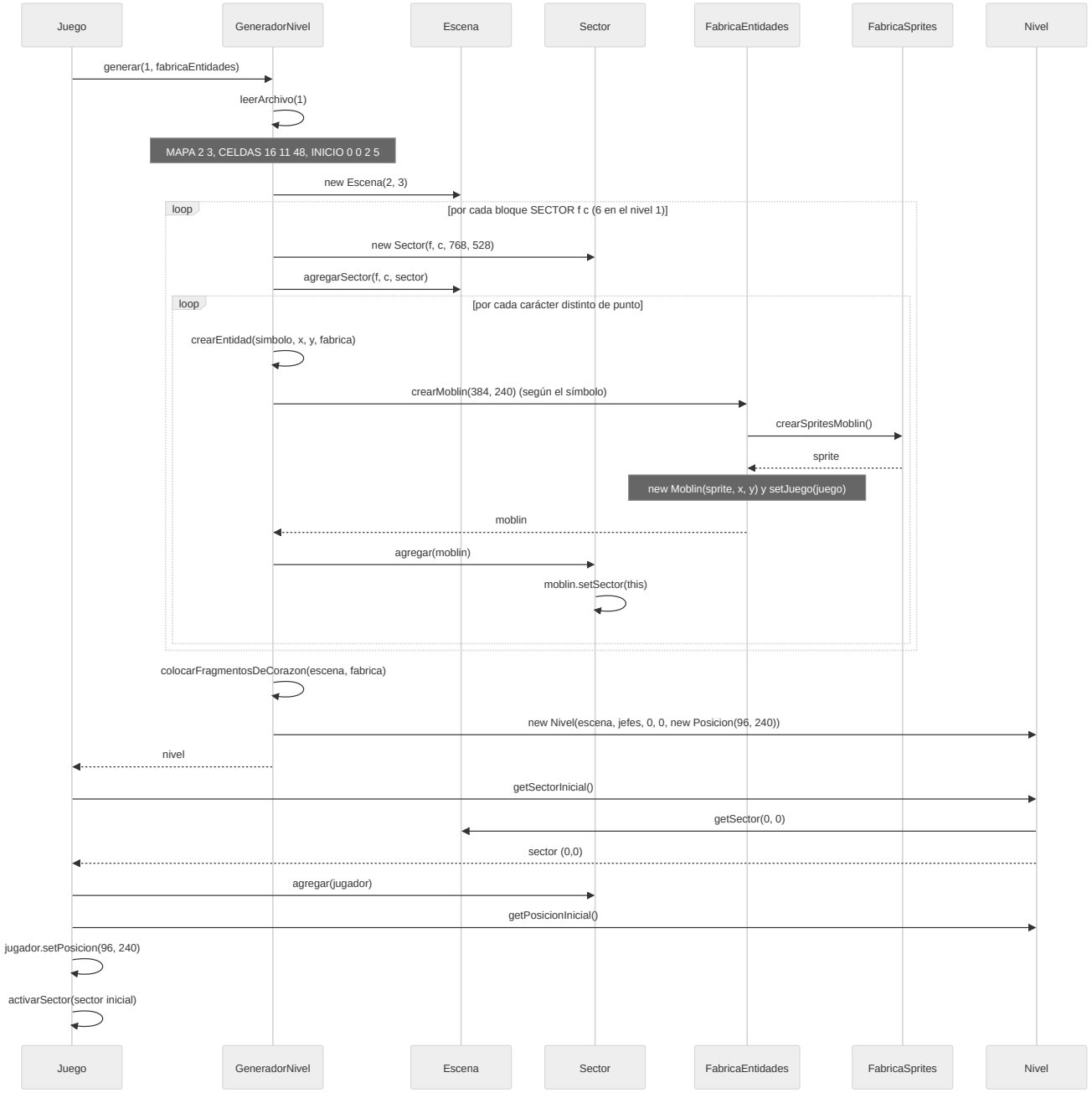
```
                Escena escena = new Escena(Integer.parseInt(mapa[1]),
Integer.parseInt(mapa[2]));
                List<Jefe> jefes = new ArrayList<>();
                int i = 3;
                while (i < lineas.size()) {                              // un bloque por sector
                    String[] cab = lineas.get(i).split(" ");             // SECTOR f c
                    int f = Integer.parseInt(cab[1]), c = Integer.parseInt(cab[2]);
                    Sector sector = new Sector(f, c, ancho * tam, alto * tam);
                    escena.agregarSector(f, c, sector);                  // Escena ◆ Sector
                    for (int r = 0; r < alto; r++) {
                        String fila = lineas.get(i + 1 + r);
                        for (int k = 0; k < ancho; k++) {
                            char s = fila.charAt(k);
                            if (s == 'S' || s == 'H') {                  // jefes: van al sector y
a la lista
                                Jefe j = (s == 'S') ? fabrica.crearMasterStalfos(k * tam, r *
tam)
                                                    : fabrica.crearStoneHinox(k * tam, r *
tam);
                                jefes.add(j);                            // Nivel -> Jefe*
(#jefes)
                                sector.agregar(j);
                            } else {
                                Entidad e = crearEntidad(s, k * tam, r * tam, fabrica);
                                if (e != null) sector.agregar(e);        // Sector -> Entidad*, y
#miSector si es animada
                            }
                        }
                    }
                    i += 1 + alto;
                }
                colocarFragmentosDeCorazon(escena, fabrica);             // 4 por nivel, máx. 1
por sector
                Posicion pos = new Posicion(Integer.parseInt(inicio[3]) * tam,
Integer.parseInt(inicio[4]) * tam);
                return new Nivel(escena, jefes, Integer.parseInt(inicio[1]),
Integer.parseInt(inicio[2]), pos);
            }

            private Entidad crearEntidad(char simbolo, int x, int y, FabricaEntidades fabrica)
{
                switch (simbolo) {                                       // texto -> objeto (no es
instanceof)
                    case 'R': return fabrica.crearRoca(x, y);
                    case 'A': return fabrica.crearAgua(x, y);
                    case 'P': return fabrica.crearPasto(x, y);
                    case 'V': return fabrica.crearCaidasAlVacio(x, y);
                    case 'Q': return fabrica.crearSuelosQuebradizo(x, y);
                    case 'M': return fabrica.crearMoblin(x, y);
                    case 'N': return fabrica.crearMoblinConEspadaYEscudo(x, y);
```

```
                    case 'E': return fabrica.crearErizoDeMar(x, y);
                    case 'O': return fabrica.crearOctorok(x, y);
                    case 'T': return fabrica.crearTrampaDeCuchillas(x, y);
                    case 'W': return fabrica.crearWizzrobe(x, y);
                    case '$': return fabrica.crearRupia(x, y);
                    case 'C': return fabrica.crearCorazon(x, y);
                    case 'X': return fabrica.crearEspada(x, y);
                    case '.': return null;                          // vacío o paso
                    default:  throw new IllegalArgumentException("Símbolo desconocido: " +
simbolo);
                }
            }
        }
```

> **Los jefes y `crearEntidad`.** En el extendido, `crearEntidad(...)` devuelve `Entidad`. Para guardar al jefe en la lista de tipo `List<Jefe>` sin `instanceof` ni *cast*, el esquema trata `S` y `H` antes de llamar a `crearEntidad` (allí se sabe que es un jefe porque se leyó la letra). Otra opción es pasarle la lista de jefes a `crearEntidad` como parámetro. Ver Observaciones.

7.9 Diagrama de secuencia de la carga



Para explicar: `Juego` le pasa al generador la fábrica del dominio elegido; el generador lee la cabecera, arma la escena, recorre cada bloque `SECTOR` creando un `Sector` y, por cada carácter, le pide la entidad a la fábrica (que a su vez le pide el `Sprite` a la `FabricaSprites*` del dominio). Al final ubica los fragmentos de corazón y devuelve el `Nivel`. `Juego` agrega a Link en el sector inicial y activa ese sector, que es cuando la vista crea las celdas.

7.10 El objeto que queda armado

Después de `generar(1, fabrica)` y de `cargarNivel()`, el grafo de objetos queda así (simplificado):

```
Juego
 ├─◇ #nivel ──────────> Nivel (filaInicial 0, columnaInicial 0)
 │                      ├─◆ #mapa ──────────> Escena (2 x 3)
 │                      │                     ├─◆ #sectores -> Sector (0,0) -#entidades-> 62
entidades + Link
 │                      │                     ├─◆ #sectores -> Sector (0,1) -#entidades-> 60
entidades (con la Espada)
```

```
 │                        │                        ├─  ...
 │                        │                        └─◆ #sectores -> Sector (1,0) -#entidades-> 63 rocas + MasterStalfos
 │                        ├──> #jefes ──────────> MasterStalfos (el mismo objeto que está en el Sector (1,0))
 │                        └──> #posicionInicial > Posicion (96, 240)
 ├──> #jugador ──────────> Link (el mismo objeto que está en el Sector (0,0))
 ├──> #generadorNivel ──> GeneradorNivel (no guardó nada del nivel)
 └──> #fabricaEntidades > FabricaEntidades ──#fabricaSprites──> FabricaSpritesOriginal (o Alternativa)
```

Cada entidad apunta además a su `Sprite` (`#sprite`), su `Posicion` (`#posicion`) y a `Juego` (`#miJuego`); las animadas, a su sector (`#miSector`) y a sus tres estrategias.

7.11 Errores del archivo y cómo se detectan

El archivo lo escribe el equipo, así que un error es un error de programación: conviene que el generador falle con un mensaje claro en lugar de armar un nivel roto.

Error	Ejemplo	Qué pasa en el parseo	Cómo se detecta
Fila con menos caracteres	`R....R` con `CELDAS 16` `...`	`charAt(k)` fuera de rango	Validar `fila.length() == ancho` y lanzar una excepción con el número de línea
Símbolo desconocido	`Z`	`crearEntidad` llega al `default`	`IllegalArgumentException("Símbolo desconocido: Z")`
Falta un bloque `SECTOR`	5 bloques con `MAPA 2 3`	Queda un `null` en la grilla de la `Escena`	Al final, verificar que `getSector(f, c)` no sea `null` para toda la grilla
Pasos que no coinciden	`.` a la derecha del (0,0) y `R` a la izquierda del (0,1)	Link pasa al vecino y aparece dentro de una roca	Validar los bordes al cargar (regla 5 del formato)
Ningún jefe	Sin `S` ni `H`	La lista `jefes` queda vacía y `#jefes 1..*` no se cumple	Verificar `!jefes.isEmpty()` antes de crear el `Nivel`
`INICIO` sobre una roca	`INICIO 0 0 0 0`	Link empieza encerrado	Verificar que la celda de inicio sea `.`

Las reglas completas de un archivo válido están en `Defensa 3/Formateo-de-niveles.md` (sección 4) y los errores comunes con ejemplos en `Defensa 3/Explicacion-formateo-de-niveles.md` (sección 7).

---

8. Fábricas y sprites (patrón Abstract Factory)

Cómo se crean las entidades y con qué imágenes: `FabricaEntidades` arma cada entidad y le pide su `Sprite` a la `FabricaSprites*` del dominio gráfico elegido.

`FabricaEntidades`

FabricaEntidades

```
# fabricaSprites : FabricaSprites
# juego : Juego
```

«constructores»

```
+ FabricaEntidades(juego : Juego, fabricaSprites : FabricaSprites)
```

«consultas»

```
+ crearLink(x : int, y : int) : Link
+ crearMoblin(x : int, y : int) : Moblin
+ crearMoblinConEspadaYEscudo(x : int, y : int) : MoblinConEspadaYEscudo
+ crearErizoDeMar(x : int, y : int) : ErizoDeMar
+ crearOctorok(x : int, y : int) : Octorok
+ crearTrampaDeCuchillas(x : int, y : int) : TrampaDeCuchillas
+ crearWizzrobe(x : int, y : int) : Wizzrobe
+ crearMasterStalfos(x : int, y : int) : MasterStalfos
+ crearStoneHinox(x : int, y : int) : StoneHinox
+ crearRupia(x : int, y : int) : Rupia
+ crearBellota(x : int, y : int) : Bellota
+ crearCorazon(x : int, y : int) : Corazon
+ crearFragmentoDeCorazon(x : int, y : int) : FragmentoDeCorazon
+ crearEspada(x : int, y : int) : Espada
+ crearEscudo(x : int, y : int) : Escudo
+ crearRoca(x : int, y : int) : Roca
+ crearAgua(x : int, y : int) : Agua
+ crearPasto(x : int, y : int) : Pasto
+ crearCaidasAlVacio(x : int, y : int) : CaidasAlVacio
+ crearSuelosQuebradizo(x : int, y : int) : SuelosQuebradizo
+ crearProyectil(x : int, y : int, d : Direccion) : Proyectil
+ crearRocaGrande(x : int, y : int, d : Direccion) : Proyectil
```

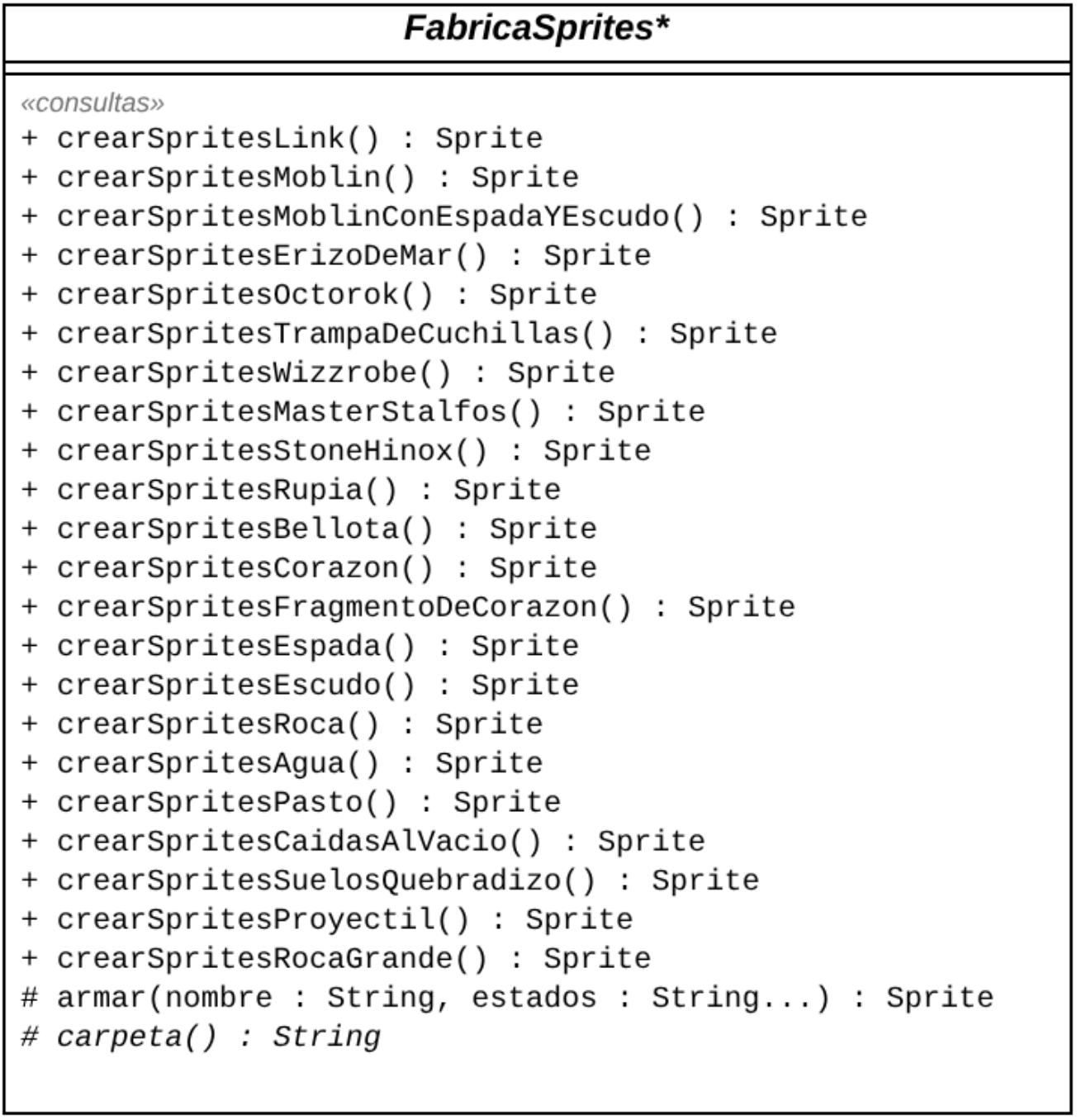
- **Utilidad:** crea todas las entidades del juego.
- **Comportamiento (según el extendido):** `FabricaEntidades(juego, fabricaSprites)` y un método `crearX(x, y)` por cada entidad (`crearLink`, `crearMoblin`, …, `crearSuelosQuebradizo`), más `crearProyectil(x, y, d)` y `crearRocaGrande(x, y, d)`. Al crear cada entidad le asigna el `Sprite` correcto, pidiéndolo a la `FabricaSprites` del dominio elegido, y le asigna `#miJuego` con `setJuego(juego)`. Así ninguna entidad sabe en qué dominio gráfico está.
- **Relaciones:** la referencia `Juego` (`#fabricaEntidades 1`); referencia a `FabricaSprites*` (→, `#fabricaSprites 1`). `GeneradorNivel` la recibe como parámetro desde `Juego` (dependencia no dibujada).
- **Por qué guarda a `Juego`:** cada entidad nueva necesita `#miJuego` para avisar sucesos de partida. La fábrica es el único lugar por donde pasan **todas** las creaciones (las del archivo de nivel y las de plena partida: drops, proyectiles, bellota), así que es el lugar natural para asignarlo. En el extendido de la defensa 4 está el atributo `# juego : Juego`, pero la flecha `FabricaEntidades → Juego` (`#juego 1`) **sigue faltando en el reducido**: conviene agregarla (ver Observaciones).

```
        public class FabricaEntidades {
            protected FabricaSprites fabricaSprites;   // #fabricaSprites 1
            protected Juego juego;                     // en el extendido; falta la flecha en
el reducido
```

```
            public FabricaEntidades(Juego juego, FabricaSprites fabricaSprites) { this.juego =
juego; this.fabricaSprites = fabricaSprites; }

            public Moblin crearMoblin(int x, int y) {
                Moblin m = new Moblin(fabricaSprites.crearSpritesMoblin(), x, y);   //
estrategias en su constructor
                m.setJuego(juego);                                                  // #miJuego
                return m;
            }
            public Proyectil crearProyectil(int x, int y, Direccion d) {
                Proyectil p = new Proyectil(fabricaSprites.crearSpritesProyectil(), x, y, d,
1);   // ½ corazón
                p.setJuego(juego);
                return p;
            }
            public Proyectil crearRocaGrande(int x, int y, Direccion d) {
                Proyectil p = new Proyectil(fabricaSprites.crearSpritesRocaGrande(), x, y, d,
2);  // 1 corazón
                p.setJuego(juego);
                return p;
            }
            // ... un método por cada entidad
        }
```

FabricaSprites*



- **Utilidad:** fábrica abstracta de gráficos (patrón **Abstract Factory**).
- **Comportamiento (según el extendido):** un método `crearSpritesX() : Sprite` por cada tipo de entidad, más `# armar(nombre, estados...) : Sprite` y `# carpeta() : String`. Lo único que redefine cada dominio es `carpeta()`.
- **Relaciones:** la referencia `FabricaEntidades`; «crea» `Sprite` (dependencia no dibujada: el equipo decidió no dibujar las dependencias en el reducido).
- **Subclases:**
  - `FabricaSpritesOriginal`: imágenes del dominio original (estilo Zelda).
  - `FabricaSpritesAlternativa`: imágenes del segundo dominio.

Elección del dominio gráfico

El dominio se elige **en la vista** (menú principal): `elegirDominio(d)` guarda la `FabricaSprites` concreta y `jugar()` la pasa por `ControladorDeVistas.iniciarPartida(modo, dominio)`; `GUI` se la pasa

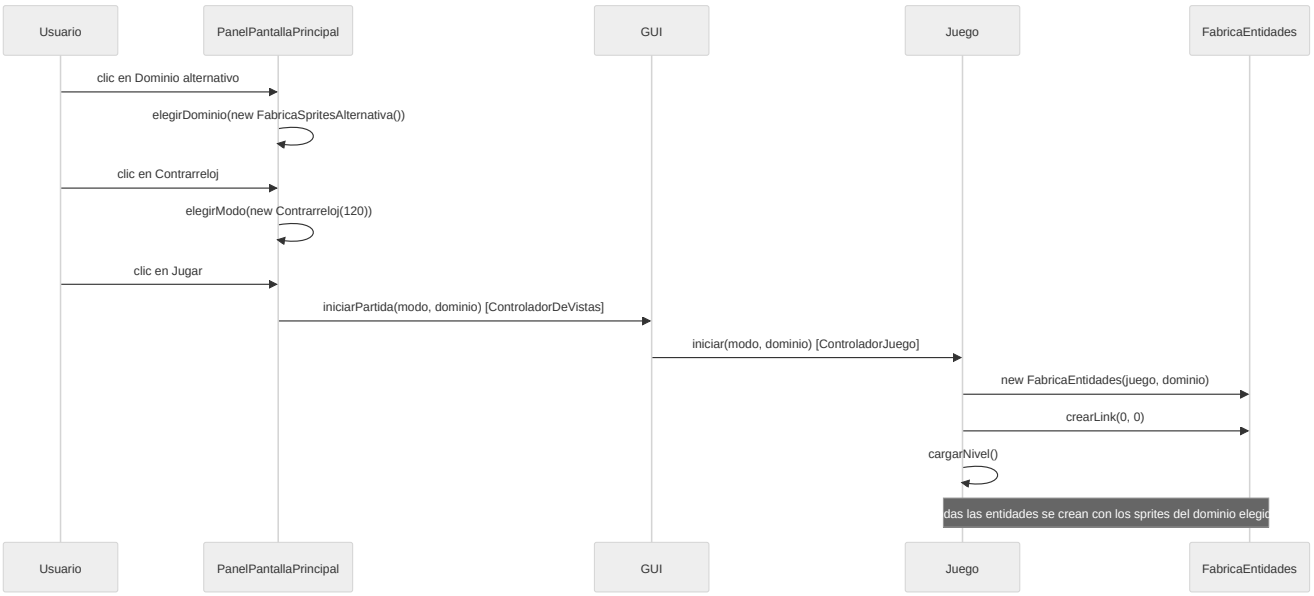
a `Juego` en `iniciar(modo, dominio)`. `Juego` arma la `FabricaEntidades` con esa fábrica, así todas las entidades salen con los sprites del dominio elegido. **No hay ningún `if` ni `switch` sobre el dominio en la lógica**, y las entidades nunca saben en qué dominio están. En el diagrama no hace falta ninguna relación nueva.

`Sprite` guarda **rutas de imágenes** (texto), no imágenes de Swing: la lógica no depende de la librería gráfica. La `CeldaGrafica` es la que carga la imagen a partir de la ruta.

```
        public abstract class FabricaSprites {
            protected abstract String carpeta();                        // lo único que cambia entre dominios
            public Sprite crearSpritesMoblin() { return armar("moblin", "quieto", "caminando"); }
            // ... uno por cada entidad
            protected Sprite armar(String nombre, String... estados) {
                Sprite s = new Sprite();
                for (String e : estados) s.agregar(e, carpeta() + "/" + nombre + "_" + e + ".png");
                return s;
            }
        }
        public class FabricaSpritesOriginal extends FabricaSprites    { protected String carpeta() { return "/imagenes/original"; } }
        public class FabricaSpritesAlternativa extends FabricaSprites { protected String carpeta() { return "/imagenes/alternativa"; } }

        // Vista: PanelPantallaPrincipal
        public void elegirDominio(FabricaSprites d) { dominio = d; }      // botón "Original" o "Alternativo"
        public void elegirModo(ModoJuego m)         { modo = m; }         // botón Clásico, Contrarreloj o NoHit
        public void jugar()                         { controladorVistas.iniciarPartida(modo, dominio); }
        // GUI
        public void iniciarPartida(ModoJuego modo, FabricaSprites dominio) {
            ventana.mostrarPanel(panelNivel);
            controladorJuego.iniciar(modo, dominio);
        }

        // Lógica: Juego
        public void iniciar(ModoJuego modo, FabricaSprites dominio) {
            modoJuego = modo;
            fabricaEntidades = new FabricaEntidades(this, dominio);   // sin if ni switch sobre el dominio
            jugador = fabricaEntidades.crearLink(0, 0);               // Link se crea una sola vez por partida
            numeroNivel = 1;
            cargarNivel();
            reloj.iniciar(); hiloJugador.iniciar(); hiloMundo.iniciar();   // defensa 5: los tres hilos
        }
```



Explicación: el usuario elige el dominio y el modo en el menú. Al jugar, la vista le pasa a la lógica la fábrica ya elegida; desde ahí, cada entidad que crea la `FabricaEntidades` (al cargar el nivel o en plena partida) recibe los sprites de ese dominio.

Alternativa descartada: llevar todo el dominio a la vista (la entidad solo informa su estado y la celda busca la imagen). Es la separación más pura, pero cambia lo decidido en el contexto (§4.5) y mueve el Abstract Factory.

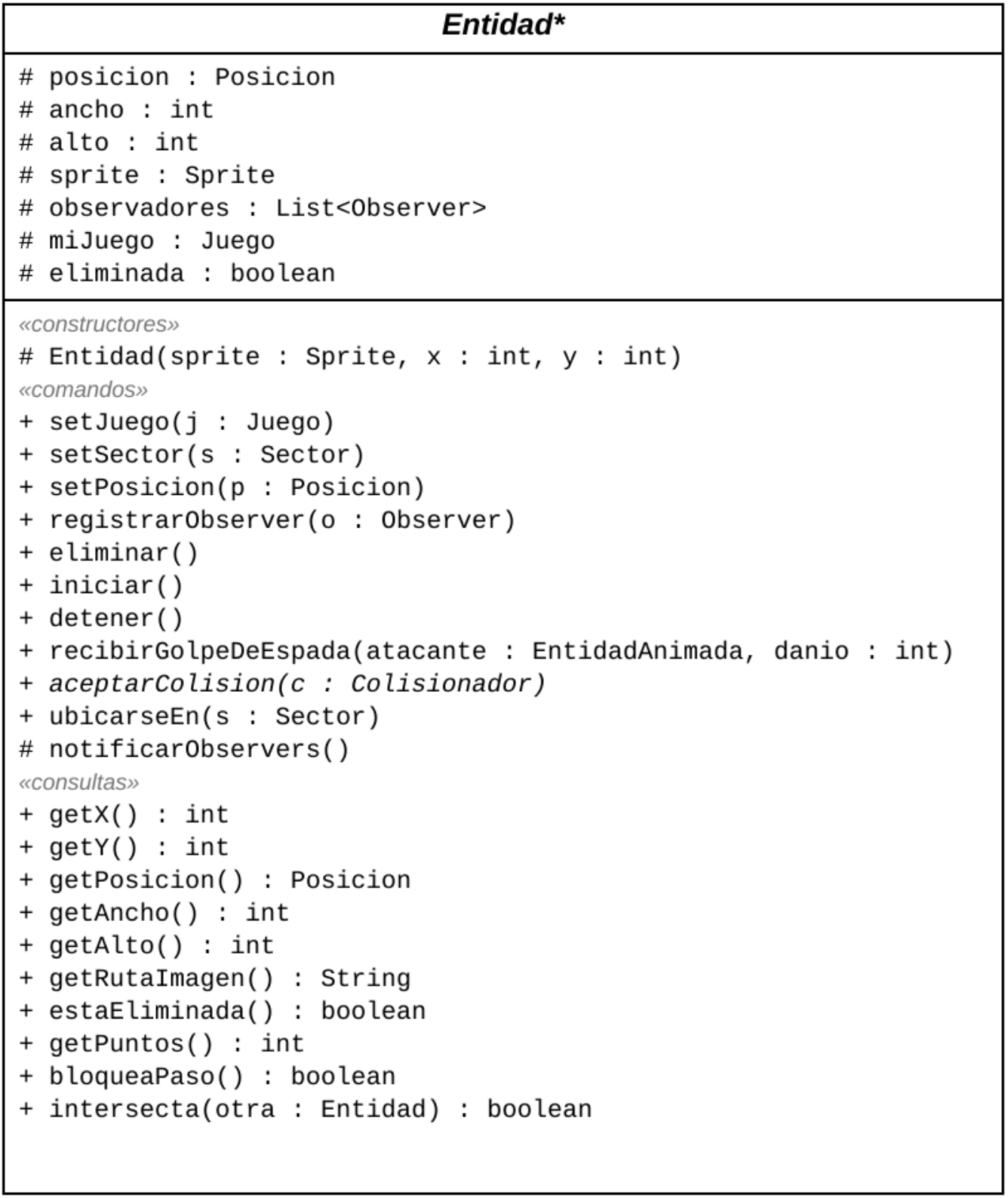
`Sprite`

- **Utilidad:** conjunto de imágenes de una entidad (una ruta por estado/dirección/animación). Es **un** objeto por entidad que guarda varias rutas; por eso el nombre en singular.
- **Comportamiento (según el extendido):** atributos `rutas : Map<String, String>` y `estadoActual : String`; `agregar(estado, ruta)`, `setEstado(estado)`, `getRutaActual()`, `getRuta(estado)`, `tieneEstado(estado)`.
- La entidad cambia el estado de su sprite (por ejemplo, `"caminando"` o `"atacando"`) y la celda gráfica muestra `getRutaImagen()`, que la entidad toma de `sprite.getRutaActual()`. Es lo **único** que cambia entre dominios: el comportamiento es idéntico.
- **Relaciones:** lo crea `FabricaSprites*`; lo referencia `Entidad*` (`#sprite 1`).

---

9. Entidades

`Entidad*`



- **Utilidad:** raíz de todo lo que aparece en un sector.
- **Comportamiento (según el extendido):** `Entidad(sprite, x, y)`; comandos `setJuego(j)`, `setSector(s)` (no hace nada; lo redefine `EntidadAnimada*`), `setPosicion(p)`, `registrarObserver(o)`, `eliminar()`, `iniciar()` y `detener()` (no hacen nada en las pasivas), `recibirGolpeDeEspada(atacante, danio)` (por defecto no pasa nada), `aceptarColision(c)` y `# notificarObservers()`; consultas `getX()`, `getY()`, `getPosicion()`, `getRutaImagen()`, `estaEliminada()`, `getPuntos()` (0 por defecto), `bloqueaPaso()` (false por defecto) e `intersecta(otra)`.

- **Posición y tamaño:** una `Posicion` (`#posicion 1`, en coordenadas **del sector** donde está) y los atributos `ancho`, `alto`. Juntos forman el rectángulo de la entidad (*hitbox*): `intersecta(otra)` compara esos rectángulos y el `Sector` aplica el doble despacho solo entre las que se tocan. La `CeldaGrafica` lee la posición por `EntidadLogica` (`getX()`, `getY()`) para ubicarse en pantalla; nunca la modifica. No se usan `java.awt.Point` ni `java.awt.Rectangle` para que la lógica no dependa de la librería gráfica.
- **Relaciones:** realiza `Colisionable` y `EntidadLogica`; referencia su `Posicion` (`#posicion 1`), su `Sprite` (`#sprite 1`) y sus observers (→ `Observer`, `#observadores 0..*`); la referencia el `Sector` donde está (`#entidades`), pero `Entidad*` no conoce a su sector (eso lo tiene solo `EntidadAnimada*`, con `#miSector`); referencia a `Juego` (→, `#miJuego 1`) para avisarle sucesos (ver Comunicación entre `Juego`, `Sector` y las entidades).
- **Subclases directas:** `Obstaculo*`, `PowerUP*`, `Objeto*`, `EntidadAnimada*`.

```
        public abstract class Entidad implements Colisionable, EntidadLogica {
            protected Posicion posicion;  protected int ancho = 48, alto = 48;
            protected Sprite sprite;                                    // #sprite 1 (en el extendido: "# sprite : Sprite")
            protected List<Observer> observadores = new ArrayList<>();
            protected Juego miJuego;                                    // #miJuego 1
            protected boolean eliminada;

            protected Entidad(Sprite sprite, int x, int y) { this.sprite = sprite; posicion = new Posicion(x, y); }
            public boolean intersecta(Entidad o) {
                return getX() < o.getX() + o.ancho && o.getX() < getX() + ancho
                    && getY() < o.getY() + o.alto  && o.getY() < getY() + alto;
            }
            public String getRutaImagen() { return sprite.getRutaActual(); }
            public void eliminar() { eliminada = true; detener(); notificarObservers(); }
            protected void notificarObservers() { for (Observer o : observadores) o.actualizar(); }
        }
```

`<<Interface>> Colisionable`

- **Utilidad:** "algo con lo que se puede chocar".
- **Comportamiento:** recibe un `Colisionador` y le avisa qué tipo concreto es, llamando al método específico (`c.colisionarCon(this)`). Esto es el **doble despacho** que reemplaza a `instanceof`. En las clases del extendido el método es `aceptarColision(c : Colisionador)` (lo tienen `Entidad*`, `Obstaculo*`, `PowerUP*`, `Objeto*`, `Enemigo*`, `Proyectil` y `Link`), y la interfaz lo declara en el extendido de la defensa 4 (observación 7 resuelta).
- **Relaciones:** la realiza `Entidad*`.

`<<Interface>> Colisionador`

- **Utilidad:** "algo que se mueve y choca".
- **Comportamiento:** tiene un método por cada tipo de `Colisionable`: en `EntidadAnimada*` están `colisionarCon(l : Link)`, `colisionarCon(e : Enemigo)`, `colisionarCon(p : Proyectil)`, `colisionarCon(o : Obstaculo)`, `colisionarCon(p : PowerUP)` y `colisionarCon(o : Objeto)`, y cada entidad animada redefine los casos que le importan (Link los de enemigo, proyectil, obstáculo, power-up y objeto; el `Proyectil` los de Link y obstáculo; la `TrampaDeCuchillas` el de obstáculo). En el extendido la interfaz declara los seis.

- **Relaciones:** la realiza `EntidadAnimada*`.

Cómo funciona el doble despacho (ejemplo con Link y un Moblin):

```
        // Sector: Link se movió y se revisa contra lo fijo (matriz) y contra las demás
animadas
        public void verificarColisiones(EntidadAnimada quien) {
            for (Entidad[] fila : entidades)                        // en la práctica, solo las
celdas que toca quien
                for (Entidad e : fila)
                    if (e != null && e.intersecta(quien)) e.aceptarColision(quien);   // 1er
despacho: por el tipo de e
            for (EntidadAnimada a : animadas)
                if (a != quien && a.intersecta(quien)) a.aceptarColision(quien);
        }
        // Enemigo
        public void aceptarColision(Colisionador c) { c.colisionarCon(this); }      // 2do
despacho: this es Enemigo
        // Link
        public void colisionarCon(Enemigo e) { recibirDanio(e.getDanioContacto(),
e.getDireccion()); }
```

Ninguna de las dos clases pregunta el tipo de la otra: Java elige `aceptarColision` según la clase de `e` y `colisionarCon(Enemigo)` según el tipo estático de `this` dentro de `Enemigo`.

Obstáculos

`Obstaculo*`

- **Utilidad:** elementos fijos del mapa que afectan el paso de Link.
- **Comportamiento (según el extendido):** `aceptarColision(c)` (llama a `c.colisionarCon(this)`) y `afectar(l : Link)`: el efecto de cada obstáculo **sobre Link**. Link, al chocar, llama a `o.afectar(this)`.
- **Relaciones:** hereda de `Entidad*`.

Clase	Comportamiento	Métodos del extendido
`Roca`	Bloquea el paso.	`bloqueaPaso()` da `true`; `afectar(l)` no hace nada.
`Agua`	Bloquea el paso (funcionalmente igual a `Roca`).	`bloqueaPaso()` da `true`.
`Pasto`	Reduce la velocidad de Link y cambia su animación. Se destruye con un golpe de espada.	`afectar(l)` llama a `l.ralentizar()`; `recibirGolpeDeEspada(atacante, danio)` hace que el atacante lo saque del sector.
`CaidasAlVacio`	Link cae: se reinicia el nivel (ambigüedad del enunciado, ver contexto §7).	`afectar(l)` llama a `l.caerAlVacio()`, y Link avisa a `Juego.jugadorCayoAlVacio()`.
`SuelosQuebradizo`	Se rompe al pisarlo y queda como `CaidasAlVacio`.	`afectar(l)` marca `pisado` y llama a `l.romperSuelo(this)`, y Link pide al sector reemplazarlo por una `CaidasAlVacio`; `estaPisado()`.

Cómo bloquea una roca: antes de avanzar, la estrategia de movimiento pregunta `sector.puedeMoverA(e, d, pasos)`; el sector revisa si en las celdas de la posición siguiente hay alguna entidad con `bloqueaPaso()`. Así la roca no necesita "empujar" a nadie: simplemente no se puede entrar.

Power-ups

`PowerUP*`

- **Utilidad:** objetos que Link recoge al tocarlos y le dan un beneficio. Suman +300 puntos (salvo la rupia).
- **Comportamiento (según el extendido):** `aceptarColision(c)`, `serRecogidoPor(l : Link)` (el efecto se aplica **sobre Link**) y `getPuntos()`.
- **Relaciones:** hereda de `Entidad*`. Cada power-up llega al mapa de una forma distinta:

Power-up	Quién lo crea	Cuándo
`Rupia`	`GeneradorDrop` (dependencia «crea») o `GeneradorNivel` (`$`)	Al morir un enemigo, si sale en la probabilidad; o fija en el mapa
`Corazon`	`GeneradorDrop` (dependencia «crea») o `GeneradorNivel` (`C`)	Al morir un enemigo, si sale en la probabilidad; o fijo en el mapa
`Bellota`	`Juego` (`aparecerBellota(x, y)`)	Cuando Link mata 4 enemigos seguidos sin recibir daño
`FragmentoDeCorazon`	`GeneradorNivel` (`colocarFragmentosDeCorazon`)	Al construir el nivel: exactamente 4, en lugares aleatorios, máx. 1 por sector

Clase	Comportamiento	Qué hace `serRecogidoPor(l)`
`Rupia`	Suma entre 1 y 5 rupias (aleatorio, `cantidad`); +10 puntos por rupia.	`l.sumarRupias(cantidad)`
`Bellota`	Reduce a ½ corazón el daño de los próximos 3 golpes. Ver Efecto de la bellota.	`l.setDefensaBase(new DefensaReducida())`
`Corazon`	+1 corazón (hit point).	`l.curar(2)` (en medios corazones)
`FragmentoDeCorazon`	Cada 4 recogidos, +1 de vida máxima.	`l.sumarFragmento()`

En todos los casos, después del efecto: `l.powerUpRecogido(this)` (Link avisa a `Juego`, que suma los puntos) y `l.quitarDelSector(this)` (el power-up sale del sector y su celda se borra).

Objetos

`Objeto*`

- **Utilidad:** ítems que Link equipa y usa.
- **Comportamiento (según el extendido):** `aceptarColision(c)`, `serRecogidoPor(l)` (se agrega al inventario), `atacarCon(l)`, `soltarAtaqueCon(l)`, `protegerCon(l)`, `soltarProteccionCon(l)` y `getNombre()` (lo muestra el HUD). Los cuatro "con Link" están en `Objeto*` sin hacer nada; `Espada` redefine los de ataque y `Escudo` los de protección.

- **Relaciones:** hereda de `Entidad*`; agregado en `Link` (◇, `#inventario 1..2`); además `Link` referencia al que tiene equipado (→, `#objetoEquipado 0..1`).
- **Por qué son dos relaciones:** el diagrama válido tenía una sola línea con los dos roles. Son dos cosas distintas:
  - `#inventario 1..2` (agregación): los objetos que Link **tiene**. Empieza solo con el escudo (1) y consigue la espada en el primer nivel (2). Es agregación porque el objeto existe antes de que Link lo tenga (la espada está en el mapa hasta que la recoge).
  - `#objetoEquipado 0..1` (asociación): cuál de esos objetos está **usando**. Puede ser ninguno (0), como pide el enunciado ("debe verse cuál está equipado, o ninguno"). Siempre apunta a uno de los objetos del inventario.

```
        // Al empezar: inventario = [escudo], objetoEquipado = null
        // Recoge la espada:  inventario = [escudo, espada]
        // cambiarObjeto():   objetoEquipado pasa de null -> escudo -> espada -> null -> ...
```

Clase	Comportamiento	Métodos que redefine
`Espada`	Ataca al presionar A. Ataque recargado: mantener 3 s y soltar. Link la consigue en el nivel 1.	`atacarCon(l)` → `l.cargarAtaque()`; `soltarAtaqueCon(l)` → `l.liberarAtaque()`
`Escudo`	Mientras se usa, Link no recibe daño de proyectiles ni de contacto. Link empieza con él.	`protegerCon(l)` → `l.activarEscudo()` (Link pasa a `DefensaPorEscudo(false)`); `soltarProteccionCon(l)` → `l.desactivarEscudo()` (vuelve a la defensa base)

Doble despacho con el objeto equipado (según el extendido): Link no pregunta qué objeto tiene equipado. Cuando el jugador ataca, Link llama a `objetoEquipado.atacarCon(this)`; si es la espada, empieza la carga; si es el escudo, la versión heredada de `Objeto*` no hace nada. Lo mismo con `protegerCon`.

```
        // Link
        public void atacar()       { if (objetoEquipado != null) objetoEquipado.atacarCon(this); }
        public void soltarAtaque() { if (objetoEquipado != null) objetoEquipado.soltarAtaqueCon(this); }
        public void usarEscudo()   { if (objetoEquipado != null) objetoEquipado.protegerCon(this); }
        public void soltarEscudo() { if (objetoEquipado != null) objetoEquipado.soltarProteccionCon(this); }
        // Espada
        public void atacarCon(Link l)       { l.cargarAtaque(); }      // AtaqueRecargado.cargar()
        public void soltarAtaqueCon(Link l) { l.liberarAtaque(); }     // el hilo de Link da el golpe
        // Escudo
        public void protegerCon(Link l)         { l.activarEscudo(); }      // Link pasa a DefensaPorEscudo(false)
        public void soltarProteccionCon(Link l) { l.desactivarEscudo(); }   // vuelve a la defensa base
```

> Esta lectura (A usa el objeto equipado) sale de los métodos del extendido. Si el equipo decide que A es siempre la espada y S siempre el escudo, `Juego` seguiría llamando a `atacar()` y `usarEscudo()` de Link, y Link buscaría el objeto correspondiente en su inventario. Conviene confirmarlo antes de implementar.

`EntidadAnimada*`

*EntidadAnimada**

```
# miSector : Sector
# movimiento : ComportamientoMovimiento
# ataque : ComportamientoAtaque
# defensa : ComportamientoDefensa
# direccion : Direccion
# vida : int
# activa : boolean
```

«constructores»

```
# EntidadAnimada(sprite : Sprite, x : int, y : int, vida : int)
```

«comandos»

```
+ setSector(s : Sector)
+ setMovimiento(m : ComportamientoMovimiento)
+ setAtaque(a : ComportamientoAtaque)
+ setDefensa(d : ComportamientoDefensa)
+ setDireccion(d : Direccion)
+ iniciar()
+ detener()
+ run()
+ avanzar(d : Direccion, pasos : int)
+ avanzarHacia(destino : Posicion, pasos : int)
+ recibirDanio(danio : int, origen : Direccion)
+ aplicarDanio(danio : int)
# morir()
# alSerGolpeado()
+ ataqueRealizado()
+ restaurarDefensa()
+ teletransportarse()
+ ataqueTerminado()
# programar(ms : long, accion : Runnable)
+ agregarAlSector(nueva : Entidad)
+ quitarDelSector(e : Entidad)
+ reemplazarEnSector(vieja : Entidad, nueva : Entidad)
+ ubicarseEn(s : Sector)
+ colisionarCon(l : Link)
+ colisionarCon(e : Enemigo)
+ colisionarCon(p : Proyectil)
+ colisionarCon(o : Obstaculo)
+ colisionarCon(p : PowerUP)
+ colisionarCon(o : Objeto)
```

«consultas»

```
+ getSector() : Sector
+ getDireccion() : Direccion
+ getVida() : int
+ estaActiva() : boolean
+ quiereAtacar() : boolean
+ soltoElAtaque() : boolean
```

```
+ soltoElAtaque() : boolean
+ linkEstaEnfrente() : boolean
+ direccionHaciaJugador() : Direccion
```

- **Utilidad:** toda entidad que se mueve, ataca o puede recibir daño.
- **Comportamiento:** delega su comportamiento en tres estrategias intercambiables en tiempo de ejecución (patrón **Strategy**): `setMovimiento(m)`, `setAtaque(a)`, `setDefensa(d)`. Se mueve con `avanzar(d, pasos)` y `avanzarHacia(destino, pasos)`; recibe daño con `recibirDanio(danio, origen)` (pasa por su defensa) y `aplicarDanio(danio)` (resta vida). Pide cosas al sector con `agregarAlSector(nueva)`, `quitarDelSector(e)` y `reemplazarEnSector(vieja, nueva)`. Consultas: `getSector()`, `getDireccion()`, `getVida()`, `estaActiva()`, `soltoElAtaque()`, `linkEstaEnfrente()` y `direccionHaciaJugador()`. **Defensa 5:** ya no tiene hilo propio. `actualizarEstado()` hace **una** vuelta de su ciclo (moverse, atacar, verificar colisiones y ejecutar las acciones vencidas) y la llama `HiloJugador` (Link) o `HiloMundo` (el resto). `# programar(ms, accion)` guarda una `AccionProgramada` que ese mismo hilo ejecuta cuando vence (en la defensa 4 arrancaba otro hilo): lo usan la trampa (`programar(600, volver)`), el Moblin con escudo (volver a `MovimientoAleatorio` después de abalanzarse), `MasterStalfos` (fin de la invulnerabilidad, 800 ms), `StoneHinox` (`embestidaTerminada` y `siguienteAtaque`) y `Link` (invulnerabilidad de 1 s y `romperSuelo`).
- **Relaciones:** hereda de `Entidad*`; realiza `Colisionador`; agrega `ComportamientoMovimiento` (◇, `#movimiento 1`), `ComportamientoAtaque` (◇, `#ataque 1`) y `ComportamientoDefensa` (◇, `#defensa 1`); compone sus acciones pendientes (◆ `AccionProgramada`, `#pendientes 0..*`); referencia a su `Direccion` (→, `#direccion 1`), hacia dónde mira o se mueve; el `Sector` la referencia en su lista (`#animadas 0..*`); referencia al `Sector` donde está (→, `#miSector 1`) para resolver colisiones y agregar, sacar o reemplazar entidades sin pasar por `Juego`. Está en `EntidadAnimada*` y no en `Entidad*` porque solo las entidades animadas inician acciones (las actualiza un hilo); obstáculos, power-ups y objetos nunca actúan por su cuenta: siempre es otra entidad la que choca con ellos.
- **Por qué agregación (◇) con las estrategias y no composición:** la estrategia se **reemplaza** durante la vida de la entidad (la trampa pasa de quieta a inducida, Link usa el escudo) y algunas se comparten o se guardan para volver a usarlas (`StoneHinox` guarda `embestida`, `lanzarRocas` y `avalancha`; Link guarda `defensaBase`).
- **Subclases directas:** `Link`, `Proyectil`, `Enemigo*`.

> **Ganchos en el extendido de la defensa 4:** una estrategia recibe una `EntidadAnimada` y solo puede llamar a lo que esa clase declara. Por eso `EntidadAnimada*` declara los ganchos (por defecto no hacen nada) y las subclases redefinen los que usan: `# morir()` (lo redefinen `Enemigo*`, `EnemigoConDrop*`, `Jefe*`, `Link` y `Proyectil`), `# alSerGolpeado()` (`MasterStalfos`), `+ ataqueRealizado()` (`Link`), `+ restaurarDefensa()` (`Link`), `+ teletransportarse()` (`Wizzrobe`) y la consulta `+ quiereAtacar() : boolean` (`MasterStalfos`; por defecto `false`). Así no hacen falta *cast* ni `instanceof` (observación 13 resuelta). `run()` también está en `EntidadAnimada*` (en la defensa 5 pasa a ser `actualizarEstado()`, ver 16.6). Quedan en las subclases `# vigilarJugador()` (`Enemigo*`, lo llama el ciclo, no una estrategia) y `ataqueTerminado()` (`StoneHinox`: lo avisa `AtaqueAvalancha.alTerminar`; para llamarlo sin *cast* convendría declararlo

> también como gancho en `EntidadAnimada*`, ver Avalancha y la observación 27). En los esquemas de este documento se marcan con un comentario `// gancho` cuando se usan.

`AccionProgramada`

- **Utilidad (defensa 5):** una espera sin hilo extra. Guarda **cuándo** (`instante`) y **qué** (`accion : Runnable`).
- **Comportamiento:** `estaVencida(ahora)` y `ejecutar()`. En cada vuelta, la entidad revisa sus pendientes y ejecuta las vencidas, dentro del mismo hilo que la actualiza.
- **Relaciones:** es parte de `EntidadAnimada*` (◆, `#pendientes 0..*`). Ejemplos y código en 16.6.

`Link`

Link

```
# vidas : int
# corazonesMaximos : int
# puntaje : int
# rupias : int
# fragmentos : int
# racha : int
# inventario : List<Objeto>
# objetoEquipado : Objeto
# caminar : MovimientoInducido
# recargado : AtaqueRecargado
# defensaBase : ComportamientoDefensa
# escudoActivo : boolean
# atacando : boolean
# soltoAtaque : boolean
# ralentizado : boolean
```

«constructores»

```
+ Link(sprite : Sprite, x : int, y : int)
```

«comandos»

```
+ mover(d : Direccion)
+ detenerMovimiento()
+ atacar()
+ soltarAtaque()
+ usarEscudo()
+ soltarEscudo()
+ activarEscudo()
+ desactivarEscudo()
+ cargarAtaque()
+ liberarAtaque()
+ cambiarObjeto()
+ agregarAlInventario(o : Objeto)
+ sumarPuntos(p : int)
+ reiniciarPuntaje()
+ sumarRupias(n : int)
+ curar(medios : int)
+ sumarFragmento()
+ incrementarRacha()
+ reiniciarRacha()
+ reiniciarVidas()
+ setDefensaBase(d : ComportamientoDefensa)
+ restaurarDefensa()
+ powerUpRecogido(p : PowerUP)
+ ralentizar()
+ avanzar(d : Direccion, pasos : int)
+ entrarPorBordeOpuesto(d : Direccion)
+ caerAlVacio()
```

```
+ caerAlVacio()
+ romperSuelo(s : SuelosQuebradizo)
+ ataqueRealizado()
+ recibirDanio(danio : int, origen : Direccion)
+ recibirGolpeDeEspada(atacante : EntidadAnimada, danio : int)
# alSerGolpeado()
+ aceptarColision(c : Colisionador)
+ colisionarCon(e : Enemigo)
+ colisionarCon(p : Proyectil)
+ colisionarCon(o : Obstaculo)
+ colisionarCon(p : PowerUP)
+ colisionarCon(o : Objeto)
# morir()
«consultas»
+ getVidas() : int
+ getCorazones() : int
+ getCorazonesMaximos() : int
+ getPuntaje() : int
+ getRupias() : int
+ getRacha() : int
+ getNombreObjetoEquipado() : String
+ getInventario() : List<Objeto>
+ quiereAtacar() : boolean
+ soltoElAtaque() : boolean
+ tieneEscudoActivo() : boolean
```

- **Utilidad:** el personaje del jugador y dueño del estado de la partida.
- **Comportamiento:** se mueve según la intención que fija el teclado (`mover(d)`, `detenerMovimiento()`: fijan la dirección de su `MovimientoInducido caminar`); `HiloJugador` actualiza la posición en cada vuelta. Guarda vidas, puntaje, rupias, corazones y fragmentos (vidas ≠ corazones). Lleva la racha de enemigos derrotados sin daño (`incrementarRacha()`, `reiniciarRacha()`, `getRacha()`) para la `Bellota`, y una **defensa base** (`setDefensaBase(d)`, `restaurarDefensa()`) a la que vuelve al soltar el escudo. Usa el objeto equipado (`atacar()`, `usarEscudo()`, `cambiarObjeto()`); el escudo lo llama de vuelta con `activarEscudo()` y `desactivarEscudo()`. Los obstáculos lo llaman con `caerAlVacio()` (`CaidasAlVacio`) y `romperSuelo(s)` (`SuelosQuebradizo`). Recibe daño con `recibirDanio(danio, origen)` (redefinido para cortar la racha y avisar a `Juego`) y redefine `recibirGolpeDeEspada(atacante, danio)` (la espada de MasterStalfos lo daña). Al quedarse sin corazones, `# morir()`. Avisa sucesos con `powerUpRecogido(p)`.
- **Relaciones:** hereda de `EntidadAnimada*`; realiza `EntidadJugador`; agrega `Objeto*` (◇, `#inventario 1..2`) y referencia al objeto que está usando (→ `Objeto*`, `#objetoEquipado 0..1`). Notifica a su `CeldaJugador` con la lista `#observadores` heredada de `Entidad*`, igual que cualquier entidad. La referencia `Juego` (`#jugador 1`) para darle las intenciones del teclado; Link le avisa sucesos a `Juego` con el `#miJuego` heredado de `Entidad*`, y le pide al sector lo espacial con el `#miSector` heredado de `EntidadAnimada*`. Lo referencia `HiloJugador` (`#jugador 1`), que lo actualiza cada ~30 ms y resuelve todas sus colisiones (16.7). Cuando cambia de

sector, `Juego` lo retira del sector viejo, lo agrega al nuevo (que le actualiza `#miSector`) y le cambia la posición, en el mismo paso.
- **Atributos de estrategia propios (extendido):** `caminar : MovimientoInducido` (el teclado le cambia la dirección), `recargado : AtaqueRecargado` (la espada) y `defensaBase : ComportamientoDefensa`. No son relaciones nuevas en el reducido: son las mismas estrategias que `EntidadAnimada*` agrega, guardadas con su tipo concreto para no hacer *cast*.

`Proyectil`

- **Utilidad:** disparos de enemigos y jefes (Octorok, Wizzrobe, rocas de StoneHinox).
- **Comportamiento (según el extendido):** `Proyectil(sprite, x, y, d, danio)`; avanza en línea recta (con `MovimientoInducido`); `colisionarCon(l : Link)` le hace daño a Link (salvo que use el escudo) y desaparece; `colisionarCon(o : Obstaculo)` desaparece. `setDestino(p)` lo usa la avalancha; `getDanio()`; `# morir()` (redefine el gancho de `EntidadAnimada*`) y `desaparecer()` (privado) lo sacan del sector.
- **Relaciones:** hereda de `EntidadAnimada*`; lo crea `GeneradorProyectil` (dependencia «crea»); referencia a `Posicion` (→, `#destino 0..1`): solo las rocas de la avalancha tienen destino (el lugar donde caen); los disparos comunes lo tienen en `null` y avanzan en su `#direccion`.

`Enemigo*`

- **Utilidad:** base de todos los enemigos.
- **Comportamiento (según el extendido):** constructor protegido `# Enemigo(sprite : Sprite, x : int, y : int, vida : int, danioContacto : int)`; `danioContacto`; `aceptarColision(c)`, `colisionarCon(l : Link)` (el enemigo se movió y tocó a Link), `recibirGolpeDeEspada(atacante, danio)` (pasa el golpe a su defensa), `# morir()` (se saca del sector y avisa a `Juego`; **no** suelta nada, así los que no tienen drop —`TrampaDeCuchillas`, los jefes— no lo necesitan); consultas `getDanioContacto()` y `getPuntos()`.
- **Relaciones:** hereda de `EntidadAnimada*`.
- **Subclases directas:** `EnemigoConDrop*`, `TrampaDeCuchillas`, `Jefe*`.

```
        public abstract class Enemigo extends EntidadAnimada {
            protected int danioContacto;
            protected Enemigo(Sprite sprite, int x, int y, int vida, int danioContacto) {
                super(sprite, x, y, vida);
                this.danioContacto = danioContacto;
            }
            public void aceptarColision(Colisionador c) { c.colisionarCon(this); }        //
doble despacho
            public void colisionarCon(Link l) { l.recibirDanio(danioContacto, getDireccion());
}
```

```
            public void recibirGolpeDeEspada(EntidadAnimada atacante, int danio) {
recibirDanio(danio, atacante.getDireccion()); }
            protected void morir() {                          // sin drop: eso lo agrega
EnemigoConDrop
                detener();
                quitarDelSector(this);
                miJuego.enemigoDerrotado(this);               // Juego suma los puntos y la
racha
            }
        }
```

`EnemigoConDrop*`

- **Utilidad:** enemigos que, al morir, pueden soltar una rupia o un corazón.
- **Comportamiento (según el extendido):** constructor protegido `# EnemigoConDrop(sprite : Sprite, x : int, y : int, vida : int, danioContacto : int)`; `generarDrop() : PowerUP` (implementa `GeneradorDrop`), `# soltarDrop()` (lo agrega al sector si salió algo), `# getProbabilidadDrop()` (cada subclase define la suya) y `# morir()`, que redefine el de `Enemigo*`: primero `super.morir()` y después `soltarDrop()`.
- **Relaciones:** hereda de `Enemigo*`; realiza `GeneradorDrop`.

```
        public abstract class EnemigoConDrop extends Enemigo implements GeneradorDrop {
            protected EnemigoConDrop(Sprite sprite, int x, int y, int vida, int danioContacto)
{
                super(sprite, x, y, vida, danioContacto);
            }
            protected void morir() { super.morir(); soltarDrop(); }   // lo de todo enemigo y
después el drop
            protected void soltarDrop() {
                PowerUP drop = generarDrop();
                if (drop != null) agregarAlSector(drop);
            }
        }
```

`<<Interface>> GeneradorDrop`

- **Utilidad:** contrato para "soltar algo al morir".
- **Comportamiento:** `generarDrop() : PowerUP` decide aleatoriamente, según la probabilidad del enemigo, si suelta un power-up y cuál. Es un **Factory Method**: fabrica una `Rupia` o un `Corazon` (o nada).
- **Relaciones:** la realiza `EnemigoConDrop*`; **crea** `Rupia` y `Corazon` (dos dependencias «crea»). Solo esos dos: según el enunciado, los enemigos sueltan "rupias o corazón". Ver Factory Method.

`<<Interface>> GeneradorProyectil`

- **Utilidad:** contrato para entidades que disparan: "sé **qué** proyectil fabrico".
- **Comportamiento:** `generarProyectil() : Proyectil`. Cada enemigo que dispara lo implementa a su manera (dirección, tipo de proyectil, daño). Es un **Factory Method**, igual que `GeneradorDrop`.
- **Relaciones:** la realizan `Octorok`, `Wizzrobe` y `StoneHinox`; **crea** `Proyectil` (dependencia «crea»); la referencia la estrategia `AtaquePorProyectil` (`#generadorProyectil 1`), que es

quien decide **cuándo** disparar. Ver [Disparo de proyectiles].

Factory Method: las relaciones «crea» y por qué son interfaces

El diagrama tiene dos **Factory Method**, y los dos se explican igual que el Abstract Factory de sprites (`FabricaSprites* «crea» Sprite`): con una dependencia «crea» hacia lo que fabrican. Esas dependencias **no se dibujan** en el reducido (decisión del equipo); en el dibujo los señalan las dos notas violetas, "Uso de patrón Factory Method (drops)" junto a `GeneradorDrop` y "Uso de patrón Factory Method (proyectiles)" junto a `GeneradorProyectil`, y las dos interfaces van en violeta.

Factory Method	Método	Quién lo realiza	Qué crea	En el diagrama
`GeneradorProyectil`	`generarProyectil()`	`Octorok`, `Wizzrobe`, `StoneHinox`	`Proyectil`	`GeneradorProyectil «crea» Proyectil` (no dibujada)
`GeneradorDrop`	`generarDrop()`	`EnemigoConDrop*`	`Rupia` o `Corazon`	`GeneradorDrop «crea» Rupia` y `GeneradorDrop «crea» Corazon` (no dibujadas)

Por qué el drop apunta a `Rupia` y `Corazon` y no a `PowerUP*`

El método devuelve un `PowerUP` (`PowerUP generarDrop()`), que es el tipo común de lo que puede salir. Pero «crea» significa "hace `new` de esta clase", y `GeneradorDrop` solo instancia `Rupia` o `Corazon`: la `Bellota` la crea `Juego` y el `FragmentoDeCorazon` lo crea `GeneradorNivel`. Si la flecha apuntara a `PowerUP*`, sugeriría que un enemigo puede soltar cualquier power-up. Apuntando a las dos clases concretas, el diagrama responde por sí solo "¿qué suelta un enemigo?".

Por qué es dependencia y no asociación

El generador **no guarda** lo que crea como atributo: lo fabrica y lo devuelve (`Proyectil generarProyectil()`). Usar un tipo de paso (como retorno, parámetro o variable local) es una **dependencia**, no una asociación. El estereotipo «crea» dice para qué la usa: para fabricarlo. Por eso, si se dibujaran, irían **punteadas**; en el reducido de la defensa 4 no se dibujan, por decisión del equipo (ver [1. Cómo leer el diagrama]).

Por qué se dibujan (si UML no obliga a dibujar todas las dependencias)

> **Hoy no se dibujan.** El equipo decidió sacar del reducido todas las dependencias, incluidas estas (UML no obliga a dibujarlas). Los argumentos de abajo eran los que justificaban dibujarlas; se dejan porque sirven igual para explicarlas en la defensa, y el punto 3 ahora lo cubren las notas violetas de Factory Method.

1. **Coherencia:** el Abstract Factory ya muestra «crea» hacia `Sprite`; los Factory Method se muestran igual.
2. **Completan la historia:** en el disparo se ve la cadena entera: `AtaquePorProyectil → GeneradorProyectil «crea» Proyectil`. En el drop: `EnemigoConDrop ⇢ GeneradorDrop «crea» Rupia / Corazon`.
3. **Defensa:** si preguntan "¿dónde está el Factory Method y qué fabrica?", el diagrama lo muestra.

Otras dependencias del mismo tipo **no se dibujan** para no recargar: `AtaquePorProyectil ⇢ Proyectil` (solo pasa el proyectil al sector) y `FabricaEntidades «crea» todas las entidades` (serían decenas de flechas; se explica en el texto).

Por qué son interfaces y no clases

1. **Java no tiene herencia múltiple de clases.** Los que generan ya tienen superclase: `Octorok` y `Wizzrobe` extienden `EnemigoConDrop`; `StoneHinox` extiende `Jefe`. Si `GeneradorProyectil` fuera una clase, `Octorok` no podría extender a las dos. Con una interfaz sí:

   ```
   public class Octorok extends EnemigoConDrop implements GeneradorProyectil { ... }
   public class StoneHinox extends Jefe implements GeneradorProyectil { ... }
   ```

2. **No hay nada en común que heredar:** cada enemigo fabrica su proyectil distinto (roca recta, magia apuntada, roca grande). Lo único común es el contrato, y eso es una interfaz.
3. **El polimorfismo funciona igual:** `AtaquePorProyectil` guarda un `GeneradorProyectil` y le sirve cualquier objeto que lo implemente, sin `instanceof`.

En `GeneradorDrop` la lógica sí es común (tirar el azar y elegir rupia o corazón); lo único que cambia es la probabilidad. Por eso la implementación vive en `EnemigoConDrop` y cada subclase solo define su probabilidad:

```
public interface GeneradorDrop {
    PowerUP generarDrop();                                  // Factory Method
}

public abstract class EnemigoConDrop extends Enemigo implements GeneradorDrop {
    public PowerUP generarDrop() {                          // lógica común
        FabricaEntidades f = miJuego.getFabricaEntidades();
        if (Math.random() < getProbabilidadDrop())
            return Math.random() < 0.5 ? f.crearRupia(getX(), getY()) :
f.crearCorazon(getX(), getY());
        return null;                                        // no suelta nada
    }
    protected abstract double getProbabilidadDrop();     // Moblin 0.20,
MoblinConEspadaYEscudo 0.50,
}                                                           // ErizoDeMar 0.20, Octorok
0.40, Wizzrobe 1.0
```

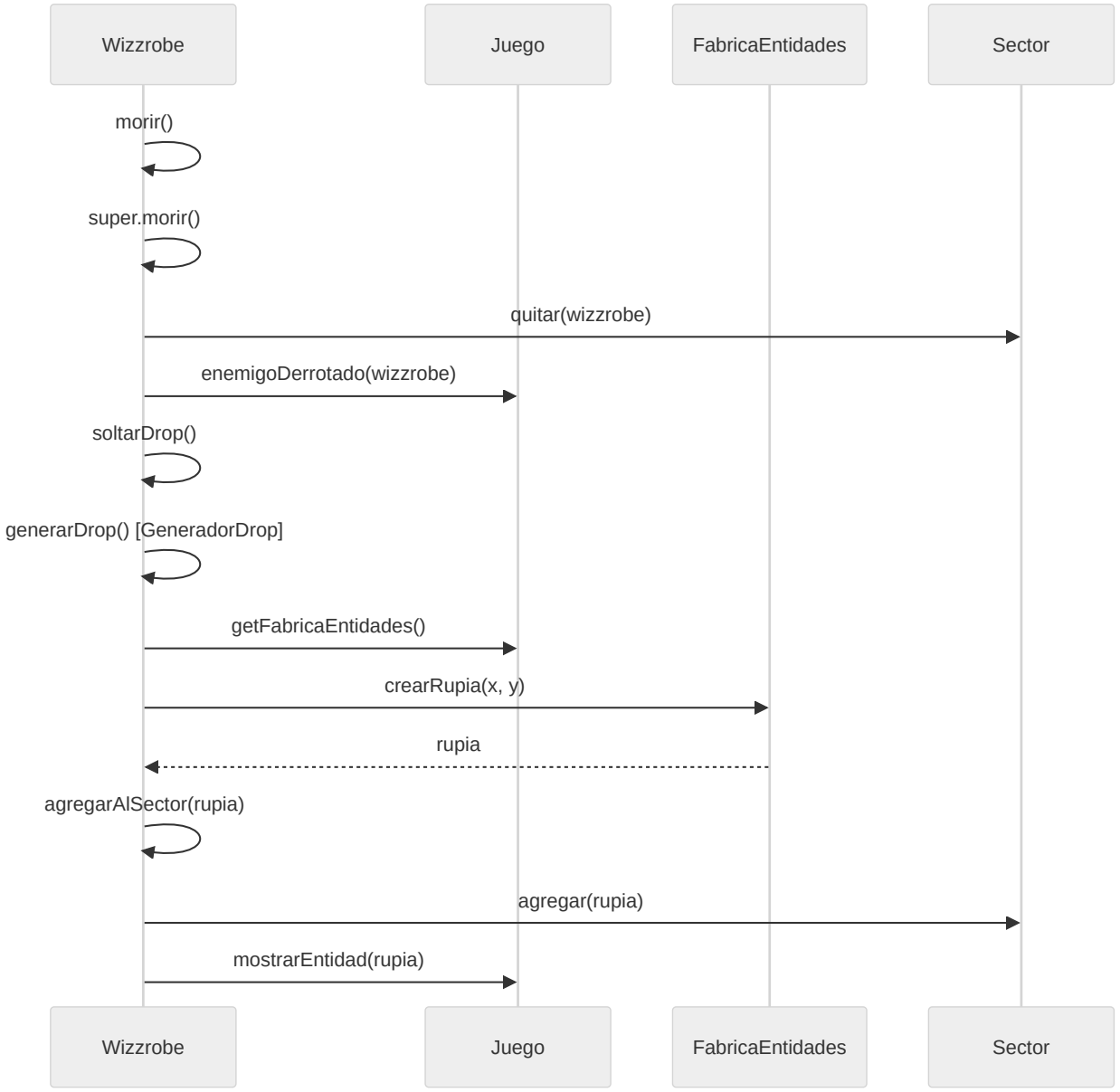
> **De dónde sale la fábrica:** las entidades no guardan la `FabricaEntidades`. La obtienen de `Juego` con `miJuego.getFabricaEntidades()` (consulta del extendido de la defensa 4), así todo lo que se crea en plena partida usa el dominio elegido.

La interfaz `GeneradorDrop` se mantiene aunque hoy la realice una sola clase: deja el Factory Method visible, queda en paralelo con `GeneradorProyectil` y permitiría que en el futuro algo que no es un enemigo (por ejemplo, el pasto al cortarlo) también suelte cosas.

Nombres: se mantienen `GeneradorProyectil` y `GeneradorDrop`. Describen el rol, forman un par coherente y no se confunden con las clases `Fabrica...` del Abstract Factory.

Ejemplos

1. **Un Octorok dispara:** `AtaquePorProyectil` llama a `generadorProyectil.generarProyectil()` → el Octorok fabrica un `Proyectil` (dependencia «crea») → la estrategia lo agrega al sector.
2. **Muere un Wizzrobe (probabilidad 100%):** `morir()` (el de `EnemigoConDrop*`) → `super.morir()` (se saca del sector y avisa `enemigoDerrotado`) → `soltarDrop()` → `generarDrop()` → sale una `Rupia` o un `Corazon` (dependencias «crea») → se agrega al sector donde murió.
3. **Muere un Moblin (probabilidad 20%):** `generarDrop()` tira el azar; 8 de cada 10 veces devuelve `null` y no se agrega nada.



Enemigos concretos

Clase	Hereda de	Comportamiento	Golpes	Puntos	Drop
`Moblin`	`EnemigoConDrop*`	Camina libremente; contacto ½ corazón.	2	100	20%

Clase	Hereda de	Comportamiento	Golpes	Puntos	Drop
`MoblinConEspadaYEscudo`	`Moblin`	Se abalanza si Link está de frente (`vigilarJugador()`, de `Enemigo*`); bloquea la espada de frente.	2	300	50%
`ErizoDeMar`	`EnemigoConDrop*`	Inmóvil; daño por contacto.	1	50	20%
`Octorok`	`EnemigoConDrop*`	Dispara un proyectil cuando está frente a Link. Realiza `GeneradorProyectil`.	1	150	40%
`Wizzrobe`	`EnemigoConDrop*`	Se teletransporta cada pocos segundos (`MovimientoTeletransporte`, `teletransportarse()`) y dispara. Invulnerable 1 s tras un golpe, hasta la próxima teletransportación. Realiza `GeneradorProyectil`.	2	500	100%
`TrampaDeCuchillas`	`Enemigo*`	Inmóvil; se abalanza si Link pasa enfrente (`vigilarJugador()`, de `Enemigo*`) y, al chocar con un obstáculo (`colisionarCon(o : Obstaculo)`), vuelve a su `#posicionOriginal` (→ `Posicion`, `1`) con `volver()`. **Indestructible**. No suelta nada.	–	–	–

Cada enemigo concreto redefine `getPuntos()` (en el extendido figura con un `+` al final, ver Observaciones) y, si suelta algo, `getProbabilidadDrop()`.

`Jefe*`

- **Utilidad:** enemigo final de cada nivel (aparece en todos los modos).
- **Comportamiento:** necesita muchos golpes, tiene períodos de invulnerabilidad y no suelta nada. Vale +5000 puntos. Hereda de `Enemigo*`, **no** de `EnemigoConDrop*`. En el extendido de la defensa 4, `Jefe*` tiene el constructor `# Jefe(sprite : Sprite, x : int, y : int, vida : int)` (cada jefe pasa su cantidad de golpes) y redefine `# morir()` para avisar `miJuego.jefeDerrotado(this)` en lugar de `enemigoDerrotado(this)` (observación 23 resuelta). Cada jefe define su `getPuntos()`.
- **Relaciones:** hereda de `Enemigo*`; lo referencia `Nivel` (`#jefes 1..*`).

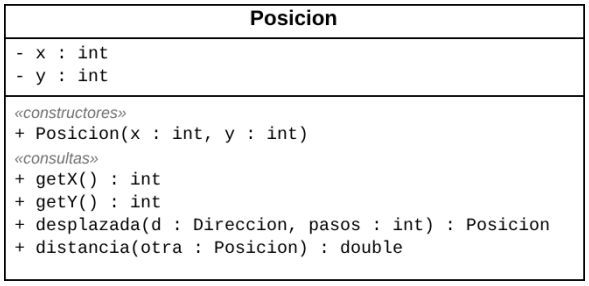
Clase	Comportamiento
`MasterStalfos`	Jefe del nivel 1. Se mueve en horizontal y vertical. Contacto ½ corazón; espada 1 corazón (`AtaqueEspada`, `quiereAtacar()` cuando Link está cerca). ≥ 5 golpes, invulnerable un rato tras cada golpe (`alSerGolpeado()`).
`StoneHinox`	Jefe del nivel 2, el que más golpes necesita. Ataques de 1 corazón: embestida (solo ahí es vulnerable, `embestidaTerminada()`), lanza tres rocas (`AtaquePorProyectil`) y avalancha (`AtaqueAvalancha`, 5–10 rocas aleatorias). Guarda sus estrategias en `embestida`, `lanzarRocas`, `avalancha` y `caminar`, y alterna con `siguienteAtaque()`. Realiza `GeneradorProyectil`.

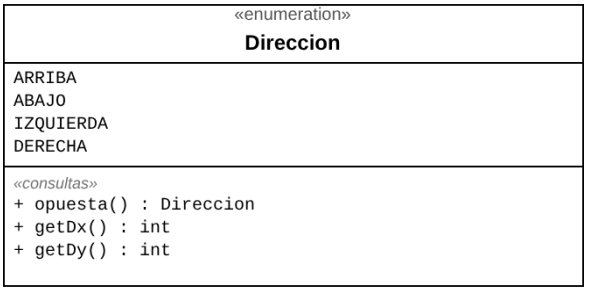
```
        public abstract class Jefe extends Enemigo {
            protected Jefe(Sprite sprite, int x, int y, int vida) { super(sprite, x, y, vida, 1); }   // contacto: ½ corazón
            protected void morir() {                    // redefine el morir() de Enemigo
                quitarDelSector(this);
                miJuego.jefeDerrotado(this);            // +5000 y, si no quedan jefes, fin del nivel
```

```
            }
        }
```

Posición y dirección (`Posicion` y `Direccion`)

Son dos tipos chicos que usa todo el modelo. Con ellos, la lógica maneja coordenadas y orientaciones **sin depender de Swing ni de AWT** (no se usan `java.awt.Point` ni `java.awt.Rectangle`) y sin `if` sobre números sueltos.





`Posicion`

- **Utilidad:** un punto `(x, y)` en píxeles, **relativo al sector** donde está la entidad (el (0,0) es la esquina superior izquierda de esa pantalla).
- **Comportamiento (según el extendido de la defensa 4):** atributos privados `x`, `y`; constructor `Posicion(x, y)`; consultas `getX()`, `getY()` (en la defensa 3 faltaba `getX()`), `desplazada(d : Direccion, pasos : int) : Posicion` (la posición que queda al avanzar `pasos` en la dirección `d`) y `distancia(otra) : double`. Conviene tratarla como **inmutable**: `desplazada` devuelve una posición nueva en lugar de modificar la actual, así dos entidades nunca comparten un punto que cambia solo.
- **Relaciones (todas asociaciones hacia `Posicion`):**

Origen	Rol	Para qué
`Entidad*`	`#posicion 1`	Dónde está cada entidad dentro de su sector. Junto con `ancho` y `alto` forma su rectángulo (*hitbox*).
`Nivel`	`#posicionInicial 1`	Dónde aparece Link dentro del sector inicial (ver Diferencia entre `posicionInicial` y `filaInicial` / `columnaInicial`). Sale de la línea `INICIO` del archivo.
`Proyectil`	`#destino 0..1`	Punto al que cae una roca de la avalancha de StoneHinox. Los disparos comunes no tienen destino (`null`): avanzan en línea recta.
`TrampaDeCuchillas`	`#posicionOriginal 1`	A dónde vuelve la trampa después de abalanzarse y chocar.
`MovimientoInducido`	`#objetivo 0..1`	Punto al que va la entidad cuando el movimiento es "hacia un lugar" (StoneHinox al centro o hacia Link).

- **Relación con `Direccion`:** `Posicion` **usa** `Direccion` en `desplazada(d, pasos)`: la recibe por parámetro y no la guarda. Ver ¿`Posicion → Direccion` es asociación o dependencia?.

`<<enumeration>> Direccion`

- **Utilidad:** las cuatro direcciones del juego: `ARRIBA`, `ABAJO`, `IZQUIERDA`, `DERECHA` (en la defensa 4 ya figuran en el extendido).
- **Comportamiento (según el extendido):** `opuesta() : Direccion`, `getDx() : int` y `getDy() : int` (cuánto cambia `x` e `y` al dar un paso). Así, avanzar es siempre `posicion.desplazada(direccion, pasos)`, sin un `switch`.
- **Relaciones (asociaciones hacia `Direccion`):**

Origen	Rol	Para qué
`EntidadAnimada*`	`#direccion 1`	Hacia dónde mira o se mueve la entidad. La usan el escudo del Moblin (bloquea de frente), la trampa y el Octorok (reaccionan si Link está enfrente) y la espada de Link (qué tiene delante).
`MovimientoInducido`	`#direccion 0..1`	La orden de movimiento: la flecha que apretó el jugador o el rumbo de un proyectil. `null` = quieto.

Además la usan como **parámetro** (sin flecha dibujada) muchos métodos: `ControladorJuego.mover(d)`, `Juego.cambiarDeSector(d)`, `Escena.getVecino(s, d)`, `Sector.puedeMoverA(e, d, pasos)`, `ControladorGrafico.transicion(d)`, `PanelPantallaNivel.desplazar(d)`, `ComportamientoDefensa.recibirDanio(e, danio, origen)` y `FabricaEntidades.crearProyectil(x, y, d)`.

¿`Posicion → Direccion` es asociación o dependencia?

Es **el único cambio** del reducido de la defensa 4 respecto del de la defensa 3: la flecha pasó de punteada a continua. Para decidir qué tipo de relación es, hay que mirar **cómo usa** `Posicion` a `Direccion` en el extendido:

Pregunta	Respuesta en el extendido de la defensa 4
¿`Posicion` tiene un atributo de tipo `Direccion`?	**No.** Sus atributos son `- x : int` y `- y : int`.
¿Recibe una `Direccion` por parámetro?	**Sí**, en `desplazada(d : Direccion, pasos : int)`.
¿La devuelve o la crea?	No.

Tipo de relación	Cuándo se usa	¿Corresponde?
Asociación (línea continua, flecha pintada, con rol y multiplicidad)	La clase de origen **guarda** una referencia permanente (un atributo)	No: no hay atributo
Dependencia (línea punteada, flecha abierta, con estereotipo)	La clase de origen **usa** a la otra de paso: parámetro, retorno o variable local	**Sí**: es un parámetro

Conclusión: debería seguir siendo dependencia (punteada, con «usa»), como en la defensa 3. **Actualización:** después el equipo decidió no dibujar dependencias en el reducido, así que la flecha se sacó del dibujo; la relación queda como **dependencia no dibujada**, que es lo que dice esta conclusión. Con línea continua se lee como asociación y alguien puede preguntar "¿cuál es el atributo de `Posicion` que guarda la dirección?", y no lo hay. Además la flecha nueva no sigue ninguna de las dos convenciones del diagrama: no tiene rol ni multiplicidad (como toda asociación) ni la línea punteada (como toda dependencia).

```
        public final class Posicion {
            private final int x, y;                          // no hay atributo Direccion: no es asociación
            public Posicion desplazada(Direccion d, int pasos) {   // la usa como parámetro:
```

```
dependencia «usa»
                return new Posicion(x + d.getDx() * pasos, y + d.getDy() * pasos);
            }
        }
```

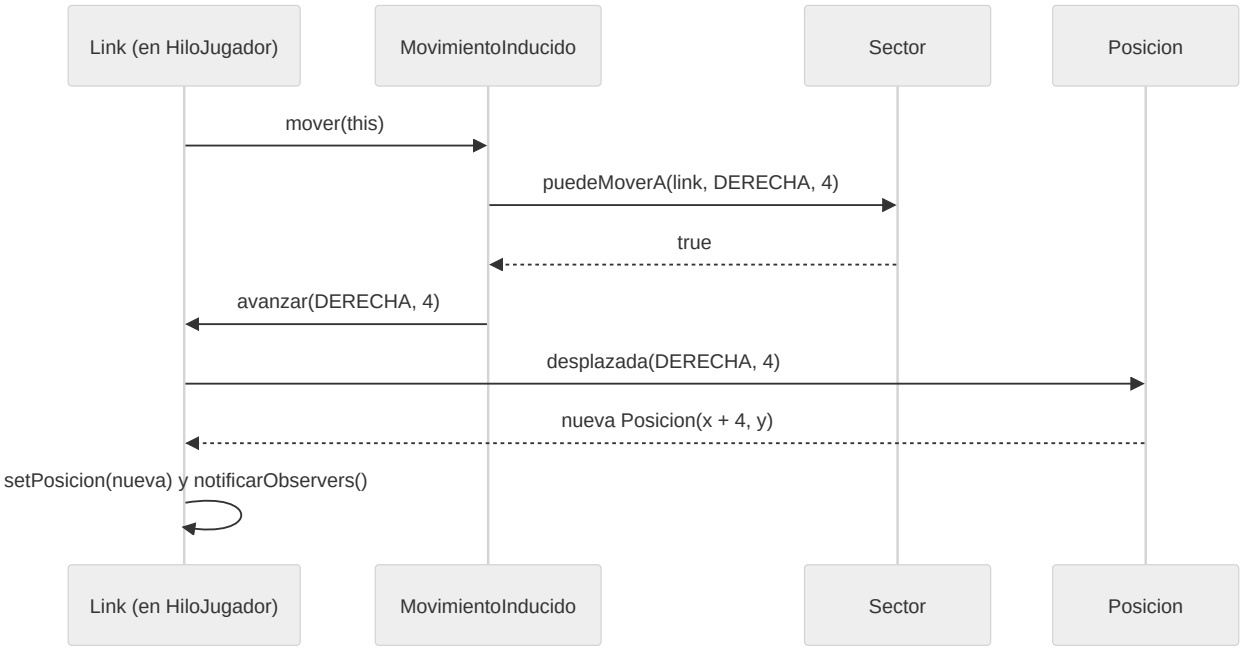
Comparación con las otras relaciones hacia `Direccion`: `EntidadAnimada* → Direccion` (`#direccion 1`) y `MovimientoInducido → Direccion` (`#direccion 0..1`) **sí** son asociaciones, porque esas clases tienen el atributo `direccion`. La diferencia entre ellas y `Posicion` es exactamente la diferencia entre asociación y dependencia.

Cómo corregirlo en el `.drawio`: en el estilo de la flecha `rel-posicion-direccion-dep`, volver a agregar `dashed=1` (la punta abierta y el estereotipo «usa» ya están bien). Si se prefiere no dibujar dependencias "de parámetro" (como no se dibuja `GeneradorNivel ⇢ FabricaEntidades`), también es válido **borrarla**; lo que no conviene es dejarla continua. **Aplicado:** se borró junto con las demás dependencias.

```
        public enum Direccion {
            ARRIBA(0, -1), ABAJO(0, 1), IZQUIERDA(-1, 0), DERECHA(1, 0);
            private final int dx, dy;
            Direccion(int dx, int dy) { this.dx = dx; this.dy = dy; }
            public int getDx() { return dx; }
            public int getDy() { return dy; }
            public Direccion opuesta() {
                switch (this) { case ARRIBA: return ABAJO; case ABAJO: return ARRIBA;
                                case IZQUIERDA: return DERECHA; default: return IZQUIERDA; }
            }
        }

        public final class Posicion {
            private final int x, y;
            public Posicion(int x, int y) { this.x = x; this.y = y; }
            public int getX() { return x; }
            public int getY() { return y; }
            public Posicion desplazada(Direccion d, int pasos) {
                return new Posicion(x + d.getDx() * pasos, y + d.getDy() * pasos);
            }
            public double distancia(Posicion otra) { return Math.hypot(x - otra.x, y - otra.y);
}
        }
```

Ejemplo: Link da un paso a la derecha



Si Link estaba en `(96, 240)`, `DERECHA.getDx()` es `1` y `getDy()` es `0`: la posición nueva es `(100, 240)`.

Ejemplo: el escudo del Moblin usa `opuesta()`

El `MoblinConEspadaYEscudo` mira a la `IZQUIERDA` (`#direccion`). Link lo ataca desde la izquierda; el golpe viaja hacia la `DERECHA`, así que `origen = DERECHA`. Como `IZQUIERDA.opuesta() == DERECHA`, el golpe viene **de frente** y `DefensaPorEscudo(true)` lo bloquea. Si Link le pega por la espalda, el origen es `IZQUIERDA`, no coincide, y el golpe pasa a la defensa `#sinEscudo`.

---

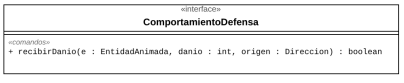
10. Comportamientos (patrón Strategy)

`EntidadAnimada*` agrega una estrategia de cada tipo y puede cambiarla en tiempo de ejecución. Así se resuelven sin `instanceof` las invulnerabilidades temporales, el escudo, el paso de "quieto" a "abalanzarse", y los distintos ataques de los jefes.

Orden de la sección: primero las tres interfaces, después **cada estrategia una por una** (con código y diagramas de secuencia) y al final el detalle de los cuatro casos que necesitaron más diseño: el disparo de proyectiles, la avalancha de StoneHinox, el teletransporte del Wizzrobe y el efecto de la bellota.







`<<Interface>> ComportamientoMovimiento`

Método: `mover(e : EntidadAnimada)`.

Implementación	Comportamiento	Ejemplos
`MovimientoAleatorio(velocidad, intervaloCambio)`	Elige direcciones al azar.	Moblin, Octorok, StoneHinox al caminar
`MovimientoInducido(d, velocidad)` / `(objetivo, velocidad)`	Se mueve hacia un objetivo o según una orden.	Link (teclado), proyectiles, trampa o Moblin al abalanzarse, StoneHinox al embestir o ir al centro. Con dirección `null` la entidad queda **quieta**: así están el ErizoDeMar (siempre) y la trampa mientras espera (detalle)
`MovimientoTeletransporte(intervalo)`	Cada pocos segundos aparece en una posición libre al azar del sector; entre saltos se queda quieto.	Wizzrobe (detalle)

`<<Interface>> ComportamientoAtaque`

Método: `atacar(atacante : EntidadAnimada)`.

Implementación	Comportamiento	Ejemplos
`AtaquePorContacto()`	No hace nada activo: daña al tocar a Link.	Moblin, ErizoDeMar, trampa, StoneHinox al embestir
`AtaqueEspada(danio, alcance)`	Golpe de espada delante de la entidad.	MasterStalfos
`AtaqueRecargado(danio, alcance)`	Hereda de `AtaqueEspada`: si se mantiene 3 s (`TIEMPO_CARGA = 3000`) y se suelta, golpe recargado.	Link
`AtaquePorProyectil(g, cantidad, intervalo)`	Decide **cuándo** disparar (Link enfrente, intervalo) y **cuántos** proyectiles; le pide a su `#generadorProyectil` **qué** proyectil crear. Define el esquema del ataque (Template Method).	Octorok, Wizzrobe, StoneHinox ("lanza rocas")
`AtaqueAvalancha(g)`	Hereda de `AtaquePorProyectil`: ataca al llegar al centro, tira entre 5 y 10 rocas y las hace caer en lugares al azar.	StoneHinox (detalle)

`<<Interface>> ComportamientoDefensa`

Método: `recibirDanio(e : EntidadAnimada, danio : int, origen : Direccion) : boolean` (devuelve si hubo daño).

Implementación	Comportamiento	Ejemplos
`DefensaVulnerable()`	Recibe el daño normalmente.	Estado normal de enemigos y Link
`DefensaInvulnerable()`	Ignora el daño.	Trampa, proyectiles, jefes y Wizzrobe tras recibir un golpe, Link 1 s después de un golpe
`DefensaPorEscudo(soloDeFrente)`	Bloquea el daño (todo, o solo el que viene de frente); si no lo cubre, delega en `#sinEscudo`.	Link con escudo (`false`), MoblinConEspadaYEscudo (`true`)

Implementación	Comportamiento	Ejemplos
`DefensaReducida()`	Cualquier golpe hace solo ½ corazón, durante 3 golpes (`golpesRestantes`).	Link después de recoger una `Bellota` (detalle)

Estrategias una por una (con diagramas de secuencia)

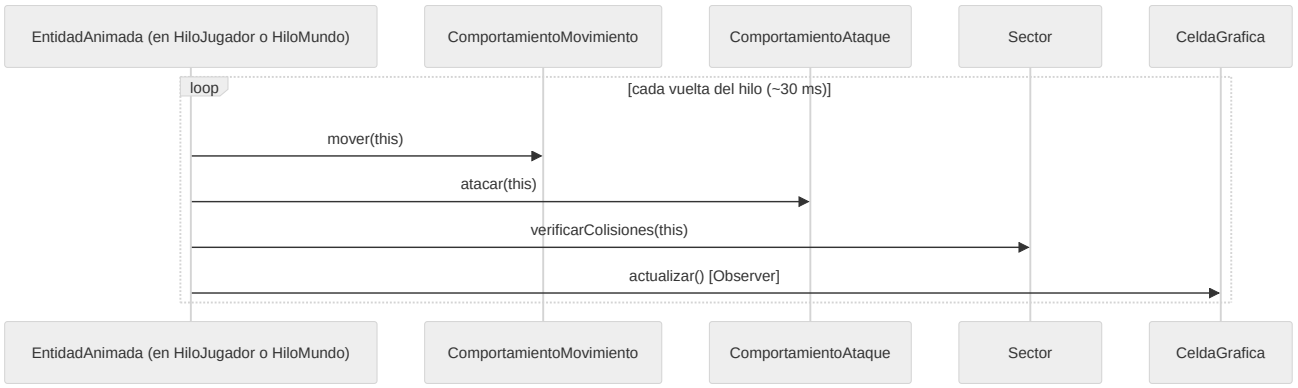
Esta sección explica **cada** estrategia del patrón Strategy con su código (esquema) y al menos un diagrama de secuencia. Las que tienen una subsección propia (`AtaquePorProyectil`, `AtaqueAvalancha`, `MovimientoTeletransporte`, `DefensaReducida`) se resumen y se enlaza su detalle.

Cómo se usan las estrategias: el ciclo de una entidad animada

Todas las estrategias se invocan desde el mismo lugar: la vuelta de cada `EntidadAnimada`, `actualizarEstado()`, que en la defensa 5 llama `HiloJugador` (Link) o `HiloMundo` (el resto) cada ~30 ms. La entidad no sabe qué estrategia concreta tiene; solo llama a la interfaz (polimorfismo).

```
        // En EntidadAnimada (defensa 5: el ciclo de run() pasa a ser una vuelta, actualizarEstado())
        protected void programar(long ms, Runnable accion) {   // # programar(ms : long, accion : Runnable)
            pendientes.add(new AccionProgramada(System.currentTimeMillis() + ms, accion));   // sin hilo nuevo
        }
        public void actualizarEstado() {              // la llama HiloJugador o HiloMundo
            movimiento.mover(this);                   // ComportamientoMovimiento
            ataque.atacar(this);                      // ComportamientoAtaque
            miSector.verificarColisiones(this);       // choques -> doble despacho
            ejecutarPendientes();                     // acciones programadas vencidas
        }                                             // la celda se redibuja cuando avanzar() notifica

        // Cuando la golpean (desde una colisión o un ataque)
        public void recibirDanio(int danio, Direccion origen) {
            if (defensa.recibirDanio(this, danio, origen))   // ComportamientoDefensa: decide si aplica aplicarDanio
                alSerGolpeado();                              // gancho (lo redefine MasterStalfos)
        }
        public void aplicarDanio(int danio) {
            vida -= danio;
            if (vida <= 0) morir();                       // gancho # morir(): lo redefinen Enemigo*, Jefe*, Link y Proyectil
        }
```



Explicación: en cada vuelta del ciclo, la entidad le pide a su estrategia de movimiento que la mueva, a su estrategia de ataque que ataque si corresponde, al sector que revise los choques y, por último, avisa a su celda para que se redibuje. La **defensa** no está en el ciclo: se usa cuando algo la golpea. Cambiar una estrategia (`setMovimiento`, `setAtaque`, `setDefensa`) cambia el comportamiento en la próxima vuelta, sin tocar el ciclo.

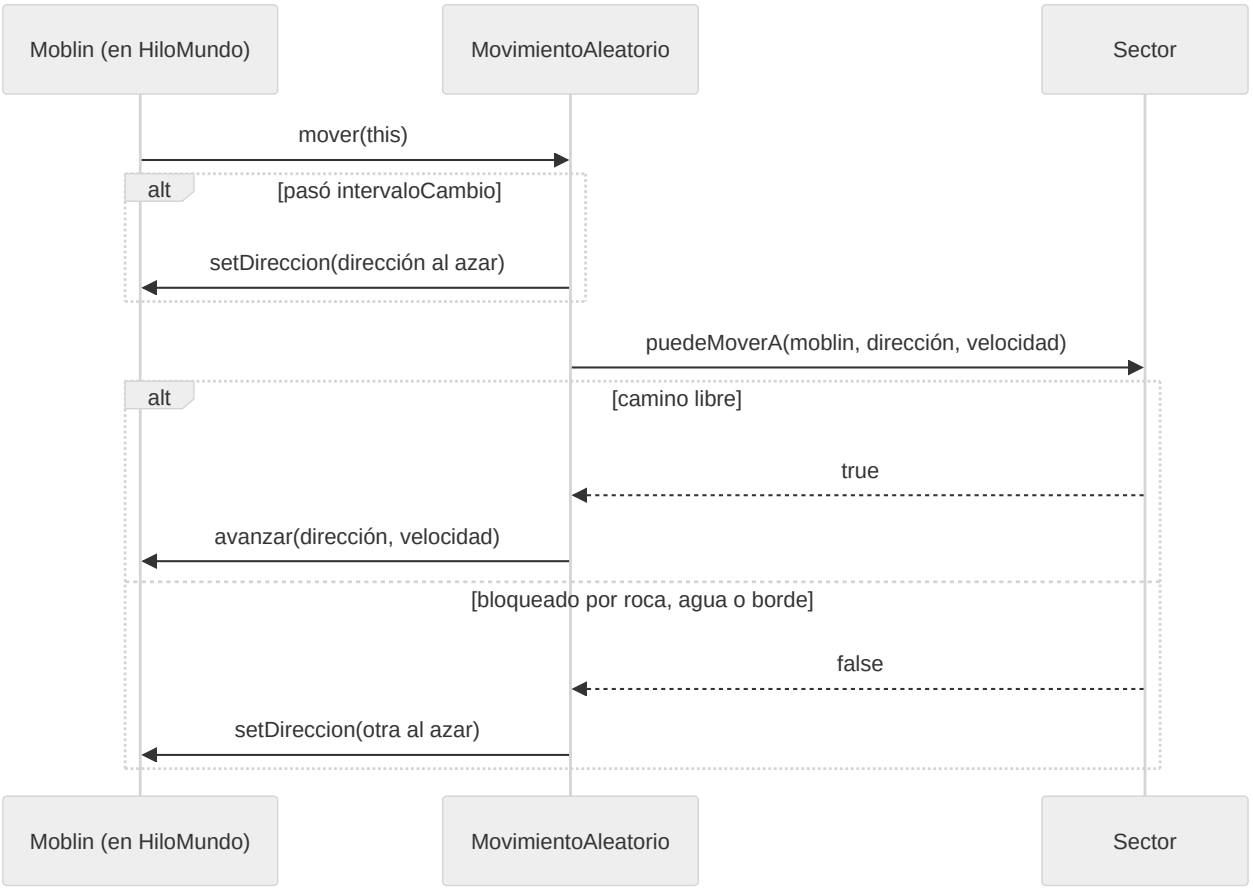
Firma común de la defensa: `boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen)`. Devuelve si hubo daño (para la racha de la bellota y el modo NoHit) y recibe de qué lado viene el golpe (para el escudo de frente). El daño se aplica con `e.aplicarDanio(danio)`, que es polimórfico: un **enemigo** resta golpes (Moblin necesita 2) y **Link** resta medios corazones.

Movimiento

`MovimientoAleatorio`

Qué es: la entidad camina sola, paso a paso, cambiando de dirección al azar cada `intervaloCambio` ms; si el camino está bloqueado (roca, agua, borde), elige otra dirección. **Quién la usa:** Moblin, Octorok, MasterStalfos (solo horizontal y vertical, que son las cuatro direcciones), StoneHinox al caminar entre ataques.

```
        public class MovimientoAleatorio implements ComportamientoMovimiento {
            private final Random azar = new Random();
            protected int velocidad;  protected long intervaloCambio, ultimoCambio;
            public MovimientoAleatorio(int velocidad, long intervaloCambio) { this.velocidad =
velocidad; this.intervaloCambio = intervaloCambio; }
            public void mover(EntidadAnimada e) {
                if (System.currentTimeMillis() - ultimoCambio >= intervaloCambio) {
                    e.setDireccion(Direccion.values()[azar.nextInt(4)]);
                    ultimoCambio = System.currentTimeMillis();
                }
                if (e.getSector().puedeMoverA(e, e.getDireccion(), velocidad))
e.avanzar(e.getDireccion(), velocidad);
                else e.setDireccion(Direccion.values()[azar.nextInt(4)]);   // bloqueado:
prueba otra
            }
        }
```

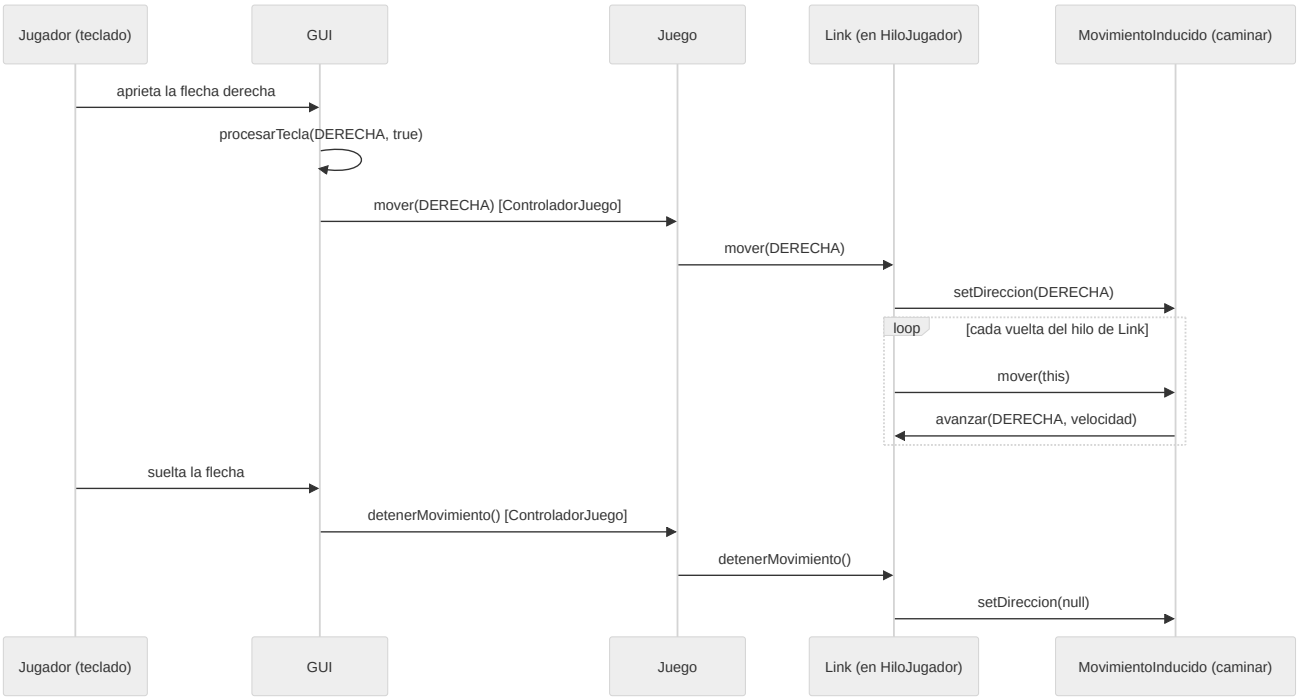


Explicación: cada cierto tiempo el Moblin cambia de rumbo al azar. En cada vuelta pregunta al sector si puede avanzar en esa dirección; si puede, da un paso; si no, elige otra dirección y lo intenta en la vuelta siguiente. El Moblin no "sabe" que camina al azar: solo llama a `mover()`.

`MovimientoInducido`

Qué es: la entidad se mueve **según una orden**: una dirección (Link con el teclado, un proyectil) o un **objetivo** (StoneHinox yendo al centro, la trampa abalanzándose). **Relaciones:** `#direccion 0..1` (→ `Direccion`) y `#objetivo 0..1` (→ `Posicion`); en cada momento se usa uno de los dos (el otro queda en `null`). Métodos: `setDireccion(d)`, `getDireccion()`, `mover(e)`.

```
        public class MovimientoInducido implements ComportamientoMovimiento {
            protected Direccion direccion;  protected Posicion objetivo;  protected int
velocidad;
            public MovimientoInducido(Direccion d, int v)  { direccion = d; velocidad = v; }
            public MovimientoInducido(Posicion obj, int v) { objetivo = obj; velocidad = v; }
            public void setDireccion(Direccion d) { direccion = d; }       // null = quieto
            public Direccion getDireccion() { return direccion; }
            public void mover(EntidadAnimada e) {
                if (objetivo != null) e.avanzarHacia(objetivo, velocidad);
                else if (direccion != null && e.getSector().puedeMoverA(e, direccion,
velocidad)) e.avanzar(direccion, velocidad);
            }
        }
```



Explicación: el teclado no mueve a Link: solo fija la **dirección** de su `MovimientoInducido`. El que mueve es el hilo de Link, que en cada vuelta llama a `mover()` y avanza en esa dirección. Al soltar la flecha, la dirección queda en `null` y Link se detiene. Así el hilo de Swing nunca mueve entidades (requisito del contexto).

Entidades quietas y que vigilan (sin `MovimientoEstatico`)

Qué cambia: el reducido ya no tiene `MovimientoEstatico`. Las entidades que no se mueven usan `MovimientoInducido` con dirección `null`, que ya significa "quieto" (es lo mismo que pasa con Link al soltar la flecha). No hace falta ninguna clase nueva.

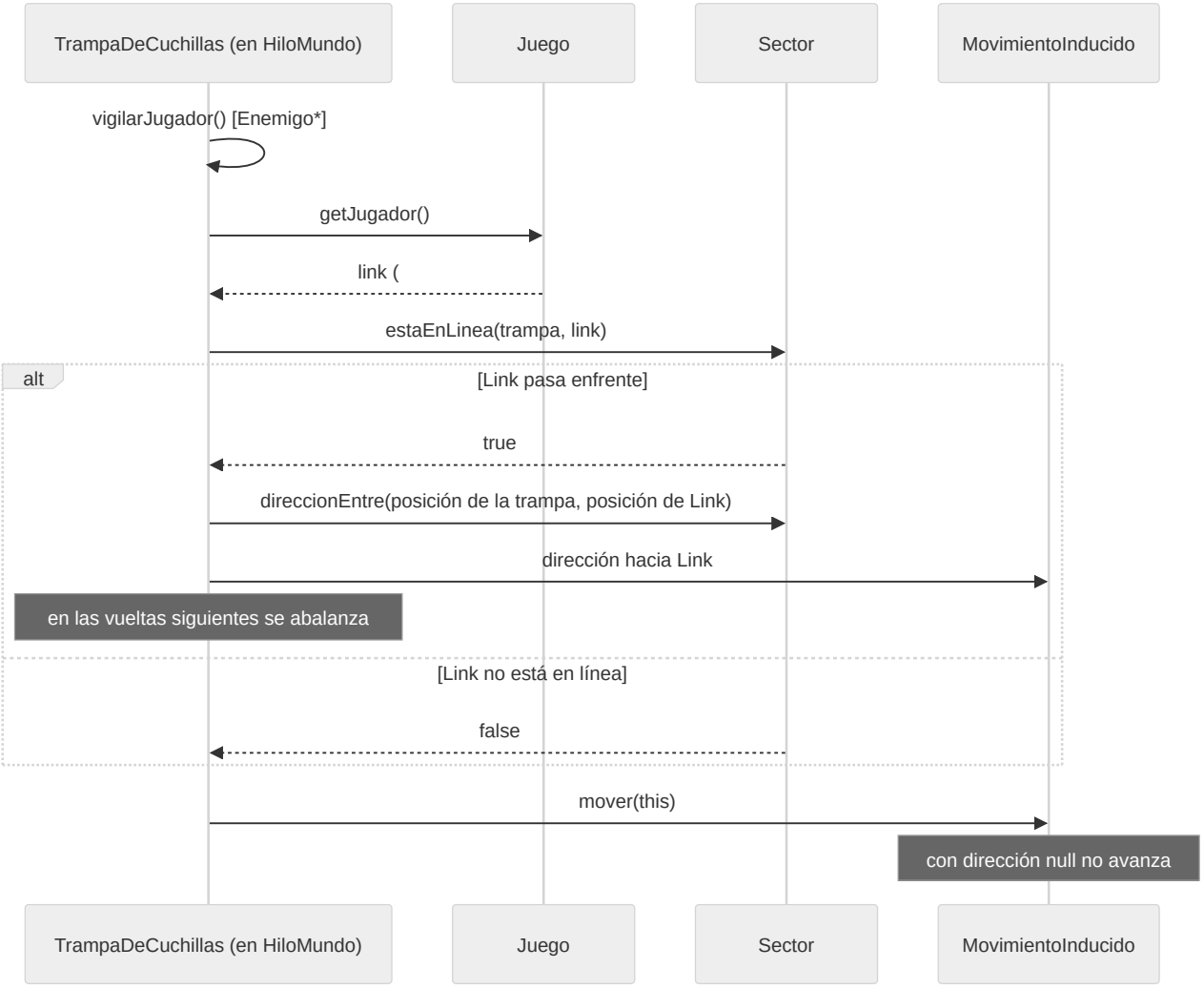
Entidad	Movimiento	Cómo cambia
ErizoDeMar	`MovimientoInducido(null, 0)`	Nunca: nadie le cambia la dirección, así que siempre está quieto.
TrampaDeCuchillas	`MovimientoInducido(null, 8)`	En su ciclo usa `vigilarJugador()`; si Link queda en línea, `setDireccion(d)` y se abalanza. Al chocar con un obstáculo vuelve a su `#posicionOriginal` y `setDireccion(null)`.
MoblinConEspadaYEscudo	`MovimientoAleatorio`	En su ciclo usa `vigilarJugador()`; si ve a Link de frente, cambia a `MovimientoInducido(d, velocidad)`.

Quién vigila a Link: `Enemigo*` tiene `# vigilarJugador()`, común a todos los enemigos: obtiene a Link y mira su posición. Con eso cada enemigo decide si realiza su ataque (la trampa y el Moblin con escudo, si se abalanzan) o hacia dónde lanza su proyectil (Octorok, Wizzrobe, StoneHinox). Ya no lo llama una estrategia de movimiento, que solo mueve, y la trampa y el Moblin ya no tienen un `vigilar()` propio.

Cómo sabe que es Link y no otra entidad: no recorre las entidades del sector. Le pide el jugador a `Juego` por su `#miJuego` (`miJuego.getJugador()`), y `Juego` devuelve su `#jugador` (`Juego → Link`), que siempre es Link. Con ese Link le pregunta al sector `estaEnLinea(this, link)` y calcula el rumbo con `direccionEntre(getPosicion(), link.getPosicion())`. Los tres métodos están en el extendido y las dos flechas (`#miJuego`, `#jugador`) en el reducido. No hace falta `instanceof`.

```
        // TrampaDeCuchillas (extendido: colisionarCon(o : Obstaculo), volver(); hereda
vigilarJugador() de Enemigo*)
        // en el constructor: setMovimiento(new MovimientoInducido(null, 8));   // quieta
        // en cada vuelta de HiloMundo (actualizarEstado() de EntidadAnimada*):
        //     vigilarJugador();          // de Enemigo*
        //     movimiento.mover(this);    // con dirección null no se mueve
        protected void vigilarJugador() {                          // Enemigo* (esquema, con
la reacción de la trampa)
            Link link = miJuego.getJugador();                      // #miJuego → Juego,
#jugador → Link
            if (miSector.estaEnLinea(this, link)) {
                movimiento.setDireccion(miSector.direccionEntre(getPosicion(),
link.getPosicion()));   // se abalanza
                programar(600, this::volver);                      // de EntidadAnimada*:
vuelve pasado ese tiempo
            }
        }
        public void colisionarCon(Obstaculo o) { volver(); }       // chocó contra una pared
        private void volver() { setPosicion(posicionOriginal); movimiento.setDireccion(null); }
// quieta otra vez
```

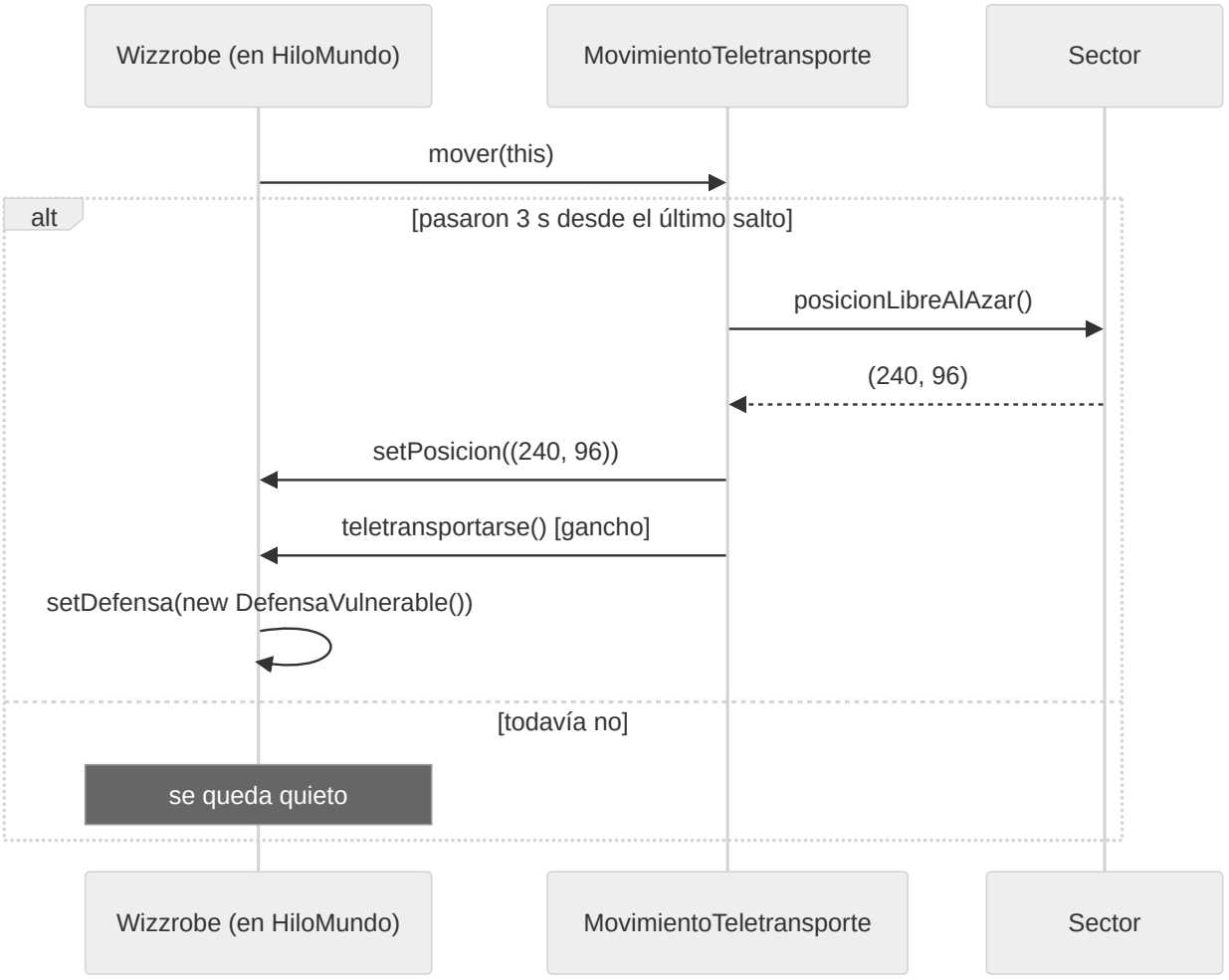
> `movimiento.setDireccion(...)` es un esquema: `#movimiento` es de tipo `ComportamientoMovimiento`, que no declara `setDireccion`. Sin *cast*, la trampa puede guardar su propio `MovimientoInducido` o crear uno nuevo con `setMovimiento(new MovimientoInducido(d, 8))` y `setMovimiento(new MovimientoInducido(null, 8))`.



Explicación: la trampa está quieta porque su `MovimientoInducido` no tiene dirección. En cada vuelta vigila; si Link queda en su misma fila o columna, le da una dirección y se abalanza. Al chocar vuelve a su posición original sin dirección. El Moblin con escudo hace lo mismo, pero partiendo de `MovimientoAleatorio`: ahí sí cambia de **estrategia** en tiempo de ejecución (Aleatorio → Inducido).

`MovimientoTeletransporte`

Qué es: cada `intervalo` ms el Wizzrobe salta a una posición libre al azar. Detalle completo en Teletransporte del Wizzrobe.



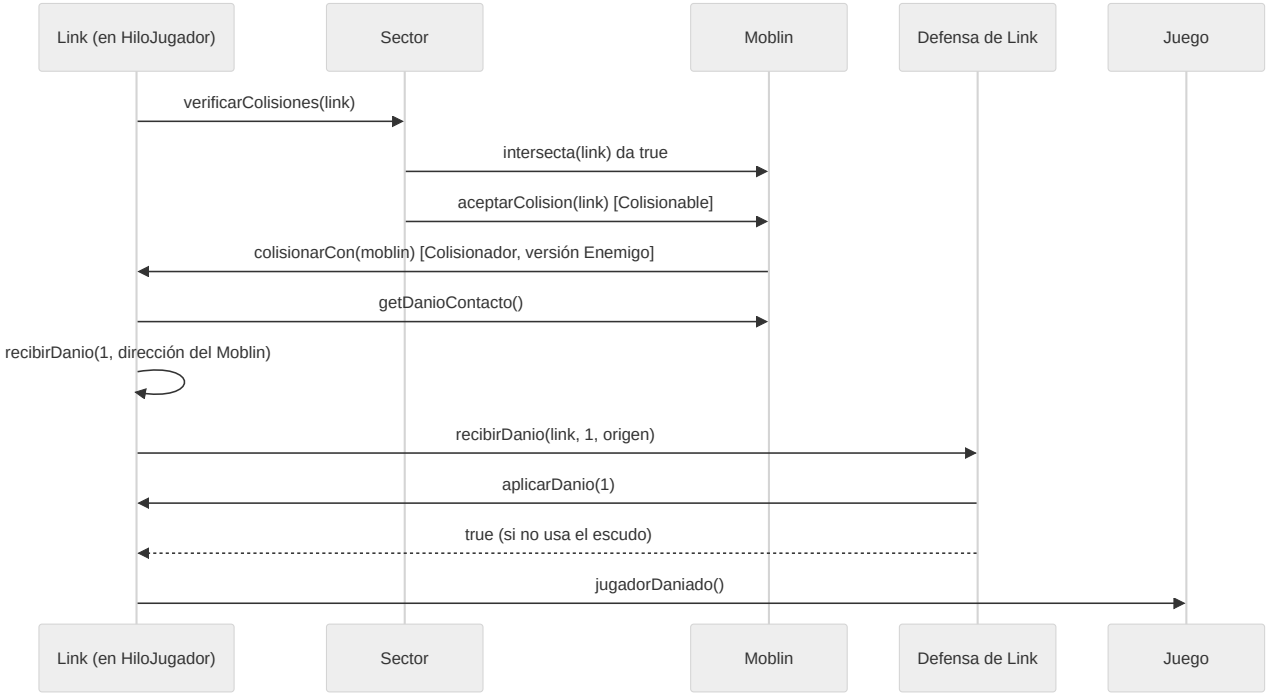
Explicación: entre saltos el Wizzrobe no se mueve. Cuando se cumple el intervalo, la estrategia le pide al sector un lugar libre, lo ubica ahí y le avisa; el Wizzrobe aprovecha para volver a ser vulnerable (si lo habían golpeado, estaba en `DefensaInvulnerable`).

Ataque

`AtaquePorContacto`

Qué es: la entidad **no hace nada activo** para atacar: el daño ocurre cuando toca a Link. La colisión se resuelve por doble despacho y el daño lo define cada enemigo (`getDanioContacto()`). La estrategia existe para que el ciclo sea uniforme (`ataque.atacar(this)` siempre se puede llamar) y para poder **cambiarla** en tiempo de ejecución (por ejemplo, StoneHinox pasa a embestida).

```
        public class AtaquePorContacto implements ComportamientoAtaque {
            public void atacar(EntidadAnimada atacante) { }   // nada activo: el daño ocurre al chocar
        }
```



Explicación: el que se movió es Link; el sector detecta que su rectángulo toca el del Moblin y resuelve el choque por doble despacho (cada uno llama al método específico del otro, sin `instanceof`). El Moblin informa su daño de contacto (½ corazón) y la **defensa de Link** decide si lo recibe: con escudo, no. Si el que se movió es el Moblin, el camino es el simétrico: `link.aceptarColision(moblin)` → `moblin.colisionarCon(link)` (versión `Link` de `Enemigo`) → `link.recibirDanio(...)`.

`AtaqueEspada`

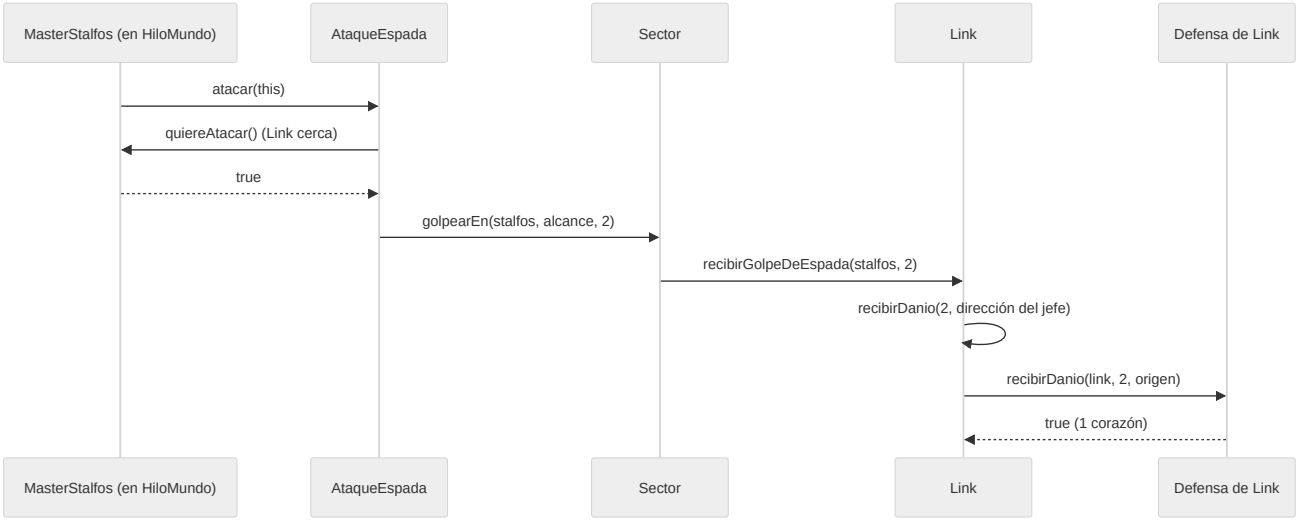
Qué es: un golpe de espada **delante** de la entidad: el sector golpea todo lo que esté a `alcance` píxeles en la dirección del atacante. **Quién la usa:** MasterStalfos (1 corazón) y, a través de `AtaqueRecargado`, Link.

```
        public class AtaqueEspada implements ComportamientoAtaque {
            protected int danio, alcance;
            public AtaqueEspada(int danio, int alcance) { this.danio = danio; this.alcance = alcance; }
            public void atacar(EntidadAnimada a) {
                if (a.soltoElAtaque() || a.quiereAtacar()) golpear(a, danio, alcance);   // ¿quiere atacar? (ver nota)
            }
            protected void golpear(EntidadAnimada a, int d, int alcance) {
                a.getSector().golpearEn(a, alcance, d);   // a cada entidad delante: recibirGolpeDeEspada(a, d)
                a.ataqueRealizado();                      // gancho de EntidadAnimada*: Link baja la marca de "soltó el ataque"
            }
        }

        // Doble despacho del golpe: cada entidad sabe qué hacerle la espada
        // Pasto:   recibirGolpeDeEspada(a, d) -> a.quitarDelSector(this)  (se corta)
        // Enemigo: recibirGolpeDeEspada(a, d) -> recibirDanio(d, a.getDireccion()) -> su
```

```
defensa decide
        // Roca:    recibirGolpeDeEspada(a, d) -> no pasa nada (versión heredada de Entidad)
```

> **Cuándo ataca:** `EntidadAnimada*` declara `soltoElAtaque()`, que Link redefine (es `true` después de soltar A), y la consulta `quiereAtacar() : boolean` (por defecto `false`), que MasterStalfos redefine (Link cerca). Como las dos están en `EntidadAnimada*`, la estrategia las pregunta sin *cast*. Lo mismo con `ataqueRealizado()`: está declarado en `EntidadAnimada*` sin hacer nada y Link lo redefine (observación 13 resuelta).



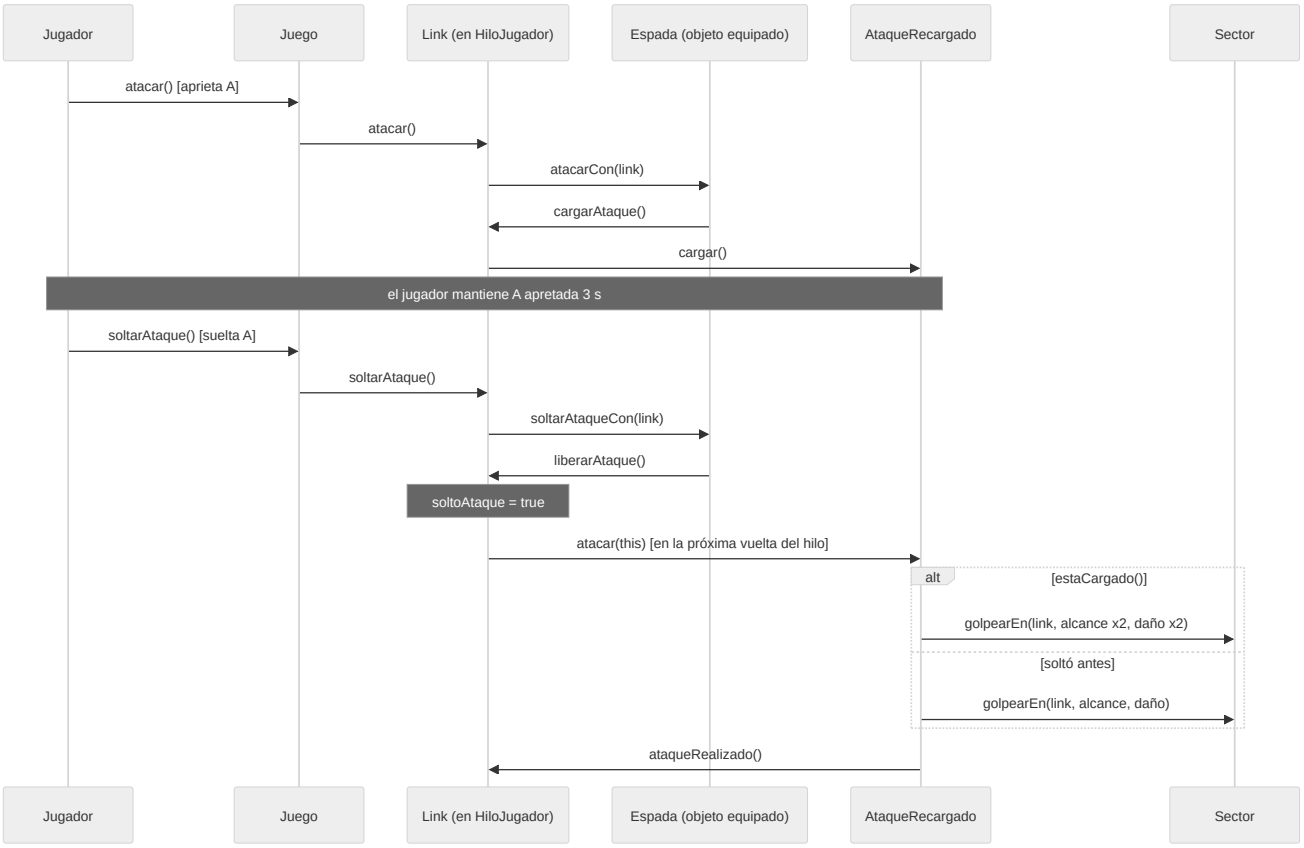
Explicación: la estrategia le pide al sector que golpee delante del atacante y el sector le avisa a cada entidad que está ahí. Cada entidad reacciona a su manera (doble despacho): Link pasa el golpe a **su** defensa, el pasto se corta, la roca no hace nada. Link redefine `recibirGolpeDeEspada` (heredado de `Entidad*`) para que la espada del jefe le haga daño.

`AtaqueRecargado`

Qué es: hereda de `AtaqueEspada`. Si se suelta A **antes** de 3 s, es un golpe normal; si se mantiene **3 s o más** (`TIEMPO_CARGA = 3000`, `estaCargado()`) y se suelta, sale el **ataque recargado** (más daño y más alcance). **Quién la usa:** Link (`recargado : AtaqueRecargado`).

```
        public class AtaqueRecargado extends AtaqueEspada {
            private static final long TIEMPO_CARGA = 3000;
            private long inicioCarga = -1;
            public AtaqueRecargado(int danio, int alcance) { super(danio, alcance); }
            public void cargar()         { inicioCarga = System.currentTimeMillis(); }
// al apretar A
            public boolean estaCargado() { return inicioCarga >= 0 &&
System.currentTimeMillis() - inicioCarga >= TIEMPO_CARGA; }
            public void atacar(EntidadAnimada a) {
                if (!a.soltoElAtaque()) return;                                  // todavía
cargando
                if (estaCargado()) golpear(a, danio * 2, alcance * 2);           // recargado
                else               golpear(a, danio, alcance);                   // normal
(lo del padre)
                inicioCarga = -1;
```

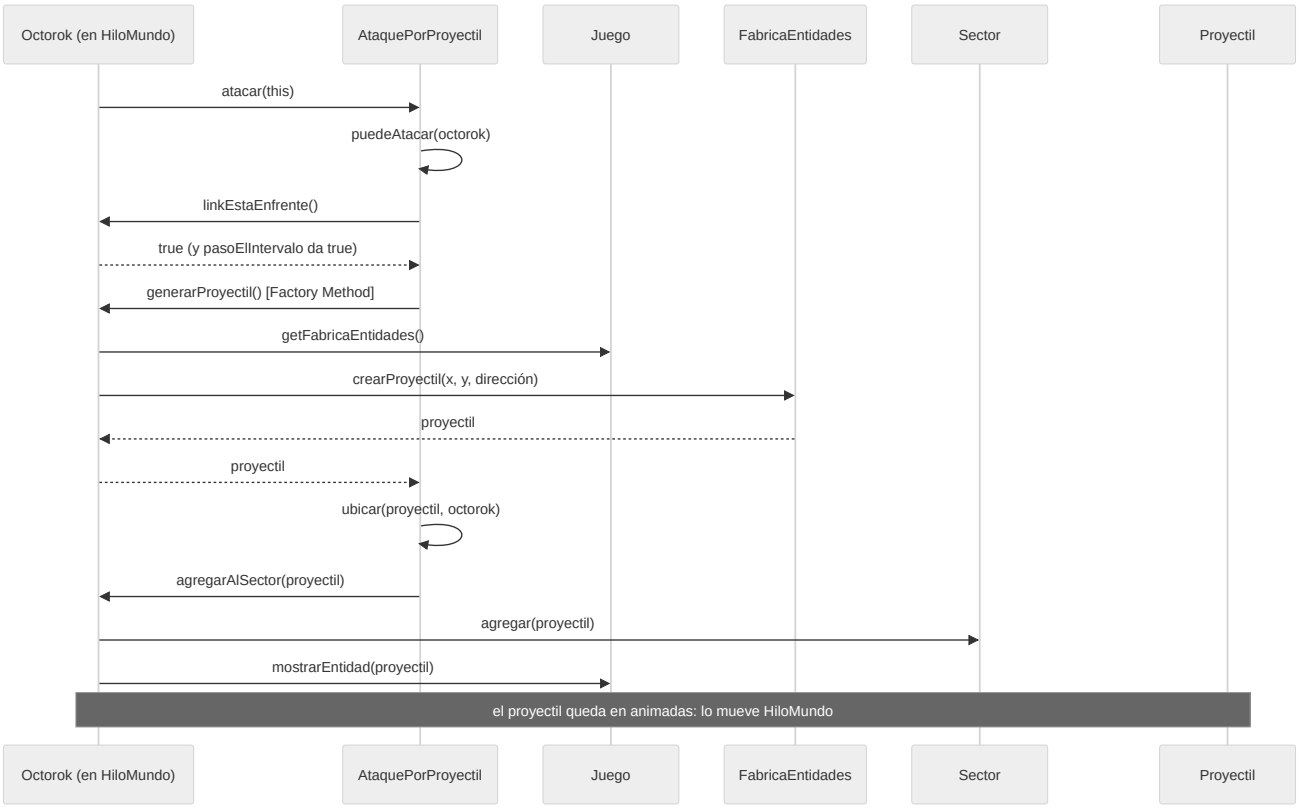
```
            }
        }
```



Explicación: apretar A empieza la carga y soltar A dispara el ataque. Por eso `ControladorJuego` tiene dos mensajes, `atacar()` y `soltarAtaque()`. Link no pregunta qué objeto tiene: se lo pide al objeto equipado (`atacarCon`, `soltarAtaqueCon`), y solo la `Espada` hace algo. La estrategia mide cuánto tiempo pasó y decide qué golpe dar; el golpe normal reutiliza lo que ya hace el padre (`golpear`).

`AtaquePorProyectil`

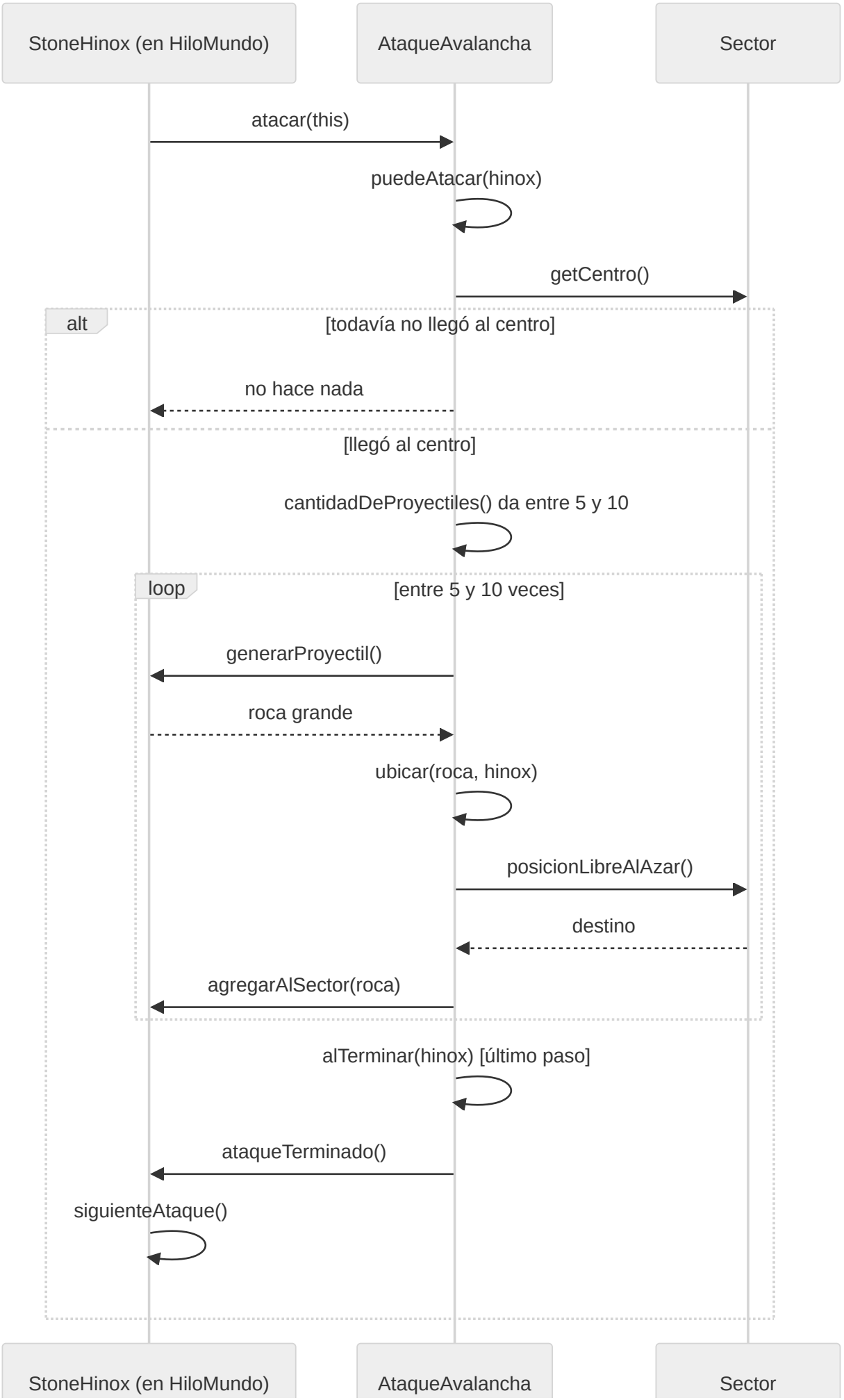
Qué es: el "gatillo" de los disparos: decide **cuándo** y **cuántos** proyectiles; el enemigo decide **qué** proyectil sale (`GeneradorProyectil`). Detalle en Disparo de proyectiles.



Explicación: la estrategia verifica la condición (Link enfrente, intervalo cumplido), le pide al Octorok el proyectil (el Octorok lo fabrica con la fábrica del dominio), lo ubica saliendo hacia Link y lo agrega al sector, donde `HiloMundo` lo mueve desde la vuelta siguiente.

`AtaqueAvalancha`

Qué es: hereda de `AtaquePorProyectil` y redefine cuatro pasos: ataca al llegar al centro, tira entre 5 y 10 rocas, las hace caer en lugares al azar y, al terminar (`alTerminar`), le avisa al StoneHinox. Detalle en Avalancha de StoneHinox.


StoneHinox (en HiloMundo)
AtaqueAvalancha
Sector
atacar(this)
puedeAtacar(hinox)
getCentro()
alt
[todavía no llegó al centro]
no hace nada
[llegó al centro]
cantidadDeProyectiles() da entre 5 y 10
loop
[entre 5 y 10 veces]
generarProyectil()
roca grande
ubicar(roca, hinox)
posicionLibreAlAzar()
destino
agregarAlSector(roca)
alTerminar(hinox) [último paso]
ataqueTerminado()
siguienteAtaque()
StoneHinox (en HiloMundo)
AtaqueAvalancha
Sector

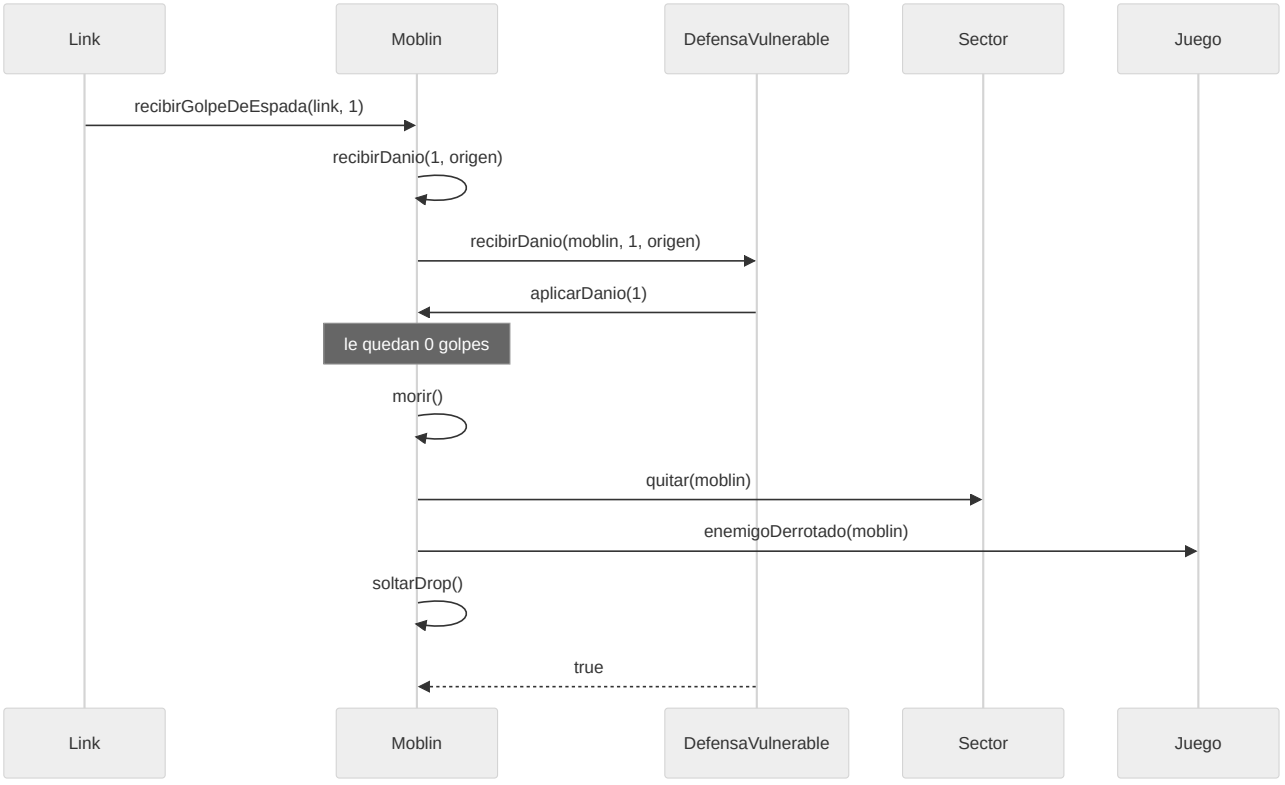
Explicación: mientras StoneHinox camina al centro (`MovimientoInducido`), la avalancha no hace nada. Al llegar, pide las rocas al jefe, elige dónde cae cada una y las agrega al sector. Al terminar, el último paso del esquema (`alTerminar`, vacío en `AtaquePorProyectil` y redefinido acá) avisa al jefe `ataqueTerminado()`, y el jefe elige su próximo ataque cambiando de estrategia.

Defensa

`DefensaVulnerable`

Qué es: la defensa normal: **recibe el daño completo**. Si la entidad se queda sin vida, muere (enemigo) o pierde una vida (Link): eso lo resuelve `aplicarDanio`, que llama al gancho `morir()`.

```
        public class DefensaVulnerable implements ComportamientoDefensa {
            public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) {
                e.aplicarDanio(danio);      // enemigo: resta golpes; Link: resta medios
corazones
                return true;
            }
        }
```



Explicación: el Moblin tenía 1 golpe restante. Su defensa vulnerable aplica el daño; al quedar sin vida, el Moblin muere: suelta su drop (si sale), se saca del sector (su celda desaparece) y avisa a `Juego`, que suma los puntos y la racha.

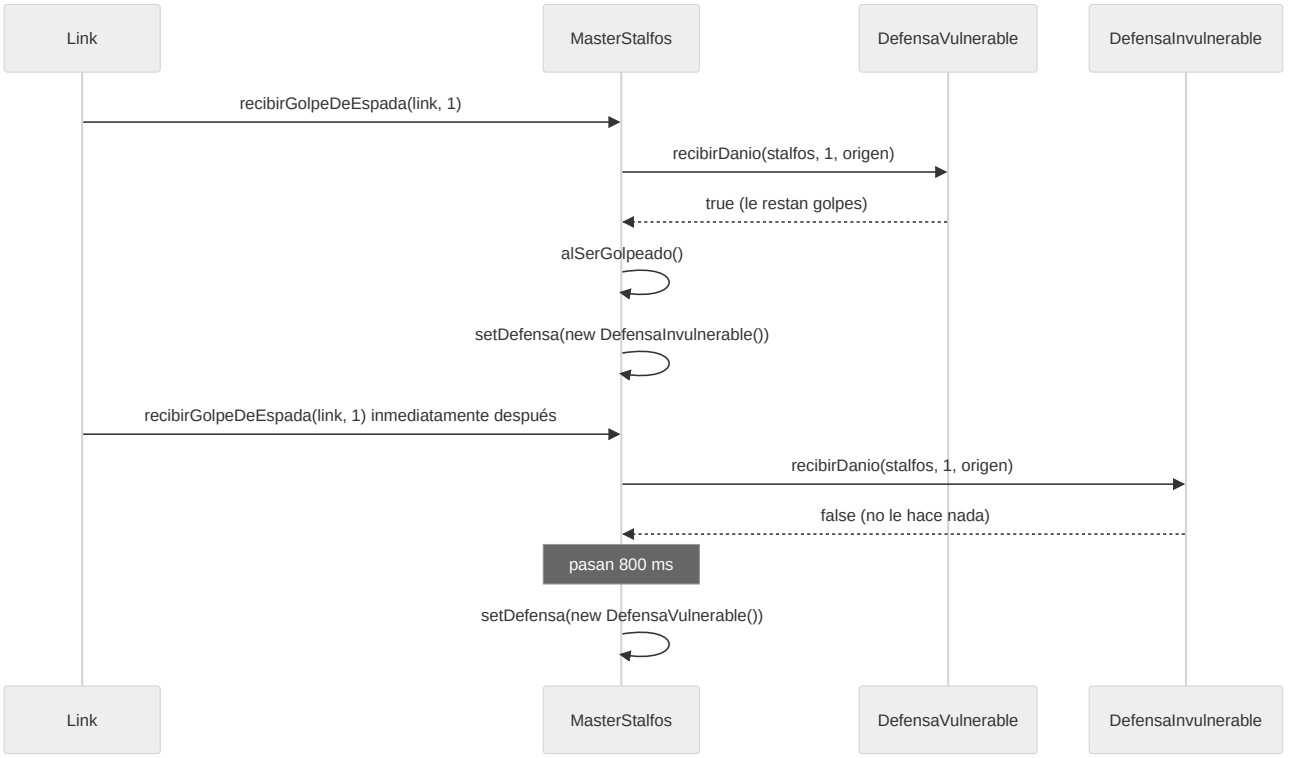
`DefensaInvulnerable`

Qué es: ignora todo daño. Se usa de forma **permanente** (TrampaDeCuchillas, Proyectil) o **temporal** (Wizzrobe hasta su próximo salto, MasterStalfos un rato después de cada golpe,

StoneHinox fuera de su ventana, Link 1 s después de recibir daño).

```
        public class DefensaInvulnerable implements ComportamientoDefensa {
            public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) { return
false; }
        }

        // EntidadAnimada* (extendido: # alSerGolpeado(), gancho que por defecto no hace nada).
        // recibirDanio lo llama cuando el golpe le quitó vida:
        public void recibirDanio(int danio, Direccion origen) {
            if (defensa.recibirDanio(this, danio, origen)) alSerGolpeado();
        }
        // MasterStalfos redefine el gancho:
        protected void alSerGolpeado() {
            setDefensa(new DefensaInvulnerable());
            programar(800, () -> setDefensa(new DefensaVulnerable()));   // de EntidadAnimada*:
pasados 800 ms
        }
```



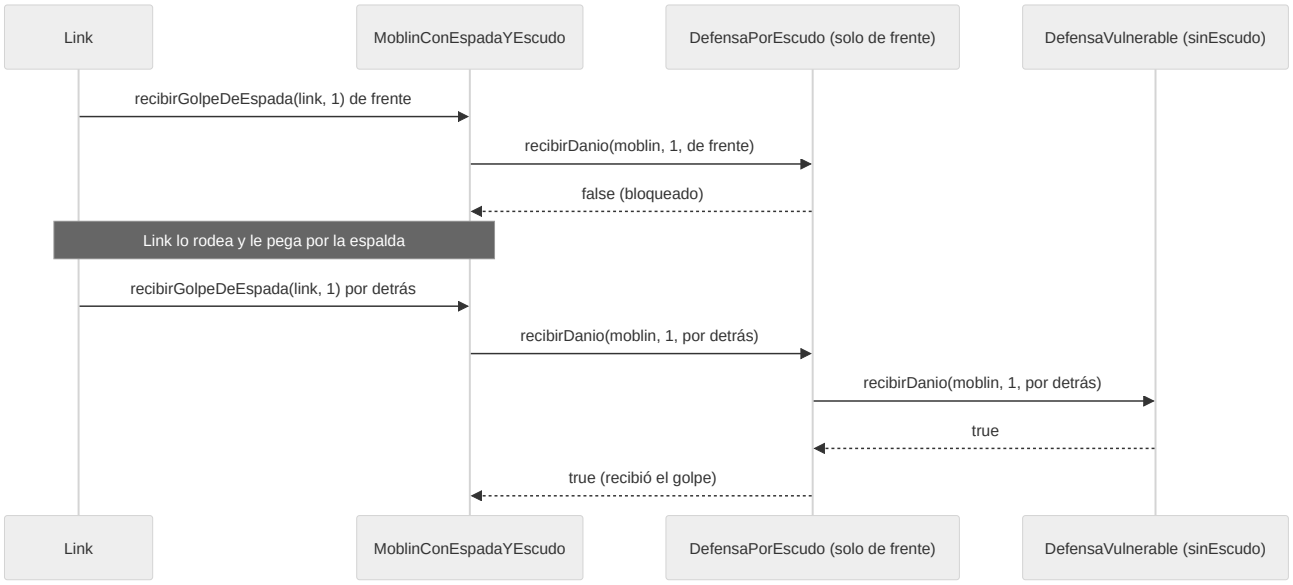
Explicación: el primer golpe entra porque MasterStalfos está vulnerable; al ser golpeado cambia su defensa a invulnerable. El segundo golpe, inmediato, rebota. Pasado el tiempo, vuelve a ser vulnerable. Así se cumple "tras cada golpe es invulnerable un corto período" sin un solo `if` sobre el tipo de enemigo.

`DefensaPorEscudo`

Qué es: bloquea el daño. Tiene dos modos (`soloDeFrente`): **total** (Link con el escudo: no recibe daño de proyectiles ni de contacto, como dice el enunciado) y **solo de frente** (MoblinConEspadaYEscudo: bloquea la espada si lo atacan de frente; por detrás o de costado recibe el golpe). **Relación:** `DefensaPorEscudo → ComportamientoDefensa` (`#sinEscudo 1`, atributo `#`

`sinEscudo : ComportamientoDefensa` del extendido): la defensa a la que delega cuando el escudo no cubre el golpe.

```
        public class DefensaPorEscudo implements ComportamientoDefensa {
            protected boolean soloDeFrente;
            protected ComportamientoDefensa sinEscudo = new DefensaVulnerable();   //
#sinEscudo 1
            public DefensaPorEscudo(boolean soloDeFrente) { this.soloDeFrente = soloDeFrente; }
            public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) {
                boolean bloquea = !soloDeFrente || origen == e.getDireccion().opuesta();   //
el golpe viene de frente
                return bloquea ? false : sinEscudo.recibirDanio(e, danio, origen);
            }
        }
```

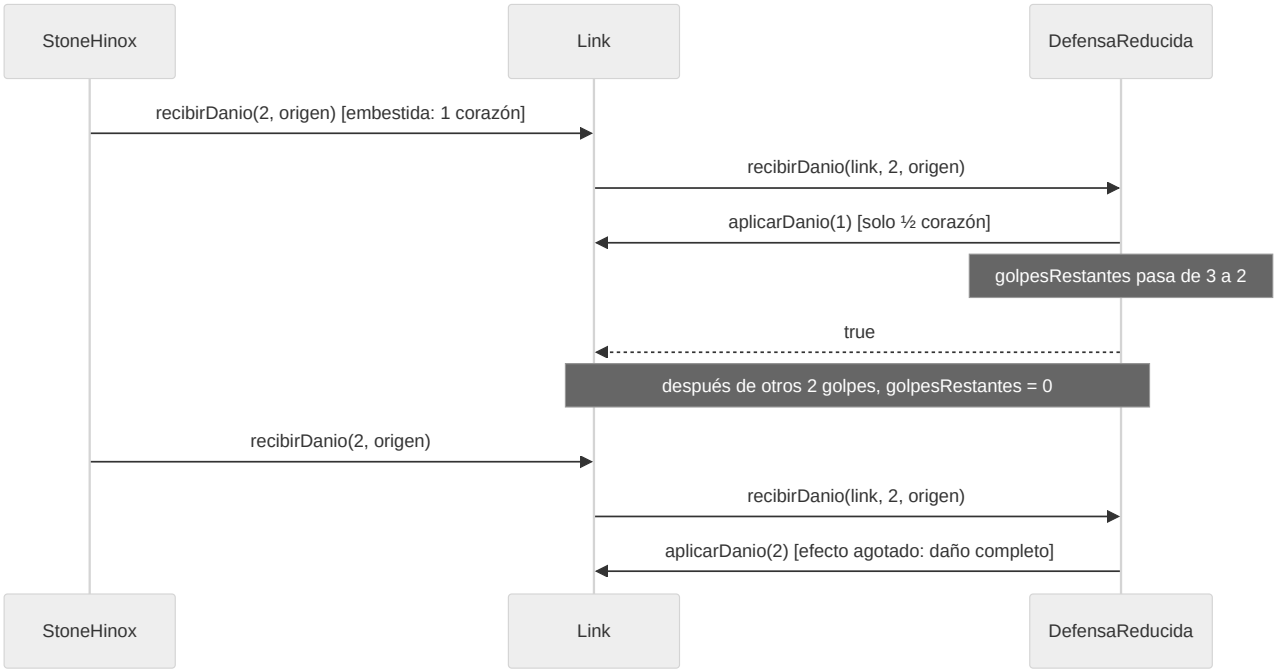


Explicación: el escudo del Moblin solo cubre el frente. Un golpe de frente se bloquea; uno por la espalda pasa a la defensa `#sinEscudo` y le resta un golpe. Con Link es más simple: mientras usa el escudo, su defensa es `DefensaPorEscudo(false)` y bloquea todo; al soltarlo vuelve a su defensa base (ver El escudo y la defensa base).

Por qué `#sinEscudo` es una asociación con `ComportamientoDefensa` y no con `DefensaVulnerable`: se programa contra la interfaz. Hoy `sinEscudo` es una `DefensaVulnerable`, pero podría ser cualquier defensa (por ejemplo, una `DefensaReducida` si un enemigo tuviera escudo y armadura). Es un pequeño **Decorator**: la defensa por escudo envuelve a otra defensa y le pasa los golpes que no cubre.

`DefensaReducida`

Qué es: el efecto de la bellota: los próximos 3 golpes (`golpesRestantes`, `getGolpesRestantes()`) hacen solo ½ corazón. Detalle en Efecto de la bellota.



Explicación: cualquier golpe, aunque valga 1 corazón, le saca solo ½ corazón a Link. La estrategia cuenta los golpes y, al llegar a cero, aplica el daño completo, igual que la defensa normal.

Resumen: quién usa qué

Entidad	Movimiento	Ataque	Defensa
Link	`MovimientoInducido` (teclado, `caminar`)	`AtaqueRecargado` (`recargado`)	`DefensaVulnerable` / `DefensaPorEscudo(false)` / `DefensaReducida` / `DefensaInvulnerable` (1 s tras un golpe)
Moblin	`MovimientoAleatorio`	`AtaquePorContacto`	`DefensaVulnerable`
MoblinConEspadaYEscudo	`MovimientoAleatorio` → `MovimientoInducido` (se abalanza)	`AtaquePorContacto`	`DefensaPorEscudo(true)`
ErizoDeMar	`MovimientoInducido(null, 0)` (quieto)	`AtaquePorContacto`	`DefensaVulnerable`
Octorok	`MovimientoAleatorio`	`AtaquePorProyectil`	`DefensaVulnerable`
TrampaDeCuchillas	`MovimientoInducido` sin dirección → con dirección hacia Link → sin dirección	`AtaquePorContacto`	`DefensaInvulnerable`
Wizzrobe	`MovimientoTeletransporte`	`AtaquePorProyectil`	`DefensaVulnerable` ↔ `DefensaInvulnerable`
MasterStalfos	`MovimientoAleatorio`	`AtaqueEspada`	`DefensaVulnerable` ↔ `DefensaInvulnerable`

Entidad	Movimiento	Ataque	Defensa
StoneHinox	`MovimientoAleatorio` / `MovimientoInducido`	`AtaquePorContacto` / `AtaquePorProyectil` / `AtaqueAvalancha`	`DefensaInvulnerable` ↔ `DefensaVulnerable`
Proyectil	`MovimientoInducido`	`AtaquePorContacto`	`DefensaInvulnerable`

Las flechas (→, ↔) son **cambios de estrategia en tiempo de ejecución**: la esencia del patrón Strategy en este juego.

> Supuestos: las entidades animadas del sector visible las actualiza `HiloMundo` (a Link, `HiloJugador`; ver la sección 16). Los métodos que no están en el extendido de la defensa 4 (el ciclo, `esperar(ms)`) se marcan en los esquemas.

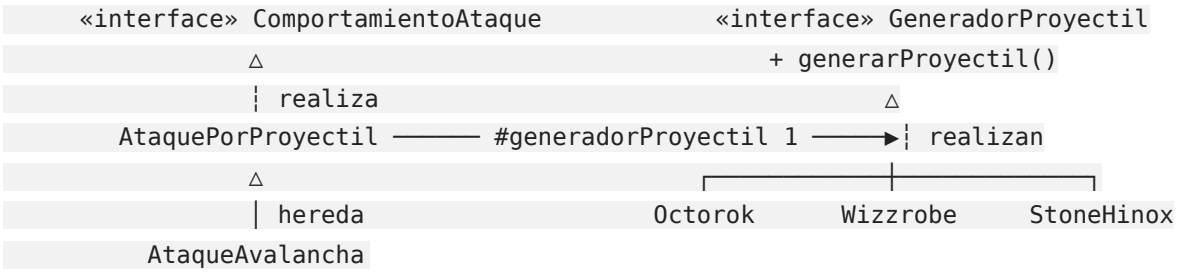
Disparo de proyectiles: `AtaquePorProyectil` + `GeneradorProyectil`

Disparar tiene dos decisiones distintas, y cada una tiene su responsable:

Decisión	Pregunta	Responsable
Cuándo y cómo atacar	¿Está Link enfrente? ¿Pasó el intervalo? ¿Uno o tres proyectiles?	La estrategia `AtaquePorProyectil` (Strategy)
Qué proyectil sale	¿Roca recta? ¿Magia apuntada a Link? ¿Roca grande de 1 corazón?	El enemigo, con `generarProyectil()` (Factory Method)

Analogía: la estrategia es el **gatillo** y el enemigo es el **cargador**. Un mismo gatillo sirve para distintos cargadores.

En el diagrama: `AtaquePorProyectil → GeneradorProyectil` (`#generadorProyectil 1`, asociación unidireccional).



Por qué no hace falta `instanceof`: la estrategia guarda el generador con tipo `GeneradorProyectil` (no `EntidadAnimada`). Al llamar `generadorProyectil.generarProyectil()`, Java ejecuta la versión del enemigo concreto. Por eso a la estrategia se le pasa el generador al construirla (`new AtaquePorProyectil(this, 1, 2000)`): si guardara la entidad como `EntidadAnimada`, tendría que preguntar si "sabe disparar".

Alternativas descartadas:

- *Solo la estrategia* (sin `GeneradorProyectil`): la estrategia tendría que saber qué proyectil crea cada enemigo.

- *Solo la interfaz* (sin `AtaquePorProyectil`): se pierde el Strategy, que es lo que permite que los jefes cambien de ataque en tiempo de ejecución.

Cómo se implementan las clases (esquema)

```
        // El gatillo (con los pasos que las subclases pueden redefinir: Template Method)
        public class AtaquePorProyectil implements ComportamientoAtaque {
            protected GeneradorProyectil generadorProyectil;   // #generadorProyectil 1
            protected int cantidad;                             // proyectiles por ataque (1, o
3 para StoneHinox)
            protected long intervalo, ultimoAtaque;             // milisegundos entre ataques

            public AtaquePorProyectil(GeneradorProyectil g, int cantidad, long intervalo) {
                generadorProyectil = g; this.cantidad = cantidad; this.intervalo = intervalo;
            }
            public void atacar(EntidadAnimada atacante) {                    // esquema fijo
                if (!puedeAtacar(atacante)) return;
                for (int i = 0; i < cantidadDeProyectiles(); i++) {
                    Proyectil p = generadorProyectil.generarProyectil();     // QUÉ sale lo
decide el enemigo
                    ubicar(p, atacante);                                     // DÓNDE y hacia
dónde
                    atacante.agregarAlSector(p);                             // sector + vista
(lo mueve HiloMundo)
                }
                ultimoAtaque = System.currentTimeMillis();
                alTerminar(atacante);                                        // último paso
(gancho)
            }
            // Pasos redefinibles (ver AtaqueAvalancha)
            protected boolean puedeAtacar(EntidadAnimada a) { return pasoElIntervalo() &&
a.linkEstaEnfrente(); }
            protected int cantidadDeProyectiles()            { return cantidad; }
            protected void ubicar(Proyectil p, EntidadAnimada a) {
                p.setPosicion(a.getPosicion());                              // sale desde el
atacante
                p.setDireccion(a.direccionHaciaJugador());                   // hacia Link
            }
            protected void alTerminar(EntidadAnimada atacante) { }          // gancho: por
defecto nada
            protected boolean pasoElIntervalo() { return System.currentTimeMillis() -
ultimoAtaque >= intervalo; }
        }

        // Los cargadores: cada enemigo decide qué proyectil sale
        public class Octorok extends EnemigoConDrop implements GeneradorProyectil {
            public Octorok(Sprite sprite, int x, int y) {
                super(sprite, x, y, 1, 1);                                   //
EnemigoConDrop(sprite, x, y, vida, danioContacto)
                setMovimiento(new MovimientoAleatorio(2, 1500));
                setAtaque(new AtaquePorProyectil(this, 1, 2000));            // se pasa a sí
```

```
mismo
                setDefensa(new DefensaVulnerable());
            }
            public Proyectil generarProyectil() {                           // roca recta
hacia donde mira
                return miJuego.getFabricaEntidades().crearProyectil(getX(), getY(),
getDireccion());
            }
        }
        public class Wizzrobe extends EnemigoConDrop implements GeneradorProyectil {
            public Proyectil generarProyectil() {                           // apuntada a Link
                return miJuego.getFabricaEntidades().crearProyectil(getX(), getY(),
direccionHaciaJugador());
            }
        }
        public class StoneHinox extends Jefe implements GeneradorProyectil {
            public Proyectil generarProyectil() {                           // roca grande: 1
corazón
                return miJuego.getFabricaEntidades().crearRocaGrande(getX(), getY(),
direccionHaciaJugador());
            }
        }

        // Lo que se crea: también es una EntidadAnimada, que mueve HiloMundo
        public class Proyectil extends EntidadAnimada {
            protected int danio;   // en medios corazones
            public void colisionarCon(Link l)     { l.recibirDanio(danio, getDireccion());
desaparecer(); }   // doble despacho
            public void colisionarCon(Obstaculo o) { if (o.bloqueaPaso()) desaparecer(); }
            private void desaparecer() { detener(); quitarDelSector(this); }
        }
```

Ejemplos de recorrido

1. **El Octorok le dispara a Link:** en la vuelta del Octorok, `HiloMundo` llama a `ataque.atacar(this)`. La estrategia verifica que Link está enfrente y que pasó el intervalo, y llama a `generadorProyectil.generarProyectil()`: se ejecuta la versión del Octorok, que pide a la fábrica una roca recta con los sprites del dominio. La estrategia la agrega al sector y `HiloMundo` la mueve desde la vuelta siguiente. Si la roca choca con Link sin escudo, Link recibe ½ corazón de daño y la roca se saca del sector.
2. **StoneHinox lanza tres rocas:** es la misma clase de estrategia con otra configuración (`new AtaquePorProyectil(this, 3, ...)`, guardada en `lanzarRocas`). La estrategia llama tres veces a `generarProyectil()` y StoneHinox crea cada roca grande apuntada a Link. No hace falta una clase nueva.
3. **StoneHinox cambia de ataque en plena pelea:** como es Strategy, alterna `setAtaque(embestida)`, `setAtaque(lanzarRocas)` y `setAtaque(avalancha)` en tiempo de ejecución (`siguienteAtaque()`).
4. **Agregar un enemigo nuevo que dispare:** alcanza con que implemente `GeneradorProyectil` y se arme su estrategia con `new AtaquePorProyectil(this, ...)`. `AtaquePorProyectil` no se toca.

Avalancha de StoneHinox: `AtaqueAvalancha`

StoneHinox (jefe del nivel 2) tiene tres ataques, todos de 1 corazón. El enunciado dice que en la avalancha *"va al centro, golpea el piso y caen entre 5 y 10 rocas en lugares aleatorios"*.

Ataque	Qué hace	Estrategia (atributo de StoneHinox)
Embestida	Embiste por contacto y queda vulnerable un momento (único momento en que recibe daño)	`embestida = new AtaquePorContacto()` (+ `MovimientoInducido` hacia Link)
Lanza rocas	Tres rocas seguidas apuntadas a Link	`lanzarRocas = new AtaquePorProyectil(this, 3, ...)`
Avalancha	Va al centro, golpea el piso y caen 5–10 rocas en lugares al azar	`avalancha = new AtaqueAvalancha(this)` (+ `MovimientoInducido` hacia el centro)

Las rocas de la avalancha **no van hacia Link** y la **cantidad es al azar**: por eso hace falta una estrategia propia. El desplazamiento al centro no necesita estrategia nueva: es un `MovimientoInducido` con el centro como objetivo.

Por qué hereda de `AtaquePorProyectil`

Pasa la prueba del "es un": **una avalancha es un ataque por proyectiles** en el que los proyectiles caen en lugares al azar. Comparte con su padre el generador y la forma de agregar los proyectiles al sector; cambian cuatro pasos (en el extendido de la defensa 4, `AtaqueAvalancha` redefine exactamente `puedeAtacar(a)`, `cantidadDeProyectiles()`, `ubicar(p, a)` y `alTerminar(atacante)`, y agrega `azar : Random`):

Paso	`AtaquePorProyectil`	`AtaqueAvalancha`
`puedeAtacar(a)`	Pasó el intervalo y Link está enfrente	StoneHinox llegó al centro del sector
`cantidadDeProyectiles()`	Fija (1, o 3 en "lanza rocas")	Al azar, entre 5 y 10
`ubicar(p, a)`	Sale del atacante hacia Link	Empieza arriba de un lugar libre al azar y cae
`alTerminar(atacante)`	Gancho vacío	Le avisa al StoneHinox `ataqueTerminado()`

`AtaquePorProyectil.atacar()` fija el esquema y deja esos pasos para redefinir: es el patrón **Template Method**. Es el mismo tipo de herencia que `AtaqueRecargado ⟶▷ AtaqueEspada`. En el diagrama: `AtaqueAvalancha ⟶▷ AtaquePorProyectil`; hereda `#generadorProyectil` y la realización de `ComportamientoAtaque`, así que no necesita otras flechas.

Implementación de las clases involucradas (esquema)

```
        public class AtaqueAvalancha extends AtaquePorProyectil {
            private final Random azar = new Random();
            public AtaqueAvalancha(GeneradorProyectil g) { super(g, 0, 0); }

            protected boolean puedeAtacar(EntidadAnimada a) {                       // llegó al centro
                return a.getPosicion().distancia(a.getSector().getCentro()) < 4;
```

```
            }
            protected int cantidadDeProyectiles() { return 5 + azar.nextInt(6); }   // 5 a 10
            protected void ubicar(Proyectil roca, EntidadAnimada a) {
                Posicion destino = a.getSector().posicionLibreAlAzar();            // dónde va
a caer
                roca.setPosicion(new Posicion(destino.getX(), 0));                 // empieza
arriba de todo
                roca.setDireccion(Direccion.ABAJO);
                roca.setDestino(destino);                                          // al
llegar, cae y desaparece
            }
            protected void alTerminar(EntidadAnimada atacante) {                    // la
avalancha solo la usa StoneHinox
                atacante.ataqueTerminado();       // gancho de EntidadAnimada*, lo redefine
StoneHinox (sin cast)
            }
        }

        public class StoneHinox extends Jefe implements GeneradorProyectil {
            private ComportamientoAtaque embestida, lanzarRocas, avalancha;
            private ComportamientoMovimiento caminar;
            private final Random azar = new Random();

            public StoneHinox(Sprite sprite, int x, int y) {
                super(sprite, x, y, 10);                                  // Jefe(sprite, x, y,
vida): 10 golpes
                embestida   = new AtaquePorContacto();
                lanzarRocas = new AtaquePorProyectil(this, 3, 0);
                avalancha   = new AtaqueAvalancha(this);
                caminar     = new MovimientoAleatorio(2, 1500);
                setDefensa(new DefensaInvulnerable());                // solo vulnerable después
de embestir
            }
            public void iniciar() { siguienteAtaque(); super.iniciar(); }   // ya tiene
#miJuego y #miSector

            private void siguienteAtaque() {                            // Strategy en tiempo de
ejecución
                switch (azar.nextInt(3)) {
                    case 0: setMovimiento(new
MovimientoInducido(miJuego.getJugador().getPosicion(), 6));
                            setAtaque(embestida);
                            programar(1500, this::embestidaTerminada);                  // de
EntidadAnimada*
                            break;
                    case 1: setMovimiento(caminar);
                            setAtaque(lanzarRocas); break;
                    case 2: setMovimiento(new MovimientoInducido(miSector.getCentro(), 3));
// primero va al centro...
                            setAtaque(avalancha);   break;
// ...la avalancha espera a llegar
```

```
                }
            }
            public void ataqueTerminado()   { siguienteAtaque(); }
            public void embestidaTerminada() {                      // vulnerable un momento, después otro ataque
                setDefensa(new DefensaVulnerable());
                programar(1000, () -> { setDefensa(new DefensaInvulnerable()); siguienteAtaque(); });
            }
        }
```

> **Quién llama a `ataqueTerminado()`:** el último paso del Template Method, `# alTerminar(atacante : EntidadAnimada)`. En `AtaquePorProyectil` es un gancho vacío y `AtaqueAvalancha` lo redefine: la avalancha solo la usa StoneHinox, así que le avisa `ataqueTerminado()`. Como `alTerminar` recibe una `EntidadAnimada`, `ataqueTerminado()` está declarado como gancho vacío en `EntidadAnimada*` y `StoneHinox` lo redefine: se llama sin *cast* (observación 27, resuelta). Como `lanzarRocas` es un `AtaquePorProyectil` común (con el `alTerminar` vacío), el paso al siguiente ataque después de las rocas lo puede agendar `siguienteAtaque` con `programar(…)`, como hace con la embestida. Ver Observaciones.

Recorrido de una avalancha

1. `siguienteAtaque()` sale `case 2`: movimiento hacia el centro y ataque `avalancha`.
2. En cada vuelta de `HiloMundo`, el `mover()` de StoneHinox lo acerca al centro y `atacar()` no hace nada porque `puedeAtacar()` da `false`.
3. Al llegar, `puedeAtacar()` da `true`: la vista muestra el golpe al piso (Observer) y `cantidadDeProyectiles()` da, por ejemplo, 7.
4. Siete veces: StoneHinox fabrica una roca grande (`generarProyectil()`), `ubicar()` la pone arriba de un lugar libre al azar con dirección hacia abajo y `setDestino(destino)`, y `agregarAlSector()` la agrega al sector y a la vista; `HiloMundo` la empieza a mover.
5. El último paso, `alTerminar(hinox)` (redefinido en `AtaqueAvalancha`), avisa `ataqueTerminado()` → `siguienteAtaque()`.
6. Cada roca baja con `HiloMundo`; al llegar a su destino, si Link está ahí (sin escudo) recibe 1 corazón. Después desaparece (`morir()` / `desaparecer()`).

Patrones que se ven

Patrón	Dónde
Strategy	StoneHinox cambia movimiento, ataque y defensa en tiempo de ejecución
Template Method	`AtaquePorProyectil.atacar()` fija el esquema; `AtaqueAvalancha` redefine cuatro pasos (`puedeAtacar`, `cantidadDeProyectiles`, `ubicar` y el gancho `alTerminar`)
Factory Method	`StoneHinox.generarProyectil()` decide qué roca sale
Abstract Factory	`FabricaEntidades` / `FabricaSprites`: sprites de la roca según el dominio
Doble despacho	`Proyectil.colisionarCon(Link)` / `colisionarCon(Obstaculo)`, sin `instanceof`

Decisiones a confirmar

1. **Cómo caen las rocas:** cada una baja desde el borde superior hasta un lugar libre al azar. Alternativa: primero una sombra en el piso y después la roca.
2. **Orden de los ataques:** al azar entre los tres. Alternativa: en ciclo (embestida → rocas → avalancha).
3. **Golpes de StoneHinox:** 10 (el enunciado solo dice que es el que más necesita; MasterStalfos necesita ≥ 5).
4. **"Golpea el piso":** animación y sonido de la vista, avisados por el Observer; no es una clase.

Teletransporte del Wizzrobe: `MovimientoTeletransporte`

El Wizzrobe no camina como los demás: **desaparece de donde está y aparece de golpe en otro lugar** del sector. El enunciado dice: *"se teletransporta cada pocos segundos a un lugar aleatorio y lanza un proyectil a Link. Tras recibir un golpe es invulnerable 1 s hasta su próxima teletransportación"*.

Comportamiento	Estrategia
Cambiar de lugar cada pocos segundos, a un punto al azar	**Movimiento:** `MovimientoTeletransporte(intervalo)`
Disparar un proyectil a Link	**Ataque:** `AtaquePorProyectil` (con `#generadorProyectil` = el propio Wizzrobe)
Invulnerable al ser golpeado, hasta la próxima teletransportación	**Defensa:** alterna `DefensaVulnerable` / `DefensaInvulnerable`

Por qué es una estrategia de movimiento propia y no hereda de `MovimientoAleatorio`

Ninguna de las existentes sirve: `MovimientoAleatorio` **camina** paso a paso al azar, `MovimientoInducido` va hacia un objetivo o según el teclado. El Wizzrobe **salta** sin recorrer el camino.

	`MovimientoAleatorio` (Moblin)	`MovimientoTeletransporte` (Wizzrobe)
Cómo se mueve	Camina paso a paso	Salta: desaparece y aparece
Qué elige al azar	La dirección del próximo paso	La posición de destino
Recorre el camino	Sí (puede chocar en el trayecto)	No: aparece directo en el destino
Cada cuánto actúa	Un paso en cada ciclo	Cada `intervalo` ms; entre saltos queda quieto

Si heredara, tendría que reescribir todo `mover()` sin reutilizar nada del padre. Distinto es `AtaqueRecargado extends AtaqueEspada`: un ataque recargado **es** un ataque con espada al que se le agrega la carga. El código común (elegir una posición libre al azar) se comparte con `Sector.posicionLibreAlAzar()`, sin herencia.

Cómo se implementa (esquema)

```
        public class MovimientoTeletransporte implements ComportamientoMovimiento {
            protected long intervalo, ultimo;            // extendido: intervalo, ultimo
            public MovimientoTeletransporte(long intervalo) { this.intervalo = intervalo; }
            public void mover(EntidadAnimada e) {
```

```
                long ahora = System.currentTimeMillis();
                if (ahora - ultimo >= intervalo) {
                    e.setPosicion(e.getSector().posicionLibreAlAzar());   // el sector sabe qué
lugares están libres
                    e.teletransportarse();                                // gancho declarado
en EntidadAnimada*
                    ultimo = ahora;
                }
            }
        }

        public class Wizzrobe extends EnemigoConDrop implements GeneradorProyectil {
            public Wizzrobe(Sprite sprite, int x, int y) {
                super(sprite, x, y, 2, 1);                                // 2 golpes, contacto
½ corazón
                setMovimiento(new MovimientoTeletransporte(3000));        // salta cada 3 s
                setAtaque(new AtaquePorProyectil(this, 1, 3000));         // dispara a Link
                setDefensa(new DefensaVulnerable());
            }
            public void recibirDanio(int danio, Direccion origen) {       // invulnerable
después de un golpe...
                super.recibirDanio(danio, origen);
                setDefensa(new DefensaInvulnerable());
            }
            public void teletransportarse() { setDefensa(new DefensaVulnerable()); }   //
...hasta el próximo salto
        }
```

Para que funcione sin `instanceof`, `teletransportarse()` está declarado en `EntidadAnimada*` sin hacer nada (`+ teletransportarse()` en el extendido de la defensa 4), y solo el Wizzrobe lo redefine.

Ejemplo

1. El Wizzrobe está en (528, 144) (sector (0,1) del nivel 2). Su `AtaquePorProyectil` le dispara una magia a Link.
2. Link lo golpea: le queda 1 golpe y pasa a `DefensaInvulnerable`.
3. Link vuelve a pegarle enseguida: no le hace nada.
4. Pasan los 3 s: `MovimientoTeletransporte` lo lleva a un lugar libre al azar y llama al gancho → vuelve a `DefensaVulnerable`. Al aparecer, vuelve a disparar.
5. Link lo alcanza y lo golpea: muere, suelta una rupia o un corazón (100%) y suma 500 puntos.

Decisiones a confirmar

1. **"Cada pocos segundos":** 3 s, como parámetro del constructor.
2. **"Lugar aleatorio":** una posición **libre** (sin rocas, agua ni otras entidades) y no encima de Link (`Sector.posicionLibreAlAzar()`).
3. **"Invulnerable 1 s hasta su próxima teletransportación":** se interpreta como que deja de ser invulnerable al teletransportarse.
4. **Efecto visual** (desaparecer y aparecer): lo dibuja la vista, cuando la celda recibe el aviso del Observer.

Efecto de la bellota: `DefensaReducida`

`DefensaReducida` es una **estrategia de defensa** que representa el efecto de la `Bellota`. El enunciado dice: *"reduce a ½ corazón el daño de los próximos 3 golpes. Aparece tras matar 4 enemigos consecutivos sin ser dañado"*.

Por qué es una estrategia: la defensa de una entidad es justamente **cómo recibe el daño**, y eso es lo que cambia con la bellota. Como el Strategy permite cambiar la defensa en tiempo de ejecución, el efecto es simplemente cambiarle la defensa a Link por un rato, sin agregar un `if (tieneBellota)` en cada golpe.

Estrategia	Cómo recibe un golpe	Ejemplo
`DefensaVulnerable`	Daño completo	Espada de MasterStalfos: 1 corazón
`DefensaInvulnerable`	Sin daño	Wizzrobe después de ser golpeado
`DefensaPorEscudo`	Bloquea golpes y proyectiles	Link con el escudo activo
`DefensaReducida`	Solo ½ corazón, durante 3 golpes	Link después de agarrar una bellota

Quién controla la bellota

Quién	Qué controla	Por qué él
`Link`	La **racha**: enemigos muertos seguidos sin recibir daño (`racha`, `incrementarRacha()`, `reiniciarRacha()`)	Es quien se entera cuando recibe daño, que corta la racha
`Juego`	**Cuándo aparece** la bellota y la crea (`aparecerBellota(x, y)`)	Es una regla de partida, como los puntos; ya recibe `enemigoDerrotado`, conoce a Link (`#jugador`) y a la fábrica
`Bellota`	Su **efecto** al ser recogida (`serRecogidoPor(l)`)	Como cualquier power-up: le cambia la defensa a Link

Paso a paso

```
        // 1. Muere un enemigo (EnemigoConDrop.morir): lo de todo enemigo y después el drop
        super.morir();                      // Enemigo.morir(): detener(), quitarDelSector(this),
miJuego.enemigoDerrotado(this)
        soltarDrop();

        // 2. Juego suma puntos y controla la racha
        public void enemigoDerrotado(Enemigo e) {
            jugador.sumarPuntos(e.getPuntos());
            jugador.incrementarRacha();
            if (jugador.getRacha() == 4) {
                aparecerBellota(e.getX(), e.getY());  // donde murió el cuarto enemigo
                jugador.reiniciarRacha();
            }
        }
        private void aparecerBellota(int x, int y) {
            Bellota b = fabricaEntidades.crearBellota(x, y);   // sprites del dominio +
#miJuego
            jugador.getSector().agregar(b);                    // el sector activo es el de
```

```
Link
            mostrarEntidad(b);                                  // la vista crea su celda
        }

        // 3. Si Link recibe daño, la racha se corta (Link redefine recibirDanio)
        public void recibirDanio(int danio, Direccion origen) {
            if (defensa.recibirDanio(this, danio, origen)) {   // la estrategia decide cuánto
daño aplica
                reiniciarRacha();                               // "consecutivos sin ser dañado"
                miJuego.jugadorDaniado();                       // por ejemplo, para NoHit
            }
        }

        // 4. Link recoge la bellota (doble despacho: link.colisionarCon(p : PowerUP) ->
p.serRecogidoPor(this))
        public void serRecogidoPor(Link l) {
            l.setDefensaBase(new DefensaReducida());   // próximos 3 golpes: ½ corazón
            l.powerUpRecogido(this);                   // Link avisa a Juego, que suma los 300
puntos
            l.quitarDelSector(this);
        }

        // 5. La estrategia
        public class DefensaReducida implements ComportamientoDefensa {
            protected int golpesRestantes = 3;
            public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) {
                if (golpesRestantes > 0) { e.aplicarDanio(1); golpesRestantes--; }   // siempre
½ corazón
                else e.aplicarDanio(danio);                                          //
agotada: daño completo
                return true;
            }
            public int getGolpesRestantes() { return golpesRestantes; }
        }
```

Cuándo vuelve Link a `DefensaVulnerable`: la estrategia recibe una `EntidadAnimada` y `setDefensaBase` es de `Link`, así que no puede devolverle la defensa base sin *cast*. El esquema lo resuelve sin devolverla: cuando se agota, `DefensaReducida` aplica el daño completo, es decir, **se comporta igual que `DefensaVulnerable`**, y queda como defensa base hasta la próxima bellota (que crea una nueva con 3 golpes). Si el equipo prefiere que Link vuelva explícitamente a `DefensaVulnerable`, la estrategia puede llamar a `e.restaurarDefensa()` sin *cast*, porque el extendido de la defensa 4 lo declara en `EntidadAnimada*` (Link lo redefine); haría falta que antes la defensa base de Link vuelva a ser una `DefensaVulnerable`.

Con el escudo activo, `DefensaPorEscudo` bloquea el golpe y devuelve `false`: no hubo daño, así que **la racha no se corta**.

Ejemplo completo

1. Link mata un Moblin, un ErizoDeMar, un Octorok y otro Moblin sin recibir daño → racha 4 → `Juego` crea una bellota donde murió el último Moblin y la racha vuelve a 0.
2. Link la recoge → `DefensaReducida` (3 golpes) y +300 puntos.

3. Un Moblin lo golpea (normalmente ½ corazón) → ½ corazón. Quedan 2.
4. StoneHinox lo embiste (normalmente 1 corazón) → **solo ½ corazón**. Queda 1.
5. Un proyectil lo golpea → ½ corazón. Se termina el efecto.
6. El próximo golpe ya hace el daño completo.

El escudo y la defensa base

Mientras Link usa el escudo, su defensa es `DefensaPorEscudo`; al soltarlo tiene que volver a la que tenía antes, que puede ser la reducida. Por eso Link guarda una **defensa base** (`defensaBase`, `setDefensaBase(d)`, `restaurarDefensa()`, `escudoActivo`):

```
        // En Link
        protected ComportamientoDefensa defensaBase = new DefensaVulnerable();
        public void setDefensaBase(ComportamientoDefensa d) { defensaBase = d; if (!escudoActivo) setDefensa(d); }
        public void restaurarDefensa() { escudoActivo = false; setDefensa(defensaBase); }   // redefine el gancho
        public void activarEscudo()    { escudoActivo = true; setDefensa(new DefensaPorEscudo(false)); }
        public void desactivarEscudo() { restaurarDefensa(); }
        // Escudo.protegerCon(l): l.activarEscudo()
        // Escudo.soltarProteccionCon(l): l.desactivarEscudo()
```

Así, si agarra la bellota con el escudo levantado, el efecto no se pierde: queda como defensa base y se aplica al soltar el escudo.

Decisiones tomadas (a confirmar)

1. **Dónde aparece:** donde murió el cuarto enemigo (como un drop). El enunciado no lo dice.
2. **Qué enemigos cuentan:** todos los que mueren. La `TrampaDeCuchillas` es indestructible y nunca suma.
3. **El drop y la bellota no se excluyen:** el cuarto enemigo puede soltar una rupia y además aparecer la bellota.
4. **Bellota con el efecto todavía activo:** el contador vuelve a 3 (se crea una `DefensaReducida` nueva).

En el diagrama solo está `DefensaReducida ⇢ ComportamientoDefensa`. No hace falta ninguna relación nueva: `Juego` ya tiene `#jugador` y `#fabricaEntidades`; la racha y la defensa base son atributos de Link.

---

11. Funcionamiento de la partida

Temas que atraviesan varias clases: quién lleva y modifica el puntaje, cómo se maneja el tiempo y las reglas de cada modo, cómo se comunican `Juego`, los sectores y las entidades, y cómo se crean, asignan y eliminan las entidades durante la partida.

Puntaje: quién lo lleva y quién lo modifica

Responsabilidad	Quién
Guardar el puntaje y mostrarlo en el HUD (Observer)	`Link` (`puntaje`, `getPuntaje()`)
Cuántos puntos vale cada cosa	Cada entidad: `getPuntos()` (Moblin 100, MoblinConEspadaYEscudo 300, ErizoDeMar 50, Octorok 150, Wizzrobe 500, jefes 5000, power-up 300, rupia 10 por unidad; el resto 0)
Sumar o resetear el puntaje	**Solo** `Juego` (`jugador.sumarPuntos(p)`, `jugador.reiniciarPuntaje()`)

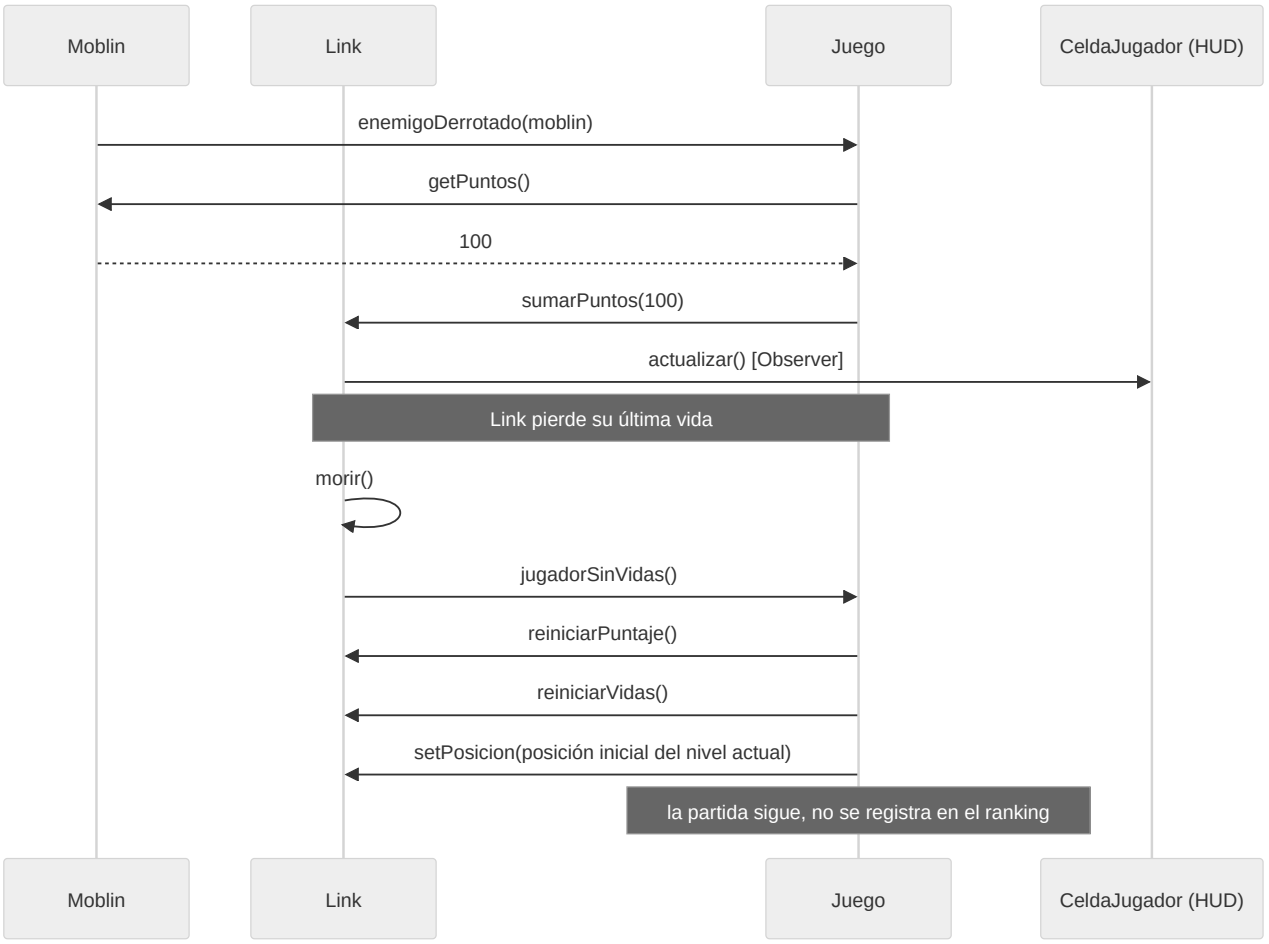
Con un único lugar que modifica el puntaje no hay sumas duplicadas ni olvidadas, el modo de juego puede intervenir si hace falta, y es fácil asegurar que la información no se pierda. Los power-ups no suman directo: Link avisa a `Juego` con `powerUpRecogido(p)`.

Perder todas las vidas (respuesta del ayudante): la partida **no termina**. Link vuelve a su posición inicial en el **nivel actual**, con **puntaje 0** y 3 vidas. Por eso **no se registra en el ranking** en ese momento: el ranking se registra solo cuando la partida termina, y eso lo decide el `ModoJuego`.

```
        public abstract class Entidad    { public int getPuntos() { return 0; } }
        public class Moblin extends EnemigoConDrop  { public int getPuntos() { return 100; } }
        public class MasterStalfos extends Jefe     { public int getPuntos() { return 5000; } }
        public abstract class PowerUP extends Entidad { public int getPuntos() { return 300; } }
}
        public class Rupia extends PowerUP          { public int getPuntos() { return 10 *
cantidad; } }

        // Link: solo guarda y notifica (lo llama Juego)
        public void sumarPuntos(int p) { puntaje += p; notificarObservers(); }
        public void reiniciarPuntaje() { puntaje = 0;  notificarObservers(); }

        // Juego: único que modifica
        public void enemigoDerrotado(Enemigo e) { jugador.sumarPuntos(e.getPuntos()); /* racha,
bellota... */ }
        public void powerUpRecogido(PowerUP p)  { jugador.sumarPuntos(p.getPuntos()); }
        public void jefeDerrotado(Jefe j) {
            jugador.sumarPuntos(j.getPuntos());
            if (!nivel.jefesDerrotados()) return;
            if (modoJuego.esUltimoNivel(numeroNivel)) terminarPartida();
            else { numeroNivel++; cargarNivel();
controladorGrafico.actualizarNivel(numeroNivel); }
        }
        public void jugadorSinVidas() {
            jugador.reiniciarPuntaje();                          // vuelve a 0
            jugador.reiniciarVidas();                            // vuelve a 3
            // vuelve a la posición inicial del nivel actual (mismo procedimiento que en
cargarNivel)
        }
```



Explicación: el Moblin no suma puntos: avisa que murió. `Juego` le pregunta cuánto vale y se lo suma a Link, que notifica al HUD. Cuando Link pierde su última vida, `Juego` lo reinicia como indicó el ayudante y la partida continúa.

Tiempo y reglas del modo

`Juego` lleva el tiempo con su `Reloj` y consulta al modo en dos momentos: cuando pasa el tiempo y cuando Link recibe daño. Cada modo redefine solo el gancho que le importa:

Momento	Método de `Juego`	Gancho de `ModoJuego*`	Quién lo redefine	Qué hace
Pasa un segundo	`tic(segundos)` (público: lo llama el `Reloj`)	`alPasarTiempo(j, segundos)`	`Contrarreloj`	Si `segundos >= tiempoLimite`, termina la partida (o lo que se decida con el ayudante)
Link recibe daño	`jugadorDaniado()`	`alRecibirDanio(j)`	`NoHit`	Termina la partida con el primer daño
Se vence al jefe	`jefeDerrotado(j)`	`esUltimoNivel(numero)`	(común)	Pasa al nivel siguiente o termina la partida

```
        // Juego
        public void tic(int segundos) {                         // lo llama Reloj.ciclo() por
#juego
            controladorGrafico.actualizarTiempo(segundos);      // HUD
            modoJuego.alPasarTiempo(this, segundos);            // solo Contrarreloj hace algo
        }
```

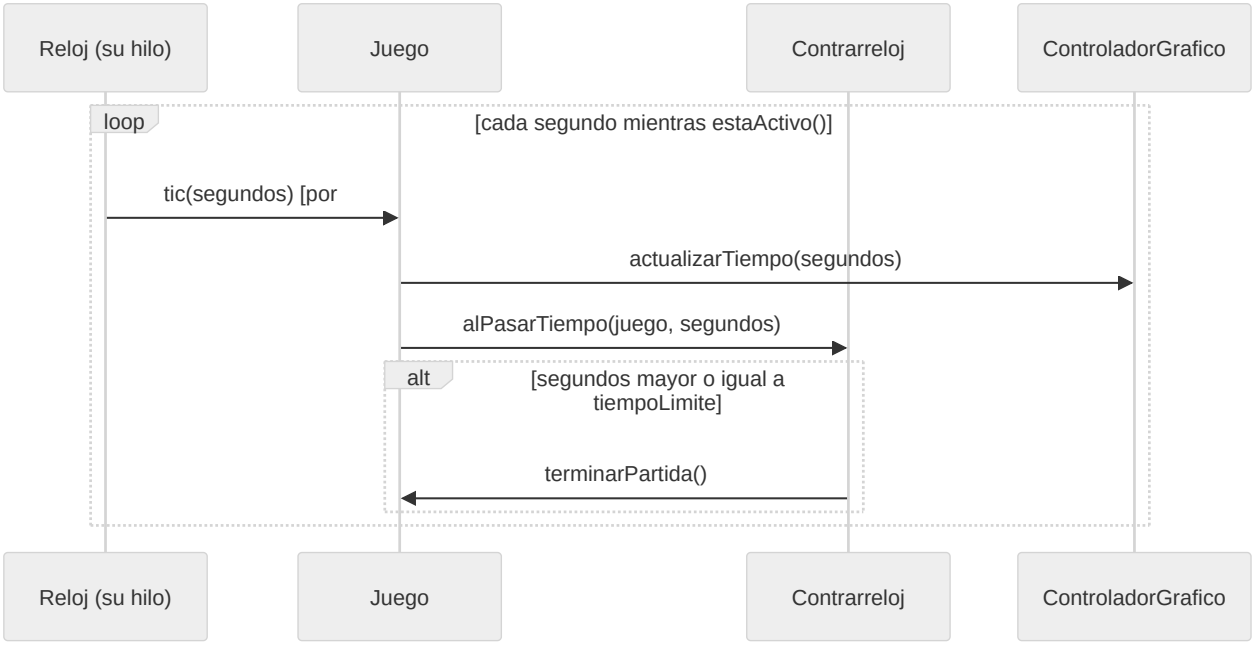
```
        public void jugadorDaniado() { modoJuego.alRecibirDanio(this); }   // solo NoHit hace algo

        // Contrarreloj
        public void alPasarTiempo(Juego j, int segundos) { if (segundos >= tiempoLimite) j.terminarPartida(); }
        // NoHit
        public void alRecibirDanio(Juego j) { j.terminarPartida(); }
```

> **Defensa 5:** para que el reloj "descuente un segundo al modo de juego vigente", como dice el diagrama de secuencia 3, `Contrarreloj` lleva `tiempoRestante` y lo baja en cada `alPasarTiempo`; si llega a 0, termina la partida (ver 16.10).

Cómo llega el segundo a `Juego`: el `Reloj` corre en su hilo (`run()`), cuenta `segundos` y le avisa a `Juego`, al que conoce por `#juego`. Había tres formas de resolverlo, y el equipo eligió la **B**:

Opción	Cómo	En el diagrama
A. Callback (como en la defensa 3)	`Timer(alTic : Runnable)`; `Juego` crea `new Timer(() -> tic(timer.getSegundos()))`	Nada nuevo (`Runnable` es de Java)
B. El reloj conoce a `Juego` (elegida)	`Reloj(j : Juego)` guarda `# juego : Juego` y llama a `tic(segundos)`, que es público; `Juego` lo crea con `new Reloj(this)`	La asociación `Reloj → Juego (#juego 1)`, dibujada junto a la composición `Juego ◆ Reloj (#reloj 1)`
C. `Juego` consulta	En cada ciclo, `Juego` lee `reloj.getSegundos()`	Nada nuevo, pero `Juego` necesita su propio ciclo



Cuándo termina la partida (y se registra el ranking): lo decide el modo. Clásico: al vencer al jefe del último nivel. Contrarreloj y NoHit: al ganar, y además al agotarse el tiempo o al recibir un golpe; qué pasa exactamente en esos dos casos (¿se registra el puntaje?) **queda para consultar al ayudante** (ver contexto §7).

Comunicación entre `Juego`, `Sector` y las entidades

Tres asociaciones resuelven cómo se hablan `Juego`, el `Sector` y las entidades:

Relación	Tipo	Sentido	Para qué
`Juego → Link (#jugador 1)`	Asociación unidireccional	`Juego` llama a `Link`	Pasarle a Link las intenciones del jugador
`Entidad* → Juego (#miJuego 1)`	Asociación unidireccional	Cada entidad llama a `Juego`	Avisar sucesos **de partida**: puntos, vidas, fin de nivel, cambio de sector
`EntidadAnimada* → Sector (#miSector 1)`	Asociación unidireccional	Cada entidad animada llama a su `Sector`	Pedir cosas **espaciales**: colisiones, agregar, sacar o reemplazar entidades
`Sector → EntidadAnimada* (#animadas 0..*)`	Asociación unidireccional	El `Sector` llama a lo que se mueve	Revisar colisiones contra las animadas, iniciarlas y detenerlas

A estas se suma la relación de ida entre el sector y sus entidades:

Relación	Tipo	Sentido	Para qué
`Sector → Entidad* (#entidades 0..*)`	Asociación unidireccional	`Sector` conoce a sus entidades	Recorrerlas para verificar colisiones, agregarlas y sacarlas

Las cuatro son **asociaciones** (no composición ni agregación): ninguna clase es dueña de la otra, solo guarda una referencia permanente para poder llamarla. Ninguna toca la vista: la vista sigue enterándose de los cambios solo por el Observer.

Analogía: `Juego` es el *árbitro* y el `Sector` es la *cancha*. Para lo que pasa en la cancha (qué hay cerca, poner o sacar algo) la entidad le habla a la cancha; para lo que afecta al partido (puntos, vidas, fin del nivel) le habla al árbitro.

`Juego → Link (#jugador 1)`

- **Por qué hace falta:** las intenciones del teclado llegan a `Juego` por `ControladorJuego` (`mover`, `atacar`, `usarEscudo`, `cambiarObjeto`). `Juego` tiene que reenviárselas a Link. Sin esta referencia, la única forma de llegar a Link sería recorrer `Nivel → Escena → Sector → Entidad`, y dentro del sector Link es una entidad más: para encontrarla habría que preguntar el tipo, lo que obligaría a usar `instanceof` (prohibido).
- **Es el mismo objeto que está en el sector:** igual que el atajo `Nivel → Jefe`, Link vive en la colección de entidades del sector activo y `#jugador` apunta a ese mismo objeto. Cuando Link cambia de sector, la referencia sigue siendo válida porque Link no se recrea.
- **Multiplicidad 1:** hay un solo jugador. Además, `Juego.getJugador()` lo expone a las entidades que necesitan saber dónde está Link (la trampa, el Wizzrobe, StoneHinox).

`Entidad* → Juego (#miJuego 1)`

- **Por qué hace falta:** las entidades son las que *detectan* los sucesos (cada entidad animada se actualiza en `HiloJugador` o `HiloMundo` y las colisiones se resuelven entre entidades por doble despacho), pero las consecuencias **de partida** las decide `Juego`: sumar puntos, llevar la racha, descontar vidas, consultar el `ModoJuego`, terminar el nivel, hacer la transición entre sectores.

- **Costo:** queda una dependencia en los dos sentidos entre `Juego` y las entidades (`Juego` llega a ellas por el mapa y por `#jugador`; ellas lo conocen por `#miJuego`). Es aceptable porque las dos son de la capa lógica.
- **Alternativa descartada:** que `Juego` observe a las entidades con otro Observer (suma interfaces y patrones sin necesidad).

`EntidadAnimada* → Sector (#miSector 1)`

- **Por qué hace falta:** muchas acciones de las entidades son sobre el lugar donde están: verificar con qué chocaron, dejar un proyectil o un drop, sacarse a sí mismas, reemplazar un suelo. El dueño natural de esa colección es el `Sector`. Sin esta referencia, todo eso tendría que pasar por `Juego`, que terminaría haciendo de todo.
- **Por qué en `EntidadAnimada*` y no en `Entidad*`:** el que *inicia* una acción siempre es una entidad animada, porque son las únicas que actualiza un hilo. Una `Roca`, un `Pasto` o una `Rupia` nunca actúan por su cuenta: cuando Link corta el pasto, el golpe lo da el hilo de Link, y es Link quien termina pidiendo `quitarDelSector(pasto)`.
- **Multiplicidad 1:** una entidad animada está en un solo sector a la vez.
- **Depende del modelo de hilos:** en la defensa 5 un solo hilo (`HiloMundo`) recorre el sector visible, pero llama `a.actualizarEstado()` sin pasarle el sector, así que `#miSector` sigue haciendo falta como asociación. Si el hilo le pasara el sector por parámetro (`actualizarEstado(s)`), `#miSector` pasaría a ser una **dependencia**.
- **Por qué `Sector → Entidad*` es asociación y no agregación ni composición:**
  - *Composición* no corresponde: la parte tendría que pertenecer a un único todo toda su vida y morir con él, y Link pasa de un sector a otro sin dejar de existir. Además, el sector no crea las entidades (las crea la fábrica y las carga `GeneradorNivel`).
  - *Agregación* sería defendible ("el sector contiene entidades"), pero en el código se traduce igual que una asociación (un atributo `Entidad[][] entidades`) y en la defensa abre la discusión de por qué no es composición.
  - *Asociación* es lo que usa el ejemplo de la cátedra para el mismo caso (`Ruta → #obstaculos`, `#power_ups`, etc.) y el rol `#entidades 0..*` ya deja claro que el sector las contiene.
- **Cuidado:** entre `Sector` y las entidades animadas hay referencias en los dos sentidos. Cuando Link cambia de sector hay que actualizar las dos cosas en el mismo paso: retirarlo del sector viejo (`retirar`), agregarlo al nuevo (`agregar`, que le cambia `#miSector`).

Quién resuelve cada suceso

Entidad que actúa	Suceso	Al `Sector (#miSector)`	A `Juego (#miJuego)`
Cualquier `EntidadAnimada`	Se mueve	`verificarColisiones(this)`	—
`Octorok`, `Wizzrobe`, `StoneHinox`	Disparan	`agregarAlSector(proyectil)`	`mostrarEntidad(proyectil)`
`Proyectil`	Choca o sale de la pantalla (`estaFuera(p)`)	Sacarse a sí mismo	—
`Enemigo`	Muere (`morir()`)	Sacarse a sí mismo y agregar el drop (si suelta)	`enemigoDerrotado(this)`: puntos y racha

Entidad que actúa	Suceso	Al `Sector (#miSector)`	A `Juego (#miJuego)`
`Jefe`	Muere	Sacarse a sí mismo	`jefeDerrotado(this)`: +5000 y fin del nivel
`Link`	Corta el `Pasto`	`quitarDelSector(pasto)`	—
`Link`	Pisa un `SuelosQuebradizo` (`romperSuelo(s)`, lo llama el suelo en `afectar(l)`)	`reemplazarEnSector(suelo, caida)`	`getFabricaEntidades()` y `mostrarEntidad(caida)`
`Link`	Recoge un `PowerUP`	`quitarDelSector(p)`	`powerUpRecogido(p)`: +300, o +10 por rupia
`Link`	Recibe daño	—	`jugadorDaniado()`: racha y modo (NoHit)
`Link`	Pierde todas las vidas (`morir()`)	—	`jugadorSinVidas()`
`Link`	Cae al vacío (`caerAlVacio()`, lo llama `CaidasAlVacio` en `afectar(l)`)	—	`jugadorCayoAlVacio()`: reiniciar el nivel
`Link`	Llega al borde del sector	—	`cambiarDeSector(d)`: transición

Ejemplos de recorrido

1. **Link camina:** flecha → `GUI.procesarTecla` → `controladorJuego.mover(d)` → `Juego` → `jugador.mover(d)` (fija la dirección en su `MovimientoInducido`). El hilo de Link actualiza la posición y pide `miSector.verificarColisiones(this)`. Link notifica a sus observers y `CeldaJugador` se redibuja. No hace falta avisarle nada a `Juego`.

2. **Un Octorok dispara:** su estrategia de ataque crea el proyectil y lo deja en la cancha con `agregarAlSector(p)`. `HiloMundo` lo mueve y, en cada vuelta, el proyectil pide `miSector.verificarColisiones(this)`. Si choca con Link (y Link no usa el escudo), Link recibe el daño y el proyectil se saca del sector.

3. **Link mata un Moblin:** A → `controladorJuego.atacar()` → `Juego` → `jugador.atacar()` → `espada.atacarCon(link)`; al soltar A, el `AtaqueRecargado` golpea con `golpearEn`. El Moblin se queda sin golpes y `morir()` reparte los avisos: suelta el drop y se saca (cancha), y avisa `enemigoDerrotado` (árbitro). `Juego` le suma los puntos a Link por `#jugador`; Link notifica y el HUD se actualiza.

4. **Link pisa un suelo quebradizo:** el choque lo detecta el hilo de Link (doble despacho: `suelo.aceptarColision(link)` → `link.colisionarCon(o : Obstaculo)` → `suelo.afectar(link)`) y el suelo pide que lo reemplacen por una `CaidasAlVacio` creada con la fábrica.

5. **Link se queda sin vidas:** un `Proyectil` lo golpea, Link pierde corazones y luego una vida; al no quedarle vidas, `morir()` avisa `jugadorSinVidas()` por `#miJuego`. `Juego` lo reinicia (puntaje 0, 3 vidas, posición inicial del nivel actual).

6. **Link cruza al sector siguiente:** al avanzar fuera del sector (`miSector.estaFuera(p)`), Link avisa `miJuego.cambiarDeSector(d)`. `Juego` detiene las entidades, le pide el vecino a la escena y hace el cambio (ver Cambio de sector).

Conversión de coordenadas: si el sector mide 768 × 528 y Link sale por la derecha en (768, 240), en el sector nuevo entra en (0, 240): cambia la `x` al borde opuesto y conserva la `y`. Si sale por arriba en (336, 0), entra en (336, 480). El sector viejo nunca se entera de cuál es su vecino: eso lo resuelve `Juego` con la `Escena`.

Creación y asignación de entidades

Qué mensajes se envían cuando se **crea** una entidad y quién le **asigna** cada una de sus referencias. Hay dos momentos: al **cargar el nivel** y **durante la partida**.

Regla: solo las entidades del **sector activo** tienen celda en pantalla. Las de los otros sectores existen en memoria pero no se dibujan hasta que su sector se activa.

Qué se le asigna a cada entidad nueva

Referencia	Quién la asigna	Cuándo
`#sprite`	`FabricaEntidades` (se lo pide a `FabricaSprites`)	Al crearla (constructor)
`#posicion`	El constructor (`x`, `y`)	Al crearla
`#miJuego`	`FabricaEntidades` con `setJuego(juego)` (conoce a `Juego`)	Al crearla
Estrategias (movimiento, ataque, defensa)	El constructor de cada clase concreta	Al crearla
`#miSector` (solo animadas)	`Sector.agregar(e)` con `e.setSector(this)`	Al agregarla a un sector
`#observadores` (su celda)	`Juego`, con la celda que devuelve la vista (`registrarObserver`)	Cuando su sector está activo
Quién la actualiza (solo animadas; defensa 5)	Nadie se lo asigna: `HiloMundo` recorre `animadas` del sector visible (`hiloMundo.setSector(s)` al activarlo); a Link lo actualiza `HiloJugador`	Cuando su sector es el visible

`setSector` está definido en `Entidad` sin hacer nada y `EntidadAnimada` lo redefine para guardar `#miSector`. Así el sector lo llama sobre cualquier entidad sin preguntar el tipo. (En la defensa 4 pasaba lo mismo con `iniciar()`; en la defensa 5 desaparece, porque las entidades ya no arrancan hilos.)

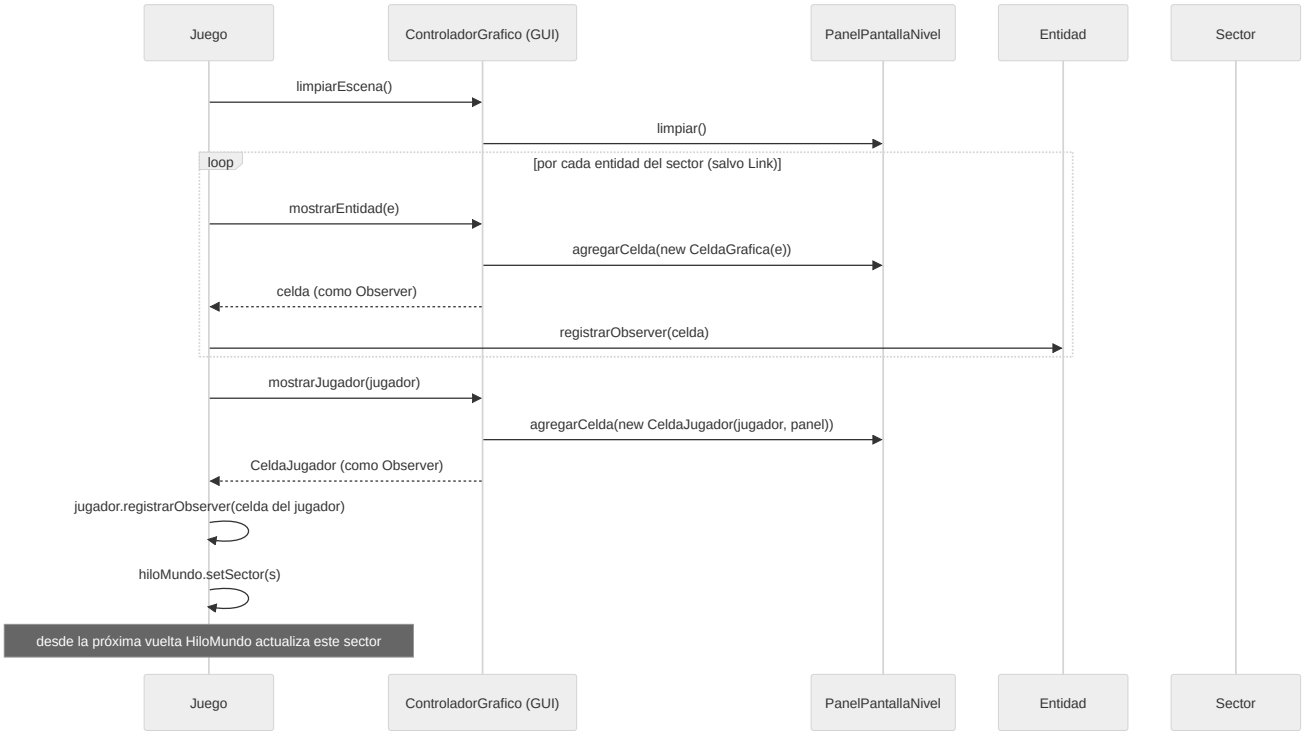
1. Carga de un nivel

El detalle línea por línea del archivo está en 7. Formato y parseo de niveles. Del lado de `Juego`:

```
        // Juego
        private void cargarNivel() {
            nivel = generadorNivel.generar(numeroNivel, fabricaEntidades);
            Sector inicio = nivel.getSectorInicial();
            inicio.agregar(jugador);                        // Link no se recrea; agregar le asigna #miSector
            jugador.setPosicion(nivel.getPosicionInicial());
            activarSector(inicio);
        }
```

2. Activar un sector (celdas e hilos)

Pasa al empezar el nivel y en cada transición de sector.




```
        // Juego
        private void activarSector(Sector s) {
            controladorGrafico.limpiarEscena();
            for (Entidad e : s.getEntidades())
                if (e != jugador)                                   // compara referencias, no tipos
                    e.registrarObserver(controladorGrafico.mostrarEntidad(e));   // la vista crea la CeldaGrafica
            jugador.registrarObserver(controladorGrafico.mostrarJugador(jugador));   // CeldaJugador (HUD)
            hiloMundo.setSector(s);                                 // defensa 5: HiloMundo pasa a actualizar este sector
        }
```

La lógica solo ve un `Observer`: nunca conoce `CeldaGrafica` ni Swing. El `if (e != jugador)` evita crearle dos celdas a Link (una común y la del jugador) y no es un `instanceof`: compara si es **el mismo objeto**.

> Detalle de implementación: al activar un sector nuevo, las celdas viejas ya no existen (`limpiarEscena`), pero las entidades siguen teniendo registrado su observer viejo. Conviene que `Entidad` permita limpiar sus observers (o que `registrarObserver` reemplace al anterior) para no notificar a celdas que ya no están en pantalla.

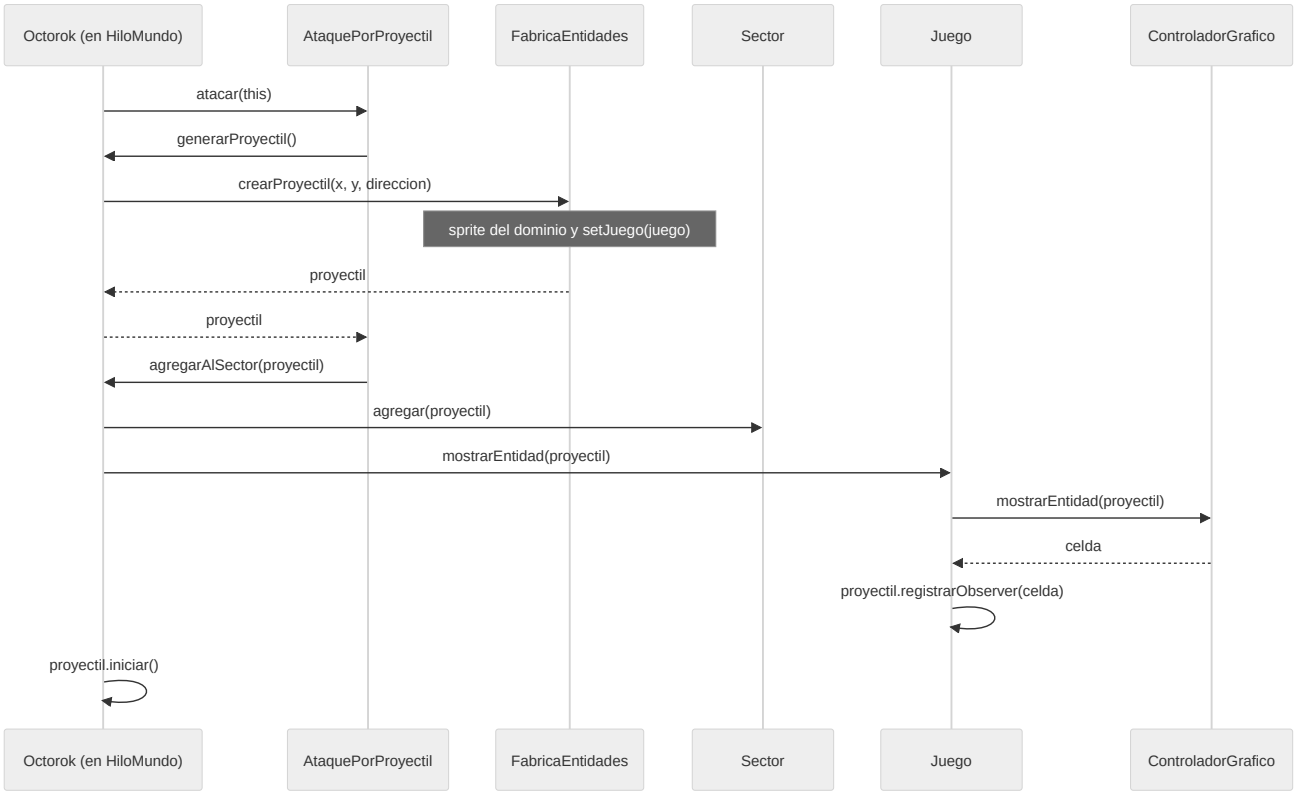
3. Crear entidades en plena partida

Siempre el mismo patrón: **crear con la fábrica → agregar al sector → mostrar** (en la defensa 4 había un paso más, iniciar su hilo). Para no repetirlo, `EntidadAnimada` tiene `agregarAlSector(nueva)`:

```
        // EntidadAnimada
        public void agregarAlSector(Entidad nueva) {
            miSector.agregar(nueva);           // asigna #miSector si es animada
            miJuego.mostrarEntidad(nueva);     // Juego pide la celda a la vista y la registra
            // defensa 5: sin iniciar(); si es animada, HiloMundo la mueve desde la vuelta siguiente
        }

        // Juego
        public void mostrarEntidad(Entidad e) {
            e.registrarObserver(controladorGrafico.mostrarEntidad(e));
        }
```

Ejemplo, un Octorok dispara:



Caso	Quién crea	Mensajes
Octorok dispara	`AtaquePorProyectil`	`generadorProyectil.generarProyectil()` → `fabrica.crearProyectil(...)` → `atacante.agregarAlSector(p)`
Enemigo muere y suelta algo	El enemigo (`EnemigoConDrop.morir()` → `soltarDrop()`, después de `super.morir()`)	`generarDrop()` → `fabrica.crearRupia(...)` o `crearCorazon(...)` → `agregarAlSector(drop)`

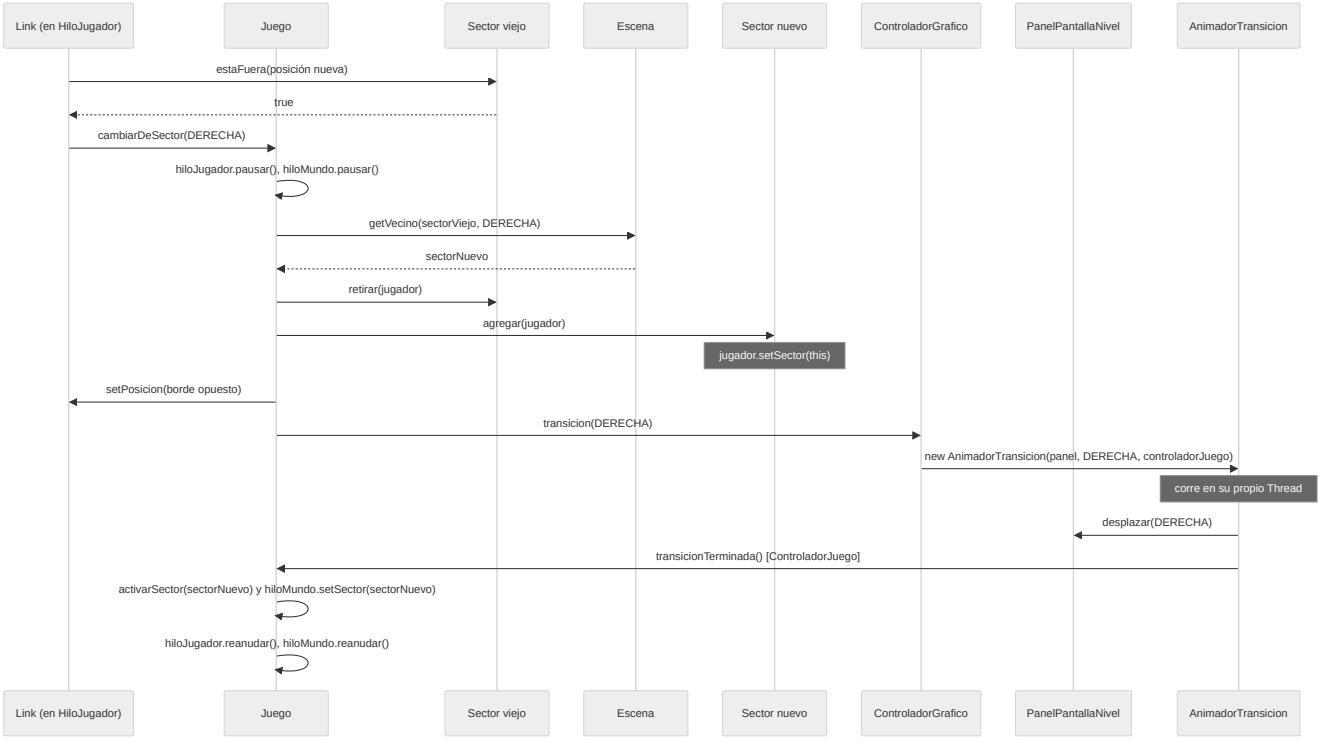
Caso	Quién crea	Mensajes
Aparece una bellota	`Juego`, al recibir `enemigoDerrotado` con racha 4	`aparecerBellota(x, y)` → `fabrica.crearBellota(...)` → `jugador.getSector().agregar(b)` → `mostrarEntidad(b)`
Link pisa un suelo quebradizo	`SuelosQuebradizo.afectar(l)`	`fabrica.crearCaidasAlVacio(...)` → `l.reemplazarEnSector(suelo, caida)` → `mostrarEntidad(caida)`

4. Eliminar una entidad (lo inverso)

```
        // Sector
        public void quitar(Entidad e) {
            // lo fijo: entidades[fila][columna] = null; lo animado: animadas.remove(e) (diferido si se está recorriendo)
            e.eliminar();
        }
        public void retirar(Entidad e) { animadas.remove(e); }    // sin eliminar: Link al cambiar de sector

        // Entidad
        public void eliminar() {
            eliminada = true;
            // defensa 5: no hay hilo que detener; actualizarEntidades() la saca de animadas
            notificarObservers();  // la CeldaGrafica ve estaEliminada() y se saca del panel
        }
```

5. Cambio de sector (Link pasa al vecino)




```
        // Juego
        public void cambiarDeSector(Direccion d) {
```

```
            Sector viejo = jugador.getSector();
            Sector nuevo = nivel.getMapa().getVecino(viejo, d);      // la escena conoce la grilla
            if (nuevo == null) return;                               // borde del mapa: no se puede salir
            hiloJugador.pausar(); hiloMundo.pausar();                // defensa 5: nadie se mueve en la transición
            viejo.retirar(jugador);                                  // sin eliminarlo
            nuevo.agregar(jugador);                                  // le asigna #miSector
            jugador.setPosicion(entrada(jugador.getPosicion(), d));  // borde opuesto (cálculo de Juego)
            controladorGrafico.transicion(d);                        // la vista desplaza la pantalla
        }
        public void transicionTerminada() {                          // lo avisa AnimadorTransicion
            activarSector(jugador.getSector());                      // incluye hiloMundo.setSector(nuevo)
            hiloJugador.reanudar(); hiloMundo.reanudar();
        }
```

`retirar` saca a Link de la lista sin llamar a `eliminar()`: Link no desaparece, solo cambia de sector. Antes, en la defensa 3, esto se llamaba `quitarSinEliminar`. Después `Juego` le pide a Link `entrarPorBordeOpuesto(d)`, que lo ubica en el borde contrario del sector nuevo (por ejemplo `x = 0` si iba a la derecha).

Resumen

Paso	Mensaje clave	Asigna
Crear	`fabrica.crearX(x, y)`	`#sprite`, `#posicion`, `#miJuego`, estrategias
Ubicar	`sector.agregar(e)`	`#entidades` del sector y `#miSector`
Mostrar	`controladorGrafico.mostrarEntidad(e)` + `e.registrarObserver(celda)`	`#observadores`
Activar	`hiloMundo.setSector(sector)`	Qué sector actualiza `HiloMundo`
Eliminar	`sector.quitar(e)` → `e.eliminar()`	Borra la celda; `actualizarEntidades()` la saca de `animadas`
Mover de sector	`sector.retirar(e)` + `otro.agregar(e)`	`#miSector` nuevo

12. Observaciones y puntos a revisar

Detalles detectados al revisar el diagrama reducido de la defensa 4 y su relación con el extendido de la defensa 4. Las que no están resueltas **no se modificaron** en el diagrama; quedan para que el equipo decida antes de la entrega. Estados:

- **Resuelta:** ya está corregida en el diagrama o en el extendido de la defensa 4.
- **Parcial:** se corrigió una parte.
- **Pendiente:** sigue igual que en la defensa 3, o es nueva.

- **Aclaración:** no es un error; queda como nota para la defensa.
- **No aplica (decisión del equipo):** la observación dejó de corresponder por una decisión del equipo.

12.1 Observaciones de la defensa 3: estado en la defensa 4

N.º	Observación (defensa 3)	Estado	Detalle en la defensa 4
1	Falta `FabricaEntidades → Juego`	**Pendiente**	El extendido sigue teniendo `# juego : Juego` y el constructor `FabricaEntidades(juego, fabricaSprites)`, pero la flecha no está en el reducido. Conviene dibujar `FabricaEntidades → Juego` con `#juego 1`: todo atributo de tipo clase del modelo es una asociación, y si no está alguien puede preguntar quién asigna `#miJuego`.
2	Dependencias con línea continua (`FabricaSprites* → Sprite`, `GeneradorNivel → Nivel`, `GeneradorDrop → Rupia`, `GeneradorDrop → Corazon`, `GeneradorProyectil → Proyectil`, `PersistenciaRanking → Ranking`)	**No aplica (decisión del equipo)**	El equipo decidió no dibujar las dependencias en el reducido: se sacaron del dibujo las siete (las seis de la defensa 3 y `Posicion → Direccion`). El texto las sigue explicando como dependencias no dibujadas. Si alguna vez se dibujan, van **punteadas** con flecha abierta (`dashed=1` en draw.io).
3	`DefensaPorEscudo → ComportamientoDefensa` con etiquetas `#sinEscudo` y `1` sueltas y punta distinta	**Pendiente**	Las dos etiquetas siguen como textos independientes (no pegados a la flecha) y la punta es la flecha "classic" de draw.io en lugar de la flecha negra pintada del resto de las asociaciones.
4	Tres líneas en diagonal (`Ranking → Juego`, `Juego → GeneradorNivel`, `GeneradorNivel → Nivel`)	**Pendiente**	La geometría del diagrama no cambió respecto de la defensa 3.
5	`Sprite` y `Sprites`	**Resuelta**	`FabricaSprites*` devuelve `Sprite`, el atributo de `Entidad*` es `# sprite : Sprite` (coincide con el rol `#sprite` del reducido) y los constructores de `EntidadAnimada`, `Link`, `Corazon` y `MasterStalfos` reciben `sprite : Sprite`, como los demás.
6	`Posicion` sin `getX()`; `Direccion` sin sus valores	**Resuelta**	`Posicion` tiene `getX()` y `getY()`; `Direccion` lista `ARRIBA`, `ABAJO`, `IZQUIERDA`, `DERECHA`.
7	`Colisionable` y `Colisionador` vacías en el extendido	**Resuelta**	`Colisionable` declara `aceptarColision(c : Colisionador)` y `Colisionador` los seis `colisionarCon(...)`.
8	`Nivel`: `posicionInicial` junto con `filaInicial` y `columnaInicial`	**Aclaración**	No es redundante (ver [la explicación en `Nivel`]).

N.º	Observación (defensa 3)	Estado	Detalle en la defensa 4
9	`#miJuego` en `Entidad*` o en `EntidadAnimada*`	**Aclaración (decisión abierta)**	Se mantiene en `Entidad*`. En el extendido actual `Link` volvió a tener `caerAlVacio()` y `romperSuelo(s)`: `CaidasAlVacio.afectar(l)` llama a `l.caerAlVacio()` y `SuelosQuebradizo.afectar(l)` a `l.romperSuelo(this)`, y Link avisa a `Juego`. Los obstáculos ya no necesitan `#miJuego`, así que moverlo a `EntidadAnimada*` sería posible (ver 13.1).
10	Ranking: conviven `getRanking(j)` y `registrarPuntaje(j, e)`	**Resuelta**	En la defensa 4 los dos trabajan juntos: `getRanking` para preguntar `entraEnTop5` antes de pedir el nombre y `registrarPuntaje` para insertar por `agregarAlRanking…(e)` → `agregar(i, e)` (ver Cómo llega el puntaje al ranking del modo). Queda el detalle de la observación 15.
11	Entrega en `Diagramas Puntuables` con nombres exactos	**Resuelta**	Los nombres se confirmaron con los requerimientos de la defensa 4: `Diagrama-reducido-puntuable-1.pdf` y `Diagrama-extendido-puntuable-1.pdf`. Los PDF están en `Diagramas Puntuables/` (copias de los de `Defensa 4/`); hay que subirlos también a la rama principal del repo de la comisión (ver Requerimientos de la defensa 4).

12.2 Observaciones nuevas de la defensa 4

12. **`Posicion → Direccion` con línea continua.** Es el único cambio del reducido y conviene revertirlo: `Posicion` no tiene un atributo `Direccion`, solo la recibe por parámetro en `desplazada(d, pasos)`; eso es una dependencia (punteada). Con línea continua se lee como asociación, y además no tiene rol ni multiplicidad, como sí tienen todas las asociaciones del diagrama. Explicación completa en ¿`Posicion → Direccion` es asociación o dependencia?. **Recomendación:** volver a agregar `dashed=1` a la flecha `rel-posicion-direccion-dep` (o borrarla). **No aplica (decisión del equipo):** el equipo decidió no dibujar dependencias en el reducido, así que la flecha se borró junto con las demás; la relación se explica en el texto como dependencia no dibujada.

13. **Ganchos que las estrategias necesitan y que `EntidadAnimada*` no declaraba.** Las estrategias reciben una `EntidadAnimada`, así que solo pueden llamar a lo que esa clase declara. En la primera versión del extendido de la defensa 4 se habían sacado de `EntidadAnimada*` y estaban en las subclases:

Gancho	Lo necesita	Estaba en	Hoy
~~`vigilar()`~~	Resuelto: ahora es `#vigilarJugador()` en `Enemigo*` y no lo llama ninguna estrategia (ver la observación 25)	`Enemigo*`	`Enemigo*`
`Teletransportarse()`	`MovimientoTeletransporte`	`Wizzrobe`	`+ teletransportarse()` en `EntidadAnimada*`, redefinido en `Wizzrobe`

Gancho	Lo necesita	Estaba en	Hoy
`ataqueTerminado()`	`AtaqueAvalancha` (con `alTerminar`)	`StoneHinox`	`+ ataqueTerminado()` en `EntidadAnimada*`, lo redefine `StoneHinox`; lo llama `AtaqueAvalancha.alTerminar`
`ataqueRealizado()`	`AtaqueEspada` / `AtaqueRecargado`	`Link`	`+ ataqueRealizado()` en `EntidadAnimada*`, redefinido en `Link`
`quiereAtacar()`	`AtaqueEspada` (MasterStalfos)	`Link`, `MasterStalfos`	`+ quiereAtacar() : boolean` en `EntidadAnimada*`, redefinido en `MasterStalfos`
`alSerGolpeado()`	`EntidadAnimada.recibirDanio`	`MasterStalfos`	`# alSerGolpeado()` en `EntidadAnimada*`, redefinido en `MasterStalfos`
`restaurarDefensa()`	`DefensaReducida` (opcional)	`Link`	`+ restaurarDefensa()` en `EntidadAnimada*`, redefinido en `Link`
`morir()`	`EntidadAnimada.aplicarDanio`	`Enemigo*` (protegido), `Link` (público); `Proyectil` usaba `alQuedarSinVida()`	`# morir()` en `EntidadAnimada*`, redefinido en `Enemigo*`, `Jefe*`, `Link` y `Proyectil`

Recomendación: declararlos en `EntidadAnimada*` sin hacer nada (o abstractos donde corresponda) y que cada subclase redefina los que usa. Así se mantiene el polimorfismo sin *cast* ni `instanceof`. **Resuelta:** el extendido declara en `EntidadAnimada*` `# morir()`, `# alSerGolpeado()`, `+ ataqueRealizado()`, `+ restaurarDefensa()`, `+ teletransportarse()` y `+ quiereAtacar() : boolean`, y las subclases los redefinen (`Wizzrobe`, `Link`, `MasterStalfos`, `Enemigo*`, `Jefe*`, `Proyectil`). También `+ ataqueTerminado()`, que redefine `StoneHinox` (ver Avalancha y la observación 27).

14. **`Wizzrobe.Teletransportarse()` con mayúscula.** En Java los métodos van en camelCase con minúscula: `teletransportarse()`. **Resuelta:** el extendido usa `teletransportarse()`, declarado en `EntidadAnimada*` y redefinido en `Wizzrobe`.

15. **`Juego.getRankingContrareloj()` con una sola r.** La clase es `Contrarreloj` y el comando es `agregarAlRankingContrarreloj(e)`. Conviene unificar a `getRankingContrarreloj()`. **Resuelta:** el extendido usa `getRankingContrarreloj()`.

16. **`Juego` no lista `getRankings()`.** `ControladorJuego` declara `getRankings() : Ranking[]` y `Juego` la realiza, pero la caja de `Juego` ya no la muestra (sí estaba en la defensa 3). El menú la usa para mostrar los tres rankings. **Resuelta:** `Juego` lista `+ getRankings() : Ranking[]` (consulta; devuelve `rankings`).

17. **`Timer` sin forma de avisar a `Juego`.** `Timer()` ya no recibe el `Runnable` de la defensa 3, no conoce a `Juego`, y `Juego.tic(segundos)` es privado. Hay que elegir cómo llega el segundo (ver Tiempo y reglas del modo); la opción más simple es volver a `Timer(alTic : Runnable)`. **Resuelta:** el equipo eligió que el timer conozca a `Juego`: `Timer(j : Juego)`, `# juego : Juego`

y `+ tic(segundos : int)` público; `Juego` lo crea con `new Timer(this)` y `run()` llama `juego.tic(segundos)` cada segundo. En el reducido se dibujó la asociación `Timer → Juego` (`#juego 1`), además de `Juego ◆ Timer`. **Defensa 5:** `Timer` se renombró `Reloj` y hereda de `HiloJuego*`; las dos flechas siguen.

18. **Hilos de las entidades.** `EntidadAnimada*` no mostraba `run()` (sí `iniciar()`, `detener()` y `estaActiva()`), mientras que `Timer` (hoy `Reloj`) sí lo tiene. **Resuelta:** el extendido agrega `+ run()` en `EntidadAnimada*`, el ciclo que mueve, ataca y verifica colisiones en cada vuelta. **Defensa 5:** ese ciclo pasa a ser `actualizarEstado()` y lo llaman `HiloJugador` y `HiloMundo` (ver la sección 16).

19. **Nombres distintos para "quedarse sin vida".** `Enemigo.morir()` (protegido), `Link.morir()` (público) y `Proyectil.alQuedarSinVida()`. Conviene un solo nombre declarado en `EntidadAnimada*` (ver la observación 13). **Resuelta:** el único nombre es `# morir()`, declarado en `EntidadAnimada*` y redefinido en `Enemigo*`, `Jefe*`, `Link` y `Proyectil`.

20. **`+` sueltos al final de algunas firmas.** Por ejemplo `getPuntos() : int +` (en los enemigos y jefes), `recibirDanio(...) : boolean +` (en las defensas) y `getRanking(j : Juego) : Ranking +` (en los modos). Si es una marca de "redefine", conviene explicarla con una nota en el diagrama; si no, borrarla.

21. **Jefes y `crearEntidad`.** `GeneradorNivel.crearEntidad(simbolo, x, y, fabrica)` devuelve `Entidad`, pero los jefes tienen que ir además a la lista `List<Jefe>` del `Nivel`. Sin `instanceof` ni *cast*, hay que tratar `S` y `H` antes de llamar a `crearEntidad` (como en el esquema del parseo) o pasarle la lista de jefes como parámetro.

22. **Nombres de los archivos del ranking.** La constante `ARCHIVOS` de `Juego` (defensa 3) ya no aparece, pero `agregar(i, e)` necesita saber qué archivo escribir. Conviene declararla de nuevo en `Juego` o en `PersistenciaRanking`.

23. **`Jefe*` vacía y aviso de jefe derrotado.** La caja de `Jefe*` quedó vacía. Como `Enemigo.morir()` avisa `enemigoDerrotado(this)`, conviene que `Jefe*` redefina `morir()` para avisar `jefeDerrotado(this)`, que es el que termina el nivel. **Resuelta:** `Jefe*` tiene `# Jefe(sprite : Sprite, x : int, y : int, vida : int)` y `# morir()`, que avisa `miJuego.jefeDerrotado(this)`.

24. **Celdas viejas registradas como observers.** `Entidad*` tiene `registrarObserver(o)` pero no una forma de sacar observers. Al volver a un sector ya visitado, sus entidades tendrían registrada la celda vieja y la nueva. Se puede resolver con un `limpiarObservers()` o haciendo que `registrarObserver` reemplace al anterior (ver Activar un sector).

25. **`MovimientoEstatico` se sacó del reducido.** El equipo lo borró a propósito junto con su realización de `ComportamientoMovimiento`. Hasta ahora lo usaban el `ErizoDeMar` (siempre quieto), la `TrampaDeCuchillas` en reposo (vigilando hasta abalanzarse) y el `MoblinConEspadaYEscudo` al vigilar. **Resuelta:** las entidades quietas usan `MovimientoInducido` con dirección `null`, y para saber dónde está Link usan `# vigilarJugador()`, común a todos los enemigos en `Enemigo*` (ver Entidades quietas y que vigilan). También se sacó `MovimientoEstatico` del extendido de la defensa 4.

26. **`Sector` con matriz y lista de animadas.** En el extendido, `Sector` guarda lo fijo en `entidades : Entidad[][]` (una entidad por celda) y lo que se mueve en `animadas : List<EntidadAnimada>`. Quedan dos detalles: el reducido no dibuja `Sector → EntidadAnimada*` (`#animadas 0..*`), y `agregar(e : Entidad)` tiene que saber si `e` va a la matriz o a la lista sin `instanceof`. **Resuelta:** el reducido dibuja `Sector → EntidadAnimada*` (`#animadas 0..*`) y el

extendido resuelve `agregar` con doble despacho (`e.ubicarseEn(this)`, que `Entidad*` responde con `agregarFija(this)` y `EntidadAnimada*` con `agregarAnimada(this)`).

27. **`ataqueTerminado()` sin *cast*.** El extendido ya tiene `# alTerminar(atacante : EntidadAnimada)` en `AtaquePorProyectil` (gancho vacío, último paso de `atacar()`) y `AtaqueAvalancha` lo redefine para avisar `ataqueTerminado()`. Pero `ataqueTerminado()` solo está en `StoneHinox` y `alTerminar` recibe una `EntidadAnimada`. **Resuelta:** `+ ataqueTerminado()` está declarado como gancho vacío en `EntidadAnimada*` y lo redefine `StoneHinox`, así `alTerminar` lo llama sin *cast*. Junto con esto se resolvieron otros mensajes del extendido: `soltarDrop()` lo llama `EnemigoConDrop.morir()` (redefinido: `super.morir()` y después `soltarDrop()`), `programar(ms, accion)` está en `EntidadAnimada*` y `Enemigo*` / `EnemigoConDrop*` tienen constructor protegido.

12.3 Resumen

Estado	Observaciones
Resueltas	5 (`Sprite` en todo el extendido), 6 (`getX()` y valores de `Direccion`), 7 (`Colisionable` y `Colisionador` con sus métodos), 10 (ranking con `getRanking` y `registrarPuntaje` juntos), 11 (nombres de los PDF y carpeta `Diagramas Puntuables`), 13 (ganchos en `EntidadAnimada*`), 14 (`teletransportarse()`), 15 (`getRankingContrarreloj()`), 16 (`getRankings()` en `Juego`), 17 (`Timer → Juego` con `#juego 1` y `tic` público), 18 (`run()` en `EntidadAnimada*`), 19 (`# morir()` como único nombre), 23 (`Jefe*` con constructor y `morir()`), 25 (entidades quietas sin `MovimientoEstatico`) y 26 (`Sector → EntidadAnimada*` y `agregar` con doble despacho), 27 (`ataqueTerminado()` como gancho en `EntidadAnimada*`)
No aplican (decisión del equipo)	2 y 12 (las dependencias no se dibujan en el reducido)
Pendientes de la defensa 3	1, 3 y 4
Pendientes nuevas (extendido)	20, 21, 22 y 24
Aclaraciones	8 y 9
Nuevas de la defensa 4	12 a 27 (la 12, la 25 y la 26 son del reducido, y la 17 terminó agregando `Timer → Juego` al reducido; el resto son del extendido y afectan a cómo se explican los mensajes)

Lo más importante para corregir antes de la defensa: en el reducido, la 1 (`FabricaEntidades → Juego`), la 3 (`#sinEscudo` pegado a su flecha) y la 4 (líneas en diagonal); en el extendido, las pendientes 20, 21, 22 y 24.

12.4 Observaciones de la defensa 5

28. **El extendido no tiene los hilos nuevos.** El reducido muestra `HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj`, `AnimadorTransicion` y `AccionProgramada`, pero el extendido sigue con un hilo por entidad animada y con `Timer`. Para que los dos diagramas coincidan hay que aplicar los cambios de 16.11. **Pendiente.**
29. **Template Method de los ataques sin nota.** Ahora hay una nota de Template Method (la de los hilos), pero el de `AtaquePorProyectil` → `AtaqueAvalancha` sigue sin nota, por la decisión del equipo de la defensa 4. Si preguntan, se explica igual (14.2). **Aclaración.**

30. **Los diagramas de secuencia de la entrega.** Tienen que ir como PDF separados en `Diagramas/Diagramas de secuencia` de la rama principal. Los diagramas 2 y 3 de 16.10 ya usan los hilos nuevos. **Pendiente.**

---

13. Sugerencias y alternativas (no aplicadas)

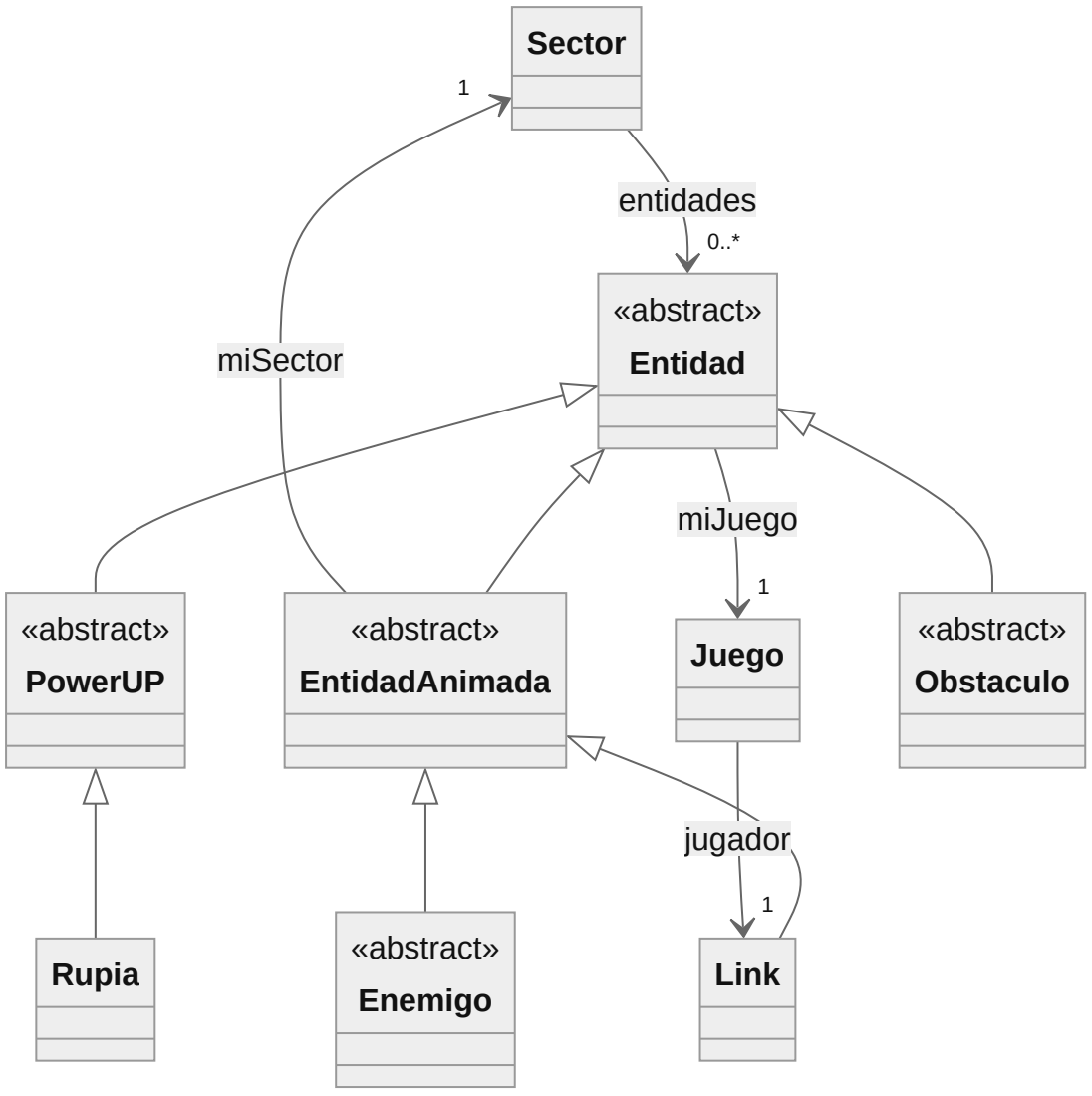
Propuestas y alternativas de modelado analizadas que **no se aplicaron al diagrama**. Quedan documentadas para que el equipo decida.

13.1 Mover `#miJuego` de `Entidad*` a `EntidadAnimada*`

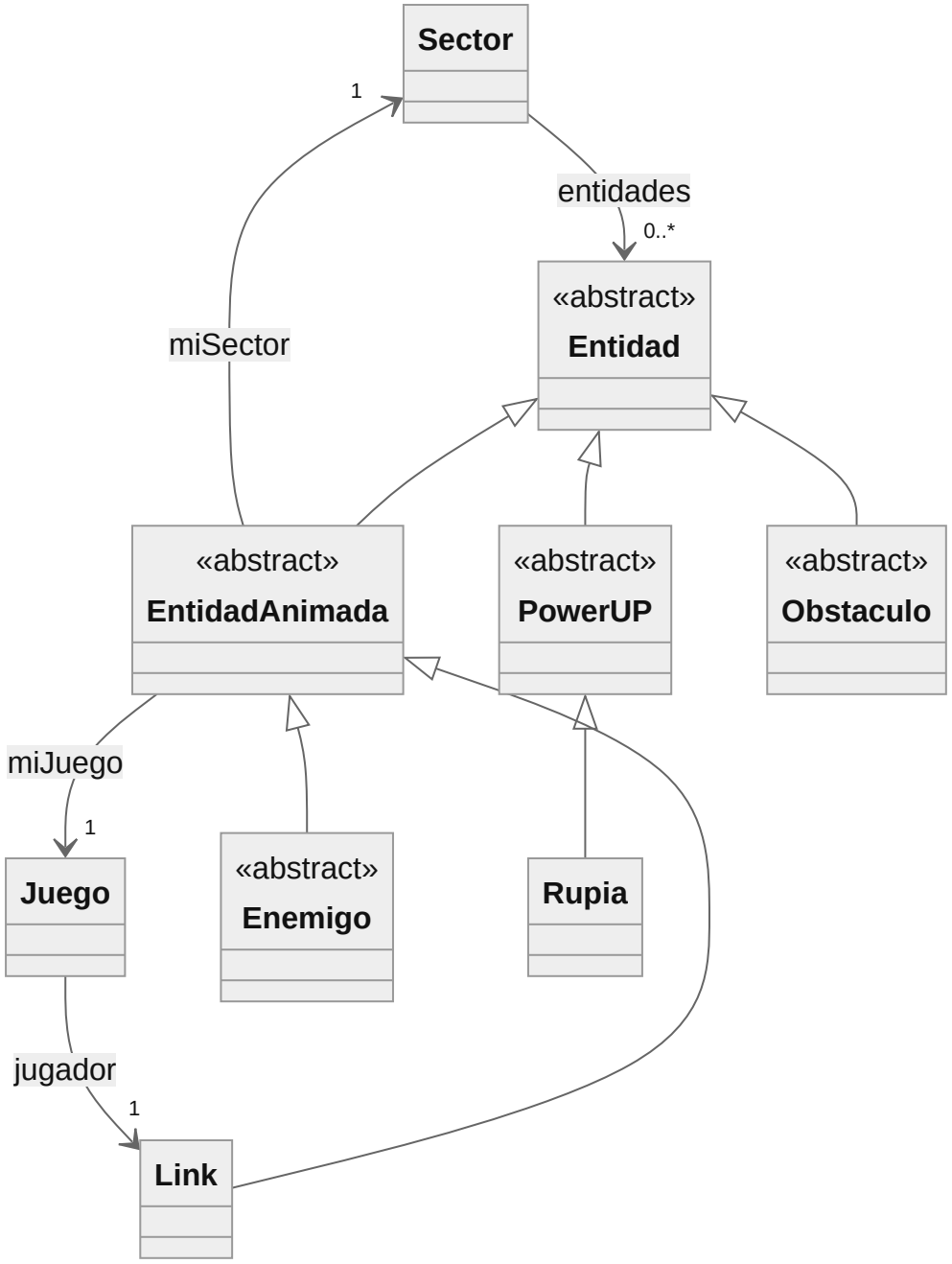
Situación actual: la asociación `Entidad* → Juego` (`#miJuego 1`) sale de la raíz, así que **todas** las entidades (cada `Roca`, `Agua`, `Pasto`, `Rupia`, `Espada`...) guardan una referencia a `Juego`.

Sugerencia: que salga de `EntidadAnimada*`, junto a `#miSector`. Solo Link, enemigos, jefes y proyectiles tendrían la referencia.

Actual:



Sugerido (solo cambia el origen de `miJuego`):



Por qué se pensó

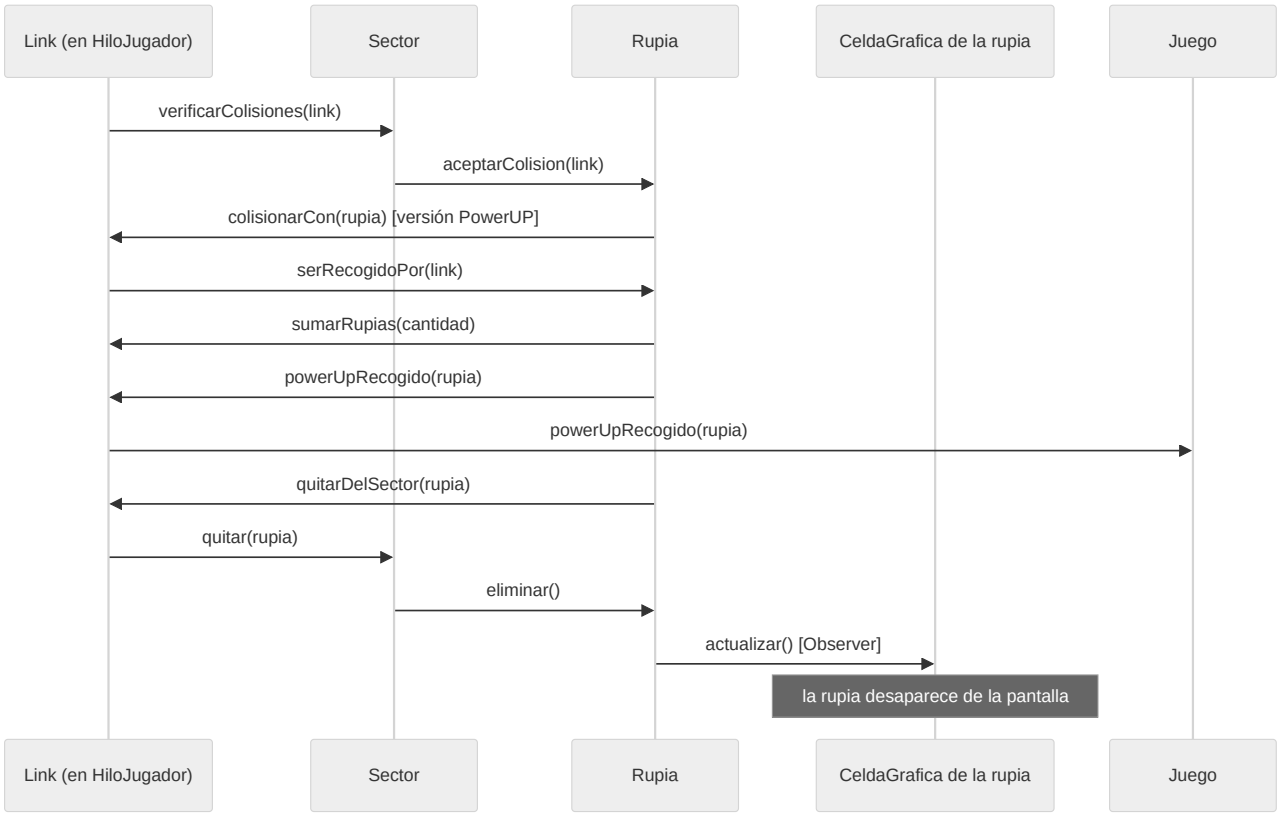
`#miJuego` sirve para que una entidad **le avise a** `Juego` algo que afecta a la partida. Lo necesita una entidad que **decide algo por su cuenta**, y eso lo hacen sobre todo las entidades animadas (las actualiza un hilo).

Caso	**¿Quién actúa?**	**¿Usa `#miJuego`?**
Recoger rupia, corazón, fragmento, bellota u objeto	Link (el power-up recibe a Link por parámetro)	No: el efecto se aplica sobre Link y Link avisa `powerUpRecogido`
Cortar pasto	Link	No: usa `#miSector`
Pisar suelo quebradizo	El suelo, en `afectar(l)`, llama a `l.romperSuelo(this)`	No: Link pide la fábrica (`getFabricaEntidades()`) con su propio `#miJuego`
Caer al vacío	`CaidasAlVacio`, en `afectar(l)`, llama a `l.caerAlVacio()`	No: Link avisa `jugadorCayoAlVacio()` con su propio `#miJuego`

Caso	¿Quién actúa?	¿Usa `#miJuego`?
Chocar con roca o agua	Link	No: solo se frena
Muere un enemigo o jefe	El enemigo	**Sí**
Link recibe daño, pierde vidas o llega al borde	Link	**Sí**
Un proyectil choca	El proyectil	No: daña a Link y se saca con `#miSector`

Qué cambió en la defensa 4: en la primera versión del extendido de la defensa 4, Link no tenía `caerAlVacio()` ni `romperSuelo(s)`, y el efecto lo resolvía el obstáculo avisando a `Juego` con su propio `#miJuego`. En la versión actual **volvieron a `Link`**: `CaidasAlVacio.afectar(l)` llama a `l.caerAlVacio()` y `SuelosQuebradizo.afectar(l)` a `l.romperSuelo(this)`, y es Link quien avisa a `Juego` (`jugadorCayoAlVacio()`, `getFabricaEntidades()`, `mostrarEntidad(caida)`). Así ningún obstáculo necesita `#miJuego`, como en la defensa 3, y la sugerencia vuelve a ser aplicable sin cambios en los obstáculos.

Ejemplo: Link captura una rupia (no usa `#miJuego` de la rupia)



Ventajas y desventajas

A favor	En contra
Las entidades pasivas no dependen de `Juego`: se crean sin pasarle `Juego`	Ninguna fuerte desde que Link volvió a tener `caerAlVacio()` y `romperSuelo(s)`: los obstáculos ya no avisan a `Juego`
`#miJuego` y `#miSector` quedan juntas, en la clase de las entidades que actúan	En el dibujo, la flecha sale de más lejos y suma cruces

A favor	En contra
El diagrama muestra con precisión quién habla con `Juego`	Cambia algo que el contexto da por decidido (§4.3: `Entidad → Juego`)

Alternativa que se mantiene hoy: `#miJuego` en `Entidad*`. Funciona igual y permite que un obstáculo, una rupia u objeto le avise directamente a `Juego`, a cambio de que todas las entidades carguen esa referencia.

13.2 Alternativa de modelado E2: `Juego` con tres rankings con nombre

> **Relación con lo aplicado:** el diagrama usa una versión simplificada: una sola composición `Juego ◆ Ranking (#rankings 3)` en lugar de tres roles con nombre. Los métodos del extendido de la defensa 4 (`agregarAlRankingClasico/Contrarreloj/NoHit` y `getRankingClasico/Contrarreloj/NoHit`) son justamente los de esta alternativa, pero apuntando a posiciones del arreglo en lugar de a tres atributos.

En esta variante, `Juego` tiene un `Ranking` por modo, cada uno con nombre propio, y el modo activo elige el suyo por **doble despacho** (sin `instanceof`).

Relación	Tipo	Rol
`Juego ◆ Ranking`	Composición	`#rankingClasico 1`
`Juego ◆ Ranking`	Composición	`#rankingContrarreloj 1`
`Juego ◆ Ranking`	Composición	`#rankingNoHit 1`
`Ranking ◆ EntradaRanking`	Composición	`#entradas 0..5`
`Juego ◇ ModoJuego*`	Agregación	`#modoJuego 1` (el modo se crea en cada partida)
`ModoJuego* ⋯> Juego`	Dependencia («usa»)	El modo recibe a `Juego` por parámetro en `getRanking(j)` y `registrarPuntaje(j, e)`

```
        public class Juego implements ControladorJuego {
            protected Ranking rankingClasico, rankingContrarreloj, rankingNoHit;   // los tres, desde que arranca
            public void agregarAlRankingClasico(EntradaRanking e)      { rankingClasico.insertar(e); }
            public void agregarAlRankingContrarreloj(EntradaRanking e) { rankingContrarreloj.insertar(e); }
            public void agregarAlRankingNoHit(EntradaRanking e)        { rankingNoHit.insertar(e); }
        }
```

A favor	En contra
El diagrama dice qué ranking es cada uno, sin depender de posiciones 0, 1, 2	Tres flechas hacia `Ranking` en lugar de una
No usa `instanceof`: el modo elige su ranking por doble despacho	Agregar un modo nuevo obliga a tocar `Juego` (atributo y métodos nuevos), igual que hoy
La lógica de ordenar, insertar y guardar sigue en `Ranking`	`getRankings()` tiene que armar el arreglo para el menú

Otras variantes que se evaluaron (tampoco aplicadas):

Variante	Idea	Por qué no se eligió
A	`Juego ◆ Ranking (#rankings 3)`, `Ranking → ModoJuego (#modo)`, `Juego ◆ ModoJuego (#modos 3)`	Cuatro relaciones para tres clases; obliga a crear los tres modos al arrancar
B	`Juego ◆ ModoJuego (#modos 3)`, `ModoJuego ◆ Ranking`	Obliga a crear los tres modos al arrancar y a reiniciar su estado en cada partida
C	`Ranking` con atributo `nombreModo`	El vínculo modo–ranking queda en un texto, no en el diagrama
D	`GestorRanking` que compone los tres rankings	Una clase más; conviene solo si la persistencia se complica
E1	`Juego` con tres listas de `EntradaRanking`, sin clase `Ranking`	`Juego` también ordenaría, insertaría y persistiría: demasiadas responsabilidades

---

14. Guía para la defensa

14.1 Qué pide la cátedra y dónde se cumple

> Requerimientos del PDF **"Requerimientos Defensa Nº 4"** (defensa del martes 13/10). Se piden dos diagramas de clases: un **reducido**, con solo nombres de clases y relaciones, y un **extendido**, con nombre, atributos y operaciones de cada clase. El reducido se concentra en los aspectos **lógicos**, el **tratamiento de datos para el parseo de niveles** y las **cuestiones gráficas**: tiene que dar una aproximación clara a la **interconexión entre la parte lógica y la gráfica**, con un mapeo uno a uno entre cada entidad gráfica y su entidad lógica, mostrar cómo un cambio en lo lógico se refleja en lo gráfico, y tener definidos **todos los patrones requeridos**. El extendido agrega la comunicación entre el motor del juego, el personaje, los power-ups y los enemigos, y la detección y resolución de colisiones. Como *tips*, la cátedra sugiere pensar qué son clases y qué interfaces, qué se factoriza, qué clases están parcial o totalmente implementadas, las entidades abstractas que no están en el enunciado (Juego, Mapa, Nivel, Ranking), los patrones, las secuencias de servicios (cargar un nivel, mover a Link, colisión Link-enemigo y Link-power-up), el modelo de 3 capas, y aclara que los hilos pueden esperar.
>
> **Defensa 5:** para el reducido pide lo mismo. El extendido suma el **uso de hilos con sus operaciones** y la entrega suma tres diagramas de secuencia (ver la parte B).

Requerimiento (defensa 4)	Dónde está en el diagrama
Reducido: solo nombres de clases y relaciones.	`Diagrama-reducido-defensa-4.drawio`: cajas sin atributos ni métodos; solo relaciones estructurales (las dependencias no se dibujan por decisión del equipo y se explican en el texto).
Foco en los aspectos **lógicos**.	Entidades (§9), estrategias (§10), `Juego` y la partida (§5 y §11).
Tratamiento de datos para el parseo de niveles.	`GeneradorNivel`, `Nivel`, `Escena`, `Sector` (§6) y la sección dedicada 7. Formato y parseo de niveles: qué línea crea qué objeto y qué relación.
Cuestiones gráficas.	Vista y controladores (§3), Observer (§4), dominios gráficos con Abstract Factory (§8).

Requerimiento (defensa 4)	Dónde está en el diagrama
Aproximación clara a la **interconexión lógica ↔ gráfica**.	`ControladorJuego` (vista → lógica) y `ControladorGrafico` (lógica → vista) (§3); la lógica nunca importa Swing.
Mapeo uno a uno entidad gráfica ↔ entidad lógica.	Cada `Entidad*` tiene su `CeldaGrafica` (Link, su `CeldaJugador`): `CeldaGrafica → EntidadLogica` (`#entidadObservada 1`) y `Entidad* → Observer` (`#observadores 0..*`) (§4 y Activar un sector).
Cómo un **cambio lógico se refleja en lo gráfico**.	Patrón Observer: la entidad cambia y llama a `notificarObservers()`; la celda consulta `getX()`, `getY()` y `getRutaImagen()` por `EntidadLogica` y se redibuja (§4; ejemplo en Mover a Link).
Todos los patrones requeridos definidos.	Tabla 14.2. El diagrama tiene seis patrones con nota: Observer (verde), Strategy (amarillo), Abstract Factory (rojo), Visitor (azul), Factory Method (violeta, dos notas) y Template Method de los hilos (naranja). El Template Method de los ataques se explica en el texto (sin nota).
Extendido: nombre, atributos y operaciones.	`Diagrama-extendido-defensa-4.drawio` (constructores, comandos y consultas de cada clase); sus nombres son los que usan los esquemas de este documento.
Comunicación motor ↔ personaje ↔ power-ups ↔ enemigos.	`Juego ▶ Link` (`#jugador`), `Entidad* ▶ Juego` (`#miJuego`), `EntidadAnimada* ▶ Sector` (`#miSector`); `PowerUP*.serRecogidoPor(l)` aplica el efecto sobre Link (Comunicación entre `Juego`, `Sector` y las entidades).
Detección y resolución de **colisiones**.	`Entidad*.intersecta(otra)` y `Sector.verificarColisiones(e)` detectan; el doble despacho `Colisionable` / `Colisionador` (Visitor) resuelve (§9, Colisionador).
Tip: clases vs. interfaces.	Interfaces: controladores, `Observer`, `EntidadLogica`, `EntidadJugador`, Strategy, `Colisionable`, `Colisionador`, `GeneradorDrop`, `GeneradorProyectil` (ver Por qué son interfaces y no clases).
Tip: qué se **factoriza** en clases más generales.	`Entidad*` → `Obstaculo*`, `PowerUP*`, `Objeto*`, `EntidadAnimada*` → `Enemigo*` → `EnemigoConDrop*`, `Jefe*`.
Tip: clases parcial o totalmente implementadas.	Abstractas (con `*`): `Entidad*`, `EntidadAnimada*`, `Enemigo*`, `EnemigoConDrop*`, `Jefe*`, `Obstaculo*`, `PowerUP*`, `Objeto*`, `ModoJuego*`, `FabricaSprites*`, `PanelVista*`. El resto son concretas.
Tip: entidades abstractas que no están en el enunciado (Juego, Mapa, Nivel, Ranking).	`Juego`, `Nivel`, `Escena` (el mapa), `Sector`, `Ranking`, `EntradaRanking`, `PersistenciaRanking`, `Reloj`, `ModoJuego*`, `GeneradorNivel`, `FabricaEntidades`, `Posicion`, `Direccion`.
Tip: secuencias de servicios (cargar nivel, mover a Link, colisión Link-enemigo, Link-power-up).	14.3 Escenarios, con un diagrama de secuencia para cada uno.
Tip: modelo de 3 capas.	Vista (`GUI`, paneles, celdas), lógica (`Juego`, entidades, estrategias) y datos (`GeneradorNivel` con los archivos de nivel, `PersistenciaRanking`), comunicadas por interfaces (§2).
Tip: los hilos pueden esperar.	Defensa 5: cuatro hilos dibujados en el reducido (`HiloJugador`, `HiloMundo`, `Reloj` y `AnimadorTransicion`), con `HiloJuego*` como base y `AccionProgramada` para las esperas (sección 16).

Requerimiento (defensa 4)	Dónde está en el diagrama
Animaciones (de la defensa 3: se pueden dejar para más adelante).	`Sprite` guarda una ruta por estado (`setEstado`, `getRutaActual`); la animación fina queda pendiente.

14.2 Patrones de diseño

Patrón	Clases	Qué problema resuelve
Strategy	`EntidadAnimada*` ◇ `ComportamientoMovimiento`, `ComportamientoAtaque`, `ComportamientoDefensa` y sus 12 implementaciones.	Cada entidad combina cómo se mueve, ataca y se defiende, y lo **cambia en ejecución** (la trampa pasa de quieta a inducida, Link usa el escudo, la bellota reduce el daño, StoneHinox alterna ataques) sin subclases por combinación ni `if`.
Observer	`Entidad*` ▶ `Observer` (`#observadores`); `CeldaGrafica`, `CeldaJugador`; `EntidadLogica`, `EntidadJugador`.	La lógica avisa que cambió **sin conocer Swing**; la celda consulta lo que necesita por `EntidadLogica`.
Abstract Factory	`FabricaEntidades` ▶ `FabricaSprites*` ← `FabricaSpritesOriginal`, `FabricaSpritesAlternativa`; «crea» `Sprite`.	Los **dos dominios gráficos**: cambiar de dominio es cambiar de fábrica, sin tocar ninguna entidad.
Factory Method	`GeneradorDrop` (lo realiza `EnemigoConDrop*`), `GeneradorProyectil` (lo realizan `Octorok`, `Wizzrobe`, `StoneHinox`). En el diagrama: notas violetas "Uso de patrón Factory Method (drops)" y "(proyectiles)", y las dos interfaces en violeta.	Cada enemigo decide **qué** soltar o disparar; el que lo usa no conoce la clase concreta.
Template Method (hilos)	`HiloJuego*` ← `HiloJugador`, `HiloMundo`, `Reloj`, con nota naranja.	`run()` fija el ciclo (repetir, pausar, dormir) y cada hilo redefine solo `ciclo()`, lo que hace en una vuelta.
Template Method	`AtaquePorProyectil` → `AtaqueAvalancha` (sin nota en el diagrama); el mismo esquema se repite en `AtaqueEspada` → `AtaqueRecargado`. Las clases siguen en amarillo porque son parte del Strategy.	El esquema del ataque (`atacar()`) está en la clase base y la subclase cambia solo algunos pasos.
Decorator (pequeño)	`DefensaPorEscudo` → `ComportamientoDefensa` (`#sinEscudo`).	El escudo envuelve a otra defensa y le pasa los golpes que no cubre.
Visitor	`Colisionable` (lo realiza `Entidad*`: `aceptarColision(c)`) y `Colisionador` (lo realiza `EntidadAnimada*`: `colisionarCon(...)`).	Cada colisión se resuelve según los **dos** tipos concretos: la entidad chocada acepta al colisionador (`aceptarColision(c)`) y le devuelve la llamada con su propio tipo (`c.colisionarCon(this)`), sin `instanceof`.
Doble despacho (técnica, no patrón GoF)	La base del Visitor de colisiones; también `recibirGolpeDeEspada`; objeto equipado (`atacarCon`, `protegerCon`); `ModoJuego.getRanking(j)` y `registrarPuntaje(j, e)`.	Resolver qué pasa entre dos tipos concretos **sin `instanceof`** (prohibido).

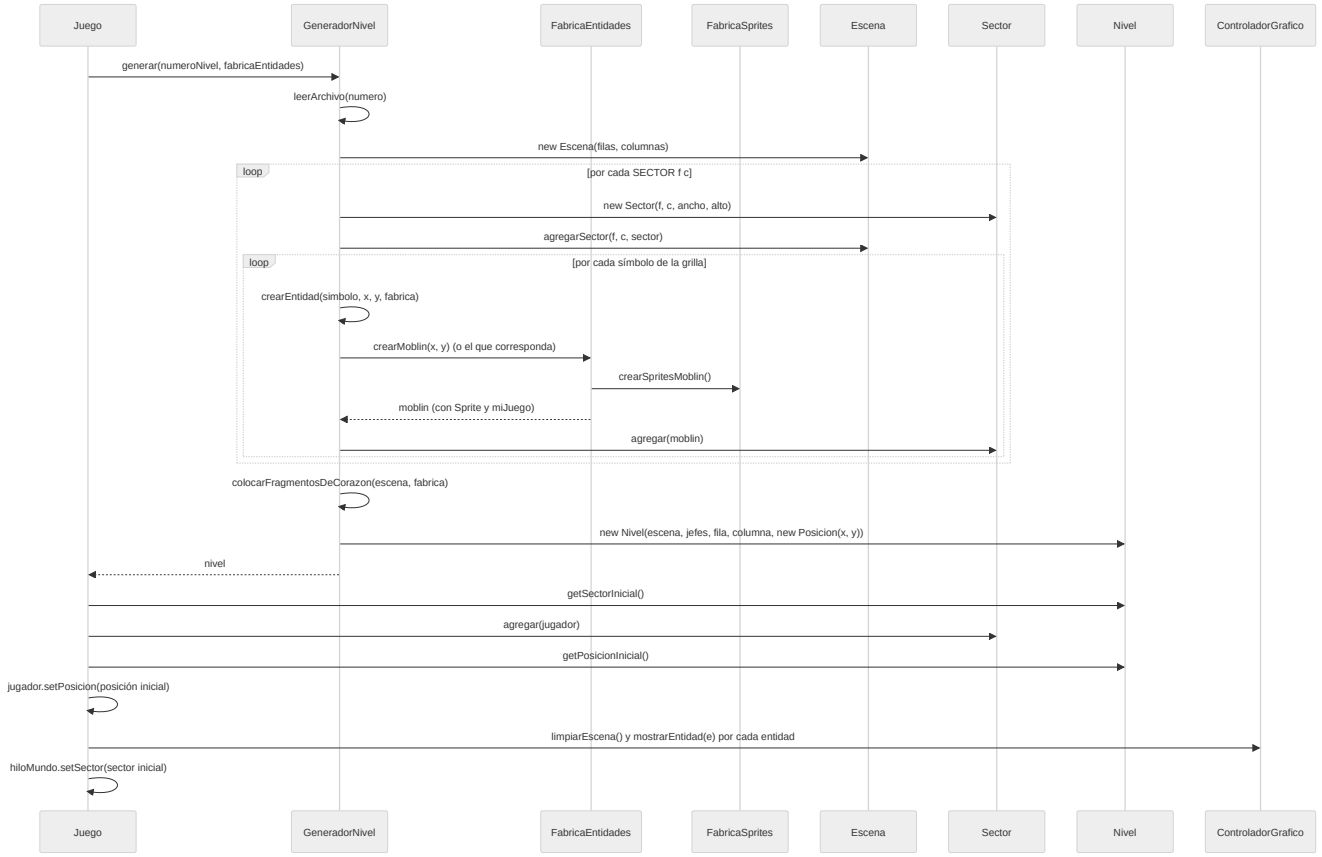
Patrón	Clases	Qué problema resuelve
MVC con interfaces	`ControladorJuego` (vista → lógica), `ControladorGrafico` (lógica → vista), `ControladorDeVistas` (paneles → GUI).	La lógica **nunca importa Swing**; se pueden probar o cambiar por separado.

14.3 Escenarios (secuencias de mensajes)

Los requerimientos sugieren pensar las operaciones a partir de escenarios. Los cuatro primeros son los que nombra la cátedra; los demás son los que suelen preguntar. Todos usan los métodos del extendido de la defensa 4 (cuando un mensaje no está en el extendido, se aclara).

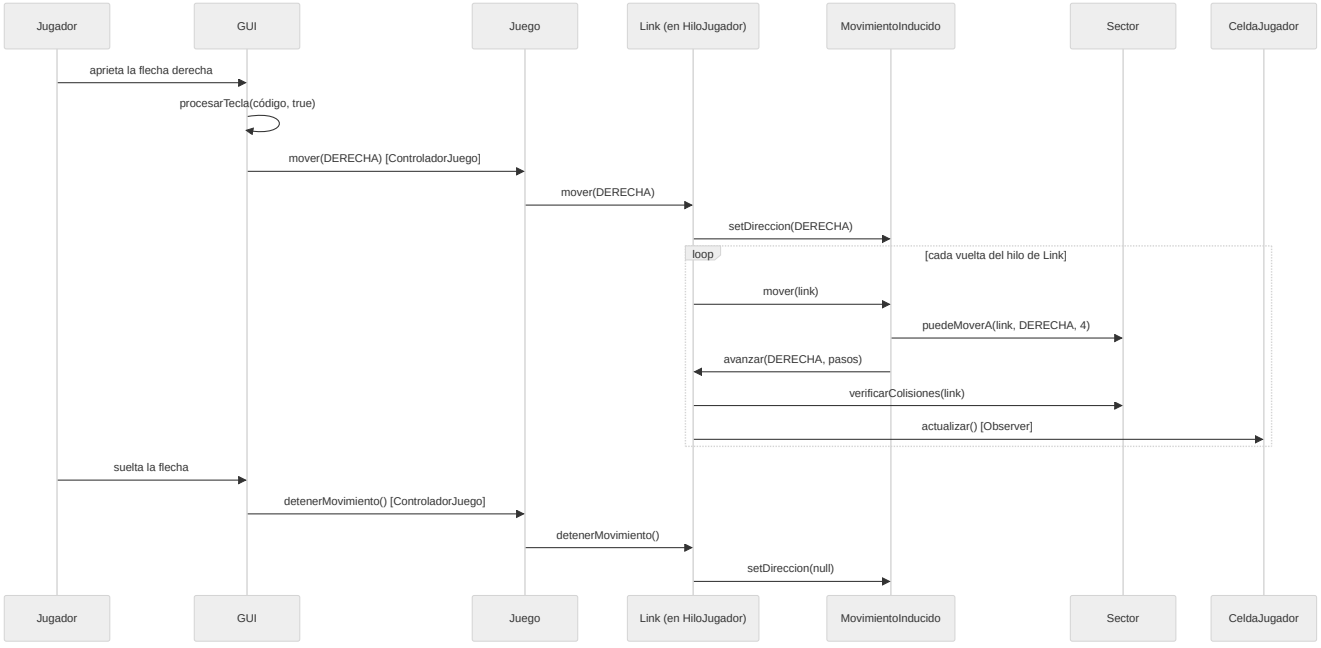
Escenario	Dónde
Cargar un nivel desde un archivo de texto	Abajo y 7.9
Mover a Link	Abajo
Colisión entre Link y un enemigo	Abajo
Colisión entre Link y un power-up	Abajo
Link recoge la espada y la equipa	Abajo
Link pisa un suelo quebradizo y cae al vacío	Abajo
Link ataca y mata a un Moblin	Abajo
Elegir dominio y modo, y empezar la partida	Elección del dominio gráfico
Cambio de sector	Cambio de sector
Fin de partida y ranking	Cómo llega el puntaje al ranking del modo
Disparo, avalancha, teletransporte, bellota	10. Comportamientos
Pasa el tiempo en Contrarreloj	Tiempo y reglas del modo

Cargar un nivel desde un archivo de texto



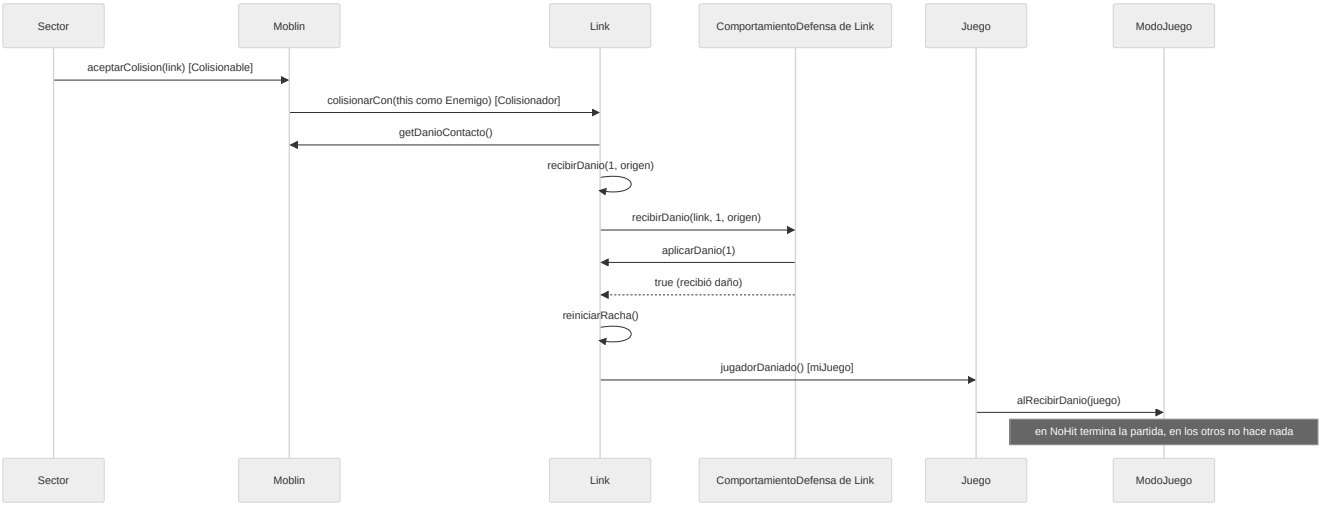
Para explicar: `Juego` le pasa al generador la fábrica del dominio elegido; el generador lee el archivo (`MAPA`, `CELDAS`, `INICIO`, bloques `SECTOR`), crea cada entidad pidiéndosela a la fábrica, arma la escena y devuelve el `Nivel`. Con `nivel1.txt`: `MAPA 2 3` crea la `Escena` de 6 sectores, `INICIO 0 0 2 5` deja a Link en (96, 240) del sector (0,0), y la `M` de la fila 5 del primer sector es `crearMoblin(384, 240)`. Detalle en 7. Formato y parseo de niveles.

Mover a Link



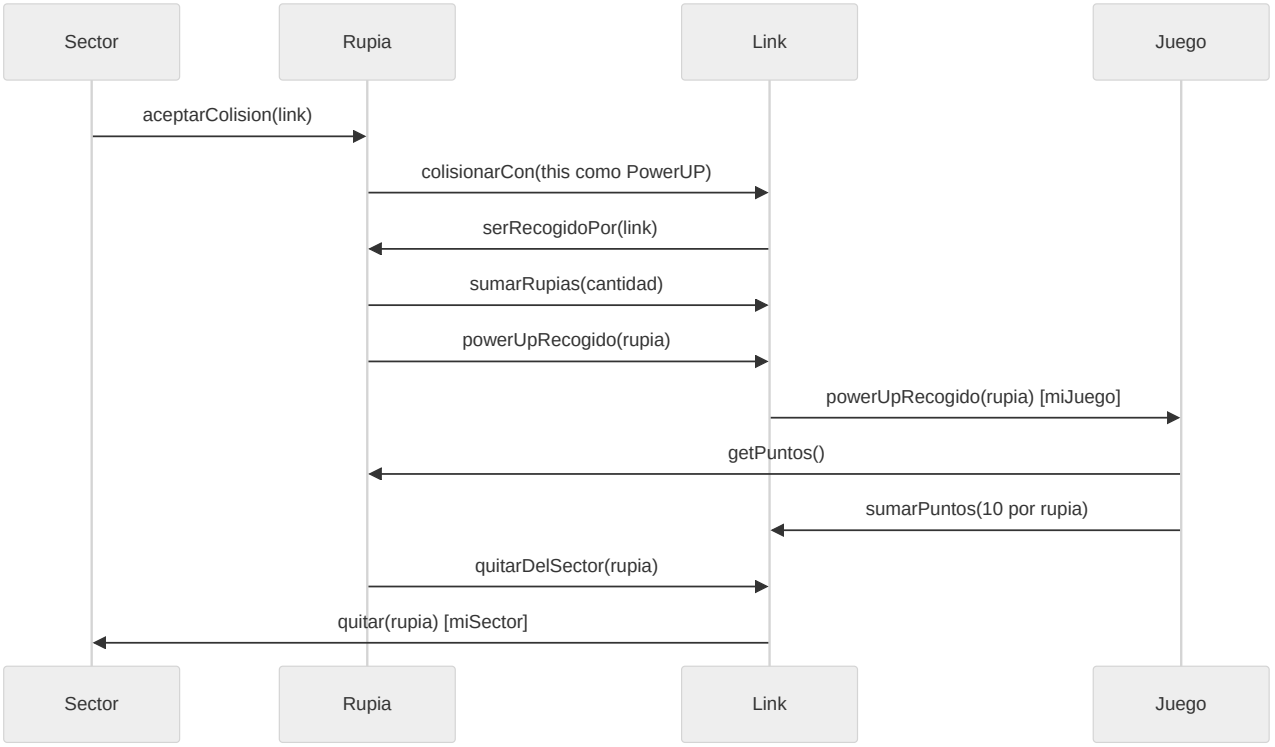
Para explicar: el teclado solo fija la dirección; el hilo de Link es el que mueve. Si Link sale por el borde, `Juego.cambiarDeSector(d)` le pide a la `Escena` el vecino (`getVecino(s, d)`) y la vista hace la transición.

Colisión entre Link y un enemigo (doble despacho)



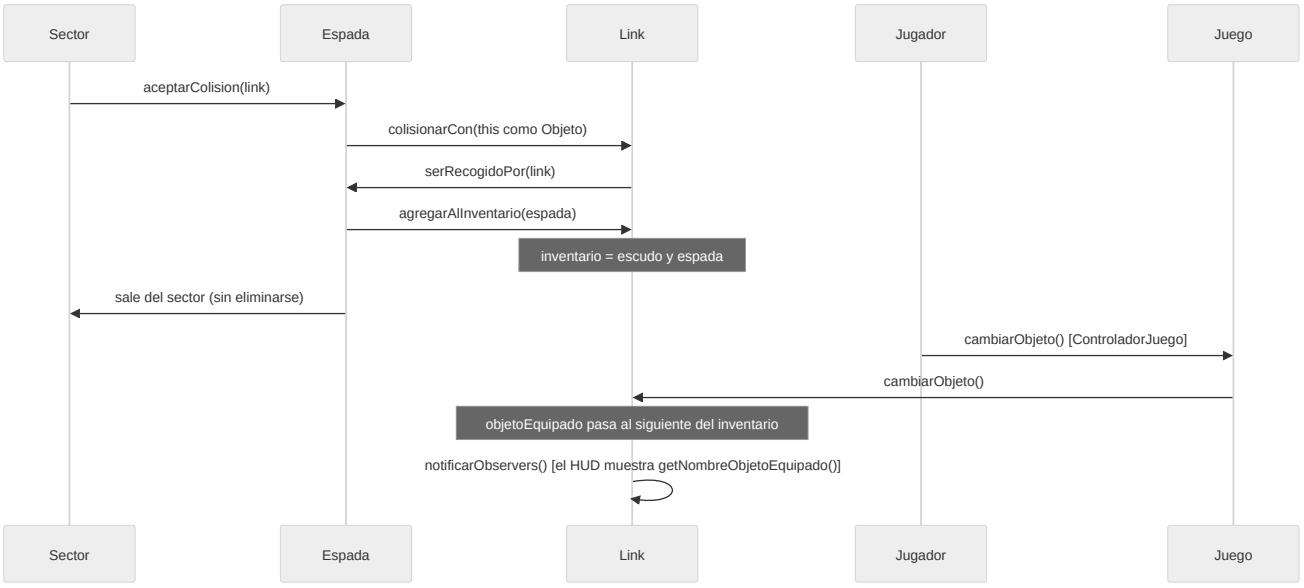
Para explicar: el sector detecta que los rectángulos se tocan (`intersecta`) y llama a `aceptarColision`. El Moblin responde "soy un enemigo" llamando a `colisionarCon(Enemigo)`: Link sabe qué hacer **sin** `instanceof`. La estrategia de defensa decide si el daño se aplica (vulnerable, escudo, invulnerable o reducida). Si hubo daño, Link corta la racha y avisa a `Juego`, que consulta al modo.

Colisión entre Link y un power-up



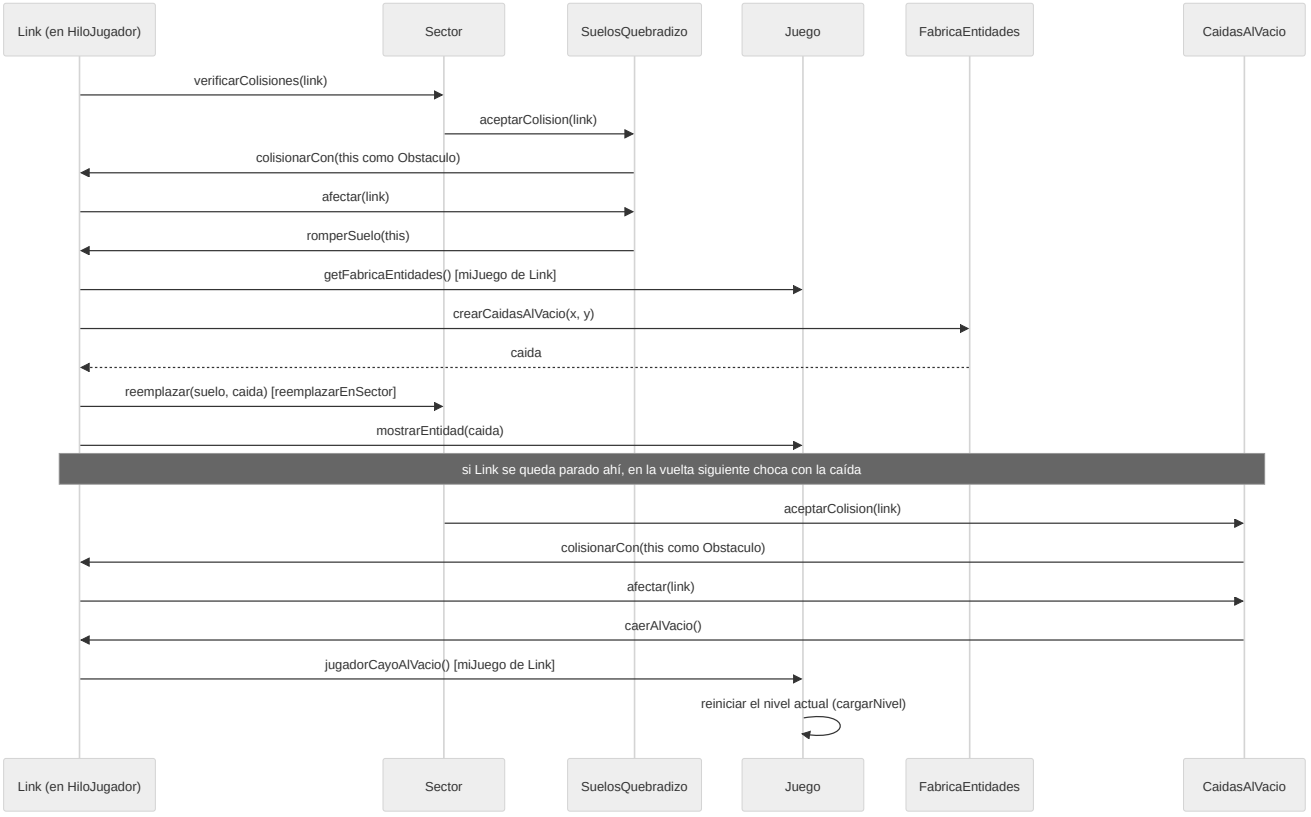
Para explicar: el efecto del power-up se aplica **sobre Link** (`serRecogidoPor(l)`), como piden los requerimientos. El puntaje lo suma solo `Juego`. La rupia sale del sector y su celda se borra (`estaEliminada()`). Otros efectos: `Corazon` llama a `curar`, `FragmentoDeCorazon` a `sumarFragmento`, y la `Bellota` le pone a Link una `DefensaReducida`.

Link recoge la espada y la equipa



Para explicar: la espada está en el mapa (la `X` del sector (0,1) del nivel 1) como cualquier entidad. Al tocarla, el doble despacho llega a `serRecogidoPor`, que la agrega a `#inventario` (agregación: el objeto existía antes de que Link lo tenga). Debe salir del sector **sin** eliminarse (para que siga existiendo en el inventario), por ejemplo con `Sector.retirar(e)`. `cambiarObjeto()` recorre el inventario y actualiza `#objetoEquipado`; el HUD lo muestra con `getNombreObjetoEquipado()`.

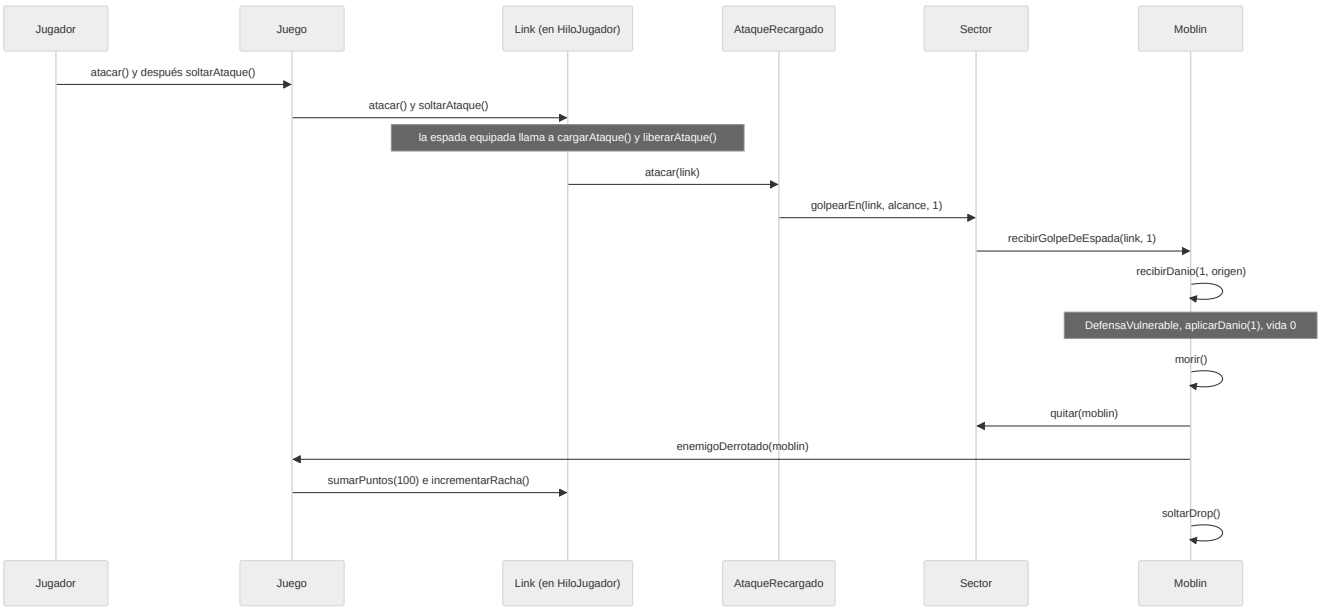
Link pisa un suelo quebradizo y cae al vacío



Para explicar: los dos obstáculos usan el mismo doble despacho (`afectar(l)`); cada uno decide su efecto. El suelo le pide a Link `romperSuelo(this)` y Link lo reemplaza en el sector por una caída al vacío; la caída le pide a Link `caerAlVacio()` y Link avisa a `Juego`, que reinicia el nivel (lo que

pasa exactamente al caer es una ambigüedad del enunciado, ver contexto §7). Como los avisos los da Link con su `#miJuego`, los obstáculos no lo necesitan (ver 13.1).

Link ataca y mata a un Moblin



Para explicar: el teclado solo marca la intención; el golpe lo da el hilo de Link. El sector reparte el golpe a lo que está delante (doble despacho con `recibirGolpeDeEspada`), la defensa del Moblin lo acepta, y al morir el Moblin avisa a la cancha (se saca) y al árbitro (`enemigoDerrotado`); después, como es un `EnemigoConDrop*`, deja el drop (`super.morir()` y luego `soltarDrop()`).

14.4 Preguntas probables y respuestas cortas

Pregunta	Respuesta
¿Qué cambió respecto del reducido de la defensa 3?	Las mismas clases y las mismas relaciones estructurales; el equipo decidió no dibujar las dependencias (se explican en el texto, por ejemplo `Posicion «usa» Direccion`, parámetro de `desplazada`) y se agregaron notas de Factory Method (violeta).
¿Por qué `Posicion → Direccion` es dependencia y `EntidadAnimada* → Direccion` es asociación?	`EntidadAnimada*` guarda un atributo `direccion` (`#direccion 1`); `Posicion` solo recibe la dirección por parámetro y no la guarda.
¿Por qué `Juego` y no `Link` lleva la partida?	`Link` guarda su estado (vidas, puntaje, rupias); `Juego` coordina: niveles, modo, ranking, tiempo y sectores. Es el único que modifica el puntaje.
¿Cómo evitan `instanceof`?	Con doble despacho: colisiones (`aceptarColision` / `colisionarCon`), golpes de espada (`recibirGolpeDeEspada`), objeto equipado (`atacarCon`, `protegerCon`) y ranking (`getRanking(j)` / `registrarPuntaje(j, e)`).
¿Cómo se comunican lógica y vista?	Por dos interfaces: `ControladorJuego` (vista → lógica) y `ControladorGrafico` (lógica → vista). Más Observer: cada entidad avisa a su celda.
¿Quién lee el archivo del nivel?	`GeneradorNivel.generar(numero, fabrica)`. `leerArchivo` trae las líneas, `crearEntidad` traduce cada símbolo pidiéndole la entidad a `FabricaEntidades`, y al final devuelve el `Nivel`.

Pregunta	Respuesta
¿Qué crea cada línea del archivo?	`MAPA` crea la `Escena`; `CELDAS` da el tamaño (no crea objetos); `INICIO` da `filaInicial`, `columnaInicial` y la `Posicion` inicial; cada `SECTOR` crea un `Sector`; cada carácter, una entidad (salvo `.`).
¿Por qué el `switch` del parseo no es un `instanceof`?	Traduce texto a objetos; no pregunta el tipo de un objeto ya creado.
¿Por qué `GeneradorNivel` no guarda la fábrica?	La recibe por parámetro de `Juego`, que es el único dueño. Así siempre usa la del dominio actual. Es una dependencia que no se dibuja.
¿Por qué `Sector` conoce a sus entidades y no al revés?	La asociación es unidireccional (`#entidades`). Solo las animadas necesitan su sector (`#miSector`), para colisiones y para agregar o quitar entidades.
¿Por qué `Nivel ◆ Escena ◆ Sector` son composiciones?	La escena es de un solo nivel y no vive sin él; el sector es de una sola escena.
¿Por qué `Juego ◇ Nivel` es agregación?	El nivel lo construye `GeneradorNivel` y se reemplaza al pasar o reiniciar el nivel.
¿Por qué las estrategias son agregación y no composición?	Se reemplazan en ejecución y algunas se guardan para reusarlas (StoneHinox, la defensa base de Link).
¿Cómo se cambia el dominio gráfico?	El menú crea la `FabricaSprites` concreta y se la pasa a `Juego.iniciar(modo, dominio)`. Ninguna entidad sabe en qué dominio está.
¿Qué guarda `Sprite`?	Rutas de imágenes por estado (texto). La celda gráfica carga la imagen. Así la lógica no depende de Swing.
¿Qué es `#sinEscudo`?	La defensa que usa `DefensaPorEscudo` cuando el escudo no cubre el golpe (por ejemplo, un golpe por la espalda al Moblin).
¿Cómo se usan los hilos?	Defensa 5: `HiloJugador` actualiza a Link, `HiloMundo` a las animadas del sector visible y `Reloj` lleva el tiempo; los tres heredan de `HiloJuego*`. La transición corre en `AnimadorTransicion`. No se usa `javax.swing.Timer` (prohibido). Al cambiar de sector se pausan los dos primeros y `HiloMundo` pasa al sector nuevo. Más preguntas en 16.12.
¿Cómo se guarda el ranking?	`PersistenciaRanking` carga y guarda un archivo de texto por modo. Se pide el nombre solo si el puntaje entra al Top 5 (`getRanking` + `entraEnTop5`); se inserta con `registrarPuntaje`.
¿Cómo se ubican los 4 fragmentos de corazón?	`GeneradorNivel.colocarFragmentosDeCorazon` los pone al azar en 4 sectores distintos, en posiciones libres. No están en el archivo.
¿Qué falta por definir?	Pasar los hilos al extendido (16.11). La regla de colisiones con hilos está en 16.7 y el aviso del `Reloj` ya está resuelto: `Reloj → Juego` (`#juego 1`) y `juego.tic(segundos)`.

---

15. Posición en píxeles y posición en la matriz

Cómo usan la posición las dos clases de entidades del diagrama extendido: las **animadas**, que se mueven de a píxeles con `Posicion`, y las **fijas**, que el `Sector` guarda en la celda de su posición dentro de la matriz `entidades : Entidad[][]` (16 × 11 celdas de `CELDA = 48` píxeles).

15.1 La idea en una frase

Todas las entidades tienen una `Posicion` en **píxeles** (su esquina superior izquierda, en coordenadas del sector). Las **fijas** no se mueven nunca, así que el `Sector` las guarda en la **celda** que corresponde a esa posición (`entidades[fila][columna]`) y las encuentra al instante. Las **animadas** cambian de posición de a pocos píxeles en cada vuelta de su hilo, pueden quedar entre dos celdas y encima de algo fijo, así que van aparte, en la lista `animadas`.

	Entidad fija	**Entidad animada**
Ejemplos	`Roca`, `Agua`, `Pasto`, `SuelosQuebradizo`, `Rupia`, `Corazon`, `Espada`	`Link`, enemigos, jefes, `Proyectil`
Dónde la guarda el `Sector`	`entidades[fila][columna]` (una por celda)	`animadas : List<EntidadAnimada>`
Su posición	Múltiplo exacto de 48 (alineada a una celda)	Cualquier píxel dentro del sector
Cambia de posición	Nunca	En cada vuelta del hilo (`avanzar`) o de golpe (`setPosicion`)
Cómo entra al sector	`agregar(e)` → `e.ubicarseEn(this)` → `agregarFija(this)`	`agregar(e)` → `e.ubicarseEn(this)` → `agregarAnimada(this)`
Para qué sirve guardarla así	Saber en tiempo constante qué hay en una celda	Recorrer solo lo que se mueve

---

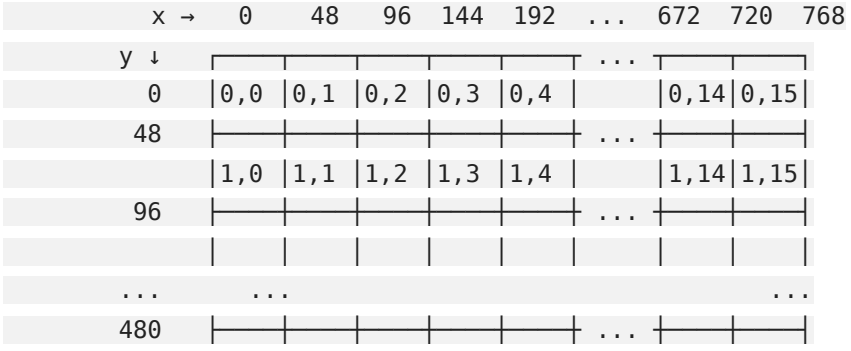
15.2 Clases y métodos que participan

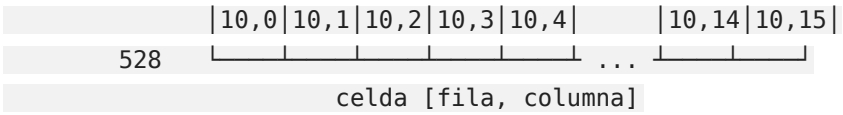
Clase	**Miembro**	**Papel en la posición**
`Posicion`	`x : int`, `y : int` (privados)	Punto en píxeles dentro del sector. Inmutable.
	`getX()`, `getY()`	Leer las coordenadas.
	`desplazada(d : Direccion, pasos : int) : Posicion`	La posición nueva tras avanzar `pasos` píxeles en `d`.
	`distancia(otra : Posicion) : double`	Distancia euclídea (golpes, avalancha, `avanzarHacia`).
`Direccion`	`getDx()`, `getDy()`	-1, 0 o 1 en cada eje: cuánto suma un paso.
	`opuesta()`	Dirección contraria (escudo, borde opuesto).
`Entidad*`	`# posicion : Posicion`, `# ancho`, `# alto`	El rectángulo que ocupa.
	`getPosicion()`, `getX()`, `getY()`, `getAncho()`, `getAlto()`, `setPosicion(p)`	Leer y cambiar la posición y el tamaño; `setPosicion` notifica a las celdas gráficas (Observer).
	`intersecta(otra : Entidad) : boolean`	Si los dos rectángulos se superponen.
	`bloqueaPaso() : boolean`	Si no se puede atravesar (`Roca`, `Agua`: true).
	`ubicarseEn(s : Sector)`	Doble despacho de `agregar`: responde `s.agregarFija(this)`.

Clase	Miembro	Papel en la posición
EntidadAnimada*	`avanzar(d, pasos)`, `avanzarHacia(destino, pasos)`	Moverse de a píxeles.
	`ubicarseEn(s : Sector)`	Redefinido: responde `s.agregarAnimada(this)`.
	`# miSector`, `# direccion`	En qué sector está y hacia dónde mira.
	`run()` (defensa 5: `actualizarEstado()`)	Una vuelta: mueve, ataca y verifica colisiones. La llama `HiloJugador` o `HiloMundo`.
Link	`entrarPorBordeOpuesto(d : Direccion)`	Al cambiar de sector, se ubica en el borde contrario.
Sector	`# entidades : Entidad[][]`	Matriz de lo fijo.
	`# animadas : List<EntidadAnimada>`	Lista de lo que se mueve.
	`# ancho`, `# alto`, `getAncho()`, `getAlto()`	Tamaño del sector en píxeles (768 × 528).
	`# CELDA : int = 48` (estática)	Lado de cada celda: para crear la matriz y pasar de píxeles a celda.
	`agregar`, `agregarFija`, `agregarAnimada`, `quitar`, `retirar`, `reemplazar`	Altas, bajas y reemplazos.
	`puedeMoverA(e, d, pasos)`, `verificarColisiones(quien)`, `estaFuera(p)`	Lo espacial durante el movimiento.
	`posicionLibreAlAzar()`, `celdaLibreCerca(p)`, `getCentro()`, `direccionEntre(desde, hasta)`, `estaEnLinea(a, b)`	Consultas sobre posiciones.
MovimientoAleatorio, MovimientoInducido, MovimientoTeletransporte	`mover(e : EntidadAnimada)`	Deciden **cuánto y hacia dónde** se mueve la entidad en cada vuelta.

15.3 Las medidas del sector: píxeles y celdas

La línea `CELDAS 16 11 48` del archivo de nivel dice que cada sector tiene **16 columnas × 11 filas** de celdas de **48 × 48 píxeles**. Por eso `GeneradorNivel` crea cada sector con `new Sector(f, c, 768, 528)`: 16 × 48 = 768 de ancho y 11 × 48 = 528 de alto.





- El **(0, 0)** en píxeles es la esquina superior izquierda **del sector**, no del mapa. Todas las posiciones son locales al sector: para saber dónde está una entidad en el mapa hace falta además saber en qué sector está.
- `x` crece hacia la derecha y es la **columna**; `y` crece hacia abajo y es la **fila**. Ojo con el orden: la matriz se indexa `entidades[fila][columna]`, o sea `entidades[y / 48][x / 48]`, al revés que `(x, y)`.
- La matriz tiene 11 filas y 16 columnas: `new Entidad[alto / 48][ancho / 48]` = `new Entidad[11][16]`.

Para qué sirve `CELDA` (= 48). En el extendido, ese 48 es la constante estática `# CELDA : int = 48` de `Sector` (subrayada). Las entidades guardan su posición en **píxeles**, pero `Sector` guarda lo fijo en una **matriz de casilleros**: `CELDA` es el número que traduce de uno a otro. Sin ella, el sector no podría saber en qué casillero cae un punto y tendría que recorrer todas sus entidades para ver cuál está cerca.

Para qué	Cómo la usa	Métodos
Armar la matriz del tamaño justo	`new Entidad[alto / CELDA][ancho / CELDA]` = 11 × 16	constructor de `Sector`
Guardar una entidad fija en su casillero	`entidades[y / CELDA][x / CELDA] = e`	`agregarFija`, `quitar`
Mirar solo los casilleros cercanos cuando algo se mueve	casilleros desde `x / CELDA` hasta `(x + ancho - 1) / CELDA`	`puedeMoverA`, `verificarColisiones`
Alinear una posición a la grilla	múltiplos de `CELDA`	`celdaLibreCerca`, `Link.entrarPorBordeOpuesto`

Ejemplo: Link está en (100, 240) y quiere moverse a la derecha. El sector calcula `columna = 100 / 48 = 2` y `fila = 240 / 48 = 5`, y mira solo `entidades[5][2]` y `entidades[5][3]` para saber si hay una roca. Es una constante (y no un atributo de cada sector) porque todas las celdas de todos los sectores miden lo mismo.

15.3.1 Pasar de celda a píxeles

Lo hace `GeneradorNivel` al leer el archivo: el carácter de la fila `r` y la columna `k` del bloque crea la entidad en

`x = k × 48` `y = r × 48`

Celda `[fila, columna]`	Píxeles `(x, y)`
`[0, 0]`	(0, 0)
`[2, 3]`	(144, 96)
`[5, 8]`	(384, 240)
`[10, 15]`	(720, 480)

15.3.2 Pasar de píxeles a celda

Lo hace el `Sector` cada vez que necesita saber en qué celda cae un punto (división entera por su constante `CELDA = 48`):

```
fila    = y / CELDA
columna = x / CELDA
```

Píxeles `(x, y)`	**Celda `[fila, columna]`**	**Por qué**
(144, 96)	`[2, 3]`	96 / 48 = 2, 144 / 48 = 3
(100, 240)	`[5, 2]`	100 / 48 = 2 (resto 4)
(143, 95)	`[1, 2]`	justo antes del borde de `[2, 3]`
(767, 527)	`[10, 15]`	el último píxel del sector

Para una entidad fija el resto siempre es 0 (su posición es múltiplo de 48). Para una animada casi nunca: una posición como (100, 240) está 4 píxeles adentro de la columna 2.

15.3.3 Qué celdas toca un rectángulo

Una entidad ocupa el rectángulo que va de `(x, y)` a `(x + ancho - 1, y + alto - 1)`, con `getAncho()` y `getAlto()`. Con entidades de 48 × 48, una animada puede tocar **hasta 4 celdas** a la vez:

```
filaDesde    = y / 48                  filaHasta    = (y + alto - 1) / 48
columnaDesde = x / 48                  columnaHasta = (x + ancho - 1) / 48
```

Link en `(x, y)`	**Filas**	**Columnas**	**Celdas que toca**
(96, 240)	5 a 5	2 a 2	solo `[5, 2]` (está alineado)
(100, 240)	5 a 5	2 a 3	`[5, 2]` y `[5, 3]`
(100, 250)	5 a 6	2 a 3	`[5, 2]`, `[5, 3]`, `[6, 2]` y `[6, 3]`

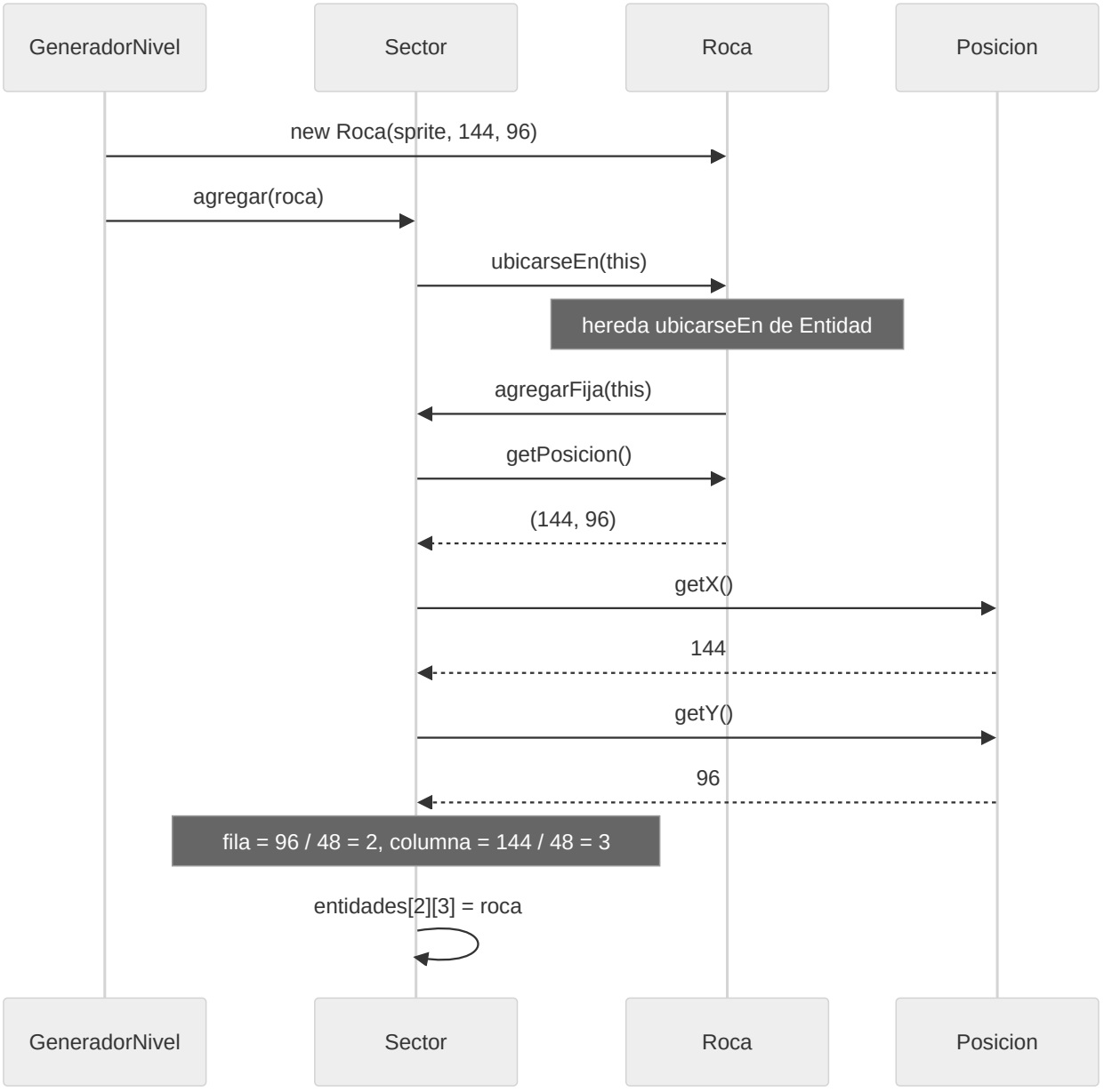
Esto es lo que usa `verificarColisiones` para no recorrer las 176 celdas: mira solo estas.

15.4 Entidades fijas: la posición en la matriz

15.4.1 Cómo llega una entidad fija a su celda

`Sector.agregar(e : Entidad)` no puede preguntar si `e` es fija o animada (no se usa `instanceof`). Le pregunta a la entidad con **doble despacho**:

1. `Sector.agregar(e)` envía `e.ubicarseEn(this)`.
2. La entidad responde según su clase:
   - `Entidad*` (todas las fijas heredan esta versión): `s.agregarFija(this)`.
   - `EntidadAnimada*` (la redefine): `s.agregarAnimada(this)`.
3. `agregarFija(e)` calcula la celda con la posición de `e` y la guarda en `entidades[fila][columna]`.



Paso	Emisor	Receptor	Mensaje	Qué pasa
1	`GeneradorNivel`	`Roca`	`new Roca(sprite, 144, 96)`	Carácter `R` en la fila 2, columna 3 del bloque: 3 × 48 = 144, 2 × 48 = 96.
2	`GeneradorNivel`	`Sector`	`agregar(roca)`	El generador no sabe ni le importa si es fija.
3	`Sector`	`Roca`	`ubicarseEn(this)`	Primer paso del doble despacho.
4	`Roca`	`Sector`	`agregarFija(this)`	La versión heredada de `Entidad*`.
5 a 10	`Sector`	`Roca`, `Posicion`	`getPosicion()`, `getX()`, `getY()`	Lee (144, 96).
11	`Sector`	`Sector`	`entidades[2][3] = roca`	Queda en su celda. No recibe `setSector`: las fijas no guardan su sector.

15.4.2 Ejemplo con el sector (0,0) de `nivel1.txt`

```
     columna: 0123456789012345
fila  0       RRRRRRRRRRRRRRRR
```

```
fila  1       R............R
fila  2       R..PPP......$..R
fila  3       R..PPP.........R
fila  4       R..............
fila  5       R.......M.......
fila  6       R..............
fila  7       R...C.....PPP..R
fila  8       R.........PPP..R
fila  9       R............R
fila 10       RRRRRRRRRRRRRRR
```

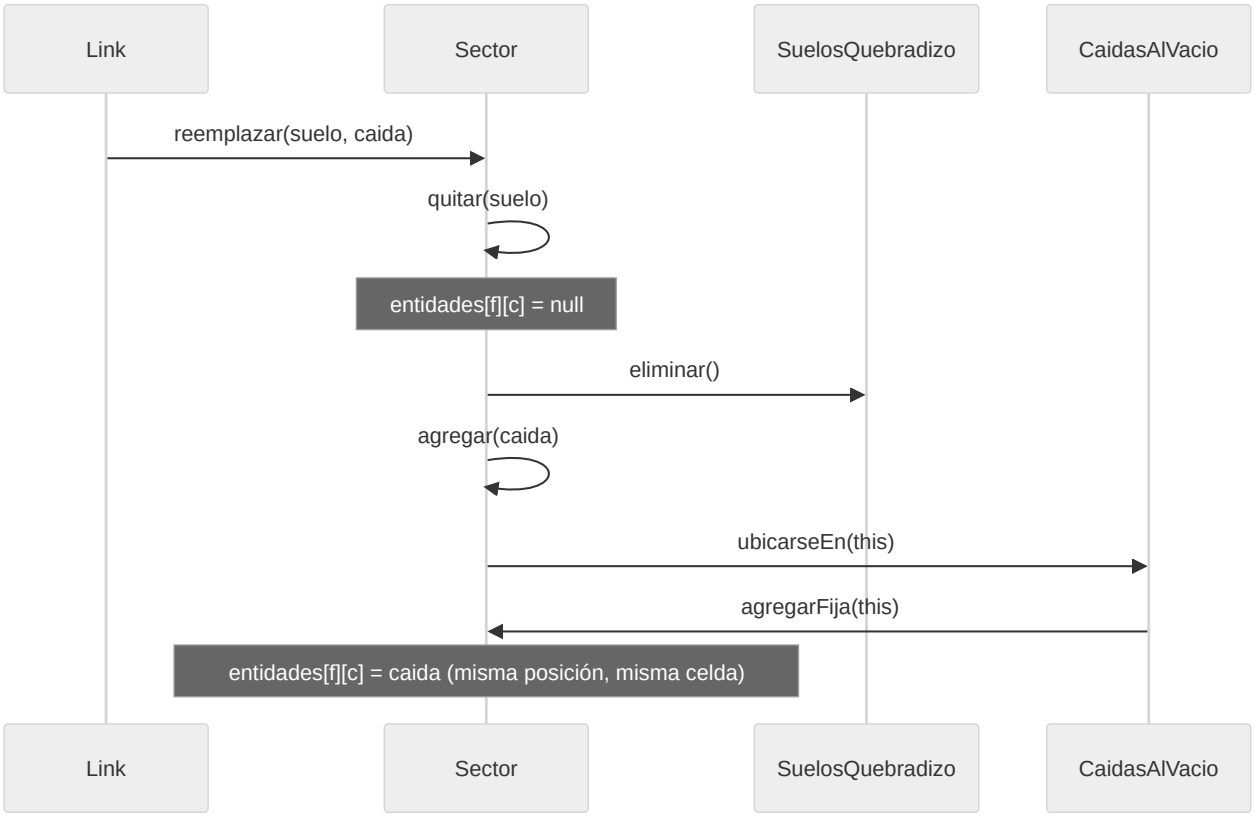
Cómo queda la matriz después de la carga (algunas celdas):

Celda	Carácter	Posición creada	Qué guarda `entidades[f][c]`
`[0, 0]` … `[0, 15]`	`R`	(0, 0) … (720, 0)	una `Roca` por celda
`[2, 3]`, `[2, 4]`, `[2, 5]`	`P`	(144, 96), (192, 96), (240, 96)	un `Pasto` cada una
`[2, 12]`	`$`	(576, 96)	una `Rupia`
`[7, 4]`	`C`	(192, 336)	un `Corazon`
`[5, 8]`	`M`	(384, 240)	`null`: el `Moblin` es animada y va a `animadas`
`[5, 2]`	`.`	—	`null`: ahí aparece Link (`INICIO 0 0 2 5`), también en `animadas`
`[4, 15]`, `[5, 15]`, `[6, 15]`	`.`	—	`null`: el paso al sector (0,1)

Al terminar: la matriz tiene las rocas de los bordes, el pasto, la rupia y el corazón; `animadas` tiene el Moblin (y Link, cuando `Juego` lo agrega al sector inicial).

15.4.3 Sacar y reemplazar una entidad fija

- **`quitar(e)`**: calcula la celda de `e` con su posición; si `entidades[fila][columna] == e`, la pone en `null`. Si no estaba ahí, la busca en `animadas`. Después `e.eliminar()`. No hace falta doble despacho: mirar en las dos estructuras alcanza.
- **`reemplazar(vieja, nueva)`**: saca la vieja y agrega la nueva **en la misma celda** (la nueva se crea con la posición de la vieja). Es el caso de `SuelosQuebradizo` → `CaidasAlVacio`:



- **Cortar pasto** con la espada: `Sector.golpearEn` encuentra el `Pasto` por distancia, el pasto responde `recibirGolpeDeEspada` y termina en `quitar(pasto)`: su celda queda en `null`.

15.4.4 Entidades fijas que aparecen durante la partida

No todas las fijas vienen del archivo:

Caso	Quién la crea	Con qué posición	Celda
Fragmento de corazón	`GeneradorNivel.colocarFragmentosDeCorazon`	`sector.posicionLibreAlAzar()`	La de esa posición libre (alineada a 48).
Drop de un enemigo (rupia, corazón)	`EnemigoConDrop.generarDrop`	`getSector().celdaLibreCerca(getPosicion())`	La celda libre más cercana a donde murió el enemigo, ya alineada a 48: el enemigo es animado y puede morir en cualquier píxel.
Bellota (racha)	`Juego.aparecerBellota(x, y)`	La que recibe	La de esa posición.

15.5 Entidades animadas: la posición por movimiento

15.5.1 `Posicion` es inmutable

`Posicion` no tiene `setX` ni `setY`. Moverse **no cambia** la posición: crea una nueva y la entidad la reemplaza con `setPosicion(p)`. Ventajas:

- Dos entidades pueden compartir el mismo objeto `Posicion` sin que una mueva a la otra por accidente (por ejemplo, el drop que se crea con la posición del enemigo).

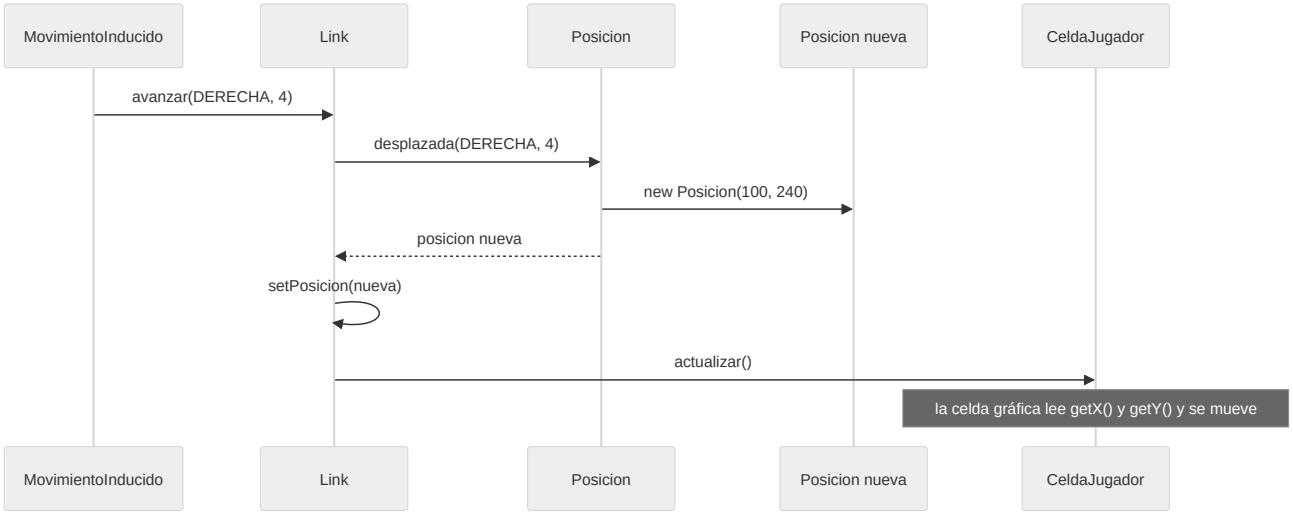
- `Nivel.posicionInicial` y `TrampaDeCuchillas.posicionOriginal` no se corrompen aunque la entidad se mueva.
- Con varios hilos (`HiloJugador`, `HiloMundo` y el de Swing) nadie ve una posición a medio actualizar: se cambia la referencia entera de una vez.

15.5.2 Un paso: `avanzar(d, pasos)`

`EntidadAnimada.avanzar(d : Direccion, pasos : int)` hace:

```
nueva = posicion.desplazada(d, pasos)      // new Posicion(x + d.getDx() × pasos, y + d.getDy() × pasos)
setPosicion(nueva)                          // guarda y notifica a sus observadores (la celda gráfica se mueve)
```

`d`	`getDx()`	`getDy()`	**Desde (96, 240) con 4 pasos**
`DERECHA`	1	0	(100, 240)
`IZQUIERDA`	-1	0	(92, 240)
`ABAJO`	0	1	(96, 244)
`ARRIBA`	0	-1	(96, 236)



15.5.3 Ejemplo: Link camina hacia la derecha

Link arranca en (96, 240), la celda `[5, 2]` del sector (0,0). El jugador mantiene la flecha derecha. Su `MovimientoInducido` tiene velocidad 4, y el hilo de Link da una vuelta cada 30 ms.

Vuelta	**Posición**	**Celdas que toca**	**Qué pasa**
0	(96, 240)	`[5, 2]`	Alineado a una celda.
1	(100, 240)	`[5, 2]`, `[5, 3]`	Ya pisa dos celdas.
2	(104, 240)	`[5, 2]`, `[5, 3]`	—
…	…	…	—

Vuelta	Posición	Celdas que toca	Qué pasa
12	(144, 240)	`[5, 3]`	Otra vez alineado: una celda entera más a la derecha.
72	(384, 240)	`[5, 8]`	Llegó a donde empezó el Moblin (si sigue ahí, chocan antes: `intersecta` da true desde x = 340, cuando x + 48 pasa 384).
168	(768, 240)	fuera	`estaFuera` da true: `Juego.cambiarDeSector(DERECHA)` y aparece en x = 0 del sector (0,1).

En 12 vueltas (360 ms) recorre una celda. La matriz no cambia en ningún momento: Link no está en ella.

Sobre el pasto. Si Link toca una celda con `Pasto`, `verificarColisiones` le envía al pasto `aceptarColision(link)`; el pasto responde `link.colisionarCon(this)` y Link se `ralentizar()`: la velocidad baja (avanza 2 píxeles en vez de 4). El pasto sigue en su celda, y Link pasa por encima porque `Pasto.bloqueaPaso()` es false.

15.5.4 Las estrategias de movimiento y la posición

Estrategia	Quién la usa	Cómo cambia la posición
`MovimientoInducido(d, velocidad)`	Link (d la fija el teclado), `Proyectil`, trampa y Moblin con escudo al abalanzarse	Si `d != null` y `puedeMoverA`: `avanzar(d, velocidad)`. Con `d == null` (Erizo, trampa en reposo) no se mueve.
`MovimientoInducido(objetivo, velocidad)`	`StoneHinox` (caminar al centro, embestir)	`avanzarHacia(objetivo, velocidad)`: `direccionEntre(posicion, objetivo)` da el eje con mayor diferencia y avanza en esa dirección.
`MovimientoAleatorio(velocidad, intervalo)`	`Moblin`, `Octorok`	Avanza en su dirección; si está bloqueado o pasó el intervalo, sortea otra.
`MovimientoTeletransporte`	`Wizzrobe`	No usa `avanzar`: `setPosicion(sector.posicionLibreAlAzar())`, un salto directo a otra posición (alineada a celda).

`avanzarHacia` con un ejemplo. StoneHinox en (96, 96) camina al centro del sector, `getCentro()` = (384, 264):

Vuelta	Posición	Diferencias (dx, dy)	Dirección elegida
0	(96, 96)	(288, 168)	`DERECHA` (dx es mayor)
…	…	…	`DERECHA` mientras dx > dy
30 (velocidad 4)	(216, 96)	(168, 168)	empate: el eje x (criterio a fijar)
…	…	…	alterna hasta llegar

15.6 Dónde se cruzan: moverse sobre la matriz

Las animadas no están en la matriz, pero la **usan** en cada paso para saber qué tienen alrededor.

15.6.1 `puedeMoverA`: ¿la celda de adelante bloquea?

`Sector.puedeMoverA(e : EntidadAnimada, d : Direccion, pasos : int) : boolean` (la estrategia le pasa su `velocidad` como `pasos`):

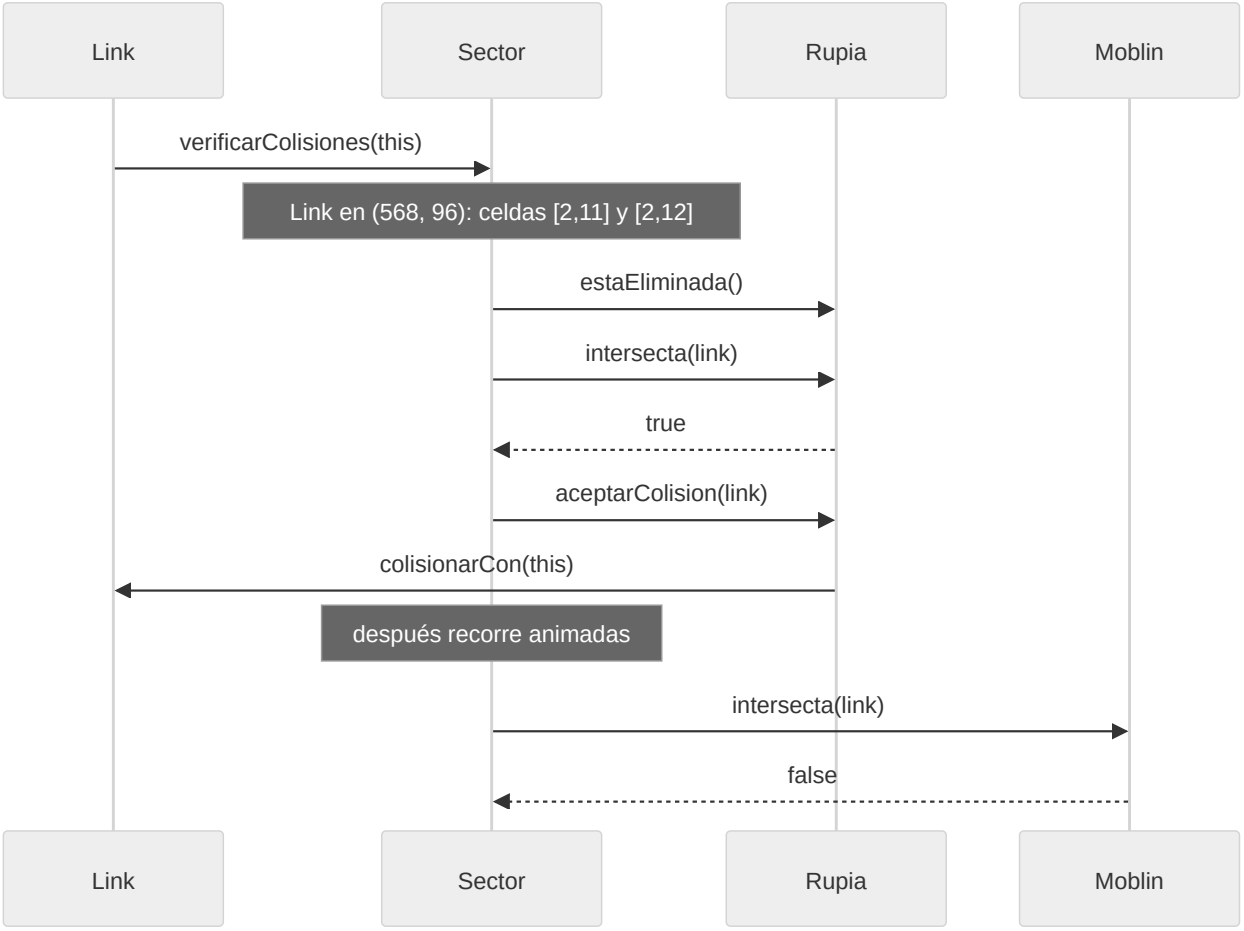
1. Calcula la posición siguiente: `e.getPosicion().desplazada(d, pasos)`.
2. Calcula las celdas que tocaría el rectángulo en esa posición (sección 3.3).
3. Si en alguna de esas celdas hay una entidad con `bloqueaPaso()` true (`Roca`, `Agua`), devuelve false.
4. Si no, true.

Ejemplo: Link en (96, 52) quiere subir 4 píxeles. La posición siguiente es (96, 48), que toca la celda `[1, 2]` (vacía): puede. Desde (96, 48) quiere subir a (96, 44), que toca `[0, 2]`, una `Roca`: **no puede**, se queda pegado al borde.

15.6.2 `verificarColisiones`: con qué se superpone

`Sector.verificarColisiones(quien : EntidadAnimada)` se llama al final de cada vuelta del hilo:

1. **Contra lo fijo:** solo las celdas que toca `quien` (hasta 4). Para cada una con entidad no eliminada que `intersecta(quien)`, envía `e.aceptarColision(quien)`.
2. **Contra lo animado:** recorre `animadas` (salvo `quien`), y hace lo mismo con cada una que se superpone.
3. Cada entidad resuelve con doble despacho: `aceptarColision(c)` → `c.colisionarCon(this)`.



Por qué la matriz ayuda. Con una lista sola, cada vuelta de cada entidad animada recorrería todas las entidades del sector (en el (0,0), más de 50 rocas y pastos). Con la matriz mira como

mucho 4 celdas más las pocas animadas.

15.6.3 Ejemplo completo de un paso del Moblin

El Moblin está en (384, 240), mirando a la `IZQUIERDA`, con `MovimientoAleatorio` de velocidad 2.

#	Mensaje	Posición / celdas	Resultado
1	`vigilarJugador()`	—	Mira dónde está Link (`miJuego.getJugador().getPosicion()`).
2	`movimiento.mover(this)`	—	Strategy.
3	`sector.puedeMoverA(moblin, IZQUIERDA, 2)`	siguiente (382, 240): celdas `[5, 7]`, `[5, 8]`	vacías: true.
4	`avanzar(IZQUIERDA, 2)`	(384, 240) → (382, 240)	`desplazada` crea la nueva, `setPosicion` notifica.
5	`sector.verificarColisiones(this)`	celdas `[5, 7]`, `[5, 8]` y las animadas	Si Link se superpone: `link.aceptarColision(moblin)` → `moblin.colisionarCon(link)` → daño a Link.
6	`CeldaGrafica.actualizar()`	—	La imagen se mueve 2 píxeles.

15.7 Casos especiales

Caso	Qué pasa con la posición	Estructura del sector
Link cambia de sector	Al pasar x = 768 (o x < 0, y fuera) `estaFuera` da true. `Juego.cambiarDeSector(d)` lo pasa al vecino y `link.entrarPorBordeOpuesto(d)` le pone x = 0 (si iba a la derecha) con la misma y.	`retirar(link)` lo saca de `animadas` del viejo **sin eliminarlo**; `agregar(link)` → `agregarAnimada` en el nuevo, que le hace `setSector`.
Proyectil que sale del sector	Su `MovimientoInducido` lo lleva fuera de 768 × 528.	Se quita de `animadas` (`quitarDelSector`) y se elimina.
Proyectil que choca con una roca	`puedeMoverA` da false.	Se quita de `animadas`. La roca sigue en su celda.
Link pierde todas las vidas o cae al vacío	`setPosicion(nivel.getPosicionInicial())` : un salto, sin `avanzar`.	`retirar` del sector actual y `agregar` al sector inicial.
Wizzrobe se teletransporta	`setPosicion(posicionLibreAlAzar())`: salto a una posición alineada a celda.	Sigue en `animadas` del mismo sector.
Trampa de cuchillas	En reposo `MovimientoInducido(null, 8)`; al ver a Link en línea pasa a `MovimientoInducido(dir, 8)`; `volver()` la lleva a `posicionOriginal`.	Siempre en `animadas`, aunque la mayor parte del tiempo esté quieta.
Erizo de mar	`MovimientoInducido(null, 0)`: su posición nunca cambia.	En `animadas` (es un `Enemigo`, que es animado), no en la matriz.
Link sobre pasto	Las dos posiciones se superponen.	El pasto en `entidades[f][c]`, Link en `animadas`: por eso no se pisan.

15.8 Pseudocódigo Java

Cómo se implementaría con los métodos del diagrama. Lo que no está en el diagrama se marca con ⚠: los auxiliares privados (`fijasQueToca`, `dentro(f, c)`, `esperar`).

```
        // ---------- Sector ----------
        protected static final int CELDA = 48;          // # CELDA : int = 48 (subrayado: estático)
        protected Entidad[][] entidades;
        protected List<EntidadAnimada> animadas;

        public Sector(int fila, int columna, int ancho, int alto) {
            this.fila = fila; this.columna = columna; this.ancho = ancho; this.alto = alto;
            entidades = new Entidad[alto / CELDA][ancho / CELDA];   // 11 × 16
            animadas = new ArrayList<>();
        }

        public void agregar(Entidad e) {
            e.ubicarseEn(this);                         // doble despacho: fija o animada
        }

        public void agregarFija(Entidad e) {
            Posicion p = e.getPosicion();
            entidades[p.getY() / CELDA][p.getX() / CELDA] = e;
        }

        public void agregarAnimada(EntidadAnimada a) {
            animadas.add(a);
            a.setSector(this);
        }

        public void quitar(Entidad e) {
            Posicion p = e.getPosicion();
            int f = p.getY() / CELDA, c = p.getX() / CELDA;
            if (dentro(f, c) && entidades[f][c] == e) entidades[f][c] = null;
            else animadas.remove(e);
            e.eliminar();
        }

        public boolean puedeMoverA(EntidadAnimada e, Direccion d, int pasos) {
            Posicion siguiente = e.getPosicion().desplazada(d, pasos);
            for (Entidad fija : fijasQueToca(siguiente, e.getAncho(), e.getAlto()))
                if (fija.bloqueaPaso()) return false;
            return true;
        }

        public void verificarColisiones(EntidadAnimada quien) {
            for (Entidad fija : fijasQueToca(quien.getPosicion(), quien.getAncho(), quien.getAlto()))
                if (!fija.estaEliminada() && fija.intersecta(quien)) fija.aceptarColision(quien);
            for (EntidadAnimada otra : new ArrayList<>(animadas))       // copia: el choque puede quitar entidades
```

```
                if (otra != quien && !otra.estaEliminada() && otra.intersecta(quien))
otra.aceptarColision(quien);
        }

        public Posicion celdaLibreCerca(Posicion p) {
            int f0 = p.getY() / CELDA, c0 = p.getX() / CELDA;
            for (int radio = 0; radio < Math.max(entidades.length, entidades[0].length);
radio++)   // anillos crecientes
                for (int f = f0 - radio; f <= f0 + radio; f++)
                    for (int c = c0 - radio; c <= c0 + radio; c++)
                        if (dentro(f, c) && entidades[f][c] == null) return new Posicion(c *
CELDA, f * CELDA);
            return null;                                        // sector lleno (no debería pasar)
        }

        private List<Entidad> fijasQueToca(Posicion p, int ancho, int alto) {     // ⚠ auxiliar
privado
            List<Entidad> res = new ArrayList<>();
            for (int f = p.getY() / CELDA; f <= (p.getY() + alto - 1) / CELDA; f++)
                for (int c = p.getX() / CELDA; c <= (p.getX() + ancho - 1) / CELDA; c++)
                    if (dentro(f, c) && entidades[f][c] != null) res.add(entidades[f][c]);
            return res;
        }

        // ---------- Entidad ----------
        public void ubicarseEn(Sector s) { s.agregarFija(this); }

        // ---------- EntidadAnimada ----------
        @Override
        public void ubicarseEn(Sector s) { s.agregarAnimada(this); }

        public void actualizarEstado() {                    // defensa 5: una vuelta; la llama
HiloJugador o HiloMundo cada 30 ms
            movimiento.mover(this);                          // la estrategia llama a
puedeMoverA(this, d, velocidad) y avanzar
            ataque.atacar(this);
            miSector.verificarColisiones(this);
            ejecutarPendientes();                            // acciones programadas vencidas
        }

        public void avanzar(Direccion d, int pasos) {
            setPosicion(posicion.desplazada(d, pasos));   // Posicion nueva; notifica a las
celdas
        }

        // ---------- Link ----------
        public void entrarPorBordeOpuesto(Direccion d) {
            Sector s = getSector();
            int x = posicion.getX(), y = posicion.getY();
            switch (d) {
                case DERECHA:    x = 0; break;
```

```
                case IZQUIERDA: x = s.getAncho() - Sector.CELDA; break;
                case ABAJO:     y = 0; break;
                case ARRIBA:    y = s.getAlto() - Sector.CELDA; break;
            }
            setPosicion(new Posicion(x, y));
        }

        // ---------- Posicion ----------
        public Posicion desplazada(Direccion d, int pasos) {
            return new Posicion(x + d.getDx() * pasos, y + d.getDy() * pasos);
        }
```

15.9 Observaciones: lo que faltaba en el diagrama

Todas las observaciones que cambian el diagrama ya están **aplicadas** en el extendido de la defensa 4.

#	Observación	Estado
1	El **tamaño de celda** (48) no estaba en ningún lado, y `Sector` lo necesita para crear la matriz y para pasar de píxeles a celda.	✅ **Aplicada:** `# CELDA : int = 48`, estática (subrayada), en `Sector`.
2	`Sector` ya tiene atributos `fila` y `columna` (su lugar en la grilla de `Escena`). Las filas y columnas de la matriz son otra cosa.	Aclaración, no cambia el diagrama: decir "celda `[f][c]` de la matriz" y "fila/columna del sector".
3	`puedeMoverA(e, d)` no recibía cuántos píxeles avanzaría (`velocidad` está en la estrategia).	✅ **Aplicada:** `puedeMoverA(e : EntidadAnimada, d : Direccion, pasos : int) : boolean`.
4	Un **drop** se creaba con la posición del enemigo muerto, que puede no ser múltiplo de 48 y caer en una celda ocupada.	✅ **Aplicada:** `celdaLibreCerca(p : Posicion) : Posicion` en `Sector`, que usa `EnemigoConDrop.generarDrop`.
5	`Entidad*` no tenía `getAncho()` ni `getAlto()`, que el `Sector` necesita para saber qué celdas toca una animada.	✅ **Aplicada:** `+ getAncho() : int` y `+ getAlto() : int` en `Entidad*`.
6	Faltaban `entrarPorBordeOpuesto(d)` en `Link` y `run()` en `EntidadAnimada*`.	✅ **Aplicada:** los dos métodos están en el extendido.
7	`Link.entrarPorBordeOpuesto` necesita el ancho y el alto del sector para el borde izquierdo y el de arriba, y `Sector` no tenía `getAncho()` ni `getAlto()`.	✅ **Aplicada:** `+ getAncho() : int` y `+ getAlto() : int` en `Sector`.

15.10 Preguntas probables en la defensa

¿Por qué las animadas no van en la matriz? Porque cambian de posición en cada vuelta de su hilo y casi nunca están alineadas a una celda: habría que moverlas de celda todo el tiempo, y una animada puede tocar 4 celdas a la vez o estar encima de algo fijo (Link sobre el pasto).

¿Cómo sabe el `Sector` si una entidad es fija o animada sin `instanceof`? Con doble despacho: `agregar(e)` llama a `e.ubicarseEn(this)`; la entidad sabe qué es y responde `agregarFija(this)` o `agregarAnimada(this)`.

¿Qué guarda `Posicion`: píxeles o celdas? Píxeles, en coordenadas del sector. La celda se calcula cuando hace falta: `fila = y / 48`, `columna = x / 48`.

¿Por qué `Posicion` es inmutable? Para que moverse sea crear una posición nueva y reemplazarla de una vez: nadie comparte una posición que cambia, y los hilos no ven estados a medias.

¿Cómo detecta un choque una entidad que se mueve? Después de avanzar, le pide a su sector `verificarColisiones(this)`. El sector mira las celdas de la matriz que toca y las demás animadas, y a las que se superponen les envía `aceptarColision`. Cada una resuelve con `colisionarCon`.

¿Qué pasa con la matriz cuando Link cambia de sector? Nada: Link no está en la matriz. Se lo saca de `animadas` del sector viejo con `retirar` y se lo agrega al nuevo con `agregar`, que por doble despacho termina en `agregarAnimada`.

---

16. Hilos, animaciones y colisiones (defensa 5)

La defensa 5 pide empezar a analizar **el uso de hilos con sus operaciones**. Esta sección explica cómo quedan los hilos en el reducido, qué clases se agregaron y cómo se resuelven las colisiones cuando hay más de un hilo. Lo nuevo del dibujo está resumido en C.1.

16.1 Resumen

- Se pasa de **un hilo por entidad animada** (más uno por cada `programar`) a **cuatro hilos propios**: el de Link (`HiloJugador`), el del mundo (`HiloMundo`), el del tiempo (`Reloj`, antes `Timer`) y el de la transición (`AnimadorTransicion`). Además está el hilo de Swing (EDT), que no se programa.
- Los tres hilos de la lógica heredan de una clase abstracta, `HiloJuego*`, que tiene un `run()` fijo y deja a cada subclase solo **qué hacer en una vuelta** (`ciclo()`). Es un **Template Method**, y tiene su nota en el diagrama.
- La transición entre sectores es una **animación de la capa gráfica**: `AnimadorTransicion`, tomada de la idea de "animador" del ejemplo de Candy Crush.
- Las esperas (la invulnerabilidad, la trampa que vuelve, etc.) ya no crean hilos: se guardan como `AccionProgramada` y las ejecuta el hilo que actualiza a la entidad.
- Las colisiones **se detectan y se resuelven igual que antes** (el `Sector` las detecta y el doble despacho las resuelve). Lo nuevo es una regla que dice **qué hilo** resuelve cada choque, para que dos hilos no apliquen el mismo daño.

16.2 Los cuatro hilos

Hilo	Clase	Capa	Período	Qué hace en cada vuelta
Link	`HiloJugador`	Lógica	~30 ms	`jugador.actualizarEstado()`: avanza según la intención del teclado, ataca o carga el ataque recargado, **resuelve todas las colisiones de Link** y ejecuta sus acciones programadas.
Mundo	`HiloMundo`	Lógica	~30 ms	`sector.actualizarEntidades()`: una vuelta (`actualizarEstado()`) de cada enemigo, jefe y proyectil del **sector visible**.

Hilo	Clase	Capa	Período	Qué hace en cada vuelta
Tiempo	`Reloj` (antes `Timer`)	Lógica	1 s	`segundos++` y `juego.tic(segundos)`: actualiza el HUD y llama `ModoJuego.alPasarTiempo` (en Contrarreloj descuenta un segundo).
Transición	`AnimadorTransicion`	Gráfica	—	Vive solo durante el desplazamiento: mueve el panel paso a paso y al terminar avisa `transicionTerminada()`.
Swing (EDT)	—	—	—	Teclado (`GUI.procesarTecla`) y dibujo. No se modela.

Por qué Link tiene su propio hilo: responde al teclado y tiene que reaccionar igual de rápido aunque el sector tenga muchos enemigos. Además, como su hilo es el único que lo actualiza, todas sus colisiones quedan en un solo lugar (16.7).

Por qué no un hilo por enemigo: con 6 a 10 enemigos y proyectiles por sector serían decenas de hilos tocando el `Sector` a la vez. Un solo hilo de mundo recorre la lista en orden, así que dos enemigos nunca se actualizan al mismo tiempo.

Enunciado: pide que los movimientos se hagan con `Thread` (nada de `javax.swing.Timer`) y al menos dos hilos simultáneos. Con este esquema hay tres siempre (Link, mundo y tiempo), más la transición cuando cambia el sector.

16.3 Clases nuevas, una por una

Los atributos y operaciones de esta sección son los que habría que agregar al extendido (todavía no está actualizado, ver 16.11). En el reducido solo aparecen los nombres y las relaciones.

`<<Interface>> Runnable`

Interfaz de `java.lang` con un único método, `run()`. Aparece en el diagrama por la misma razón que `JPanel` y `JLabel`: muestra de quién dependen las clases propias. La realizan `HiloJuego*` (y por herencia sus tres subclases) y `AnimadorTransicion`. Un objeto que realiza `Runnable` se le pasa a un `Thread` (`new Thread(this).start()`), y ese hilo ejecuta su `run()`.

`HiloJuego*` (abstracta, realiza `Runnable`)

Base común de los tres hilos de la lógica. Arranca, pausa, reanuda y detiene; cada subclase define solo qué hacer en una vuelta.

Elemento	Detalle
`# hilo : Thread`	El hilo que ejecuta `run()`.
`# activo : boolean`	`volatile`: lo cambia otro hilo (por ejemplo `Juego` al terminar la partida).
`# pausado : boolean`	`volatile`: lo usa la transición.
`# periodo : long`	Milisegundos entre vueltas (30 o 1000).
`# HiloJuego(periodo : long)`	«constructor»
`+ iniciar()`	`activo = true; hilo = new Thread(this); hilo.start();`
`+ detener()`	`activo = false` (termina en la vuelta siguiente).

Elemento	Detalle
`+ pausar()` / `+ reanudar()`	`pausado = true` / `false`.
`+ run()`	Fijo (Template Method): `while (activo) { if (!pausado) ciclo(); Thread.sleep(periodo); }`
`# ciclo()`	Abstracto: una vuelta. Lo redefinen `HiloJugador`, `HiloMundo` y `Reloj`.

```
        public abstract class HiloJuego implements Runnable {
            protected Thread hilo;
            protected volatile boolean activo, pausado;
            protected long periodo;

            protected HiloJuego(long periodo) { this.periodo = periodo; }
            public void iniciar()  { activo = true; hilo = new Thread(this); hilo.start(); }
            public void detener()  { activo = false; }
            public void pausar()   { pausado = true; }
            public void reanudar() { pausado = false; }

            public final void run() {                       // el esquema fijo (Template
Method)
                while (activo) {
                    if (!pausado) ciclo();                  // el paso que cambia
                    try { Thread.sleep(periodo); } catch (InterruptedException e) { activo =
false; }
                }
            }
            protected abstract void ciclo();
        }
```

`HiloJugador`

Elemento	Detalle
`# jugador : Link`	Rol `#jugador 1` en el diagrama.
`+ HiloJugador(l : Link)`	`super(30)`.
`# ciclo()`	`jugador.actualizarEstado()`.

Relaciones: hereda de `HiloJuego*`; es parte de `Juego` (◆, `#hiloJugador 1`); referencia a `Link` (→, `#jugador 1`).

`HiloMundo`

Elemento	Detalle
`# sector : Sector`	El sector visible. Rol `#sector 1` en el diagrama.
`+ HiloMundo()`	`super(30)`.
`+ setSector(s : Sector)`	Lo llama `Juego.activarSector(s)`: desde la vuelta siguiente se actualiza ese sector.
`# ciclo()`	`sector.actualizarEntidades()`.

Relaciones: hereda de `HiloJuego*`; es parte de `Juego` (◆, `#hiloMundo 1`); referencia al `Sector` visible (→, `#sector 1`). La referencia cambia en cada transición.

`Reloj` (antes `Timer`)

Elemento	Detalle
`# segundos : int`, `# juego : Juego`	Como antes.
`+ Reloj(j : Juego)`	`super(1000)`. `Juego` lo crea con `new Reloj(this)`.
`+ reiniciar()`, `+ getSegundos()`	Como antes.
`# ciclo()`	`segundos++; juego.tic(segundos);`

Por qué el nombre nuevo: el enunciado prohíbe usar "Timers" para los movimientos. La clase nunca fue `javax.swing.Timer`, pero el nombre invitaba a la pregunta. **Relaciones:** hereda de `HiloJuego*`; es parte de `Juego` (◆, `#reloj 1`, antes `#timer 1`) y referencia a `Juego` (→, `#juego 1`), igual que en la defensa 4.

`AccionProgramada`

Elemento	Detalle
`# instante : long`	Cuándo ejecutarla (ms, reloj del sistema).
`# accion : Runnable`	Qué ejecutar. Se ejecuta **dentro** de la vuelta de la entidad, no en un hilo nuevo.
`+ AccionProgramada(instante : long, accion : Runnable)`	«constructor»
`+ estaVencida(ahora : long) : boolean`	«consulta»
`+ ejecutar()`	`accion.run()`.

Relaciones: es parte de `EntidadAnimada*` (◆, `#pendientes 0..*`). Ver 16.6.

`AnimadorTransicion` (capa gráfica, realiza `Runnable`)

Elemento	Detalle
`# panel : PanelPantallaNivel`	El panel que se desplaza. Rol `#panel 1`.
`# direccion : Direccion`	Hacia dónde.
`# controlador : ControladorJuego`	A quién avisar al terminar. Rol `#controlador 1`.
`+ AnimadorTransicion(p, d, c)`	«constructor»
`+ run()`	`panel.desplazar(direccion)` paso a paso y, al terminar, `controlador.transicionTerminada()`.

Relaciones: realiza `Runnable`; referencia a `PanelPantallaNivel` (→, `#panel 1`) y a `ControladorJuego` (→, `#controlador 1`). Conoce solo cosas de la vista y la **interfaz** de la lógica, así que respeta las tres capas. En la defensa 4 `GUI.transicion(d)` armaba un `Thread` anónimo; ahora hace `new`

`Thread(new AnimadorTransicion(panelNivel, d, controladorJuego)).start()` y el hilo queda visible en el diagrama.

16.4 Relaciones nuevas

Relación	Tipo	Rol / multiplicidad
`HiloJuego*` ⇢▷ `Runnable`	Realización	—
`HiloJugador`, `HiloMundo`, `Reloj` ⟶▷ `HiloJuego*`	Herencia (3)	—
`Juego` ◆ `HiloJugador`	Composición	`#hiloJugador 1`
`Juego` ◆ `HiloMundo`	Composición	`#hiloMundo 1`
`Juego` ◆ `Reloj`	Composición (ya estaba)	`#reloj 1` (antes `#timer 1`)
`HiloJugador` → `Link`	Asociación	`#jugador 1`
`HiloMundo` → `Sector`	Asociación	`#sector 1`
`Reloj` → `Juego`	Asociación (ya estaba)	`#juego 1`
`EntidadAnimada*` ◆ `AccionProgramada`	Composición	`#pendientes 0..*`
`AnimadorTransicion` ⇢▷ `Runnable`	Realización	—
`AnimadorTransicion` → `PanelPantallaNivel`	Asociación	`#panel 1`
`AnimadorTransicion` → `ControladorJuego`	Asociación	`#controlador 1`

Por qué composición (◆) entre `Juego` y sus hilos: los hilos se crean con el juego y no tienen sentido sin él, igual que el reloj. **Por qué composición con `AccionProgramada`:** cada acción pertenece a una sola entidad y desaparece con ella. **Por qué asociación de `HiloMundo` con `Sector`:** el sector no es del hilo, solo lo recorre, y cambia en cada transición.

Lo que no cambia: `Entidad*` sigue realizando `Colisionable` y `EntidadAnimada*` sigue realizando `Colisionador`; el Observer (`Entidad* → Observer`, `CeldaGrafica`, `CeldaJugador`) queda igual; `Sector → EntidadAnimada*` (`#animadas 0..*`) queda igual. La lógica sigue sin conocer clases gráficas, y la gráfica solo conoce interfaces de la lógica.

16.5 Animaciones: qué se toma del ejemplo de Candy Crush

Cómo es en Candy Crush. La lógica cambia **a saltos**: un caramelo pasa de una celda a otra de golpe. Para que se vea suave, la gráfica tiene animadores (`AnimadorMovimiento`, `AnimadorCaida`, `AnimadorIntercambio`...). Cada uno es un `Thread` que mueve el `JLabel` paso a paso. Una `CentralAnimaciones`, con un `HashMap` celda → animaciones, encola las de una misma celda para que no se pisen.

Por qué en Zelda no hace falta un animador por movimiento. En Zelda la lógica **ya avanza de a pocos píxeles** (2 a 4 por vuelta), así que cada aviso del Observer es un paso chico y el movimiento se ve suave. Un animador por movimiento crearía cientos de hilos para algo que ya funciona.

Qué se toma:

1. **El animador en la capa gráfica**, para lo que es solo visual y no tiene equivalente en la lógica: el **desplazamiento entre sectores** (`AnimadorTransicion`).

2. **La separación de capas**: el animador conoce paneles y celdas, nunca entidades lógicas. El aviso de fin vuelve a la lógica por `ControladorJuego`.

Sprites animados (caminar, atacar): siguen como en la defensa 4. La entidad cambia el estado de su `Sprite` en su vuelta, notifica, y la celda carga la imagen de `getRutaImagen()`. No necesitan hilo.

Si más adelante se quieren efectos de una sola vez que se puedan superponer (una explosión al morir, el giro del ataque recargado), se puede agregar una `CentralAnimaciones` con un animador por celda, como en Candy Crush. Hoy no hace falta, porque esos efectos se resuelven con estados del `Sprite`.

16.6 Acciones programadas (sin hilos extra)

Problema. En la defensa 4 varias entidades esperan un rato y después hacen algo con `programar(ms, accion)`, que crea un `Thread` que duerme y ejecuta:

Quién	Qué espera
`Link`	Fin de la invulnerabilidad (1 s); el suelo quebradizo se rompe (0,5 s).
`TrampaDeCuchillas`	Volver a su lugar (600 ms).
`MasterStalfos`	Fin de la invulnerabilidad (800 ms).
`StoneHinox`	Fin de la embestida (1,5 s), vulnerable 1 s y siguiente ataque.
`MoblinConEspadaYEscudo`	Volver a caminar al azar.

Eso suma hilos, y la acción corre **al mismo tiempo** que la vuelta de la entidad, así que los dos pueden tocar la misma defensa.

Solución. `programar` guarda un recordatorio y el hilo que actualiza a la entidad lo ejecuta cuando vence:

```
        public class AccionProgramada {
            protected long instante;     // cuándo ejecutarla (ms)
            protected Runnable accion;   // qué hacer
            public AccionProgramada(long instante, Runnable accion) { this.instante = instante;
this.accion = accion; }
            public boolean estaVencida(long ahora) { return ahora >= instante; }
            public void ejecutar() { accion.run(); }
        }

        // En EntidadAnimada
        protected List<AccionProgramada> pendientes = new ArrayList<>();      // #pendientes
0..*

        protected void programar(long ms, Runnable accion) {                  // misma firma
que en la defensa 4
            pendientes.add(new AccionProgramada(System.currentTimeMillis() + ms, accion));
        }
        private void ejecutarPendientes() {
            long ahora = System.currentTimeMillis();
            Iterator<AccionProgramada> it = pendientes.iterator();
            while (it.hasNext()) {
```

```
                AccionProgramada a = it.next();
                if (a.estaVencida(ahora)) { it.remove(); a.ejecutar(); }
            }
        }

        // Una vuelta de la entidad (reemplaza a run(); la llama HiloJugador o HiloMundo)
        public void actualizarEstado() {
            vigilarJugador();                       // gancho (vacío salvo en Enemigo*)
            movimiento.mover(this);                 // Strategy
            ataque.atacar(this);                    // Strategy
            miSector.verificarColisiones(this);     // detectar choques -> doble despacho
            ejecutarPendientes();                   // esperas vencidas
        }
```

Ejemplo: Link recibe un golpe cuando el reloj del sistema marca 10 000 ms

Instante	Qué pasa (todo en `HiloJugador`)
10 000 ms	`alSerGolpeado()` → `setDefensa(new DefensaInvulnerable())` y `programar(1000, () -> restaurarDefensa())`. Se guarda una acción con instante 11 000.
10 030, 10 060, …	En cada vuelta, `ejecutarPendientes()` ve que la acción no venció: no hace nada y Link sigue invulnerable.
11 010 ms	La acción venció: se ejecuta `restaurarDefensa()` y la acción sale de la lista. Link vuelve a poder recibir daño.

Ventajas. Los hilos siguen siendo cuatro, y la acción corre en el mismo hilo que el resto de la entidad. Si el hilo se pausa (transición), las acciones también esperan. La firma de `programar` no cambia, así que las clases que lo usan tampoco. La acción se ejecuta con hasta ~30 ms de demora, que no se nota.

16.7 Colisiones con hilos

16.7.1 Detectar y resolver (no cambia)

Paso	Quién	Cómo
Evitar antes de moverse	`Sector.puedeMoverA(e, d, pasos)`	Mira las celdas de la matriz que tocaría el rectángulo de `e` en la posición siguiente; si alguna entidad tiene `bloqueaPaso()` (roca, agua), no se mueve.
Detectar después de moverse	`Sector.verificarColisiones(quien)`	Revisa las celdas que toca `quien` y las animadas del sector; a cada entidad no eliminada que `intersecta(quien)` le envía `e.aceptarColision(quien)`.
Resolver	Doble despacho (Visitor)	`e.aceptarColision(c)` responde `c.colisionarCon(this)`. Como `this` tiene su tipo concreto, Java elige la sobrecarga justa: `colisionarCon(Enemigo)`, `colisionarCon(PowerUP)`, etc. Sin `instanceof`.
Golpe de espada (no es choque)	`Sector.golpearEn(atacante, alcance, danio)`	A cada entidad delante del atacante y a su alcance: `recibirGolpeDeEspada(atacante, danio)`. Cada una decide (el pasto se corta, el enemigo recibe daño, el Moblin con escudo bloquea de frente).

16.7.2 La regla nueva: qué hilo resuelve cada choque

Con dos hilos moviendo cosas, un mismo choque Link–enemigo puede detectarse **dos veces**: una desde el hilo de Link, que termina en `Link.colisionarCon(Enemigo)`, y otra desde el hilo del mundo, que termina en `Enemigo.colisionarCon(Link)`. Si los dos aplicaran daño casi en el mismo instante, Link perdería dos corazones.

Regla: todo choque en el que participa Link lo resuelve `HiloJugador`. `HiloMundo` solo resuelve los choques que no involucran a Link.

Choque	Lo resuelve	Método que actúa	Efecto
Link – enemigo	`HiloJugador`	`Link.colisionarCon(e : Enemigo)`	`recibirDanio(e.getDanioContacto(), e.getDireccion())`.
Link – proyectil	`HiloJugador`	`Link.colisionarCon(p : Proyectil)`	`recibirDanio(p.getDanio(), …)` y el proyectil sale del sector.
Link – obstáculo	`HiloJugador`	`Link.colisionarCon(o : Obstaculo)` → `o.afectar(this)`	Pasto: `ralentizar()`. Caída: `caerAlVacio()`. Suelo quebradizo: `romperSuelo(s)`.
Link – power-up	`HiloJugador`	`Link.colisionarCon(p : PowerUP)` → `p.serRecogidoPor(this)`	Efecto sobre Link, puntos en `Juego` y el power-up sale del sector.
Link – objeto	`HiloJugador`	`Link.colisionarCon(o : Objeto)` → `o.serRecogidoPor(this)`	Va al inventario.
Proyectil – roca/agua	`HiloMundo`	`Proyectil.colisionarCon(o : Obstaculo)`	Si `bloqueaPaso()`, desaparece.
Trampa – bloqueo	`HiloMundo`	`TrampaDeCuchillas.colisionarCon(o : Obstaculo)`	Vuelve a su lugar.

Cómo se cumple la regla, sin `instanceof`:

- **`HiloMundo` no actualiza a Link.** Link sigue en la lista `animadas` del sector (no cambia nada de `agregar`, `retirar` ni del doble despacho `ubicarseEn`), pero `Sector.actualizarEntidades()` llama a cada animada un gancho de vuelta que **`Link` redefine vacío**, porque a Link lo actualiza `HiloJugador`. Es el mismo recurso de ganchos que ya usa `EntidadAnimada*` (ver `EntidadAnimada*`).
- **El lado del enemigo no hace nada.** `Enemigo.colisionarCon(l : Link)` y `Proyectil.colisionarCon(l : Link)` quedan como el gancho vacío heredado de `EntidadAnimada*`. Si el choque se detecta desde `HiloMundo`, el doble despacho llega a ese gancho y no pasa nada; el daño solo lo aplica el lado de Link.
- **Aunque Link esté quieto**, su hilo da una vuelta cada 30 ms y llama a `verificarColisiones(link)`, así que un proyectil que lo alcanza se detecta igual, como mucho 30 ms después.

16.7.3 Golpes que cruzan de un hilo a otro

Hay casos en los que un hilo cambia una entidad que actualiza el otro:

Caso	Hilo que golpea	Entidad golpeada	Protección
Link ataca con la espada	`HiloJugador` (`AtaqueEspada` / `AtaqueRecargado` → `golpearEn`)	Enemigos (los actualiza `HiloMundo`)	`EntidadAnimada.recibirDanio` y `aplicarDanio` son `synchronized`.
MasterStalfos ataca con la espada	`HiloMundo`	Link (lo actualiza `HiloJugador`)	Igual: `Link.recibirDanio` es `synchronized`.
Un enemigo muere por la espada y suelta un drop	`HiloJugador`	El sector (lo recorre `HiloMundo`)	`Sector.agregar` es `synchronized` y `actualizarEntidades` recorre una copia de `animadas`.

16.7.4 Ejemplo: una vuelta de `HiloJugador` en la que Link junta un corazón

Paso	Emisor → Receptor	Mensaje
1	`HiloJugador` → `Link`	`actualizarEstado()`
2	`Link` → `MovimientoInducido`	`mover(this)` (dirección que fijó el teclado)
3	`MovimientoInducido` → `Sector`	`puedeMoverA(link, d, 4)` → true
4	`MovimientoInducido` → `Link`	`avanzar(d, 4)` → `setPosicion` → `notificarObservers()` → `CeldaJugador.actualizar()`
5	`Link` → `AtaqueRecargado`	`atacar(this)` (no pasa nada si no soltó la tecla)
6	`Link` → `Sector`	`verificarColisiones(link)`
7	`Sector` → `Corazon`	`aceptarColision(link)` (se superponen)
8	`Corazon` → `Link`	`colisionarCon(this)` → sobrecarga `colisionarCon(PowerUP)`
9	`Link` → `Corazon`	`serRecogidoPor(this)` → `link.curar(2)`, `powerUpRecogido`, `quitarDelSector`
10	`Link` → `Link`	`ejecutarPendientes()`

16.8 Concurrencia: qué se sincroniza y por qué

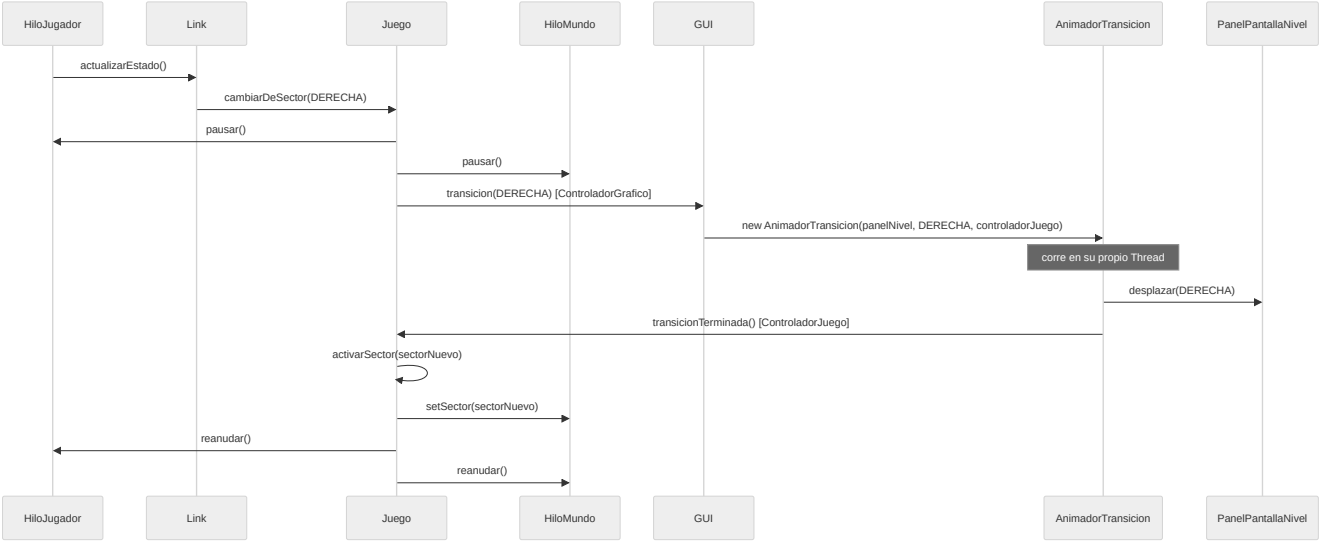
Recurso compartido	Lo tocan	Medida
`Sector.entidades` / `animadas`	`HiloJugador`, `HiloMundo` y `Juego` (al activar sectores)	`synchronized` en `agregar`, `agregarAnimada`, `quitar`, `retirar`, `reemplazar`, `verificarColisiones`, `golpearEn` y `actualizarEntidades`. `actualizarEntidades` recorre una **copia** de `animadas`, porque un proyectil o un drop nuevos pueden agregarse en medio.
Vida y defensa de una entidad animada	El hilo que la actualiza y el hilo que la golpea	`synchronized` en `recibirDanio` y `aplicarDanio`.
`activo`, `pausado` de los hilos	El hilo y quien lo detiene o pausa	`volatile`.

Recurso compartido	Lo tocan	Medida
Intenciones de Link (`direccion`, `atacando`, `soltoAtaque`, `escudoActivo`)	El EDT (teclado) las escribe y `HiloJugador` las lee	`volatile`.
Componentes Swing	Los dibuja el EDT	Todo cambio gráfico con `SwingUtilities.invokeLater` (ya está en `CeldaGrafica.actualizar()`).
`Juego` (puntaje, vidas, `tic`)	`HiloJugador` (power-ups, daño), `HiloMundo` (enemigo muerto por un proyectil o una caída), `Reloj`	`synchronized` en los métodos de `Juego` que cambian estado (`powerUpRecogido`, `enemigoDerrotado`, `jugadorDaniado`, `tic`, `terminarPartida`).

Arreglo relacionado: `Link.cargarAtaque()` tiene que empezar a cargar **solo si** `!atacando`. Swing repite `keyPressed` mientras la tecla está apretada; sin esa guarda la carga se reinicia cada vez y nunca llega a 3 s.

16.9 Transición entre sectores

Paso	Qué pasa
1	En su vuelta, `Link.avanzar` detecta que salió del sector y avisa `miJuego.cambiarDeSector(d)`.
2	`Juego` hace `hiloJugador.pausar()` y `hiloMundo.pausar()`. Nadie se mueve durante el desplazamiento; el `Reloj` sigue.
3	`Juego` pasa a Link al sector vecino (`retirar` / `agregar`, `entrarPorBordeOpuesto`) y llama `controladorGrafico.transicion(d)`.
4	`GUI.transicion(d)` hace `new Thread(new AnimadorTransicion(panelNivel, d, controladorJuego)).start()`.
5	En el hilo del animador, `panel.desplazar(d)` mueve la pantalla paso a paso (cada paso con `invokeLater`).
6	Al terminar, `controlador.transicionTerminada()` → `Juego.activarSector(nuevo)` arma las celdas y hace `hiloMundo.setSector(nuevo)`; después `reanudar()` en los dos hilos.



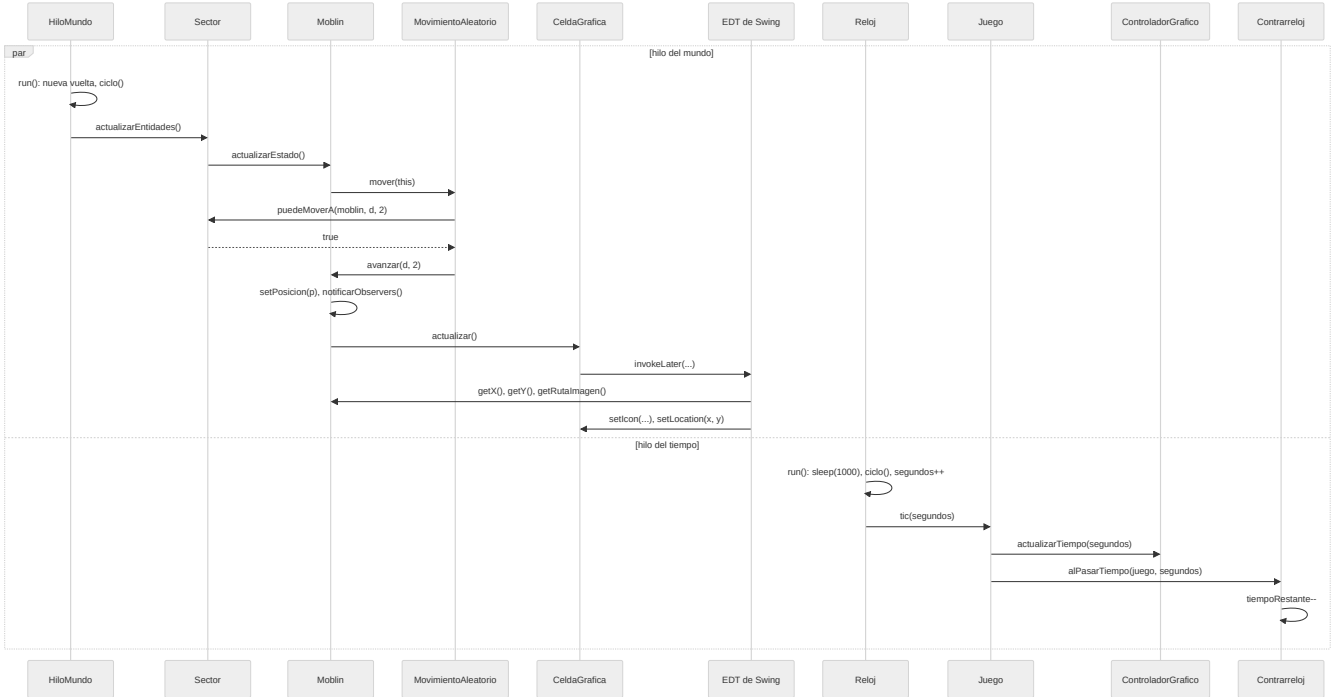
Como `HiloJugador` está pausado durante la transición, las acciones programadas de Link también esperan.

16.10 Cómo quedan los diagramas de secuencia 2 y 3 de la defensa 5

Diagrama 2: Link equipa la espada y hace el ataque recargado. El EDT solo deja la **intención**; el golpe lo da `HiloJugador`. Conviene marcar con una nota el cambio de hilo entre los pasos 4 y 5.

Paso	Hilo	Emisor → Receptor	Mensaje
1	EDT	`GUI`	`keyPressed(e)` → `procesarTecla(TECLA_CAMBIO, true)`
2	EDT	`GUI` → `ControladorJuego (Juego)`	`cambiarObjeto()` → `Link.cambiarObjeto()` → `notificarObservers()` → `CeldaJugador.actualizar()` (HUD: "Espada")
3	EDT	`GUI` → `ControladorJuego`	`procesarTecla(A, true)` → `atacar()` → `Link.atacar()` → `Espada.atacarCon(l)` → `Link.cargarAtaque()` → `AtaqueRecargado.cargar()`
			Nota: pasan 3 s; los `keyPressed` repetidos no reinician la carga
4	EDT	`GUI` → `ControladorJuego`	`procesarTecla(A, false)` → `soltarAtaque()` → `Link.soltarAtaque()` → `Espada.soltarAtaqueCon(l)` → `Link.liberarAtaque()`
5	`HiloJugador`	`HiloJugador` → `Link`	`actualizarEstado()` → `AtaqueRecargado.atacar(link)` → `estaCargado()` = true → `golpearEn(link, 64, 4)` → `Link.ataqueRealizado()` → `notificarObservers()` → `CeldaJugador.actualizar()`

Diagrama 3: un enemigo se mueve mientras el reloj descuenta un segundo. El "hilo responsable de actualizar las entidades" es `HiloMundo` y "el hilo que lleva el tiempo" es `Reloj`. Van en un fragmento `par`:



Para que el reloj **descuente** un segundo al modo, como pide el enunciado, `Contrarreloj` necesita `# tiempoRestante : int` (se inicia con `tiempoLimite` al cargar el nivel) y `alPasarTiempo` lo baja en uno; si llega a 0, llama `j.terminarPartida()`.

16.11 Impacto en el extendido

El extendido **no se cambió** para esta entrega. Para que coincida con este reducido habría que hacer estos cambios:

Clase	Cambio
Nuevas	`Runnable`, `HiloJuego*`, `HiloJugador`, `HiloMundo`, `AccionProgramada` y `AnimadorTransicion`, con los atributos y operaciones de 16.3. `Timer` pasa a llamarse `Reloj` y hereda de `HiloJuego*`.
`Entidad*`	Salen `iniciar()` y `detener()`.
`EntidadAnimada*`	Salen `iniciar()`, `detener()`, `run()` y `activa`. Entran `+ actualizarEstado()` (una vuelta), `# pendientes : List<AccionProgramada>` y `- ejecutarPendientes()`. `programar(ms, accion)` mantiene la firma. `recibirDanio` y `aplicarDanio` pasan a ser `synchronized`. Gancho vacío `# vigilarJugador()`.
`Link`	Redefine vacío el gancho de vuelta que llama `HiloMundo`; `cargarAtaque()` con guarda `!atacando`; intenciones `volatile`.
`Enemigo*`, `Proyectil`	`colisionarCon(l : Link)` sale: queda el gancho vacío heredado (el choque lo resuelve Link).
`Sector`	Salen `iniciarEntidades()` y `detenerEntidades()`. Entra `+ actualizarEntidades()`. Métodos `synchronized` (16.8).
`Juego`	`# reloj : Reloj`, `# hiloJugador : HiloJugador`, `# hiloMundo : HiloMundo`. Cambian `iniciar`, `activarSector`, `cambiarDeSector`, `transicionTerminada` y `terminarPartida` (arrancar, pausar, reanudar y detener los hilos).
`ModoJuego*` / `Contrarreloj`	Gancho `alIniciarNivel()`; `Contrarreloj` con `# tiempoRestante : int` y `getTiempoRestante()`.
`GUI`	`transicion(d)` crea un `AnimadorTransicion` en vez de un `Thread` anónimo.

16.12 Preguntas probables en la defensa

Pregunta	Respuesta corta
¿Cuántos hilos hay y quién los arranca?	Tres fijos: `HiloJugador`, `HiloMundo` y `Reloj`. Los arranca `Juego.iniciar()` y los detiene `terminarPartida()`. Más `AnimadorTransicion` durante cada transición y el EDT de Swing.
¿Por qué Link tiene su propio hilo?	Responde al teclado sin depender de cuántos enemigos haya, y así sus colisiones se resuelven en un solo lugar.
¿Por qué no un hilo por enemigo?	Serían decenas de hilos tocando el sector a la vez. Uno solo los recorre en orden.
¿Cómo evitan que un choque haga daño dos veces?	Todo choque con Link lo resuelve el hilo de Link; desde el lado del enemigo o del proyectil, `colisionarCon(Link)` es un gancho vacío.
¿Qué pasa si la espada de Link golpea a un enemigo que se está moviendo?	`recibirDanio` es `synchronized`: el daño y el movimiento no se pisan.

Pregunta	Respuesta corta
¿Por qué `invokeLater`?	Swing solo se toca desde su propio hilo y los avisos llegan desde `HiloJugador` o `HiloMundo`.
¿Cómo esperan sin `Reloj` ni hilos extra?	Con `AccionProgramada`: el hilo que actualiza a la entidad revisa en cada vuelta si ya venció.
¿Qué patrón usa `HiloJuego*`?	Template Method: `run()` fijo y `ciclo()` redefinido por cada hilo.
¿Por qué `Runnable` y no heredar de `Thread`?	`HiloJuego*` y `AnimadorTransicion` describen **qué** se ejecuta; el `Thread` es solo el que lo ejecuta. Así se puede crear un hilo nuevo al reiniciar sin cambiar la clase.
¿Qué toman del ejemplo de animadores?	El animador gráfico para la transición. Para el movimiento no hace falta, porque la lógica ya avanza de a pocos píxeles.
¿Por qué `AnimadorTransicion` está en la vista y no en la lógica?	Mueve el panel, que es Swing. Solo conoce la interfaz `ControladorJuego` para avisar que terminó.

Glosario

Término	Significado
Mensaje	Invocación de un método de otro objeto (o del mismo).
Comando / consulta	Método que cambia el estado / que devuelve información sin cambiarlo.
Polimorfismo	El mismo mensaje ejecuta código distinto según la clase real del receptor.
Doble despacho	Dos llamadas encadenadas para elegir el código según el tipo de dos objetos, sin `instanceof`.
Gancho	Método vacío que una subclase redefine para reaccionar a algo (`alSerGolpeado`).
Strategy	Patrón: el comportamiento (moverse, atacar, defenderse) está en un objeto aparte e intercambiable.
Template Method	Patrón: un método de la superclase fija los pasos de un algoritmo y las subclases redefinen algunos (`HiloJuego.run()` y el `ciclo()` de cada hilo; `AtaquePorProyectil.atacar` y los pasos que redefine `AtaqueAvalancha`).
Observer	Patrón: la entidad avisa `actualizar()` a sus celdas cuando cambia.
Abstract Factory	Patrón: una fábrica por dominio gráfico crea los sprites de todas las entidades.
Factory Method	Patrón: `generarProyectil()` / `generarDrop()` deciden qué objeto crear.
Hilo (`Thread`)	Ejecución en paralelo. En la defensa 5 hay cuatro propios: `HiloJugador` (Link), `HiloMundo` (el sector visible), `Reloj` (el tiempo) y `AnimadorTransicion` (la transición).
`Runnable`	Interfaz de Java con `run()`: lo que ejecuta un `Thread`.
EDT	El hilo de Swing (*Event Dispatch Thread*): atiende el teclado y dibuja. Solo él toca componentes gráficos, por eso `invokeLater`.
Acción programada	Algo que una entidad hace después de un tiempo (`programar(ms, accion)`), guardado como `AccionProgramada` y ejecutado por el hilo que la actualiza.
`synchronized` / `volatile`	`synchronized`: un solo hilo a la vez ejecuta ese método sobre el objeto. `volatile`: el cambio de un atributo lo ven enseguida los otros hilos.

Término	Significado
Parseo	Leer un texto con formato (el archivo del nivel) y convertirlo en objetos (escena, sectores y entidades).
Sector / Escena	Una pantalla del mapa / el mapa completo (grilla de sectores).
Dominio	El conjunto de imágenes del juego (original o alternativo).
HUD	Barra con nivel, puntaje, tiempo, vidas, rupias y objeto equipado.
Marca `+` al final de una firma	Notación del diagrama de la defensa 4 para algunos métodos que redefinen uno heredado. No es UML estándar.
⚠	En este documento: un mensaje que hace falta para la comunicación descrita pero que el diagrama de la defensa 4 no tiene. Hoy no queda ninguno: todos los mensajes descritos existen.
Píxel	Unidad de la posición de las entidades animadas: `Posicion(x, y)` se mide en píxeles dentro del sector.
Celda	Casillero de la matriz `entidades : Entidad[][]` del `Sector`; mide `CELDA = 48` píxeles de lado.
Puntuable	Entregable que evalúa la cátedra: los PDF de esta carpeta, que no se modifican después de la defensa.

Registro de cambios

Fecha	Cambio
2026-10-13	Creación: une el resumen del enunciado, los requerimientos de la defensa 4, la explicación del diagrama reducido de `Defensa 4/` y la explicación de la posición en píxeles y en la matriz, con índice general.
2026-10-19	Versión de la defensa 5 (`Defensa 5/`): parte B con los requerimientos de la defensa 5; C.1 con los cambios del reducido (seis clases y doce relaciones de hilos, `Timer` → `Reloj`) y la historia de la defensa 4 como C.2; fichas de `Reloj`, `HiloJuego*`, `HiloJugador`, `HiloMundo`, `AnimadorTransicion` y `AccionProgramada`; ciclo de las entidades como `actualizarEstado()`; transición con `AnimadorTransicion`; observaciones 28 a 30; nueva sección 16 (hilos, animaciones y colisiones). La regla de colisiones deja a Link en `animadas` y usa ganchos vacíos en vez de sacarlo de la lista.